

ANALYSE DU PROJET DWWWM

FICHER : .devcontainer/app/devcontainer.json

```
{
  "name": "Secret-de-Hammam - app",
  "service": "app",
  "dockerComposeFile": "../../../docker-compose.dev.yaml",
  "workspaceFolder": "/app",
  "customizations": {
    "vscode": {
      "extensions": [
        "christian-kohler.npm-intellisense",
        "christian-kohler.path-intellisense",
        "biomejs.biome",
        "dsznajder.es7-react-js-snippets",
        "VASubasRaj.flashpost",
        "ritwickdey.LiveServer"
      ],
      "settings": {
        "[typescript]": {
          "editor.defaultFormatter": "biomejs.biome",
          "editor.formatOnSave": true,
          "editor.formatOnType": true
        },
        "[typescriptreact]": {
          "editor.defaultFormatter": "biomejs.biome",
          "editor.formatOnSave": true,
          "editor.formatOnType": true
        },
        "[javascript]": {
          "editor.defaultFormatter": "biomejs.biome",
          "editor.formatOnSave": true,
          "editor.formatOnType": true
        },
        "[javascriptreact]": {
          "editor.defaultFormatter": "biomejs.biome",
          "editor.formatOnSave": true,
          "editor.formatOnType": true
        },
        "[css]": {
          "editor.defaultFormatter": "biomejs.biome",
          "editor.formatOnSave": true,
          "editor.formatOnType": true
        },
        "editor.codeActionsOnSave": {
          "source.fixAll": "always"
        }
      }
    }
  }
}
```

```
    }  
  }  
}
```

FICHER : .devcontainer/mongodb/devcontainer.json

```
// Reprendre les infos du fichier yaml  
{  
  "name": "Secret-de-Hammam - MongoDB",  
  "service": "mongodb", // reprendre strictement le nom du fichier  
  "dockerComposeFile": "../../docker-compose.dev.yaml", // reprendre  
    strictement le nom du fichier yaml  
  "workspaceFolder": "/app" // reprendre strictement le nom de  
    working_dir  
}
```

FICHER : .devcontainer/mysql/devcontainer.json

```
{  
  "name": "Secret-de-Hammam - MySQL",  
  "service": "mysql",  
  "dockerComposeFile": "../../docker-compose.dev.yaml",  
  "workspaceFolder": "/app",  
  "customizations": {  
    "vscode": {  
      "extensions": [],  
      "settings": {}  
    }  
  }  
}
```

FICHER : README.md

```
# Secret-de-Hammam
```

FICHER : bundle-code.js

```

const fs = require('fs');
const path = require('path');

// 1. Configuration des fichiers/dossiers à ignorer
const IGNORE_LIST = [
  'node_modules', '.git', 'dist', 'build', '.next',
  'package-lock.json', 'yarn.lock', '.gitignore', '.env', 'mysql-data',
  'venv', '.vscode'
];

// 2. Extensions de fichiers que nous voulons inclure
const ALLOWED_EXTENSIONS = ['.ts', '.tsx', '.js', '.jsx', '.css', '.json',
  '.html', '.md'];

const outputFile = 'mon_projet_certification.md';

/**
 * Fonction principale qui parcourt les dossiers récursivement
 */
function generateMarkdown(dir, fileList = []) {
  const files = fs.readdirSync(dir);

  files.forEach(file => {
    const filePath = path.join(dir, file);
    const stat = fs.statSync(filePath);

    // Si c'est un dossier et qu'il n'est pas dans la liste noire, on
    descend dedans
    if (stat.isDirectory()) {
      if (!IGNORE_LIST.includes(file)) {
        generateMarkdown(filePath, fileList);
      }
    } else {
      // Si c'est un fichier, on vérifie son extension
      const ext = path.extname(file);
      if (ALLOWED_EXTENSIONS.includes(ext) &&
!IGNORE_LIST.includes(file)) {
        const content = fs.readFileSync(filePath, 'utf8');
        const relativePath = path.relative(process.cwd(),
filePath);

        // On formate le fichier pour le Markdown
        const markdownBlock = `\n---\n### FICHER :
${relativePath}\n\n\`\`\`${ext.replace('.', '')}\n${content}\n\`\`\`\n`;
        fs.appendFileSync(outputFile, markdownBlock);
      }
    }
  });
}

// Nettoyage du fichier de sortie s'il existe déjà
if (fs.existsSync(outputFile)) fs.unlinkSync(outputFile);

```

```
fs.writeFileSync(outputFile, "# ANALYSE DU PROJET DWWM\n");
console.log("Extraction en cours...");
generateMarkdown('.');
console.log(`Terminé ! Votre fichier est prêt : ${outputFile}`);
```

FICHER : code/README.md

Vite + RSC

This example shows how to set up a React application with [Server Component](<https://react.dev/reference/rsc/server-components>) features on Vite using [`@vitejs/plugin-rsc`](<https://github.com/vitejs/vite-plugin-react/tree/main/packages/plugin-rsc>).

[![Open in StackBlitz](https://developer.stackblitz.com/img/open_in_stackblitz.svg)](<https://stackblitz.com/github/vitejs/vite-plugin-react/tree/main/packages/plugin-rsc/examples/starter>)

```
```sh
```

```
run dev server
npm run dev
```

```
build for production and preview
npm run build
npm run preview
```

## API usage

See [@vitejs/plugin-rsc](#) for the documentation.

- `vite.config.ts`
  - `@vitejs/plugin-rsc/plugin`
- `./src/framework/entry.rsc.tsx`
  - `@vitejs/plugin-rsc/rsc`
  - `import.meta.viteRsc.loadModule`
- `./src/framework/entry.ssr.tsx`
  - `@vitejs/plugin-rsc/ssr`
  - `import.meta.viteRsc.loadBootstrapScriptContent`
  - `rsc-html-stream/server`
- `./src/framework/entry.browser.tsx`
  - `@vitejs/plugin-rsc/browser`
  - `rsc-html-stream/client`

## Notes

- `./src/framework/entry.{browser,rsc,ssr}.tsx` (with inline comments) provides an overview of how low level RSC (React flight) API can be used to build RSC framework.
- You can use `vite-plugin-inspect` to understand how `"use client"` and `"use server"` directives are transformed internally.

## Deployment

See [vite-plugin-rsc-deploy-example](#)

```

FICHER : code/Test.ts

```ts
//REPOSITORY

// sélectionner un enregistrement
// data représente une partie des propriétés du type
public
selectOne = async (data: Partial<Role>): Promise<Role | unknown> => {
  // connexion au serveur MySQL
  const connection = await new MySQLService().connect(); // requete SQL
  // where category.id=....
  // SELECT level.* FROM arabizy_dev.level;
  // variable de requete : précédée d'un :, suivi du nom de la variable
  // requetes préparées :sécurité;(utilisation des varibales de
  requetes)la requete est exécutée si elle ne représente pas de risque de
  sécurité
  const sql = SELECT;
  $;
  this.table;
  .*
  FROM $
  process.env.MYSQL_DATABASE;
  .$
  this.table;
  where;
  $;
  this.table;
  .id= :id
  // try / catch: récupérer les résultats de la requete ou un erreur
  try {
    // exécution de la requete
    // si la requete possède des variables, utiliser le paramètre de
    la méthode
    const [query] = await connection.execute(sql, data);
    // récupérer le premier indice d'un array
    const result = (query as Role[]).shift();
    // retourner les résultats
    return result;
  } catch (error) {
    return error;
  }
}

```

```

    }
};

//controllers;

public
selectOne = async (req: Request, res: Response) => {
    // récupérer la variable de route
    // req.params :rrécupère les variables de route
    console.log(req.params);
    // récupération des résultats de la requete
    const results = await new RoleRepository().selectOne(req.params);
    // si la requete renvoie une erreur
    if (results instanceof Error) {
        res.status(400).json({
            status: 400,
            message:
                process.env.NODE_ENV === "production" ? "Error" :
results.message,
        });
        return;
    }
    // envoyer une réponse
    // res.send("coucou");
    // renvoyer une réponse avec un code de statut HTTP et au format JSON
    res.status(200).json({
        status: 200,
        message: "ok level",
        data: results,
    });
};

//router;
// // variable de route:précédée par un ;; suivie du nom de la variable
// appel 1
this.router.get("/:id", new RoleController().selectOne);

```

FICHER : code/_flashpost/flashpost-collection-body_part.json

```

{"app": "Flashpost", "id": "f4c9fbc8-0fec-4e25-bab3-b2f42db98c17", "name": "body_part", "version": "1.0", "type": "collections", "createdAt": "05-Dec-2025 14:16:23", "exportedDate": "05-Dec-2025 14:16:48", "collections": [{"id": "f4c9fbc8-0fec-4e25-bab3-b2f42db98c17", "parent": "0", "text": "body_part", "createdAt": "05-Dec-2025 14:16:23", "draggable": true, "data": {"treeNodeType": "collection", "variableId": ""}, "relativeIndex": 0},

```

```

{"id": "874c7867-14c1-4060-b6d1-073ec44b5235", "parent": "f4c9fbc8-0fec-4e25-bab3-b2f42db98c17", "text": "127.0.0.1:3000/api/body_part/3", "createdTime": "05-Dec-2025 15:13:11", "draggable": false, "data": {"id": "f8e1841f-5a2c-4cea-8008-0519516254fb", "method": "get", "name": "127.0.0.1:3000/api/body_part/3", "url": "127.0.0.1:3000/api/body_part/3", "createdTime": "05-Dec-2025 15:13:11", "$loki": 14, "treeNodeType": "request"}}, {"id": "4ffb410a-3504-4131-a613-c442e14174de", "parent": "f4c9fbc8-0fec-4e25-bab3-b2f42db98c17", "text": "127.0.0.1:3000/api/body_part", "createdTime": "05-Dec-2025 15:12:35", "draggable": false, "data": {"id": "af61a4c0-f02e-4d24-9528-f839684ac4b0", "method": "get", "name": "127.0.0.1:3000/api/body_part", "url": "127.0.0.1:3000/api/body_part", "createdTime": "05-Dec-2025 15:12:35", "$loki": 13, "treeNodeType": "request"}}], "requests": [{"id": "874c7867-14c1-4060-b6d1-073ec44b5235", "url": "127.0.0.1:3000/api/body_part/3", "name": "127.0.0.1:3000/api/body_part/3", "createdTime": "05-Dec-2025 15:13:11", "method": "get", "params": [{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth": {"authType": "noauth", "userName": "", "password": "", "addTo": "queryParams", "showPwd": false, "tokenPrefix": "", "aws": {"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "none", "formdata": [{"isChecked": false, "key": "", "value": ""}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"}, {"id": "4ffb410a-3504-4131-a613-c442e14174de", "url": "127.0.0.1:3000/api/body_part", "name": "127.0.0.1:3000/api/body_part", "createdTime": "05-Dec-2025 15:12:35", "method": "get", "params": [{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth": {"authType": "noauth", "userName": "", "password": "", "addTo": "queryParams", "showPwd": false, "tokenPrefix": "", "aws": {"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "none", "formdata": [{"isChecked": false, "key": "", "value": ""}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"}]}

```

FICHER : code/_flashpost/flashpost-collection-category.json

```
{
  "app": "Flashpost",
  "id": "b6bd3bce-70c8-43a3-8cd7-11c8ecd2e917",
  "name": "category",
  "version": "1.0",
  "type": "collections",
  "createdAt": "05-Dec-2025 13:38:19",
  "exportedDate": "05-Dec-2025 13:39:54",
  "collections": [
    {
      "id": "b6bd3bce-70c8-43a3-8cd7-11c8ecd2e917",
      "parent": "0",
      "text": "category",
      "createdAt": "05-Dec-2025 13:38:19",
      "draggable": true,
      "data": {
        "treeNodeType": "collection",
        "variableId": "",
        "relativeIndex": 0
      },
      "id": "c86f2434-3063-4230-b343-acf21492a1af",
      "parent": "b6bd3bce-70c8-43a3-8cd7-11c8ecd2e917",
      "text": "127.0.0.1:3000/api/category/1",
      "createdAt": "05-Dec-2025 14:38:57",
      "draggable": false,
      "data": {
        "id": "177799a8-d92f-4480-87a5-37b24b6afd6d",
        "method": "get",
        "name": "127.0.0.1:3000/api/category/1",
        "url": "127.0.0.1:3000/api/category/1",
        "createdAt": "05-Dec-2025 14:38:57",
        "$loki": 7,
        "treeNodeType": "request"
      },
      "id": "dd92977e-8538-45c9-8889-b7366d8898c2",
      "parent": "b6bd3bce-70c8-43a3-8cd7-11c8ecd2e917",
      "text": "127.0.0.1:3000/api/category",
      "createdAt": "05-Dec-2025 14:37:38",
      "draggable": false,
      "data": {
        "id": "2715445d-c94c-4135-aa26-8c43797b6fbe",
        "method": "get",
        "name": "127.0.0.1:3000/api/category",
        "url": "127.0.0.1:3000/api/category/1",
        "createdAt": "05-Dec-2025 14:37:38",
        "$loki": 6,
        "treeNodeType": "request"
      }
    },
    {
      "id": "c86f2434-3063-4230-b343-acf21492a1af",
      "url": "127.0.0.1:3000/api/category/1",
      "name": "127.0.0.1:3000/api/category/1",
      "createdAt": "05-Dec-2025 14:38:57",
      "method": "get",
      "params": {
        "isChecked": false,
        "key": "",
        "value": ""
      },
      "pathParams": [],
      "auth": {
        "authType": "noauth",
        "userName": "",
        "password": "",
        "addTo": "queryParams",
        "showPwd": false,
        "tokenPrefix": "",
        "aws": {
          "service": "",
          "region": "",
          "accessKey": "",
          "secretAccessKey": "",
          "sessionToken": ""
        },
        "headers": [
          {
            "key": "Accept",
            "value": "*/*",
            "isChecked": true
          },
          {
            "key": "User-Agent",
            "value": "Flashpost",
            "isChecked": true
          },
          {
            "key": "",
            "value": "",
            "isChecked": false
          }
        ],
        "body": {
          "bodyType": "none",
          "formdata": {
            "isChecked": false,
            "key": "",
            "value": ""
          },
          "urlencoded": {
            "isChecked": false,
            "key": "",
            "value": ""
          },
          "raw": {
            "data": "",
            "lang": "json",
            "binary": {
              "fileName": "",
              "data": {}
            },
            "contentTypeOption": "manual",
            "graphql": {
              "query": "",
              "variables": ""
            },
            "tests": [
              {
                "parameter": "",
                "action": "",
                "expectedValue": ""
              },
              {
                "parameter": "",
                "key": "",
                "variableName": ""
              }
            ],
            "notes": "",
            "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"
          },
          "id": "dd92977e-8538-45c9-8889-b7366d8898c2",
          "url": "127.0.0.1:3000/api/category/1",
          "name": "127.0.0.1:3000/api/category",
          "createdAt": "05-Dec-2025 14:37:38",
          "method": "get",
          "params": {
            "isChecked": false,
            "key": "",
            "value": ""
          },
          "pathParams": [],
          "auth": {
            "authType": "noauth",
            "userName": "",
            "password": "",
            "addTo": "queryParams",
            "showPwd": false,
            "tokenPrefix": "",
            "aws": {
              "service": "",
              "region": "",
              "accessKey": "",
              "secretAccessKey": "",
              "sessionToken": ""
            },
            "headers": [
              {
                "key": "Accept",
                "value": "*/*",
                "isChecked": true
              },
              {
                "key": "User-Agent",
                "value": "Flashpost",
                "isChecked": true
              },
              {
                "key": "",
                "value": "",
                "isChecked": false
              }
            ],
            "body": {
              "bodyType": "none",
              "formdata": {
                "isChecked": false,
                "key": "",
                "value": ""
              },
              "urlencoded": {
                "isChecked": false,
                "key": "",
                "value": ""
              },
              "raw": {
                "data": "",
                "lang": "json",
                "binary": {
                  "fileName": "",
                  "data": {}
                },
                "contentTypeOption": "manual",
                "graphql": {
                  "query": "",
                  "variables": ""
                },
                "tests": [
                  {
                    "parameter": "",
                    "action": "",
                    "expectedValue": ""
                  },
                  {
                    "parameter": "",
                    "key": "",
                    "variableName": ""
                  }
                ],
                "notes": "",
                "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"
              }
            }
          }
        }
      }
    }
  ]
}
```



```

owPwd": false, "tokenPrefix": "", "aws":
{"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "none", "formdata": [{"isChecked": false, "key": "", "value": ""}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"]}]

```

FICHER : code/_flashpost/flashpost-collection-homepage.json

```

{"app": "Flashpost", "id": "05ca44cd-054a-44b6-a922-9b7e8830b5a3", "name": "homepage", "version": "1.0", "type": "collections", "createdAt": "05-Dec-2025 12:51:22", "exportedDate": "05-Dec-2025 12:52:48", "collections": [{"id": "05ca44cd-054a-44b6-a922-9b7e8830b5a3", "parent": "0", "text": "homepage", "createdAt": "05-Dec-2025 12:51:22", "draggable": true, "data": {"treeNodeType": "collection", "variableId": ""}, "relativeIndex": 0}, {"id": "bb2a5775-780f-4ea9-946c-33a60527eef3", "parent": "05ca44cd-054a-44b6-a922-9b7e8830b5a3", "text": "127.0.0.1:3000/api", "createdAt": "05-Dec-2025 13:50:00", "draggable": false, "data": {"id": "5b7dd5ed-c995-447f-bf5c-cec9018e9fee", "method": "get", "name": "127.0.0.1:3000/api", "url": "127.0.0.1:3000/api", "createdAt": "05-Dec-2025 13:50:00", "$loki": 3, "treeNodeType": "request"}}, {"requests": [{"id": "bb2a5775-780f-4ea9-946c-33a60527eef3", "url": "127.0.0.1:3000/api", "name": "127.0.0.1:3000/api", "createdAt": "05-Dec-2025 13:50:00", "method": "get", "params": [{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth": {"authType": "noauth", "userName": "", "password": "", "addTo": "queryParams", "showPwd": false, "tokenPrefix": "", "aws": {"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "none", "formdata": [{"isChecked": false, "key": "", "value": ""}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"}]}]}

```

FICHER : code/_flashpost/flashpost-collection-pack.json

```
{
  "app": "Flashpost",
  "id": "91a7ecfc-2514-4439-a207-63a88d331b5e",
  "name": "pack",
  "version": "1.0",
  "type": "collections",
  "createdTime": "05-Dec-2025 14:09:39",
  "exportedDate": "05-Dec-2025 14:10:41",
  "collections": [
    {
      "id": "91a7ecfc-2514-4439-a207-63a88d331b5e",
      "parent": "0",
      "text": "pack",
      "createdTime": "05-Dec-2025 14:09:39",
      "droppable": true,
      "data": {
        "treeNodeType": "collection",
        "variableId": "",
        "relativeIndex": 0
      },
      "id": "6b422a11-ffde-42cf-a832-016146ea36dd",
      "parent": "91a7ecfc-2514-4439-a207-63a88d331b5e",
      "text": "127.0.0.1:3000/api/pack/2",
      "createdTime": "05-Dec-2025 15:10:04",
      "droppable": false,
      "data": {
        "id": "d8413749-9b22-44b6-8ef9-da49d25ebac5",
        "method": "get",
        "name": "127.0.0.1:3000/api/pack/2",
        "url": "127.0.0.1:3000/api/pack/2",
        "createdTime": "05-Dec-2025 15:10:04",
        "$loki": 12,
        "treeNodeType": "request"
      }
    },
    {
      "id": "ddd3583a-8387-4871-ba69-f7e551ec5c92",
      "parent": "91a7ecfc-2514-4439-a207-63a88d331b5e",
      "text": "127.0.0.1:3000/api/pack",
      "createdTime": "05-Dec-2025 15:09:10",
      "droppable": false,
      "data": {
        "id": "7f1e9c16-60c5-4d92-82d1-52076346fcc2",
        "method": "get",
        "name": "127.0.0.1:3000/api/pack",
        "url": "127.0.0.1:3000/api/pack/2",
        "createdTime": "05-Dec-2025 15:09:10",
        "$loki": 10,
        "treeNodeType": "request"
      }
    }
  ],
  "requests": [
    {
      "id": "6b422a11-ffde-42cf-a832-016146ea36dd",
      "url": "127.0.0.1:3000/api/pack/2",
      "name": "127.0.0.1:3000/api/pack/2",
      "createdTime": "05-Dec-2025 15:10:04",
      "method": "get",
      "params": [
        {
          "isChecked": false,
          "key": "",
          "value": ""
        }
      ],
      "pathParams": [],
      "auth": {
        "authType": "noauth",
        "userName": "",
        "password": "",
        "addTo": "queryparams",
        "showPwd": false,
        "tokenPrefix": "",
        "aws": {
          "service": "",
          "region": "",
          "accessKey": "",
          "secretAccessKey": "",
          "sessionToken": ""
        }
      },
      "headers": [
        {
          "key": "Accept",
          "value": "*/*",
          "isChecked": true
        },
        {
          "key": "User-Agent",
          "value": "Flashpost",
          "isChecked": true
        },
        {
          "key": "",
          "value": "",
          "isChecked": false
        }
      ],
      "body": {
        "bodyType": "none",
        "formdata": [
          {
            "isChecked": false,
            "key": "",
            "value": ""
          }
        ],
        "urlencoded": [
          {
            "isChecked": false,
            "key": "",
            "value": ""
          }
        ],
        "raw": {
          "data": "",
          "lang": "json",
          "binary": {
            "fileName": "",
            "data": {}
          },
          "contentTypeOption": "manual",
          "graphql": {
            "query": "",
            "variables": ""
          },
          "tests": [
            {
              "parameter": "",
              "action": "",
              "expectedValue": ""
            }
          ],
          "setvar": [
            {
              "parameter": "",
              "key": "",
              "variableName": ""
            }
          ],
          "notes": "",
          "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"
        },
        {
          "id": "ddd3583a-8387-4871-ba69-f7e551ec5c92",
          "url": "127.0.0.1:3000/api/pack/2",
          "name": "127.0.0.1:3000/api/pack",
          "createdTime": "05-Dec-2025 15:09:10",
          "method": "get",
          "params": [
            {
              "isChecked": false,
              "key": "",
              "value": ""
            }
          ],
          "pathParams": [],
          "auth": {
            "authType": "noauth",
            "userName": "",
            "password": "",
            "addTo": "queryparams",
            "showPwd": false,
            "tokenPrefix": "",
            "aws": {
              "service": "",
              "region": "",
              "accessKey": "",
              "secretAccessKey": "",
              "sessionToken": ""
            }
          },
          "headers": [
            {
              "key": "Accept",
              "value": "*/*",
              "isChecked": true
            },
            {
              "key": "User-Agent",
              "value": "Flashpost",
              "isChecked": true
            },
            {
              "key": "",
              "value": "",
              "isChecked": false
            }
          ],
          "body": {

```

```
{
  "bodyType": "none",
  "formdata": [
    {
      "isChecked": false,
      "key": "",
      "value": ""
    },
    {
      "isChecked": false,
      "key": "",
      "value": ""
    }
  ],
  "urlencoded": [
    {
      "isChecked": false,
      "key": "",
      "value": ""
    }
  ],
  "raw": {
    "data": "",
    "lang": "json",
    "binary": {
      "fileName": "",
      "data": {}
    },
    "contentTypeOption": "manual",
    "graphql": {
      "query": "",
      "variables": ""
    },
    "tests": [
      {
        "parameter": "",
        "action": "",
        "expectedValue": ""
      }
    ],
    "setvar": [
      {
        "parameter": "",
        "key": "",
        "variableName": ""
      }
    ],
    "notes": "",
    "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"
  ]
}
```

FICHER : code/_flashpost/flashpost-collection-product.json

```
{
  "app": "Flashpost",
  "id": "40583afe-8a76-4b94-9d8f-51ed03cac317",
  "name": "product",
  "version": "1.0",
  "type": "collections",
  "createdAt": "05-Dec-2025 14:19:35",
  "exportedDate": "18-Dec-2025 21:25:09",
  "collections": [
    {
      "id": "40583afe-8a76-4b94-9d8f-51ed03cac317",
      "parent": "0",
      "text": "product",
      "createdTime": "05-Dec-2025 14:19:35",
      "draggable": true,
      "data": {
        "treeNodeType": "collection",
        "variableId": "",
        "relativeIndex": 0
      }
    },
    {
      "id": "2f95e3ef-8be8-459f-9adb-0990aa71a20a",
      "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
      "text": "127.0.0.1:3000/api/product",
      "createdTime": "18-Dec-2025 10:06:39",
      "draggable": false,
      "data": {
        "id": "db85cbbd-ff4f-4497-92a1-df1f7ad6e7f6",
        "method": "put",
        "name": "127.0.0.1:3000/api/product",
        "url": "127.0.0.1:3000/api/product",
        "createdTime": "18-Dec-2025 10:06:39",
        "$loki": 23,
        "treeNodeType": "request"
      }
    },
    {
      "id": "b7031931-d679-4564-8a5a-cdb95b0c7a8d",
      "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
      "text": "127.0.0.1:3000/api/product",
      "createdTime": "18-Dec-2025 14:19:36",
      "draggable": false,
      "data": {
        "id": "8ddcffad-3c8f-4435-ba2a-72aab86b7019",
        "method": "delete",
        "name": "127.0.0.1:3000/api/product",
        "url": "127.0.0.1:3000/api/product",
        "createdTime": "18-Dec-2025 14:19:36",
        "$loki": 25,
        "treeNodeType": "request"
      }
    },
    {
      "id": "2c6b49dc-0897-42c0-8a60-c6fcd27e1e11",
      "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
      "text": "127.0.0.1:3000/api/product",
      "createdTime": "17-Dec-2025 09:19:47",
      "draggable": false,
      "data": {
        "id": "2335fdca-2643-4f01-ab74-42a6c954be5c",
        "method": "post",
        "name": "127.0.0.1:3000/api/product",
        "url": "127.0.0.1:3000/api/product",
        "createdTime": "17-Dec-2025 09:19:47",
        "$loki": 18,
        "treeNodeType": "request"
      }
    },
    {
      "id": "a177c61d-ee44-4994-948f-fc99f49323d6",
      "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
      "text": "127.0.0.1:3000/api/product/7",
      "createdTime": "05-Dec-2025 15:19:21",
      "draggable": false,
      "data": {
        "id": "47a0b5d5-5c07-4571-b5ce-d267a5e1b34c",
        "method": "get",
        "name": "127.0.0.1:3000/api/product/7",
        "url": "127.0.0.1:3000/api/product/7",
        "createdTime": "05-Dec-2025 15:19:21",
        "$loki": 16,
        "treeNodeType": "request"
      }
    },
    {
      "id": "c4ca3ced-fec0-4814-96f5-4bd0c0b34bed",
      "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
      "text": "127.0.0.1:3000/api/product",
      "createdTime": "05-Dec-2025 15:19:05",
      "draggable": false,
      "data": {
        "id": "7778d6e4-3918-4f2e-9436-1740adb140e6",
        "method": "get",
        "name": "127.0.0.1:3000/api/product",
        "url": "127.0.0.1:3000/api/product",
        "createdTime": "05-Dec-2025 15:19:05",
        "$loki": 15,
        "treeNodeType": "request"
      }
    }
  ],
  "requests": [
    {
      "id": "40583afe-8a76-4b94-9d8f-51ed03cac317",
      "parent": "0",
      "text": "product",
      "createdTime": "05-Dec-2025 14:19:35",
      "draggable": true,
      "data": {
        "treeNodeType": "collection",
        "variableId": "",
        "relativeIndex": 0
      }
    },
    {
      "id": "2f95e3ef-8be8-459f-9adb-0990aa71a20a",
      "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
      "text": "127.0.0.1:3000/api/product",
      "createdTime": "18-Dec-2025 10:06:39",
      "draggable": false,
      "data": {
        "id": "db85cbbd-ff4f-4497-92a1-df1f7ad6e7f6",
        "method": "put",
        "name": "127.0.0.1:3000/api/product",
        "url": "127.0.0.1:3000/api/product",
        "createdTime": "18-Dec-2025 10:06:39",
        "$loki": 23,
        "treeNodeType": "request"
      }
    },
    {
      "id": "b7031931-d679-4564-8a5a-cdb95b0c7a8d",
      "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
      "text": "127.0.0.1:3000/api/product",
      "createdTime": "18-Dec-2025 14:19:36",
      "draggable": false,
      "data": {
        "id": "8ddcffad-3c8f-4435-ba2a-72aab86b7019",
        "method": "delete",
        "name": "127.0.0.1:3000/api/product",
        "url": "127.0.0.1:3000/api/product",
        "createdTime": "18-Dec-2025 14:19:36",
        "$loki": 25,
        "treeNodeType": "request"
      }
    },
    {
      "id": "2c6b49dc-0897-42c0-8a60-c6fcd27e1e11",
      "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
      "text": "127.0.0.1:3000/api/product",
      "createdTime": "17-Dec-2025 09:19:47",
      "draggable": false,
      "data": {
        "id": "2335fdca-2643-4f01-ab74-42a6c954be5c",
        "method": "post",
        "name": "127.0.0.1:3000/api/product",
        "url": "127.0.0.1:3000/api/product",
        "createdTime": "17-Dec-2025 09:19:47",
        "$loki": 18,
        "treeNodeType": "request"
      }
    },
    {
      "id": "a177c61d-ee44-4994-948f-fc99f49323d6",
      "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
      "text": "127.0.0.1:3000/api/product/7",
      "createdTime": "05-Dec-2025 15:19:21",
      "draggable": false,
      "data": {
        "id": "47a0b5d5-5c07-4571-b5ce-d267a5e1b34c",
        "method": "get",
        "name": "127.0.0.1:3000/api/product/7",
        "url": "127.0.0.1:3000/api/product/7",
        "createdTime": "05-Dec-2025 15:19:21",
        "$loki": 16,
        "treeNodeType": "request"
      }
    },
    {
      "id": "c4ca3ced-fec0-4814-96f5-4bd0c0b34bed",
      "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
      "text": "127.0.0.1:3000/api/product",
      "createdTime": "05-Dec-2025 15:19:05",
      "draggable": false,
      "data": {
        "id": "7778d6e4-3918-4f2e-9436-1740adb140e6",
        "method": "get",
        "name": "127.0.0.1:3000/api/product",
        "url": "127.0.0.1:3000/api/product",
        "createdTime": "05-Dec-2025 15:19:05",
        "$loki": 15,
        "treeNodeType": "request"
      }
    }
  ]
}
```

```
[{"id": "2f95e3ef-8be8-459f-9adb-0990aa71a20a", "url": "127.0.0.1:3000/api/product", "name": "127.0.0.1:3000/api/product", "createdTime": "18-Dec-2025 10:06:39", "method": "put", "params": [{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth": {"authType": "noauth", "userName": "", "password": "", "addTo": "queryParams", "showPwd": false, "tokenPrefix": "", "aws": {"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "Content-Type", "value": "multipart/form-data", "isChecked": true, "isFixed": false}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "formdata", "formdata": [{"isChecked": true, "key": "id", "value": "48"}, {"isChecked": true, "key": "name", "value": "update product", "type": "Text"}, {"isChecked": true, "key": "image", "value": "update img", "type": "Text"}, {"isChecked": true, "key": "description", "value": "update desc", "type": "Text"}, {"isChecked": true, "key": "price", "value": "12", "type": "Text"}, {"isChecked": true, "key": "category_id", "value": "2", "type": "Text"}, {"isChecked": true, "key": "skin_ids", "value": "3,4", "type": "Text"}, {"isChecked": true, "key": "pack_ids", "value": "3,4", "type": "Text"}, {"isChecked": true, "key": "body_part_ids", "value": "3,4", "type": "Text"}, {"isChecked": false, "key": "", "value": "", "type": "Text"}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"}, {"id": "b7031931-d679-4564-8a5a-cdb95b0c7a8d", "url": "127.0.0.1:3000/api/product", "name": "127.0.0.1:3000/api/product", "createdTime": "18-Dec-2025 14:19:36", "method": "delete", "params": [{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth": {"authType": "noauth", "userName": "", "password": "", "addTo": "queryParams", "showPwd": false, "tokenPrefix": "", "aws": {"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "Content-Type", "value": "application/json", "isChecked": true, "isFixed": false}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "raw", "formdata": [{"isChecked": false, "key": "", "value": ""}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "{\r\n    \id\r\n    : 48\r\n}", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"}, {"id": "2c6b49dc-0897-42c0-8a60-c6fcd27e1e11", "url": "127.0.0.1:3000/api/product", "name": "127.0.0.1:3000/ap
```

```

i/product", "createdTime": "17-Dec-2025 09:19:47", "method": "post", "params":
[{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth":
{"authType": "noauth", "userName": "", "password": "", "addTo": "queryparams", "sh
owPwd": false, "tokenPrefix": "", "aws":
{"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToke
n": ""}}, "headers": [{"isChecked": true, "key": "Content-
Type", "value": "multipart/form-data", "isFixed": false},
{"key": "", "value": "", "isChecked": false}], "body":
{"bodyType": "formdata", "formdata":
[{"isChecked": true, "key": "name", "value": "nouveau product 3"},
{"isChecked": true, "key": "image", "value": "image .jpg", "type": "Text"},
{"isChecked": true, "key": "description", "value": "desc nouveau product
", "type": "Text"},
{"isChecked": true, "key": "price", "value": "10", "type": "Text"},
{"isChecked": true, "key": "category_id", "value": "1", "type": "Text"},
{"isChecked": true, "key": "skin_ids", "value": "1,2", "type": "Text"},
{"isChecked": true, "key": "body_part_ids", "value": "1,2", "type": "Text"},
{"isChecked": true, "key": "pack_ids", "value": "1,2", "type": "Text"},
{"isChecked": false, "key": "", "value": "", "type": "Text"}], "urlencoded":
[{"isChecked": false, "key": "", "value": ""}], "raw":
{"data": "", "lang": "json"}, "binary": {"fileName": "", "data":
{}}, "contentTypeOption": "manual", "graphql":
{"query": "", "variables": ""}}, "tests":
[{"parameter": "", "action": "", "expectedValue": ""}], "setvar":
[{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironm
entId": "5cfb74f5-0183-401a-a720-9780a2b57fd4", {"id": "a177c61d-ee44-4994-
948f-
fc99f49323d6", "url": "127.0.0.1:3000/api/product/7", "name": "127.0.0.1:3000/
api/product/7", "createdTime": "05-Dec-2025
15:19:21", "method": "get", "params":
[{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth":
{"authType": "noauth", "userName": "", "password": "", "addTo": "queryparams", "sh
owPwd": false, "tokenPrefix": "", "aws":
{"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToke
n": ""}}, "headers": [{"key": "Accept", "value": "*//*", "isChecked": true},
{"key": "User-Agent", "value": "Flashpost", "isChecked": true},
{"key": "", "value": "", "isChecked": false}], "body":
{"bodyType": "none", "formdata":
[{"isChecked": false, "key": "", "value": ""}], "urlencoded":
[{"isChecked": false, "key": "", "value": ""}], "raw":
{"data": "", "lang": "json"}, "binary": {"fileName": "", "data":
{}}, "contentTypeOption": "manual", "graphql":
{"query": "", "variables": ""}}, "tests":
[{"parameter": "", "action": "", "expectedValue": ""}], "setvar":
[{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironm
entId": "5cfb74f5-0183-401a-a720-9780a2b57fd4", {"id": "c4ca3ced-fec0-4814-
96f5-
4bd0c0b34bed", "url": "127.0.0.1:3000/api/product", "name": "127.0.0.1:3000/ap
i/product", "createdTime": "05-Dec-2025 15:19:05", "method": "get", "params":
[{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth":
{"authType": "noauth", "userName": "", "password": "", "addTo": "queryparams", "sh
owPwd": false, "tokenPrefix": "", "aws":
{"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToke
n": ""}}, "headers": [{"key": "Accept", "value": "*//*", "isChecked": true},

```



```
{
  "key": "User-Agent", "value": "Flashpost", "isChecked": true,
  "key": "", "value": "", "isChecked": false,
  "body": {
    "bodyType": "none", "formdata": {
      "isChecked": false, "key": "", "value": ""
    }, "urlencoded": {
      "isChecked": false, "key": "", "value": ""
    }, "raw": {
      "data": "", "lang": "json", "binary": {
        "fileName": "", "data": {}
      }, "contentTypeOption": "manual", "graphql": {
        "query": "", "variables": ""
      }, "tests": {
        "parameter": "", "action": "", "expectedValue": ""
      }, "setvar": {
        "parameter": "", "key": "", "variableName": ""
      }, "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"
    }
  }
}
```

FICHER : code/_flashpost/flashpost-collection-role.json

```
{
  "app": "Flashpost", "id": "56fc6175-1f58-456f-b478-f4068571010b", "name": "role", "version": "1.0", "type": "collections", "createdAt": "05-Dec-2025 12:54:38", "exportedDate": "05-Dec-2025 12:57:50", "collections": [
    {
      "id": "56fc6175-1f58-456f-b478-f4068571010b", "parent": "0", "text": "role", "createdAt": "05-Dec-2025 12:54:38", "draggable": true, "data": {
        "treeNodeType": "collection", "variableId": "", "relativeIndex": 0,
        "id": "50293731-cdbc-4eef-a4a7-23406b494626", "parent": "56fc6175-1f58-456f-b478-f4068571010b", "text": "127.0.0.1:3000/api/role/1", "createdAt": "05-Dec-2025 13:55:26", "draggable": false, "data": {
          "id": "57399b58-95ca-482e-a09c-272a3fa41fe7", "method": "get", "name": "127.0.0.1:3000/api/role/1", "url": "127.0.0.1:3000/api/role/1", "createdAt": "05-Dec-2025 13:55:26", "$loki": 5, "treeNodeType": "request"
        }
      }, {
        "id": "8c4982af-b0e4-4784-ae83-676d3182d87c", "parent": "56fc6175-1f58-456f-b478-f4068571010b", "text": "127.0.0.1:3000/api/role", "createdAt": "05-Dec-2025 13:54:16", "draggable": false, "data": {
          "id": "b8f097a1-15ba-4935-9ddc-864c49927d4a", "method": "get", "name": "127.0.0.1:3000/api/role", "url": "127.0.0.1:3000/api/role", "createdAt": "05-Dec-2025 13:54:16", "$loki": 4, "treeNodeType": "request"
        }
      }
    ], "requests": [
      {
        "id": "50293731-cdbc-4eef-a4a7-23406b494626", "url": "127.0.0.1:3000/api/role/1", "name": "127.0.0.1:3000/api/role/1", "createdAt": "05-Dec-2025 13:55:26", "method": "get", "params": {
          "isChecked": false, "key": "", "value": ""
        }, "pathParams": [], "auth": {
          "authType": "noauth", "userName": "", "password": "", "addTo": "queryParams", "showPwd": false, "tokenPrefix": "", "aws": {
            "service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""
          }
        }, "headers": [
          {
            "key": "Accept", "value": "*/*", "isChecked": true
          },
          {
            "key": "User-Agent", "value": "Flashpost", "isChecked": true
          },
          {
            "key": "", "value": "", "isChecked": false
          }
        ], "body": {
          "bodyType": "none", "formdata": {
            "isChecked": false, "key": "", "value": ""
          }, "urlencoded": {
            "isChecked": false, "key": "", "value": ""
          }, "raw": {
            "data": "", "lang": "json", "binary": {
              "fileName": "", "data": {}
            }, "contentTypeOption": "manual", "graphql": {
              "query": "", "variables": ""
            }, "tests": {}
          }
        }
      }
    ]
  }
}
```

```
[{"parameter":"","action":"","expectedValue":""},"setvar":
[{"parameter":"","key":"","variableName":""},"notes":"","selectedEnvironm
entId":"5cfb74f5-0183-401a-a720-9780a2b57fd4"}, {"id":"8c4982af-b0e4-4784-
ae83-
676d3182d87c","url":"127.0.0.1:3000/api/role","name":"127.0.0.1:3000/api/r
ole","createdTime":"05-Dec-2025 13:54:16","method":"get","params":
[{"isChecked":false,"key":"","value":""},"pathParams":[],"auth":
{"authType":"noauth","userName":"","password":"","addTo":"queryparams","sh
owPwd":false,"tokenPrefix":"","aws":
{"service":"","region":"","accessKey":"","secretAccessKey":"","sessionToke
n":""}}, {"headers":[{"key":"Accept","value":"*/*","isChecked":true},
{"key":"User-Agent","value":"Flashpost","isChecked":true},
{"key":"","value":"","isChecked":false}],"body":
{"bodyType":"none","formdata":
[{"isChecked":false,"key":"","value":""},"urlencoded":
[{"isChecked":false,"key":"","value":""},"raw":
{"data":"","lang":"json"},"binary":{"fileName":"","data":
{},"contentTypeOption":"manual"},"graphql":
{"query":"","variables":""}},"tests":
[{"parameter":"","action":"","expectedValue":""},"setvar":
[{"parameter":"","key":"","variableName":""},"notes":"","selectedEnvironm
entId":"5cfb74f5-0183-401a-a720-9780a2b57fd4"}]]}
```

FICHER : code/_flashpost/flashpost-collection-skin.json

```
{"app":"Flashpost","id":"ca3fda4a-c6da-49c4-8d25-
9f082e9b41b7","name":"skin","version":"1.0","type":"collections","createdT
ime":"05-Dec-2025 14:07:23","exportedDate":"05-Dec-2025
14:08:03","collections":[{"id":"ca3fda4a-c6da-49c4-8d25-
9f082e9b41b7","parent":"0","text":"skin","createdTime":"05-Dec-2025
14:07:23","draggable":true,"data":
{"treeNodeType":"collection","variableId":"","relativeIndex":0},
{"id":"bd4747dd-aed2-4b63-ae15-2c94229cda94","parent":"ca3fda4a-c6da-49c4-
8d25-9f082e9b41b7","text":"127.0.0.1:3000/api/skin/3","createdTime":"05-
Dec-2025 15:06:53","draggable":false,"data":{"id":"40b38ee2-51cd-4f67-
9272-
46ce7ee8dbc2","method":"get","name":"127.0.0.1:3000/api/skin/3","url":"127
.0.0.1:3000/api/skin/3","createdTime":"05-Dec-2025
15:06:53","$loki":9,"treeNodeType":"request"}}, {"id":"ba924c8c-9d2c-4e83-
b07a-94e20736bbcf","parent":"ca3fda4a-c6da-49c4-8d25-
9f082e9b41b7","text":"127.0.0.1:3000/api/skin","createdTime":"05-Dec-2025
15:06:14","draggable":false,"data":{"id":"d642455b-7ee4-4a5a-9950-
a4b32f56758f","method":"get","name":"127.0.0.1:3000/api/skin","url":"127.0
.0.1:3000/api/skin","createdTime":"05-Dec-2025
15:06:14","$loki":8,"treeNodeType":"request"}]]},"requests":
[{"id":"bd4747dd-aed2-4b63-ae15-
2c94229cda94","url":"127.0.0.1:3000/api/skin/3","name":"127.0.0.1:3000/api
/skin/3","createdTime":"05-Dec-2025 15:06:53","method":"get","params":
[{"isChecked":false,"key":"","value":""},"pathParams":[],"auth":
{"authType":"noauth","userName":"","password":"","addTo":"queryparams","sh
```

```

owPwd": false, "tokenPrefix": "", "aws":
{"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "none", "formdata": [{"isChecked": false, "key": "", "value": ""}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"}, {"id": "ba924c8c-9d2c-4e83-b07a-94e20736bbcf", "url": "127.0.0.1:3000/api/skin", "name": "127.0.0.1:3000/api/skin", "createdTime": "05-Dec-2025 15:06:14", "method": "get", "params": [{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth": {"authType": "noauth", "userName": "", "password": "", "addTo": "queryparams", "showPwd": false, "tokenPrefix": "", "aws": {"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "none", "formdata": [{"isChecked": false, "key": "", "value": ""}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"}]}

```

FICHER : code/_flashpost/flashpost-collection-user.json

```

{"app": "Flashpost", "id": "28f0a610-03ad-46b9-9ed5-a916d659558d", "name": "user", "version": "1.0", "type": "collections", "createdTime": "15-Dec-2025 11:43:21", "exportedDate": "15-Dec-2025 11:43:43", "collections": [{"id": "28f0a610-03ad-46b9-9ed5-a916d659558d", "parent": "0", "text": "user", "createdTime": "15-Dec-2025 11:43:21", "draggable": true, "data": {"treeNodeType": "collection", "variableId": "", "relativeIndex": 0}, {"id": "e40e4cee-d331-4c7e-a890-10d1545385ee", "parent": "28f0a610-03ad-46b9-9ed5-a916d659558d", "text": "127.0.0.1:3000/api/user", "createdTime": "15-Dec-2025 12:42:14", "draggable": false, "data": {"id": "6e284180-86ed-4742-9085-e40fb31ec1a0", "method": "get", "name": "127.0.0.1:3000/api/user", "url": "127.0.0.1:3000/api/user", "createdTime": "15-Dec-2025 12:42:14", "$loki": 17, "treeNodeType": "request"}}, {"requests": [{"id": "e40e4cee-d331-4c7e-a890-

```



```
10d1545385ee", "url": "127.0.0.1:3000/api/user", "name": "127.0.0.1:3000/api/u
ser", "createdTime": "15-Dec-2025 12:42:14", "method": "get", "params":
[{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth":
{"authType": "noauth", "userName": "", "password": "", "addTo": "queryParams", "sh
owPwd": false, "tokenPrefix": "", "aws":
{"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToke
n": ""}}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true},
{"key": "User-Agent", "value": "Flashpost", "isChecked": true},
{"key": "", "value": "", "isChecked": false}], "body":
{"bodyType": "none", "formdata":
[{"isChecked": false, "key": "", "value": ""}], "urlencoded":
[{"isChecked": false, "key": "", "value": ""}], "raw":
{"data": "", "lang": "json"}, "binary": {"fileName": "", "data":
{}}, "contentTypeOption": "manual", "graphql":
{"query": "", "variables": ""}}, "tests":
[{"parameter": "", "action": "", "expectedValue": ""}], "setvar":
[{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironm
entId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"]}]}
```

FICHER : code/models/body_part.ts

```
import type { Product } from "../product";

// Reprendre strictement les noms des colonnes de la table SQL
type Body_part = {
  id: number;
  name: string;

  // liste concaténée des identifiants des produits
  body_part_ids: string;
  body_parts: Body_part[];

  product_ids: string;
  products: Product[];
};

export type { Body_part };
```

FICHER : code/models/category.ts

```
// Reprendre strictement les noms des colonnes de la table SQL
type Category = {
  id: number;
  name: string;
};
```

```
export type { Category };
```

FICHER : code/models/pack.ts

```
import type { Product } from "../product";

// Reprendre strictement les noms des colonnes de la table SQL
type Pack = {
  id: number;
  name: string;
  price: number;

  // liste concaténée des identifiants des produits
  product_ids: string;
  products: Product[];
};

export type { Pack };
```

FICHER : code/models/product.ts

```
import type { Body_part } from "../body_part";
import type { Category } from "../category";
import type { Pack } from "../pack";
import type { Skin } from "../skin";

// Reprendre strictement les noms des colonnes de la table SQL
type Product = {
  id: number;
  name: string;
  image: string;
  description: string;
  price: number;

  category: Category;
  category_id: number;

  skin_ids: string | string[];
  skins: Skin[];

  body_part_ids: string | string[];
  body_parts: Body_part[];

  pack_ids: string | string[];
  packs: Pack[];
```

```
};  
  
export type { Product };
```

FICHIER : code/models/role.ts

```
// Reprendre strictement les noms des colonnes de la table SQL  
type Role = {  
  id: number;  
  name: string;  
};  
  
export type { Role };
```

FICHIER : code/models/skin.ts

```
import type { Product } from "../product";  
  
// Reprendre strictement les noms des colonnes de la table SQL  
type Skin = {  
  id: number;  
  type: string;  
  
  // liste concaténée des identifiants des produits  
  skin_ids: string ;  
  skins: Skin[];  
  
  product_ids: string;  
  products: Product[];  
};  
  
export type { Skin };
```

FICHIER : code/models/user.ts

```
// Reprendre strictement les noms des colonnes de la table SQL  
type User = {  
  id: number;  
  email: string;  
  password: string;  
};
```

```
export type { User };
```

FICHER : code/package.json

```
{
  "name": "@vitejs/plugin-rsc-examples-starter",
  "version": "0.0.0",
  "private": true,
  "license": "MIT",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "server": "npx tsx watch --env-file .env.development server/index.ts"
  },
  "dependencies": {
    "@types/cors": "^2.8.19",
    "@types/express": "^5.0.6",
    "@types/multer": "^2.0.0",
    "cors": "^2.8.5",
    "express": "^5.2.1",
    "file-type": "^21.3.0",
    "multer": "^2.0.2",
    "mysql2": "^3.15.3",
    "react": "^19.2.0",
    "react-dom": "^19.2.0",
    "react-hook-form": "^7.70.0",
    "react-icons": "^5.5.0",
    "react-router": "^7.10.1",
    "tsx": "^4.21.0",
    "zod": "^4.3.5"
  },
  "devDependencies": {
    "@biomejs/biome": "2.3.8",
    "@types/react": "^19.2.7",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "latest",
    "@vitejs/plugin-rsc": "latest",
    "rsc-html-stream": "^0.0.7",
    "vite": "^7.2.4"
  }
}
```

FICHER : code/server/controller/body_part_controller.ts

```

import type { Request, Response } from "express";
import Body_partRepository from "../repository/body_part_repository";

class Body_PartController {
  // Méthode relié à la route en GET située dans le routeur
  public index = async (_req: Request, res: Response) => {
    // Récupération des relultats de la rête
    const results = await new Body_partRepository().selectAll();

    // Si la requête renvoie une erreur
    if (results instanceof Error) {
      res.status(400).json({
        status: 400,
        message:
          process.env.NODE_ENV === `production` ? `Error` :
results.message,
      });
      return; // Permet de stopper l'execution de la fonction
    }

    // renvoyer une reponse avec un code de status HTTP et au format
    json
    res.status(200).json({
      status: 200,
      message: "OK",
      data: results,
    });
  };

  public selectOne = async (req: Request, res: Response) => {
    // récupérer la variable de route
    // req.params : récupère les variables de route
    console.log(req.params);
    // récupération des resultats de la requete
    const results = await new
Body_partRepository().selectOne(req.params);
    // Si la raquete renvoi une erreur
    if (results instanceof Error) {
      res.status(400).json({
        status: 400,
        message:
          process.env.NODE_ENV === "production" ? "Error" :
results.message,
      });
      return;
    }
    // envoyer une réponse
    // res.send("coucou");
    // renvoyer une réponse avec un code de statut HTTP et au format
    JSON
    res.status(200).json({
      status: 200,

```

```

        message: "ok body_part",
        data: results,
    });
};
}

export default Body_PartController;

```

FICHER : code/server/controller/category_controller.ts

```

import type { Request, Response } from "express";
import CategoryRepository from "../repository/category_repository";

class CategoryController {
    // Méthode relié à la route en GET située dans le routeur
    public index = async (_req: Request, res: Response) => {
        // Récupération des relultats de la rêquete
        const results = await new CategoryRepository().selectAll();

        // Si la requête renvoie une erreur
        if (results instanceof Error) {
            res.status(400).json({
                status: 400,
                message:
                    process.env.NODE_ENV === `production` ? `Error` :
results.message,
            });
            return; // Permet de stopper l'execution de la fonction
        }

        // renvoyer une reponse avec un code de status HTTP et au format
json
        res.status(200).json({
            status: 200,
            message: "OK",
            data: results,
        });
    };

    public selectOne = async (req: Request, res: Response) => {
        // récupérer la variable de route
        // req.params : récupère les variables de route
        console.log(req.params);
        // récupération des resultats de la requete
        const results = await new
CategoryRepository().selectOne(req.params);
        // Si la raquete renvoi une erreur
        if (results instanceof Error) {
            res.status(400).json({

```



```
        status: 400,
        message:
            process.env.NODE_ENV === "production" ? "Error" :
results.message,
    });
    return;
}
//    envoyer une réponse
// res.send("coucou");
// renvoyer une réponse avec un code de statut HTTP et au format
JSON
res.status(200).json({
    status: 200,
    message: "ok category",
    data: results,
});
};
}

export default CategoryController;
```

FICHER : code/server/controller/homepage_controller.ts

```
import type { Request, Response } from "express";

class HomepageController {
    // Méthode relié à la route en GET située dans le routeur
    public index = (_req: Request, res: Response) => {
        // renvoyer une reponse avec un code de status HTTP et au format
        json
        res.status(200).json({
            status: 200,
            message: "OK",
        });
    };
}

export default HomepageController;
```

FICHER : code/server/controller/pack_controller.ts

```
import type { Request, Response } from "express";
import PackRepository from "../repository/pack_repository";

class PackController {
```

```

// Méthode relié à la route en GET située dans le routeur
public index = async (_req: Request, res: Response) => {
  // Récupération des relultats de la rêquete
  const results = await new PackRepository().selectAll();

  // Si la requête renvoie une erreur
  if (results instanceof Error) {
    res.status(400).json({
      status: 400,
      message:
        process.env.NODE_ENV === `production` ? `Error` :
results.message,
    });
    return; // Permet de stopper l'execution de la fonction
  }

  // renvoyer une reponse avec un code de status HTTP et au format
json
  res.status(200).json({
    status: 200,
    message: "OK",
    data: results,
  });
};

public selectOne = async (req: Request, res: Response) => {
  // récupérer la variable de route
  // req.params : récupère les variables de route
  console.log(req.params);
  // récupération des resultats de la requete
  const results = await new PackRepository().selectOne(req.params);
  // Si la raquete renvoi une erreur
  if (results instanceof Error) {
    res.status(400).json({
      status: 400,
      message:
        process.env.NODE_ENV === "production" ? "Error" :
results.message,
    });
    return;
  }
  // envoyer une réponse
  // res.send("coucou");
  // renvoyer une réponse avec un code de statut HTTP et au format
JSON
  res.status(200).json({
    status: 200,
    message: "ok pack",
    data: results,
  });
};
}

```

```
export default PackController;
```

FICHER : code/server/controller/product_controller.ts

```
import type { Request, Response } from "express";
import ProductRepository from "../repository/product_repository";
import FileService from "../service/file_service";
import type { Product } from "../../models/product";

class ProductController {
  // Méthode relié à la route en GET située dans le routeur
  public index = async (_req: Request, res: Response) => {
    // Récupération des relultats de la requête
    const results = await new ProductRepository().selectAll();

    // Si la requête renvoie une erreur
    if (results instanceof Error) {
      res.status(400).json({
        status: 400,
        message:
          process.env.NODE_ENV === `production` ? `Error` :
results.message,
      });
      return; // Permet de stopper l'execution de la fonction
    }

    // renvoyer une reponse avec un code de status HTTP et au format
    json
    res.status(200).json({
      status: 200,
      message: "OK",
      data: results,
    });
  };

  public selectOne = async (req: Request, res: Response) => {
    // récupérer la variable de route
    // req.params : récupère les variables de route
    console.log(req.params);
    // récupération des resultats de la requete
    const results = await new
ProductRepository().selectOne(req.params);
    // Si la raquete renvoi une erreur
    if (results instanceof Error) {
      res.status(400).json({
        status: 400,
        message:
          process.env.NODE_ENV === "production" ? "Error" :
results.message,
```

```

        });
        return;
    }
    //    envoyer une réponse
    // res.send("coucou");
    // renvoyer une réponse avec un code de statut HTTP et au format
JSON
    res.status(200).json({
        status: 200,
        message: "ok product",
        data: results,
    });
};

public insert = async (req: Request, res: Response) => {
    // req.files: permet de récupérer les fichiers transférés
    // console.log(req.files);
    // const file = req.files[0];
    const file = (req.files as Express.Multer.File[]).shift() as
Express.Multer.File;

    // Instancier le service de fichiers
    const fileService = new FileService();

    // Renommer le fichier transféré et on recupere le nom complet
avec extension
    const fullname = await fileService.rename(file);

    // req.body : récupérer les données contenues dans la requête HTTP
    // console.log(req.body);

    // insertion d'un enregistrement
    const results = await new ProductRepository().insert({...req.body,
image: fullname,});

    // Si la raquete renvoi une erreur
    if (results instanceof Error) {
        res.status(400).json({
            status: 400,
            message:
                process.env.NODE_ENV === "production" ? "Error" :
results.message,
        });
        return;
    }
    //    envoyer une réponse
    // res.send("coucou");
    // renvoyer une réponse avec un code de statut HTTP et au format
JSON
    res.status(201).json({
        status: 201,
        message: "created product",
        data: results,

```

```
});  
};  
  
public update = async (req: Request, res: Response) => {  
    // req.body : récupérer les données contenues dans la requête HTTP  
    // console.log(req.body);  
    const file = (req.files as Express.Multer.File[]).shift() as  
Express.Multer.File;  
  
    // Instancier le service de fichiers  
    const fileService = new FileService();  
  
    let fullname: string;  
  
    if (file) {  
        // Renommer le fichier transféré et on recupere le nom complet  
avec extension  
        fullname = await fileService.rename(file);  
  
    } else {  
        fullname = (await new ProductRepository().selectOne(req.body) as  
Product).image;  
    }  
  
    console.log(fullname);  
  
    // modification d'un enregistrement  
    const results = await new ProductRepository().update({...req.body,  
image: fullname,});  
    // console.log(file);  
  
    // Si la raquete renvoi une erreur  
    if (results instanceof Error) {  
        res.status(400).json({  
            status: 400,  
            message:  
                process.env.NODE_ENV === "production" ? "Error" :  
results.message,  
        });  
        return;  
    }  
    //    envoyer une réponse  
    // res.send("coucou");  
    // renvoyer une réponse avec un code de statut HTTP et au format  
JSON  
    res.status(200).json({  
        status: 200,  
        message: "update product",  
        data: results,  
    });  
};
```

```

};

public delete = async (req: Request, res: Response) => {
  // req.body : récupérer les données contenues dans la requête HTTP
  console.log(req.body);

  // insertion d'un enregistrement
  const results = await new ProductRepository().delete(req.body);

  // Si la requête renvoie une erreur
  if (results instanceof Error) {
    res.status(400).json({
      status: 400,
      message:
        process.env.NODE_ENV === "production" ? "Error" :
results.message,
    });
    return;
  }
  // envoyer une réponse
  // res.send("coucou");
  // renvoyer une réponse avec un code de statut HTTP et au format
JSON
  res.status(201).json({
    status: 201,
    message: "delete product",
    data: results,
  });
};
}

export default ProductController;

```

FICHER : code/server/controller/role_controller.ts

```

import type { Request, Response } from "express";
import RoleRepository from "../repository/role_repository";

class RoleController {
  // Méthode relié à la route en GET située dans le routeur
  public index = async (_req: Request, res: Response) => {
    // Récupération des résultats de la requête
    const results = await new RoleRepository().selectAll();

    // Si la requête renvoie une erreur
    if (results instanceof Error) {
      res.status(400).json({
        status: 400,
        message:

```

```

        process.env.NODE_ENV === `production` ? `Error` :
results.message,
    });
    return; // Permet de stopper l'execution de la fonction
}

// renvoyer une reponse avec un code de status HTTP et au format
json
res.status(200).json({
    status: 200,
    message: "OK",
    data: results,
});
});

public selectOne = async (req: Request, res: Response) => {
    // récupérer la variable de route
    // req.params : récupère les variables de route
    console.log(req.params);
    // récupération des resultats de la requete
    const results = await new RoleRepository().selectOne(req.params);
    // Si la raquete renvoi une erreur
    if (results instanceof Error) {
        res.status(400).json({
            status: 400,
            message:
                process.env.NODE_ENV === "production" ? "Error" :
results.message,
        });
        return;
    }
    //    envoyer une réponse
    // res.send("coucou");
    // renvoyer une réponse avec un code de statut HTTP et au format
JSON
res.status(200).json({
    status: 200,
    message: "ok role",
    data: results,
});
});
}

export default RoleController;

```

FICHER : code/server/controller/skin_controller.ts

```

import type { Request, Response } from "express";
import SkinRepository from "../repository/skin_repository";

```



```

class SkinController {
  // Méthode relié à la route en GET située dans le routeur
  public index = async (_req: Request, res: Response) => {
    // Récupération des relultats de la requête
    const results = await new SkinRepository().selectAll();

    // Si la requête renvoie une erreur
    if (results instanceof Error) {
      res.status(400).json({
        status: 400,
        message:
          process.env.NODE_ENV === `production` ? `Error` :
results.message,
      });
      return; // Permet de stopper l'execution de la fonction
    }

    // renvoyer une reponse avec un code de status HTTP et au format
json
    res.status(200).json({
      status: 200,
      message: "OK skin",
      data: results,
    });
  };

  public selectOne = async (req: Request, res: Response) => {
    // récupérer la variable de route
    // req.params : récupère les variables de route
    console.log(req.params);
    // récupération des resultats de la requete
    const results = await new SkinRepository().selectOne(req.params);
    // Si la raquete renvoi une erreur
    if (results instanceof Error) {
      res.status(400).json({
        status: 400,
        message:
          process.env.NODE_ENV === "production" ? "Error" :
results.message,
      });
      return;
    }
    // envoyer une réponse
    // res.send("coucou");
    // renvoyer une réponse avec un code de statut HTTP et au format
JSON
    res.status(200).json({
      status: 200,
      message: "ok skin",
      data: results,
    });
  };
}

```

```
export default SkinController;
```

FICHER : code/server/controller/user_controller.ts

```
import type { Request, Response } from "express";
import UserRepository from "../repository/user_repository";

class UserController {
  // Méthode relié à la route en GET située dans le routeur
  public index = async (_req: Request, res: Response) => {
    // Récupération des résultats de la requête
    const results = await new UserRepository().selectAll();

    // Si la requête renvoie une erreur
    if (results instanceof Error) {
      res.status(400).json({
        status: 400,
        message:
          process.env.NODE_ENV === `production` ? `Error` :
results.message,
      });
      return; // Permet de stopper l'exécution de la fonction
    }

    // renvoyer une réponse avec un code de status HTTP et au format
    json
    res.status(200).json({
      status: 200,
      message: "OK",
      data: results,
    });
  };

  public selectOne = async (req: Request, res: Response) => {
    // récupérer la variable de route
    // req.params : récupère les variables de route
    console.log(req.params);
    // récupération des résultats de la requête
    const results = await new UserRepository().selectOne(req.params);
    // Si la requête renvoie une erreur
    if (results instanceof Error) {
      res.status(400).json({
        status: 400,
        message:
          process.env.NODE_ENV === "production" ? "Error" :
results.message,
      });
      return;
    }
  };
}
```

```
    }  
    //    envoyer une réponse  
    // res.send("coucou");  
    // renvoyer une réponse avec un code de statut HTTP et au format  
JSON  
    res.status(200).json({  
        status: 200,  
        message: "ok user",  
        data: results,  
    });  
};  
}  
  
export default UserController;
```

FICHER : code/server/core/server.ts

```
import express from "express";  
import cors from "cors";  
import Body_partRouter from "../router/body_part_router";  
import CategoryRouter from "../router/category_router";  
import HomepageRouter from "../router/homepage_router";  
import PackRouter from "../router/pack_router";  
import ProductRouter from "../router/product_router";  
import RoleRouter from "../router/role_router";  
import SkinRouter from "../router/skin_router";  
import UserRouter from "../router/user_router";  
  
class Server {  
    // Propriétés  
    private app = express();  
    private router = express.Router();  
  
    // constructeur  
    constructor() {  
        // middleware JSON permet de récupérer la propriété body en JSON  
de la requête HTTP  
        this.app.use(express.json());  
  
        // intégrer le middleare CORS qui permet d'autoriser l'accès aux  
ressources à des origines différents (protocoles, port, sous-domaine)  
        this.app.use(cors({  
            origin: process.env.ORIGINS?.split(",")  
        }));  
  
        // relier le router à l'application  
        this.app.use(this.router);  
  
        // appel des routeurs
```

```
        this.routersList();
    }

    // Méthodes
    // listes des routeurs
    private routersList = () => {
        // créer un préfixe à toutes les routes incluses dans un routeur
        this.router.use("/api", new HomepageRouter().getRoutes());
        this.router.use("/api/role", new RoleRouter().getRoutes());
        this.router.use("/api/category", new
CategoryRouter().getRoutes());
        this.router.use("/api/skin", new SkinRouter().getRoutes());
        this.router.use("/api/pack", new PackRouter().getRoutes());
        this.router.use("/api/body_part", new
Body_partRouter().getRoutes());
        this.router.use("/api/product", new ProductRouter().getRoutes());
        this.router.use("/api/user", new UserRouter().getRoutes());
    };

    // créer méthode pour démarrer le serveur
    public startServer = () => {
        return this.app;
    };
}

export default Server;
```

FICHER : code/server/index.ts

```
import Server from "../core/server";

// Créer le serveur (instanciation)
const server = new Server().startServer();

// Démarrer le serveur
// process.env.VAR : acceder aux variables d'environnement
server.listen(process.env.PORT);
```

FICHER : code/server/repository/body_part_repository.ts

```
import type { Body_part } from "../../models/body_part";
import type { Product } from "../../models/product";
import MySQLService from "../service/mysql_service";
import ProductRepository from "../product_repository";
```

```

class Body_partRepository {
  // Nom de la table SQL
  private table = "body_part";

  // Selectionner tous les enregistrements
  public selectAll = async (): Promise<Body_part[] | unknown> => {
    // connexion au server SQL
    const connection = await new MySQLService().connect();

    // requête SQL : SELECT role.* FROM secretsDeHamam_dev.body_part;
    /* const sql = `SELECT ${this.table}.*
      FROM ${process.env.MYSQL_DATABASE}.${this.table};`
    */

    // requete SQL
    /*
      SELECT
        body_part.*,
        GROUP_CONCAT(product_id) AS product_ids
      FROM
        secretsDeHamam_dev.body_part
      JOIN
        secretsDeHamam_dev.product_body_part
      ON
        product_body_part.body_part_id = body_part.id
      JOIN
        secretsDeHamam_dev.product
      ON
        product.id = product_body_part.product_id
      GROUP BY
        body_part.id `
    */

    const sql = `
      SELECT ${this.table}.*,
      GROUP_CONCAT(product_id) AS product_ids
      FROM ${process.env.MYSQL_DATABASE}.${this.table}
      JOIN ${process.env.MYSQL_DATABASE}.product_body_part
      ON product_body_part.body_part_id = ${this.table}.id
      JOIN ${process.env.MYSQL_DATABASE}.product
      ON product.id = product_body_part.product_id
      GROUP BY ${this.table}.id ;
    `;

    // Try / Catch : récupérer les résultats de la requête ou un erreur
    try {
      // Execution de la requete
      const [query] = await connection.execute(sql);

      for (let i = 0; i < (query as Body_part[]).length; i++) {
        // Récupérer le resultat
        const result = (query as Body_part[])[i] as Body_part;
      }
    }
  }
}

```

```

        // tables de jointure
        result.products = (await new
ProductRepository().selectInList(
            result.product_ids,
        )) as Product[];
    }

    // Retourner reultats
    return query;
} catch (error) {
    return error;
}
};

// Sélectionner un enregistrement
// data représente une partie des propriété du type
public selectOne = async (
    data: Partial<Body_part>,
): Promise<Body_part | unknown> => {
    // connexion au server MySQL
    const connection = await new MySQLService().connect();
    // requete SQL
    // where category.id=....
    // SELECT level.* FROM secretsDeHamam_dev;
    // variable de requete : précédée d'un :, suivi du nom de la
variable
    // requetes préparées :sécurité;(utilisation des varibales de
requetes)la requete est exécutée si elle ne représente pas de risque de
sécurité
    // const sql = `
    // SELECT ${this.table}.*
    // FROM ${process.env.MYSQL_DATABASE}.${this.table}
    // WHERE ${this.table}.id = :id;
    // `;

    const sql = `
    SELECT ${this.table}.*,
GROUP_CONCAT(product_id) AS product_ids
FROM ${process.env.MYSQL_DATABASE}.${this.table}
JOIN ${process.env.MYSQL_DATABASE}.product_body_part
ON product_body_part.body_part_id = ${this.table}.id
JOIN ${process.env.MYSQL_DATABASE}.product
ON product.id = product_body_part.product_id
WHERE ${this.table}.id = :id
GROUP BY ${this.table}.id ;
    `;

    // try / catch pour récupérer les resultats de la requete ou une
erreur
    try {
        // execution de la requete
        // si la requete possede des variables, utiliser le parametre
de la méthode
        const [query] = await connection.execute(sql, data);

```

```

        // récupérer le premier indice d'un array
        const result = (query as Body_part[]).shift() as Body_part;
        result.products = (await new ProductRepository().selectInList(
            result.product_ids,
        )) as Product[];
        // retourner les résultats
        return result;
    } catch (error) {
        return error;
    }
};

public selectInList = async (list: string): Promise<Body_part[] |
unknown> => {
    // connexion au server SQL
    const connection = await new MySQLService().connect();

    // requête SQL : SELECT role.* FROM secretsDeHamman_dev.product;
    const sql = `SELECT ${this.table}.*
        FROM ${process.env.MYSQL_DATABASE}.${this.table}
        WHERE ${this.table}.id IN (${list})
        `;

    // Try / Catch : récupérer les résultats de la requête ou un erreur
    try {
        // Execution de la requete
        const [query] = await connection.execute(sql);

        // Retourner reultats
        return query;
    } catch (error) {
        return error;
    }
};
}

export default Body_partRepository;

```

FICHER : code/server/repository/category_repository.ts

```

import type { Category } from "../../models/category";
import MySQLService from "../service/mysql_service";

class CategoryRepository {
    // Nom de la table SQL
    private table = "category";

    // Selectionner tous les enregistrements

```

```

public selectAll = async (): Promise<Category[] | unknown> => {
  // connexion au server SQL
  const connection = await new MySQLService().connect();

  // requête SQL : SELECT role.* FROM secretsDeHamam_dev.catégorie;
  const sql = `SELECT ${this.table}.*
               FROM ${process.env.MYSQL_DATABASE}.${this.table}`;

  // Try / Catch : récupérer les résultats de la requête ou un erreur
  try {
    // Execution de la requete
    const [query] = await connection.execute(sql);

    // Retourner reultats
    return query;
  } catch (error) {
    return error;
  }
};

// Sélectionner un enregistrement
// data représente une partie des propriété du type
public selectOne = async (
  data: Partial<Category>,
): Promise<Category | unknown> => {
  // connexion au server MySQL
  const connection = await new MySQLService().connect();
  // requete SQL
  // where category.id=....
  // SELECT level.* FROM secretsDeHamam_dev;
  // variable de requete : précédée d'un :, suivi du nom de la
variable
  // requetes préparées :sécurité;(utilisation des varibales de
requetes)la requete est exécutée si elle ne représente pas de risque de
sécurité
  const sql = `
SELECT ${this.table}.*
FROM ${process.env.MYSQL_DATABASE}.${this.table}
WHERE ${this.table}.id = :id;
`;
  // try / catch pour récupérer les resultats de la requete ou une
erreur
  try {
    // execution de la requete
    // si la requete possede des variables, utiliser le parametre
de la méthode
    const [query] = await connection.execute(sql, data);
    // récupérer le premier indice d'un array
    const result = (query as Category[]).shift();
    // retourner les résultats
    return result;
  } catch (error) {
    return error;
  }
}

```



```

    };
}

export default CategoryRepository;

```

FICHER : code/server/repository/pack_repository.ts

```

import type { Pack } from "../../models/pack";
import type { Product } from "../../models/product";
import MySQLService from "../service/mysql_service";
import ProductRepository from "../product_repository";

class PackRepository {
  // Nom de la table SQL
  private table = "pack";

  // Selectionner tous les enregistrements
  public selectAll = async (): Promise<Pack[] | unknown> => {
    // connexion au server SQL
    const connection = await new MySQLService().connect();

    /* requête SQL : SELECT role.* FROM secretsDeHamam_dev.pack;
    const sql = `SELECT ${this.table}.*
                  FROM ${process.env.MYSQL_DATABASE}.${this.table};`;
    */

    // requete SQL
    /*
      SELECT
        pack.*,
        GROUP_CONCAT(product_id) AS product_ids
      FROM
        secretsDeHamam_dev.pack
      JOIN
        secretsDeHamam_dev.product_pack
      ON
        product_pack.pack_id = pack.id
      JOIN
        secretsDeHamam_dev.product
      ON
        product.id = product_pack.product_id
      GROUP BY
        pack.id `
    */

    const sql = `
      SELECT ${this.table}.*,
      GROUP_CONCAT(product_id) AS product_ids
      FROM ${process.env.MYSQL_DATABASE}.${this.table}

```

```

JOIN ${process.env.MYSQL_DATABASE}.product_pack
ON product_pack.pack_id = ${this.table}.id
JOIN ${process.env.MYSQL_DATABASE}.product
ON product.id = product_pack.product_id
GROUP BY ${this.table}.id ;
    `;

// Try / Catch : récupérer les résultats de la requête ou un erreur
try {
    // Execution de la requete
    const [query] = await connection.execute(sql);

    for (let i = 0; i < (query as Pack[]).length; i++) {
        // Récupérer le resultat
        const result = (query as Pack[])[i] as Pack;

        // tables de jointure
        result.products = (await new
ProductRepository().selectInList(
            result.product_ids,
            )) as Product[];
    }

    // Retourner reultats
    return query;
} catch (error) {
    return error;
}
};

// Sélectionner un enregistrement
// data représente une partie des propriété du type
public selectOne = async (data: Partial<Pack>): Promise<Pack |
unknown> => {
    // connexion au server MySQL
    const connection = await new MySQLService().connect();
    // requete SQL
    // where category.id=....
    // SELECT level.* FROM secretsDeHamam_dev;
    // variable de requete : précédée d'un :, suivi du nom de la
variable
    // requetes préparées :sécurité;(utilisation des varibales de
requetes)la requete est exécutée si elle ne représente pas de risque de
sécurité
    // const sql = `
    // SELECT ${this.table}.*
    // FROM ${process.env.MYSQL_DATABASE}.${this.table}
    // WHERE ${this.table}.id = :id;
    // `;

    const sql = `
    SELECT ${this.table}.*,
GROUP_CONCAT(product_id) AS product_ids

```

```

FROM ${process.env.MYSQL_DATABASE}.${this.table}
JOIN ${process.env.MYSQL_DATABASE}.product_pack
ON product_pack.pack_id = ${this.table}.id
JOIN ${process.env.MYSQL_DATABASE}.product
ON product.id = product_pack.product_id
WHERE ${this.table}.id = :id
GROUP BY ${this.table}.id ;
`;

// try / catch pour récupérer les resultats de la requete ou une
erreur
try {
  // execution de la requete
  // si la requete possede des variables, utiliser le parametre
de la méthode
  const [query] = await connection.execute(sql, data);
  // récupérer le premier indice d'un array
  const result = (query as Pack[]).shift() as Pack;
  result.products = (await new ProductRepository().selectInList(
    result.product_ids,
  )) as Product[];
  // retourner les résultats
  return result;
} catch (error) {
  return error;
}

};

public selectInList = async (list: string): Promise<Pack[] | unknown>
=> {
  // connexion au server SQL
  const connection = await new MySQLService().connect();

  // requête SQL : SELECT role.* FROM secretsDeHamam_dev.product;
  const sql = `SELECT ${this.table}.*
    FROM ${process.env.MYSQL_DATABASE}.${this.table}
    WHERE ${this.table}.id IN (${list})
  `;

  // Try / Catch : récupérer les résultats de la requête ou un erreur
  try {
    // Execution de la requete
    const [query] = await connection.execute(sql);

    // Retourner resultats
    return query;
  } catch (error) {
    return error;
  }
};

```

```
}  
  
export default PackRepository;
```

FICHER : code/server/repository/product_repository.ts

```
import type { QueryResult } from "mysql2";  
import type { Category } from "../../models/category";  
import type { Product } from "../../models/product";  
import MySQLService from "../service/mysql_service";  
import CategoryRepository from "../category_repository";  
import SkinRepository from "../skin_repository";  
import type { Skin } from "../../models/skin";  
import PackRepository from "../pack_repository";  
import Body_partRepository from "../body_part_repository";  
import type { Body_part } from "../../models/body_part";  
import type { Pack } from "../../models/pack";  
  
class ProductRepository {  
  // Nom de la table SQL  
  private table = "product";  
  
  // Selectionner tous les enregistrements  
  public selectAll = async (): Promise<Product[] | unknown> => {  
    // connexion au server SQL  
    const connection = await new MySQLService().connect();  
  
    // requête SQL : SELECT role.* FROM secretsDeHamman_dev.product;  
    const sql = `SELECT ${this.table}.*,  
      GROUP_CONCAT(DISTINCT skin.id) AS skin_ids,  
      GROUP_CONCAT(DISTINCT pack.id) AS pack_ids,  
      GROUP_CONCAT(DISTINCT body_part.id) AS body_part_ids  
    FROM ${process.env.MYSQL_DATABASE}.${this.table}  
  
    JOIN ${process.env.MYSQL_DATABASE}.product_skin  
    ON product_skin.product_id = product.id  
    JOIN ${process.env.MYSQL_DATABASE}.skin  
    ON product_skin.skin_id = skin.id  
  
    JOIN ${process.env.MYSQL_DATABASE}.product_pack  
    ON product_pack.product_id = product.id  
    JOIN ${process.env.MYSQL_DATABASE}.pack  
    ON product_pack.pack_id = pack.id  
  
    JOIN ${process.env.MYSQL_DATABASE}.product_body_part  
    ON product_body_part.product_id = product.id  
    JOIN ${process.env.MYSQL_DATABASE}.body_part  
    ON product_body_part.body_part_id = body_part.id`  
  }  
}
```

```

        GROUP BY ${this.table}.id;
    `;

    // Try / Catch : récupérer les résultats de la requête ou un erreur
    try {
        // Execution de la requete
        const [query] = await connection.execute(sql);

        // Boucler sur les resultats pour recuperer les objets en
        relation (composition en Programmation Orientée Objet)
        for (let i = 0; i < (query as Product[]).length; i++) {
            // Récupérer le resultat
            const result = (query as Product[])[i] as Product;

            // Cles etrangère
            result.category = (await new
CategoryRepository().selectOne({
                id: result.category_id,
            }) as Category;

            // tables de jointure
            result.skins = (await new SkinRepository().selectInList(
                result.skin_ids as string
            )) as Skin[];

        }

        // Retourner reultats
        return query;
    } catch (error) {
        return error;
    }
};

// Sélectionner un enregistrement
// data représente une partie des propriété du type
public selectOne = async (
    data: Partial<Product>,
): Promise<Product | unknown> => {
    // connexion au server MySQL
    const connection = await new MySQLService().connect();
    // requete SQL
    // where category.id=....
    // SELECT level.* FROM secretsDeHamam_dev;
    // variable de requete : précédée d'un :, suivi du nom de la
variable
    // requetes préparées :sécurité;(utilisation des varibales de
requetes)la requete est exécutée si elle ne représente pas de risque de
sécurité
    // DISTINCT: Evite les doublons
    const sql = `SELECT ${this.table}.*,
        GROUP_CONCAT(DISTINCT skin.id) AS skin_ids,
        GROUP_CONCAT(DISTINCT pack.id) AS pack_ids,

```

```

GROUP_CONCAT(DISTINCT body_part.id) AS body_part_ids
FROM ${process.env.MYSQL_DATABASE}.${this.table}

JOIN ${process.env.MYSQL_DATABASE}.product_skin
ON product_skin.product_id = product.id
LEFT JOIN ${process.env.MYSQL_DATABASE}.skin
ON product_skin.skin_id = skin.id

JOIN ${process.env.MYSQL_DATABASE}.product_pack
ON product_pack.product_id = product.id
LEFT JOIN ${process.env.MYSQL_DATABASE}.pack
ON product_pack.pack_id = pack.id

JOIN ${process.env.MYSQL_DATABASE}.product_body_part
ON product_body_part.product_id = product.id
JOIN ${process.env.MYSQL_DATABASE}.body_part
ON product_body_part.body_part_id = body_part.id

WHERE ${this.table}.id = :id
GROUP BY ${this.table}.id;
`;

// try / catch pour récupérer les résultats de la requete ou une
erreur
try {
  // execution de la requete
  // si la requete possede des variables, utiliser le parametre
de la méthode
  const [query] = await connection.execute(sql, data);
  // récupérer le premier indice d'un array
  const result = (query as Product[]).shift() as Product;

  // Cles étrangère
  result.category = (await new CategoryRepository().selectOne({
    id: result.category_id,
  })) as Category;

  // tables de jointure
  result.skins = (await new SkinRepository().selectInList(
    result.skin_ids as string
  )) as Skin[];

  result.packs = (await new PackRepository().selectInList(
    result.pack_ids as string
  )) as Pack[];

  result.body_parts = (await new
Body_partRepository().selectInList(
    result.body_part_ids as string
  )) as Body_part[];

  // retourner les résultats
  return result;

```

```

        } catch (error) {
            return error;
        }
    };

    // Selectionner plusieurs éléments dans une liste
    public selectInList = async (list: string): Promise<Product[] |
unknown> => {
        // connexion au server SQL
        const connection = await new MySQLService().connect();

        // requête SQL : SELECT role.* FROM secretsDeHamam_dev.product;
        const sql = `SELECT ${this.table}.*
                    FROM ${process.env.MYSQL_DATABASE}.${this.table}
                    WHERE ${this.table}.id IN (${list})
                    `;

        // Try / Catch : récupérer les résultats de la requête ou un erreur
        try {
            // Execution de la requete
            const [query] = await connection.execute(sql);

            // Boucler sur les resultats pour recuperer les objets en
            relation (composition en Programmation Orientée Objet)
            for (let i = 0; i < (query as Product[]).length; i++) {
                // Récupérer le resultat
                const result = (query as Product[])[i] as Product;

                // Cles étrangère
                result.category = (await new
CategoryRepository().selectOne({
                    id: result.category_id,
                })) as Category;
            }

            // Retourner reultats
            return query;
        } catch (error) {
            return error;
        }
    };

    // Insérer un enregistrement
    public insert = async (
        data: Partial<Product>,
    ): Promise<QueryResult | unknown> => {
        // connexion au server SQL
        const connection = await new MySQLService().connect();

        // requête SQL : SELECT role.* FROM secretsDeHamam_dev.product;
        let sql = `
            INSERT INTO ${process.env.MYSQL_DATABASE}.${this.table}
            VALUE (

```

```

        NULL,
        :name,
        :image,
        :description,
        :price,
        :category_id
    );
`;

// Try / Catch : récupérer les résultats de la requête ou un erreur
try {
    // Demarrer une transaction
    connection.beginTransaction();

    // Execution de la première requete
    await connection.execute(sql, data);

    // Seconde Requete
    sql = `SET @id = LAST_INSERT_ID();`;
    await connection.execute(sql, data);

    // Troisième requete
    /*
    1,2,3
    >>>
    INSERT INTO secretsDeHamman_dev.product_pack
    VALUES
    (1, @id),
    (2, @id),
    (3, @id)
    ;

    split: extraire les données d'une chaine de caractères en
array
    1,2,3 >>>> [1,2,3]
    map
        [1,2,3] >> [(1, @id), (2, @id), (3, @id)]
        join
        [(1, @id), (2, @id), (3, @id)] >> (1, @id), (2, @id), (3,
@id)

    */

    let joinIds = (data.skin_ids as string)
        ?.split(',')
        .map((value) => `(@id, ${value})`)
        .join();

    sql = `
        INSERT INTO
            ${process.env.MYSQL_DATABASE}.product_skin
        VALUES
            ${joinIds}
        ;
    `;

```



```
    await connection.execute(sql, data);

    joinIds = (data.pack_ids as string)
      ?.split(',')
      .map((value) => `(@id, ${value})`)
      .join();

    sql = `
      INSERT INTO
        ${process.env.MYSQL_DATABASE}.product_pack
      VALUES
        ${joinIds}
    `;

    await connection.execute(sql, data);

    joinIds = (data.body_part_ids as string)
      ?.split(',')
      .map((value) => `(@id, ${value})`)
      .join();

    sql = `
      INSERT INTO
        ${process.env.MYSQL_DATABASE}.product_body_part
      VALUES
        ${joinIds}
    `;

    // query doit etre placer sur la dernière requete
    const [query] = await connection.execute(sql);

    // Valider transaction SQL
    connection.commit();

    // Retourner reultats
    return query;
  } catch (error) {
    // Annuler une transaction
    connection.rollback();

    return error;
  }
};

public update = async (
  data: Partial<Product>,
): Promise<QueryResult | unknown> => {
  // connexion au server SQL
  const connection = await new MySQLService().connect();

  // requête SQL : SELECT role.* FROM secretsDeHamam_dev.product;
```

```
let sql = `
  Update
    ${process.env.MYSQL_DATABASE}.${this.table}
  set
    ${this.table}.name = :name,
    ${this.table}.image = :image,
    ${this.table}.description = :description,
    ${this.table}.price = :price,
    ${this.table}.category_id = :category_id
  where
    ${this.table}.id = :id;
`;

// Try / Catch : récupérer les résultats de la requête ou un erreur
try {
  // Demarrer une transaction
  connection.beginTransaction();

  // Execution de la première requete
  await connection.execute(sql, data);

  sql = `
    DELETE FROM
      ${process.env.MYSQL_DATABASE}.product_skin
    WHERE
      product_skin.product_id = :id;
  `;
  await connection.execute(sql, data);

  sql = `
    DELETE FROM
      ${process.env.MYSQL_DATABASE}.product_pack
    WHERE
      product_pack.product_id = :id;
  `;
  await connection.execute(sql, data);

  sql = `
    DELETE FROM
      ${process.env.MYSQL_DATABASE}.product_body_part
    WHERE
      product_body_part.product_id = :id;
  `;
  await connection.execute(sql, data);

  let joinIds = (data.skin_ids as string)
    ?.split(',')
    .map((value) => `(:id, ${value})`)
    .join();

  sql = `
    INSERT INTO
      ${process.env.MYSQL_DATABASE}.product_skin
    VALUES
```

```

        ${joinIds}
    `;

    await connection.execute(sql, data);

    joinIds = (data.pack_ids as string)
        ?.split(',')
        .map((value) => `(:id, ${value})`)
        .join();

    sql = `
        INSERT INTO
            ${process.env.MYSQL_DATABASE}.product_pack
        VALUES
            ${joinIds}
    `;

    await connection.execute(sql, data);

    joinIds = (data.body_part_ids as string)
        ?.split(',')
        .map((value) => `(:id, ${value})`)
        .join();

    sql = `
        INSERT INTO
            ${process.env.MYSQL_DATABASE}.product_body_part
        VALUES
            ${joinIds}
    `;

    // // Seconde Requete
    // sql = `
    // DELETE FROM
    //     ${process.env.MYSQL_DATABASE}.${this.table}
    // WHERE
    //     ${this.table}.id = :id;
    // `;

    // // Seconde Requete
    // sql = `SET @id = LAST_INSERT_ID();`;
    // await connection.execute(sql);

    // Troisième requete
    /*
    1,2,3
    >>>
    INSERT INTO secretsDeHammam_dev.product_pack
    VALUES
    (1, @id),
    (2, @id),

```

```

        (3, @id)
        ;

        split: extraire les données d'une chaîne de caractères en
array
        1,2,3 >>>> [1,2,3]
        map
            [1,2,3] >> [(1, @id), (2, @id), (3, @id)]
            join
            [(1, @id), (2, @id), (3, @id)] >> (1, @id), (2, @id), (3,
@id)

        */

        // const joinIds = data.skin_ids
        // ?.split(',')
        // .map((value) => `(@id, ${value})`)
        // .join();

        // sql = `
        // INSERT INTO
        //     ${process.env.MYSQL_DATABASE}.product_skin
        // VALUES
        //     ${joinIds}
        // ;
        // `;

        // query doit être placé sur la dernière requête
        const [query] = await connection.execute(sql, data);

        // Valider transaction SQL
        connection.commit();

        // Retourner résultats
        return query;
    } catch (error) {
        // Annuler une transaction
        connection.rollback();

        return error;
    }
};

public delete = async (
    data: Partial<Product>,
): Promise<QueryResult | unknown> => {
    // connexion au serveur SQL
    const connection = await new MySQLService().connect();

    // requête SQL : SELECT role.* FROM secretsDeHamam_dev.product;
    let sql = `
        DELETE FROM
            ${process.env.MYSQL_DATABASE}.product_skin
        WHERE
            product_skin.product_id = :id;
    `;

```

```
`;  
  
// Try / Catch : récupérer les résultats de la requête ou un erreur  
try {  
  // Demarrer une transaction  
  connection.beginTransaction();  
  
  // Execution de la première requete  
  await connection.execute(sql, data);  
  
  sql = `  
    DELETE FROM  
      ${process.env.MYSQL_DATABASE}.product_pack  
    WHERE  
      product_pack.product_id = :id;  
  `;  
  await connection.execute(sql, data);  
  
  sql = `  
    DELETE FROM  
      ${process.env.MYSQL_DATABASE}.product_body_part  
    WHERE  
      product_body_part.product_id = :id;  
  `;  
  await connection.execute(sql, data);  
  
  // // Seconde Requete  
  sql = `  
    DELETE FROM  
      ${process.env.MYSQL_DATABASE}.${this.table}  
    WHERE  
      ${this.table}.id = :id;  
  `;  
  
  // query doit etre placer sur la dernière requete  
  const [query] = await connection.execute(sql, data);  
  
  // Valider transaction SQL  
  connection.commit();  
  
  // Retourner reultats  
  return query;  
} catch (error) {  
  // Annuler une transaction  
  connection.rollback();  
  
  return error;  
}  
};  
}  
  
export default ProductRepository;
```

FICHER : code/server/repository/role_repository.ts

```
import type { Role } from "../../models/role";
import MySQLService from "../service/mysql_service";

class RoleRepository {
  // Nom de la table SQL
  private table = "role";

  // Selectionner tous les enregistrements
  public selectAll = async (): Promise<Role[] | unknown> => {
    // connexion au server SQL
    const connection = await new MySQLService().connect();

    // requête SQL : SELECT role.* FROM secretsDeHamam_dev.role;
    const sql = `SELECT ${this.table}.*
      FROM ${process.env.MYSQL_DATABASE}.${this.table}`;

    // Try / Catch : récupérer les résultats de la requête ou un erreur
    try {
      // Execution de la requete
      const [query] = await connection.execute(sql);

      // Retourner reultats
      return query;
    } catch (error) {
      return error;
    }
  };

  // Sélectionner un enregistrement
  // data représente une partie des propriété du type
  public selectOne = async (data: Partial<Role>): Promise<Role |
unknown> => {
    // connexion au server MySQL
    const connection = await new MySQLService().connect();
    // requete SQL
    // where category.id=....
    // SELECT level.* FROM secretsDeHamam_dev;
    // variable de requete : précédée d'un :, suivi du nom de la
variable
    // requetes préparées :sécurité;(utilisation des varibales de
requetes)la requete est exécutée si elle ne représente pas de risque de
sécurité
    const sql = `
      SELECT ${this.table}.*
      FROM ${process.env.MYSQL_DATABASE}.${this.table}
      WHERE ${this.table}.id = :id;
    `;
    // try / catch pour récupérer les resultats de la requete ou une

```

```

    erreur
    try {
        // execution de la requete
        // si la requete possede des variables, utiliser le parametre
de la méthode
        const [query] = await connection.execute(sql, data);
        // récupérer le premier indice d'un array
        const result = (query as Role[]).shift();
        // retourner les résultats
        return result;
    } catch (error) {
        return error;
    }
};
}

export default RoleRepository;

```

FICHER : code/server/repository/skin_repository.ts

```

import type { Product } from "../../models/product";
import type { Skin } from "../../models/skin";
import MySQLService from "../service/mysql_service";
import ProductRepository from "../product_repository";

class SkinRepository {
    // Nom de la table SQL
    private table = "skin";

    // Selectionner tous les enregistrements
    public selectAll = async (): Promise<Skin[] | unknown> => {
        // connexion au server SQL
        const connection = await new MySQLService().connect();

        /* requête SQL : SELECT role.* FROM secretsDeHamam_dev.skin;
const sql = `SELECT ${this.table}.*
                FROM ${process.env.MYSQL_DATABASE}.${this.table}`;
        */

        // requete SQL
        /*
        SELECT
            skin.*,
            GROUP_CONCAT(product_id) AS product_ids
        FROM
            secretsDeHamam_dev.skin
        JOIN
            secretsDeHamam_dev.product_skin
        ON

```

```

        product_skin.skin_id = skin.id
    JOIN
        secretsDeHamam_dev.product
    ON
        product.id = product_skin.product_id
    GROUP BY
        skin.id `
    */

    const sql = `
        SELECT ${this.table}.*,
        GROUP_CONCAT(product_id) AS product_ids
    FROM ${process.env.MYSQL_DATABASE}.${this.table}
    JOIN ${process.env.MYSQL_DATABASE}.product_skin
    ON product_skin.skin_id = ${this.table}.id
    JOIN ${process.env.MYSQL_DATABASE}.product
    ON product.id = product_skin.product_id
    GROUP BY ${this.table}.id ;
    `;

    // Try / Catch : récupérer les résultats de la requête ou un erreur
    try {
        // Execution de la requete
        const [query] = await connection.execute(sql);

        for (let i = 0; i < (query as Skin[]).length; i++) {
            // Récuperer le resultat
            const result = (query as Skin[])[i] as Skin;

            // tables de jointure
            result.products = (await new
ProductRepository().selectInList(
                result.product_ids,
            )) as Product[];
        }

        // Retourner reultats
        return query;
    } catch (error) {
        return error;
    }
};

// Sélectionner un enregistrement
// data représente une partie des propriété du type
public selectOne = async (data: Partial<Skin>): Promise<Skin |
unknown> => {
    // connexion au server MySQL
    const connection = await new MySQLService().connect();
    // requete SQL
    // where category.id=....
    // SELECT level.* FROM secretsDeHamam_dev;
    // variable de requete : précédée d'un :, suivi du nom de la

```



```

variable
    // requetes préparées :sécurité;(utilisation des varibales de
    requetes)la requete est exécutée si elle ne représente pas de risque de
    sécurité
    // const sql = `
    // SELECT ${this.table}.*
    // FROM ${process.env.MYSQL_DATABASE}.${this.table}
    // WHERE ${this.table}.id = :id;
    // `;

    const sql = `
    SELECT ${this.table}.*,
    GROUP_CONCAT(product_id) AS product_ids
    FROM ${process.env.MYSQL_DATABASE}.${this.table}
    JOIN ${process.env.MYSQL_DATABASE}.product_skin
    ON product_skin.skin_id = ${this.table}.id
    JOIN ${process.env.MYSQL_DATABASE}.product
    ON product.id = product_skin.product_id
    WHERE ${this.table}.id = :id
    GROUP BY ${this.table}.id ;
    `;

    // try / catch pour récupérer les resultats de la requete ou une
    erreur
    try {
        // execution de la requete
        // si la requete possede des variables, utiliser le parametre
    de la méthode
        const [query] = await connection.execute(sql, data);
        // récupérer le premier indice d'un array
        const result = (query as Skin[]).shift() as Skin;
        result.products = (await new ProductRepository().selectInList(
            result.product_ids,
        )) as Product[];
        // retourner les résultats
        return result;
    } catch (error) {
        return error;
    }
};

public selectInList = async (list: string): Promise<Skin[] | unknown>
=> {
    // connexion au server SQL
    const connection = await new MySQLService().connect();

    // requête SQL : SELECT role.* FROM secretsDeHamman_dev.product;
    const sql = `SELECT ${this.table}.*
        FROM ${process.env.MYSQL_DATABASE}.${this.table}
        WHERE ${this.table}.id IN (${list})
        `;

```

```

        // Try / Catch : récupérer les résultats de la requête ou un erreur
        try {
            // Execution de la requete
            const [query] = await connection.execute(sql);

            // Retourner reultats
            return query;
        } catch (error) {
            return error;
        }
    };
}

export default SkinRepository;

```

FICHER : code/server/repository/user_repository.ts

```

import type { User } from "../../models/user";
import MySQLService from "../service/mysql_service";

class UserRepository {
    // Nom de la table SQL
    private table = "user";

    // Selectionner tous les enregistrements
    public selectAll = async (): Promise<User[] | unknown> => {
        // connexion au server SQL
        const connection = await new MySQLService().connect();

        // requête SQL : SELECT role.* FROM secretsDeHamam_dev.pack;
        const sql = `SELECT ${this.table}.*
                    FROM ${process.env.MYSQL_DATABASE}.${this.table};`;

        // Try / Catch : récupérer les résultats de la requête ou un erreur
        try {
            // Execution de la requete
            const [query] = await connection.execute(sql);

            // Retourner reultats
            return query;
        } catch (error) {
            return error;
        }
    };

    // Sélectionner un enregistrement
    // data représente une partie des propriété du type
    public selectOne = async (data: Partial<User>): Promise<User |
unknown> => {

```

```

        // connexion au server MySQL
        const connection = await new MySQLService().connect();
        // requete SQL
        // where category.id=....
        // SELECT level.* FROM secretsDeHamam_dev;
        // variable de requete : précédée d'un :, suivi du nom de la
variable
        // requetes préparées :sécurité;(utilisation des varibales de
requetes)la requete est exécutée si elle ne représente pas de risque de
sécurité
        const sql = `
        SELECT ${this.table}.*
        FROM ${process.env.MYSQL_DATABASE}.${this.table}
        WHERE ${this.table}.id = :id;
        `;
        // try / catch pour récupérer les resultats de la requete ou une
erreur
        try {
            // execution de la requete
            // si la requete possede des variables, utiliser le parametre
de la méthode
            const [query] = await connection.execute(sql, data);
            // récupérer le premier indice d'un array
            const result = (query as User[]).shift();
            // retourner les résultats
            return result;
        } catch (error) {
            return error;
        }
    };
}

export default UserRepository;

```

FICHER : code/server/router/body_part_router.ts

```

import express from "express";
import Body_PartController from "../controller/body_part_controller";

class Body_partRouter {
    // router express
    private router = express.Router();

    // liste des routes
    public getRoutes = () => {
        // Créer une route en GET simple
        this.router.get("/", new Body_PartController().index);

        // variable de route: précédé par ;; suivi du nom de la variable

```

```
        // appel 1
        this.router.get("/:id", new Body_PartController().selectOne);

        // retourner le routeur
        return this.router;
    };
}

export default Body_partRouter;
```

FICHER : code/server/router/category_router.ts

```
import express from "express";
import CategoryController from "../controller/category_controller";

class CategoryRouter {
    // router express
    private router = express.Router();

    // liste des routes
    public getRoutes = () => {
        // Créer une route en GET simple
        this.router.get("/", new CategoryController().index);

        // variable de route: précédé par ;; suivi du nom de la variable
        // appel 1
        this.router.get("/:id", new CategoryController().selectOne);

        // retourner le routeur
        return this.router;
    };
}

export default CategoryRouter;
```

FICHER : code/server/router/homepage_router.ts

```
import express from "express";
import HomepageController from "../controller/homepage_controller";

class HomepageRouter {
    // router express
    private router = express.Router();

    // liste des routes
```

```
    public getRoutes = () => {
        // Créer une route en GET simple
        this.router.get("/", new HomepageController().index);

        // retourner le routeur
        return this.router;
    };
}

export default HomepageRouter;
```

FICHER : code/server/router/pack_router.ts

```
import express from "express";
import PackController from "../controller/pack_controller";

class PackRouter {
    // router express
    private router = express.Router();

    // liste des routes
    public getRoutes = () => {
        // Créer une route en GET simple
        this.router.get("/", new PackController().index);

        // variable de route: précédé par ;; suivi du nom de la variable
        // appel 1
        this.router.get("/:id", new PackController().selectOne);

        // retourner le routeur
        return this.router;
    };
}

export default PackRouter;
```

FICHER : code/server/router/product_router.ts

```
import express from "express";
import multer from "multer";
import ProductController from "../controller/product_controller";

class ProductRouter {
    // router express
    private router = express.Router();
```

```
// multer permet de gérer le transfert de fichier
// private multer = multer({ dest: "public" });
// Si fichier ranger dans sous dossier, modifier le chemin
private multer = multer({ dest:
`${process.env.PUBLIC_DIR}/img/products` });

// liste des routes
public getRoutes = () => {
  // Créer une route en GET simple
  this.router.get("/", new ProductController().index);

  // variable de route: précédé par ;; suivi du nom de la variable
  // appel 1
  this.router.get("/:id", new ProductController().selectOne);

  // middleware multer
  // Creation: inserer un enregistrement => Post
  this.router.post("/", this.multer.any(), new
ProductController().insert);
  // Modification: mettre à jour un enregistrement => Put
  this.router.put("/", this.multer.any(), new
ProductController().update);
  // Supprimer => Delete
  this.router.delete("/", new ProductController().delete);

  // retourner le routeur
  return this.router;
};
}

export default ProductRouter;
```

FICHER : code/server/router/role_router.ts

```
import express from "express";
import RoleController from "../controller/role_controller";

class RoleRouter {
  // router express
  private router = express.Router();

  // liste des routes
  public getRoutes = () => {
    // Créer une route en GET simple
    this.router.get("/", new RoleController().index);

    // variable de route: précédé par ;; suivi du nom de la variable
    // appel 1
  }
}
```

```
        this.router.get("/:id", new RoleController().selectOne);

        // retourner le routeur
        return this.router;
    };
}

export default RoleRouter;
```

FICHER : code/server/router/skin_router.ts

```
import express from "express";
import SkinController from "../controller/skin_controller";

class SkinRouter {
    // router express
    private router = express.Router();

    // liste des routes
    public getRoutes = () => {
        // Créer une route en GET simple
        this.router.get("/", new SkinController().index);

        // variable de route: précédé par ;; suivi du nom de la variable
        // appel 1
        this.router.get("/:id", new SkinController().selectOne);

        // retourner le routeur
        return this.router;
    };
}

export default SkinRouter;
```

FICHER : code/server/router/user_router.ts

```
import express from "express";
import UserController from "../controller/user_controller";

class UserRouter {
    // router express
    private router = express.Router();

    // liste des routes
    public getRoutes = () => {
```

```

    // Créer une route en GET simple
    this.router.get("/", new UserController().index);

    // variable de route: précédé par ;; suivi du nom de la variable
    // appel 1
    this.router.get("/:id", new UserController().selectOne);

    // retourner le routeur
    return this.router;
  };
}

export default UserRouter;

```

FICHER : code/server/service/file_service.ts

```

// fs = file system
import { fileTypeFromFile } from 'file-type';
import fs from 'node:fs/promises';

//
// [
//   {
//     fieldname: 'image',
//     originalname: 'Argile_et_Rhassoul.png',
//     encoding: '7bit',
//     mimetype: 'image/png',
//     destination: 'public/img/products',
//     filename: 'bde194790217794769729886886c077b',
//     path: 'public/img/products/bde194790217794769729886886c077b',
//     size: 3468635
//   }
// ]

class FileService {

  // fonction qui renomme le fichier transféré en ajoutant l'extension
  // Cette fonction doit retourner le nom complet du fichier
  // C'est a nous de le faire pour des raisons de sécurité
  public rename = async (file: Express.Multer.File): Promise<string> =>
  {
    // Ajouter l'extension au fichier
    const fullname = `${file.filename}.${(await
fileTypeFromFile(file.path)).ext}`;
    console.log(fullname);
    // Renommer le fichier avec son nom complet
    await fs.rename(file.path, `${file.destination}/${fullname}`);
    // Retourner le nom complet
    return fullname;
  }
}

```



```
    }  
  
  }  
  
  export default FileService;
```

FICHER : code/server/service/mysql_service.ts

```
import mysql, { type PoolConnection } from "mysql2/promise";  
  
class MySQLService {  
  // propriété statique : accessible sans objet  
  private static instance: PoolConnection;  
  
  // Créer fonction de connexion au server MySQL  
  public connect = async () => {  
    // si aucune connexion existe  
    if (!MySQLService.instance) {  
      MySQLService.instance = await mysql  
        .createPool({  
          host: process.env.MYSQL_HOST,  
          user: process.env.MYSQL_USER,  
          password: process.env.MYSQL_PASSWORD,  
          database: process.env.MYSQL_DATABASE,  
          namedPlaceholders: true,  
        })  
        .getConnection();  
    }  
  
    // Retourner la connexion  
    return MySQLService.instance;  
  };  
}  
  
export default MySQLService;
```

FICHER : code/src/assets/css/admin_css/add.module.css

```
/* Conteneur principal pour centrer le formulaire */  
.formContainer {  
  max-width: 850px;  
  margin: 0 auto;  
  animation: fadeIn 0.5s ease-in-out;  
}  
  
/* En-tête du formulaire */
```

```
.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 2.5rem;
}

.header h1 {
  font-family: 'Playfair Display', serif;
  color: #4a3f35;
  margin: 0;
}

.header p {
  color: #8c725b;
  margin: 5px 0 0 0;
  font-style: italic;
}

/* Bouton Retour / Annuler */
.backBtn {
  text-decoration: none;
  color: #8c725b;
  border: 1px solid #eaddca;
  padding: 10px 20px;
  border-radius: 8px;
  font-size: 0.9rem;
  transition: all 0.3s ease;
  background: white;
}

.backBtn:hover {
  background: #fdf2f2;
  color: #d9534f;
  border-color: #fca5a5;
}

/* Le bloc formulaire */
.productForm {
  background: white;
  padding: 2.5rem;
  border-radius: 15px;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.05);
  border: 1px solid #f3efe4;
}

/* Groupes de champs (Label + Input) */
.formGroup {
  display: flex;
  flex-direction: column;
  margin-bottom: 1.5rem;
}

.formGroup label {
```

```
    font-weight: 600;
    color: #645945;
    margin-bottom: 0.6rem;
    font-size: 0.95rem;
}

/* Style des Inputs, Select et Textarea */
.formGroup input,
.formGroup select,
.formGroup textarea {
    padding: 12px 15px;
    border: 1px solid #eaddca;
    border-radius: 8px;
    font-size: 1rem;
    color: #4a3f35;
    background-color: #fff;
    transition: border-color 0.3s ease, box-shadow 0.3s ease;
}

.formGroup input:focus,
.formGroup select:focus,
.formGroup textarea:focus {
    outline: none;
    border-color: #c5a059;
    box-shadow: 0 0 0 3px rgba(197, 160, 89, 0.1);
}

/* Mise en page sur deux colonnes pour Prix et Stock */
.formRow {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 20px;
}

/* Bouton de validation final */
.submitBtn {
    width: 100%;
    padding: 15px;
    background-color: #c5a059;
    color: white;
    border: none;
    border-radius: 8px;
    font-size: 1.1rem;
    font-weight: bold;
    cursor: pointer;
    margin-top: 1rem;
    transition: background 0.3s ease, transform 0.2s active;
}

.submitBtn:hover {
    background-color: #a68648;
}

.submitBtn:active {
```

```
        transform: scale(0.98);
    }

    /* Petit effet d'entrée */
    @keyframes fadeIn {
        from { opacity: 0; transform: translateY(10px); }
        to { opacity: 1; transform: translateY(0); }
    }

    /* Responsive pour les petits écrans */
    @media (max-width: 600px) {
        .formRow {
            grid-template-columns: 1fr;
        }
    }
}
```

FICHER : code/src/assets/css/admin_css/admin_product_index.module.css

```
.crudContainer {
    padding: 20px;
    max-width: 1200px;
    margin: 0 auto;
}

.crudHeader {
    display: flex;
    justify-content: space-between;
    align-items: flex-end;
    margin-bottom: 30px;
    gap: 20px;
}

.addButton {
    background-color: #645945;
    color: white;
    padding: 12px 24px;
    border-radius: 8px;
    text-decoration: none;
    font-weight: bold;
    transition: all 0.3s;
    white-space: nowrap; /*Empêche le texte du bouton de revenir à la
ligne*/
    display: inline-block;
}

.addButton:hover {
    background-color: #c5a059;
    transform: translateY(-2px);
}
```

```
.tableWrapper {
  background: white;
  padding: 20px;
  border-radius: 12px;
  box-shadow: 0 4px 20px rgba(0,0,0,0.05);
  overflow-x: auto;
}

.crudTable {
  width: 100%;
  border-collapse: collapse;
}

.crudTable th {
  text-align: left;
  padding: 15px;
  border-bottom: 2px solid #f3efe4;
  color: #8c725b;
}

.crudTable td {
  padding: 15px;
  border-bottom: 1px solid #f3efe4;
}

.productName {
  font-weight: 600;
  color: #8c725b;
}

.actions {
  display: flex;
  gap: 10px;
}

.modif {
  color: #8c725b;
}

.suppl {
  color: red;
}

.editBtn, .deleteBtn {
  padding: 6px 12px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 0.85rem;
}

.editBtn {
```

```
    background-color: #eaddca;
    color: #645945;
}

.deleteBtn {
    background-color: #fdf2f2;
    color: #d9534f;
}

.deleteBtn:hover {
    background-color: #d9534f;
    color: white;
}
```

FICHER : code/src/assets/css/admin_css/sidebar.module.css

```
*{
    box-sizing: border-box;
}

.sidebar {
    width: 260px;
    height: 100vh;
    background-color: #645945;
    color: #f3efe4;
    text-decoration: none;
    font-family: 'Playfair Display', serif;
    font-size: 1.1rem;
    display: flex;
    flex-direction: column;
    position: fixed;
    left: 0;
    top: 0;
    padding: 2rem 1rem;
}

.logoSection {
    margin-bottom: 3rem;
    text-align: center;
}

.logoSection h2 {
    color: #c5a059;
    font-family: 'Playfair Display', serif;
    margin: 0;
}

.badge {
    font-size: 0.7rem;
    text-transform: uppercase;
    letter-spacing: 2px;
}
```

```
    color: #8c725b;
}

.navLinks {
    display: flex;
    flex-direction: column;
    gap: 0.8rem;
}

.link, .active {
    display: flex;
    align-items: center;
    padding: 12px 15px;
    text-decoration: none;
    border-radius: 8px;
    transition: 0.3s;
    color: #a89f91;
}

.link:hover {
    background: rgba(197, 160, 89, 0.1);
    color: #c5a059;
}

.active {
    background: #c5a059;
    color: white;
}

.icon {
    margin-right: 12px;
    font-size: 1.2rem;
}

.sidebarFooter {
    margin-top: auto;
    padding-top: 1rem;
    border-top: 1px solid #333;
}

.exitLink {
    background-color: #c5a059;
    color: white;
    display: flex; /* Aligne le texte comme les autres liens */
    align-items: center;
    padding: 12px 15px; /* Même espacement interne que .link */
    text-decoration: none;
    font-family: 'Playfair Display', serif;
    font-size: 1.1rem;
    border-radius: 8px;
    transition: 0.3s;
}

.exitLink:hover {
```

```
background-color: #d9534f; /* Devient rouge plein au survol */
color: white;
} /* Texte blanc au survol */
```

FICHER : code/src/assets/css/admin_layout.module.css

```
/* Container principal */
.container {
  padding: 2rem;
  color: #4a3f35;
  background-color: #fcfaf7;
  min-height: 100%;
}

.adminLayout {
  display: flex;
  width: 100%;
  min-height: 100vh;
}

.mainContent {
  /* Épouse la largeur restante de l'écran */
  flex: 1;

  /* TRÈS IMPORTANT : Pousse le contenu pour qu'il commence
   après la sidebar qui fait 260px */
  margin-left: 260px;

  background-color: #fcfaf7; /* Fond clair pour le confort visuel */
  padding: 40px;
  box-sizing: border-box;
  min-height: 100vh;
}

/* En-tête de la page */
.header {
  margin-bottom: 3rem;
  border-bottom: 2px solid #eaddca;
  padding-bottom: 1rem;
}

.header h1 {
  font-family: 'Playfair Display', serif; /* Si tu l'as importée, sinon
  serif */
  font-size: 2.2rem;
  color: #645945;
  margin: 0;
}

.header p {
  font-style: italic;
```



```
    color: #8c725b;
}

/* Grille de statistiques (les cartes) */
.statsGrid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 1.5rem;
    margin-bottom: 3rem;
}

.card {
    background: white;
    padding: 1.5rem;
    border-radius: 15px;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.05);
    border: 1px solid #f3efe4;
    transition: transform 0.2s ease;
}

.card:hover {
    transform: translateY(-5px);
    border-color: #c5a059;
}

.card h3 {
    font-size: 1rem;
    text-transform: uppercase;
    letter-spacing: 1px;
    color: #8c725b;
    margin-bottom: 1rem;
}

.number {
    font-size: 2.5rem;
    font-weight: bold;
    color: #645945;
    margin: 0;
}

.warning {
    color: #b33a3a; /* Pour les alertes de stock */
}

/* Zone d'activité récente */
.recentActivity {
    background: white;
    padding: 2rem;
    border-radius: 15px;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.05);
}

.recentActivity h2 {
    font-size: 1.4rem;
}
```

```
    color: #645945;
    margin-bottom: 1.5rem;
  }

.recentActivity ul {
  list-style: none;
  padding: 0;
}

.recentActivity li {
  padding: 1rem 0;
  border-bottom: 1px solid #f3efe4;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.recentActivity li:last-child {
  border-bottom: none;
}

.date {
  font-size: 0.85rem;
  color: #a89f91;
  background: #f3efe4;
  padding: 4px 10px;
  border-radius: 20px;
}

/* Responsive */
@media (max-width: 768px) {
  .container {
    padding: 1rem;
  }

  .header h1 {
    font-size: 1.8rem;
  }
}
```

FICHER : code/src/assets/css/base.css

```
/* Polices */

/* Variables CSS */
:root {
  --background-color: #f3efe4;
  --padding-left: 2rem;
  --padding-right: 2rem;
  --margin: 10px 0 10px 0;
```

```
--color: #7e5835;
--font-weight: bold;
--font-style: italic;
--line-height: 32px;
}

/* Balises HTML */
* {
  box-sizing: border-box;
}

img {
  width: 100%;
}

body {
  background-color: var(--background-color);
  color: var(--color);
}
```

FICHER : code/src/assets/css/featured_products.module.css

```
.featuredProducts {
  max-width: 1200px;
  margin: 100px auto;
  padding: 0 20px;
}

.sectionHeader {
  text-align: center;
  margin-bottom: 60px;
}

.sectionHeader span {
  color: #c5a059;
  text-transform: uppercase;
  letter-spacing: 3px;
  font-size: 0.85rem;
  font-weight: 700;
  display: block;
  margin-bottom: 10px;
}

.sectionHeader h2 {
  font-family: "Playfair Display", serif;
  color: #645945;
  font-size: 2.8rem;
  margin: 0;
}
```

```
.productGrid {
  display: grid;
  /* Création automatique de colonnes : minimum 280px par carte */
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 50px 30px; /* 50px d'espace vertical, 30px horizontal */
}

/* Version Tablette / Mobile */
@media (max-width: 768px) {
  .sectionHeader h2 {
    font-size: 2rem;
  }
  .featuredProducts {
    margin: 60px auto;
  }
}
```

FICHER : code/src/assets/css/features.module.css

```
.featuresSection {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 30px;
  max-width: 1200px;
  margin: -50px auto 80px;
  position: relative;
  z-index: 10;
  padding: 0 20px;
}

.featureCard {
  background: white;
  padding: 40px;
  border-radius: 20px;
  text-align: center;
  box-shadow: 0 20px 40px rgba(100, 89, 69, 0.1);
  transition:
    transform 0.4s ease,
    box-shadow 0.4s ease;
}

.featureCard:hover {
  transform: translateY(-8px);
  box-shadow: 0 30px 60px rgba(100, 89, 69, 0.15);
}

.icon {
  font-size: 2.5rem;
```

```
    display: block;
    margin-bottom: 15px;
}

.featureCard h3 {
    font-family: "Playfair Display", serif;
    color: #645945;
    margin-bottom: 10px;
}

.featureCard p {
    font-weight: 300;
    color: #8a7d6a;
    font-size: 0.95rem;
    margin: 0;
}

@media (max-width: 768px) {
    .featuresSection {
        margin: -30px auto 40px;
        grid-template-columns: 1fr;
    }
}
```

FICHER : code/src/assets/css/footer.module.css

```
.footer {
    background-color: #645945;
    color: #f3efe4;
    padding: 40px 20px;
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 30px;
    margin-top: 50px; /* Espace avec le contenu de la page */
}

/* Logo */
.logo {
    width: 100px;
    transition: transform 0.3s ease;
}

.logo img {
    width: 100%;
    height: auto;
    filter: brightness(0) invert(1); /* Rend le logo blanc pour ressortir
sur le marron */
}
```

```
.logo:hover {
  transform: scale(1.05);
}

/* Navigation */
.nav {
  width: 100%;
}

.menu {
  list-style: none;
  padding: 0;
  margin: 0;
  display: flex;
  flex-wrap: wrap; /* Pour mobile : les liens passent à la ligne */
  justify-content: center;
  gap: 20px;
}

.menu li a {
  color: #f3efe4;
  text-decoration: none;
  font-size: 0.9rem;
  text-transform: uppercase;
  letter-spacing: 1px;
  transition: color 0.3s ease;
}

.menu li a:hover {
  color: #c5a059;
}

/* Icône Sociale */
.icone {
  font-size: 1.8rem;
  color: #f3efe4;
  cursor: pointer;
  transition: all 0.3s ease;
}

.icone:hover {
  color: #c5a059;
  transform: translateY(-3px);
}

/* Responsive Ordinateur */
@media (min-width: 768px) {
  .footer {
    flex-direction: row;
    justify-content: space-between;
    padding: 40px 60px;
  }
}
```

```
.menu {  
  gap: 30px;  
}  
  
.nav {  
  width: auto;  
}  
}
```

FICHER : code/src/assets/css/form.module.css

```
.formWrapper {  
  max-width: 500px;  
  margin: 60px auto;  
  padding: 40px;  
  background: #ffffff;  
  border-radius: 20px;  
  /* Ombre douce pour un effet aérien */  
  box-shadow: 0 15px 35px rgba(140, 114, 91, 0.1);  
  border: 1px solid rgba(197, 160, 89, 0.1);  
  position: relative;  
  overflow: hidden;  
}  
  
/* Petit motif floral ou organique en fond (optionnel) */  
.formWrapper::after {  
  content: "";  
  position: absolute;  
  top: -50px;  
  right: -50px;  
  width: 150px;  
  height: 150px;  
  background: rgba(197, 160, 89, 0.05);  
  border-radius: 50%;  
}  
  
.formWrapper h2 {  
  font-family: 'Playfair Display', serif;  
  color: #5d6d5e; /* Vert sauge profond */  
  text-align: center;  
  font-size: 2rem;  
  margin-bottom: 30px;  
  font-weight: 600;  
}  
  
.orientalForm {  
  display: flex;  
  flex-direction: column;  
  gap: 20px;  
}
```

```
/* Style des labels s'ils sont ajoutés, ou placeholder */
.orientalForm input,
.orientalForm textarea {
  font-family: 'Poppins', sans-serif;
  width: 100%;
  padding: 14px;
  border: 1px solid #e0e0e0;
  border-radius: 10px;
  background-color: #faf9f7; /* Couleur sable très clair */
  font-size: 0.95rem;
  transition: all 0.4s ease;
  color: #4a4a4a;
}

.orientalForm input:focus,
.orientalForm textarea:focus {
  border-color: #c5a059; /* Or brossé */
  background-color: #ffffff;
  box-shadow: 0 4px 12px rgba(197, 160, 89, 0.1);
  outline: none;
}

/* Style spécifique pour le bouton "Rituel de Beauté" */
.orientalForm button {
  font-family: 'Poppins', sans-serif;
  background: linear-gradient(135deg, #8c725b 0%, #a68b74 100%);
  color: white;
  padding: 16px;
  border: none;
  border-radius: 50px; /* Bouton arrondi pour la douceur */
  font-size: 1rem;
  font-weight: 500;
  letter-spacing: 1.5px;
  text-transform: uppercase;
  cursor: pointer;
  transition: transform 0.2s, box-shadow 0.2s;
  margin-top: 10px;
}

.orientalForm button:hover {
  transform: translateY(-2px);
  box-shadow: 0 7px 20px rgba(140, 114, 91, 0.3);
}

/* Texte d'aide ou liens sous le formulaire */
.formFooter {
  text-align: center;
  margin-top: 20px;
  font-family: 'Poppins', sans-serif;
  font-size: 0.85rem;
  color: #8c725b;
}
```



```
/*TEMPORAIRE*/

.groupTitle {
  font-family: 'Playfair Display', serif;
  font-size: 0.85rem;
  color: #8c725b;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 1px;
  margin-bottom: 10px;
}

/* Style spécifique pour le SELECT dans le formGroup */
.formGroup select {
  width: 100%;
  border: none;
  background: transparent;
  font-family: 'Poppins', sans-serif;
  font-size: 1rem;
  color: #4a4a4a;
  cursor: pointer;
  outline: none;
  padding: 5px 0;
}

/* Conteneur pour aligner les checkbox proprement */
.checkboxGrid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(140px, 1fr));
  gap: 10px;
  padding-top: 5px;
}

/* Style de chaque ligne de checkbox */
.checkboxItem {
  display: flex;
  align-items: center;
  gap: 10px;
  cursor: pointer;
  font-size: 0.9rem;
  color: #4a4a4a;
  transition: color 0.2s;
}

.checkboxItem:hover {
  color: #c5a059;
}

.checkboxItem input[type="checkbox"] {
  width: auto; /* Empêche la checkbox de prendre 100% de largeur */
  accent-color: #c5a059; /* Colore la case en doré */
}
```

```
    cursor: pointer;
}
```

FICHER : code/src/assets/css/header.module.css

```
.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 10px 40px;
  background-color: #ffffff;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.05);
  position: sticky;
  top: 0;
  z-index: 1000;
}

h1 {
  display: none;
}

/* RESPONSIVE ECRAN ORDINATEUR
@media only screen and (min-width: 769px) {
  .header {
  }
} */
```

FICHER : code/src/assets/css/hero.module.css

```
.hero {
  height: 80vh;
  display: flex;
  align-items: center;
  justify-content: center;
  background:
    linear-gradient(rgba(0, 0, 0, 0.4), rgba(0, 0, 0, 0.4)),
    url("/img/products/Hammam.png") center / cover no-repeat;
  color: #645945;
  text-align: center;
  border-radius: 0 0 40px 40px;
}

.heroContent {
  color: white;
}
```

```
    max-width: 800px;
    padding: 20px;
    position: relative;
    z-index: 5;
  }

.heroContent h2 {
  font-family: "Playfair Display", serif;
  font-size: 4rem;
  font-weight: 700;
  margin-bottom: 25px;
  letter-spacing: 2px;
  line-height: 1.1;
  text-shadow: 0 2px 15px rgba(0, 0, 0, 0.2);
}

.heroContent p {
  font-size: 1.2rem;
  font-weight: 300;
  line-height: 1.8;
  margin-bottom: 45px;
  max-width: 650px;
  margin: 0 auto 45px;
  color: rgba(255, 255, 255, 0.9);
}

.ctaButton {
  background-color: #c5a059;
  color: white;
  padding: 15px 40px;
  border-radius: 50px;
  text-decoration: none;
  font-weight: bold;
  display: inline-block;
  transition: 0.3s ease;
}

.ctaButton:hover {
  background-color: white;
  color: #c5a059;
}

@media (max-width: 768px) {
  .hero {
    height: 70vh;
    border-radius: 0 0 30px 30px;
  }
  .heroContent h2 {
    font-size: 2.2rem;
  }
}
```

FICHER : code/src/assets/css/homepage.module.css

```
.homeWrapper {
  background-color: #fffd9;
}

.hero {
  height: 80vh;
  display: flex;
  align-items: center;
  justify-content: center;
  background:
    linear-gradient(rgba(0, 0, 0, 0.4), rgba(0, 0, 0, 0.4)),
    url("/img/products/Hammam.png") center / cover no-repeat;
  color: #645945;
  text-align: center;
  border-radius: 0 0 40px 40px;
  margin-bottom: 0;
}

.heroContent {
  color: white;
  max-width: 800px;
  padding: 20px;
  position: relative;
  z-index: 5;
}

.heroContent h2 {
  font-family: "Playfair Display", serif;
  font-size: 4rem;
  font-weight: 700;
  margin-bottom: 25px;
  letter-spacing: 2px;
  line-height: 1.1;
  text-align: center;
  text-shadow: 0 2px 15px rgba(0, 0, 0, 0.2);
}

.heroContent p {
  font-size: 1.2rem;
  font-weight: 300;
  line-height: 1.8;
  margin-bottom: 45px;
  max-width: 650px;
  margin-left: auto;
  margin-right: auto;
  color: rgba(255, 255, 255, 0.9);
}

.ctaButton {
```

```
    background-color: #c5a059;
    color: white;
    padding: 15px 40px;
    border-radius: 50px;
    text-decoration: none;
    font-weight: bold;
    display: inline-block;
    transition: 0.3s ease;
}

.ctaButton:hover {
    background-color: white;
}

.featuresSection {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 30px;
    max-width: 1200px;
    margin: -50px auto 80px;
    position: relative;
    z-index: 10;
    padding: 0 20px;
}

.featureCard {
    background: white;
    padding: 40px;
    border-radius: 20px;
    text-align: center;
    box-shadow: 0 20px 40px rgba(100, 89, 69, 0.1);
    transition:
        transform 0.4s ease,
        box-shadow 0.4s ease;
    position: relative;
    z-index: 10;
}

.featureCard p {
    font-weight: 300;
    color: #8a7d6a;
    font-size: 0.95rem;
}

.featureCard:hover {
    transform: translateY(-8px);
    box-shadow: 0 30px 60px rgba(100, 89, 69, 0.15);
}

.icon {
    font-size: 2.5rem;
    display: block;
    margin-bottom: 15px;
}
```

```
.featureCard h3 {
  font-family: "Playfair Display", serif;
  color: #645945;
  margin-bottom: 10px;
}

.shopPreview {
  padding: 100px 20px;
  text-align: center;
  background-color: #f3efe4;
}

.secondaryLink {
  color: #645945;
  text-decoration: none;
  font-weight: bold;
  border-bottom: 2px solid #c5a059;
  padding-bottom: 5px;
}

.featuredProducts {
  max-width: 1200px;
  margin: 100px auto;
  padding: 0 20px;
}

.sectionHeader {
  text-align: center;
  margin-bottom: 60px;
}

.sectionHeader span {
  color: #c5a059;
  text-transform: uppercase;
  letter-spacing: 3px;
  font-size: 0.9rem;
  font-weight: bold;
}

.sectionHeader h2 {
  font-family: "Playfair Display", serif;
  color: #645945;
  font-size: 2.5rem;
  margin-top: 10px;
}

.productGrid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  gap: 40px;
}

.productCard {
```

```
    text-align: center;
    transition: 0.3s;
}

.imageContainer {
    background-color: #f9f6f0;
    border-radius: 15px;
    padding: 40px;
    margin-bottom: 20px;
    position: relative;
    overflow: hidden;
}

.imageContainer img {
    width: 100%;
    height: 300px;
    object-fit: contain; /* Garde les proportions du flacon/produit */
    transition: transform 0.5s ease;
}

.productCard:hover img {
    transform: scale(1.05);
}

.badge {
    position: absolute;
    top: 15px;
    right: 15px;
    background-color: #645945;
    color: white;
    padding: 5px 12px;
    font-size: 0.7rem;
    border-radius: 20px;
    text-transform: uppercase;
}

.productCard h4 {
    font-family: "Playfair Display", serif;
    font-size: 1.3rem;
    color: #333;
    margin-bottom: 8px;
}

.productCard p {
    color: #8a7d6a;
    font-weight: 300;
    margin-bottom: 20px;
}

.productLink {
    text-decoration: none;
    color: #c5a059;
    font-weight: bold;
    border-bottom: 1px solid transparent;
```

```
    transition: 0.3s;
  }

.productLink:hover {
  border-bottom: 1px solid #c5a059;
}

@media (max-width: 768px) {
  .hero {
    height: 70vh;
    border-radius: 0 0 30px 30px;
  }

  .heroContent h2 {
    font-size: 2.2rem;
    padding: 0 10px;
  }

  .heroContent p {
    font-size: 1rem;
    margin-bottom: 30px;
    padding: 0 15px;
  }

  .featuresSection {
    margin: -30px auto 40px;
    grid-template-columns: 1fr;
    gap: 20px;
  }

  .featureCard {
    padding: 30px 20px;
  }
}
```

FICHER : code/src/assets/css/icon_navbar.module.css

```
.icon_navbar {
  display: flex;
  gap: 20px;
  align-items: center;
  color: #645945;
}

.icon_navbar svg {
  font-size: 1.3rem;
  cursor: pointer;
  transition: color 0.3s;
}
```



```
.icon_navbar svg:hover {  
    color: #c5a059;  
}
```

FICHER : code/src/assets/css/logo.module.css

```
.logo {  
    display: flex;  
    justify-content: center;  
    align-items: center;  
    width: auto;  
    height: 50px;  
    transition: transform 0.3s ease;  
}  
  
.logo img {  
    width: auto;  
    height: 100%;  
    display: block;  
    object-fit: contain;  
}  
  
.logo h1 {  
    display: none;  
}
```

FICHER : code/src/assets/css/navbar.module.css

```
/* Polices */  
  
/* Variables CSS */  
  
/* Balises HTML */  
/* MOBILE FIRST */  
  
* {  
    box-sizing: border-box;  
}  
  
.navContainer {  
    display: flex;  
    align-items: center;  
}  
  
.navContainer button {
```

```
    display: flex;
    flex-direction: column;
    gap: 0.35rem;
    background: none;
    border: none;
    cursor: pointer;
    padding: 10px;
    z-index: 1100;
  }

  .line {
    width: 1.8rem;
    height: 2px;
    background-color: #645945;
    transition:
      transform 0.3s ease,
      opacity 0.3s ease;
  }

  .one {
    transform-origin: left top;
    transform: rotate(45deg) translate(0px, -3px);
  }

  .two {
    opacity: 0;
  }

  .three {
    transform-origin: left bottom;
    transform: rotate(-45deg) translate(0px, 3px);
  }

  .navbar {
    position: fixed;
    top: 125px;
    left: 0;
    width: 250px;
    height: calc(100vh-125px);
    background-color: #645945;
    transition: transform 0.4s ease;
    z-index: 1050;
    box-shadow: 5px 0 15px rgba(0,0,0,0.1);
  }

  .nav-visible {
    transform: translateX(-100%);
  }

  .menu li a {
    display: block;
    padding: 15px 30px;
    color: #f3efe4;
    text-decoration: none;
    font-family: 'Playfair Display', serif;
    font-size: 1.1rem;
```

```
    transition: all 0.3s;
}

.menu li a:hover {
    background-color: rgba(197, 160, 89, 0.2);
    color: #c5a059;
    padding-left: 40px;
}

.adminLink {
    margin-top: 20px;
    border-top: 1px solid rgba(243, 239, 228, 0.2);
    font-weight: bold;
    color: #c5a059 !important;
}
```

FICHER : code/src/assets/css/product_card.module.css

```
.productCard {
    display: flex;
    flex-direction: column;
    text-align: center;
    transition: transform 0.3s ease;
}

.imageContainer {
    background-color: #fcfaf7; /* Fond crème très doux */
    border-radius: 20px;
    margin-bottom: 20px;
    position: relative;
    overflow: hidden;
    aspect-ratio: 1 / 1; /* Force un carré parfait pour l'unité visuelle */
    /*
    display: flex;
    align-items: center;
    justify-content: center;
    */
}

.imageContainer img {
    width: 100%;
    height: 100%;
    object-fit: cover;
    transition: transform 0.6s cubic-bezier(0.33, 1, 0.68, 1);
}

/* Effet au survol de la carte */
.productCard:hover .imageContainer img {
    transform: scale(1.1);
}
```

```
.productInfo h4 {
  font-family: "Playfair Display", serif;
  font-size: 1.25rem;
  color: #2d2d2d;
  margin: 0 0 8px 0;
}

.productInfo p {
  color: #8a7d6a;
  font-weight: 300;
  font-size: 0.95rem;
  margin-bottom: 15px;
}

.productLink {
  text-decoration: none;
  color: #c5a059;
  font-weight: 700;
  font-size: 0.9rem;
  text-transform: uppercase;
  letter-spacing: 1px;
  display: inline-block;
  transition: color 0.3s ease;
}

.productLink:hover {
  color: #645945;
}
```

FICHER : code/src/assets/css/product_skeleton.module.css

```
/* src/components/products/product_skeleton.module.css */
.skeletonCard {
  background: #fdfcf9;
  border-radius: 8px;
  overflow: hidden;
  border: 1px solid #eee;
}

.skeletonImage {
  width: 100%;
  height: 200px;
  background: #e0e0e0;
}

.skeletonInfo {
  padding: 15px;
  display: flex;
  flex-direction: column;
```

```
    gap: 10px;
  }

.skeletonTitle,
.skeletonPrice,
.skeletonButton {
  background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0
75%);
  background-size: 200% 100%;
  animation: shimmer 1.5s infinite;
  border-radius: 4px;
}

.skeletonTitle {
  height: 20px;
  width: 80%;
}
.skeletonPrice {
  height: 15px;
  width: 40%;
}
.skeletonButton {
  height: 35px;
  width: 100%;
  margin-top: 10px;
}

@keyframes shimmer {
  0% {
    background-position: 200% 0;
  }
  100% {
    background-position: -200% 0;
  }
}
```

FICHER : code/src/assets/css/products.module.css

```
.productsSection {
  padding: 60px 20px;
  max-width: 1200px;
  margin: 0 auto;
}

.mainTitle {
  text-align: center;
  font-family: "Playfair Display", serif;
  color: #645945;
  font-size: 2.5rem;
}
```

```
    margin-bottom: 40px;
    position: relative;
}

/* Petit trait décoratif sous le titre */
.mainTitle::after {
    content: "";
    display: block;
    width: 60px;
    height: 3px;
    background: #c5a059;
    margin: 10px auto;
}

/* Grille Responsive */
.productsGrid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
    gap: 20px;
}

/* Carte Produit */
.productCard {
    background: white;
    border-radius: 15px;
    overflow: hidden;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.05);
    transition:
        transform 0.3s ease,
        box-shadow 0.3s ease;
    display: flex;
    flex-direction: column;
}

.productCard:hover {
    transform: translateY(-10px);
    box-shadow: 0 10px 25px rgba(100, 89, 69, 0.1);
}

/* Image */
.imageWrapper {
    height: auto;
    width: 100%;
    background-color: #f3efe4;
}

.imageWrapper img {
    width: 100%;
    height: 250px;
    object-fit: cover;
    transition: transform 0.5s ease;
}

.productCard:hover .imageWrapper img {
```

```
        transform: scale(1.1);
    }

    /* Infos */
    .productInfo {
        padding: 15px 20px;
        text-align: left;
        align-items: stretch;
        display: flex;
        flex-direction: column;
        gap: 8px;
    }

    .productInfo h3 {
        font-family: "Playfair Display", serif;
        color: #4a3f35;
        font-size: 1.1rem;
        margin: 0;
    }

    .productInfo a {
        text-decoration: none;
    }

    .price {
        color: #c5a059;
        font-weight: bold;
        font-size: 1rem;
    }

    /* Bouton */
    .viewButton {
        margin-top: auto;
        background-color: #645945;
        color: white;
        border: none;
        padding: 8px;
        border-radius: 5px;
        cursor: pointer;
        transition: background 0.3s;
        font-family: "Playfair Display", serif;
        font-size: 0.9rem;
    }

    .viewButton:hover {
        background-color: #c5a059;
    }

    /* centre la fiche */
    .productDetailContainer {
        max-width: 600px;
        margin: 0 auto;
    }
}
```

```
.productDetailContainer .productCard {
  flex-direction: row-reverse;
}

.descriptionText {
  color: #4a3f35;
  font-size: 1rem;
  line-height: 1.7;
  padding: 15px 0;
  text-align: left;
}

.backLink {
  display: inline-block;
  margin-top: 20px;
  color: #645945;
  text-decoration: none;
  font-size: 0.9rem;
  font-family: "Playfair Display", serif;
  transition: color 0.3s;
}

.backLink:hover {
  color: #c5a059;
}

.descriptionText h2 {
  font-family: "Playfair Display", serif;
  color: #645945;
  font-size: 1.8rem;
  margin-top: 30px;
  border-bottom: 1px solid #f3efe4;
  padding-bottom: 10px;
}

.descriptionText h3 {
  font-family: "Playfair Display", serif;
  color: #c5a059;
  font-size: 1.3rem;
  margin-top: 25px;
  margin-bottom: 10px;
}

.descriptionText p {
  margin-bottom: 15px;
}

.descriptionText ul {
  list-style: none;
  padding-left: 10px;
  margin-bottom: 20px;
}

.descriptionText li {
```



```
    position: relative;
    padding-left: 25px;
    margin-bottom: 10px;
  }

.descriptionText li::before {
  content: "◆";
  position: absolute;
  left: 0;
  color: #c5a059;
}

.descriptionText strong {
  color: #645945;
  font-weight: 600;
}

span {
  gap: 15px;
}
```

FICHER : code/src/assets/css/public_layout.module.css

```
.appContainer {
  display: flex;
  flex-direction: column;
  /* Force le site à faire au moins la hauteur de l'écran */
  min-height: 100vh;
}

.mainContent {
  /* Dit au contenu de prendre tout l'espace vide, poussant le footer en
bas */
  flex: 1;
}
```

FICHER : code/src/assets/css/reset.css

```
/* http://meyerweb.com/eric/tools/css/reset/
v2.0 | 20110126
License: none (public domain)
*/

html,
body,
div,
```

span,
applet,
object,
iframe,
h1,
h2,
h3,
h4,
h5,
h6,
p,
blockquote,
pre,
a,
abbr,
acronym,
address,
big,
cite,
code,
del,
dfn,
em,
img,
ins,
kbd,
q,
s,
samp,
small,
strike,
strong,
sub,
sup,
tt,
var,
b,
u,
i,
center,
dl,
dt,
dd,
ol,
ul,
li,
fieldset,
form,
label,
legend,
table,
caption,
tbody,
tfoot,

```
thead,
tr,
th,
td,
article,
aside,
canvas,
details,
embed,
figure,
figcaption,
footer,
header,
hgroup,
menu,
nav,
output,
ruby,
section,
summary,
time,
mark,
audio,
video {
    margin: 0;
    padding: 0;
    border: 0;
    font: inherit;
    font-size: 100%;
    vertical-align: baseline;
}
/* HTML5 display-role reset for older browsers */
article,
aside,
details,
figcaption,
figure,
footer,
header,
hgroup,
menu,
nav,
section {
    display: block;
}
body {
    line-height: 1;
}
ol,
ul {
    list-style: none;
}
blockquote,
q {
```

```
    quotes: none;
  }
  blockquote:before,
  blockquote:after,
  q:before,
  q:after {
    content: none;
  }
  table {
    border-collapse: collapse;
    border-spacing: 0;
  }
```

FICHER : code/src/assets/css/shop_preview.module.css

```
.shopPreview {
  padding: 100px 20px;
  text-align: center;
  background-color: #f3efe4;
}

.shopPreview h2 {
  font-family: "Playfair Display", serif;
  color: #645945;
  margin-bottom: 15px;
}

.shopPreview p {
  color: #8a7d6a;
  margin-bottom: 30px;
}

.secondaryLink {
  color: #645945;
  text-decoration: none;
  font-weight: bold;
  border-bottom: 2px solid #c5a059;
  padding-bottom: 5px;
  transition: color 0.3s;
}

.secondaryLink:hover {
  color: #c5a059;
}
```

FICHER : code/src/components/admin_formulaire.tsx

```
"use client";

import { useEffect, useId, useState } from "react";
import { useForm } from "react-hook-form";
import { useNavigate } from "react-router";
import type { ZodIssue } from "zod/v3";
import type { Product } from "../../models/product";
import styles from "../../assets/css/form.module.css";
import type { AdminProductsAddProps } from
"../../models/props/admin_products_add_props";
import ProductApiService from "../../services/product_api_service";

const AdminForm = ({
  categories,
  skins,
  bodyparts,
  packs,
  validator,
  dataToUpdate,
}: AdminProductsAddProps) => {
  const idId = useId();
  const nameId = useId();
  const imageId = useId();
  const priceId = useId();
  const descriptionId = useId();
  const categoryId = useId();

  // console.log(dataToUpdate);

  // useNaviagate hook: permet de créer une redirection
  const navigate = useNavigate();

  // Stocker les messages d'erreur de validation côté serveur
  const [serverErrors, setserverErrors] = useState<Partial<Product>>>();

  // Message lié à la soumission du formulaire en cas d'echec
  const [message, setMessage] = useState<string>("");

  const {
    register,
    handleSubmit,
    reset,
    formState: { errors },
  } = useForm<Partial<Product>>>();

  // Pré-remplir le formulaire avant l'affichage du composant
  // Use effect ne s'execute que dans un composantclient: "use client";

  useEffect(() => {
    // Si des données sont à mettre à jour (update)
    if (dataToUpdate) {
      // Normaliser les données de saisies: se baser sur les données
```

```

testees dans flashpost pour que les donnés soient identiques.
    // Modifier le models si besoin
    const normalizedData = {
        ...dataToUpdate,
        skin_ids: (dataToUpdate.skin_ids as string).split(","),
        pack_ids: (dataToUpdate.pack_ids as string).split(","),
        body_part_ids: (dataToUpdate.body_part_ids as
string).split(","),
    };
    reset(normalizedData);
}
}, [dataToUpdate, reset]);

const submitForm = async (data: Partial<Product>) => {
    // Normaliser les donnés de saisies: se baser sur les donnés
testees dans flashpost pour que les donnés soient identiques.
    const normalizedData = {
        ...data,
        skin_ids: (data.skin_ids as unknown as string[]).join(),
        pack_ids: (data.pack_ids as unknown as string[]).join(),
        body_part_ids: (data.body_part_ids as unknown as
string[]).join(),
        image: (data.image as string)[0],
    };

    // validation de la saisie avec le validateur côté serveur
    const validation = await validator(normalizedData);

    // Si la validation échoue
    if (validation instanceof Error) {
        // Stocker les messages d'erreurs dans un objet
        let errors = {};

        // récupérer les messages d'erreur
        (JSON.parse(validation.message) as ZodIssue[]).map((item) => {
            errors = { ...errors, [item.path.shift() as string]:
item.message };
            return errors;
        });

        // Définir l'état
        setserverErrors(errors);

        return;
    }

    // Si la validation réussie
    // // console.log(normalizedData);
    // Si le formulaire contient un champs de fichier: envoyer vers
l'API un objet de type formData

    const formData = new FormData();
    // Reprendre strictement le nom des champs de formulaire tester
sur flashpost

```

```

        formData.set("id", normalizedData.id as unknown as string);
        formData.set("name", normalizedData.name as unknown as string);
        formData.set("image", normalizedData.image as unknown as string);
        formData.set(
            "description",
            normalizedData.description as unknown as string,
        );
        formData.set("price", normalizedData.price as unknown as string);
        formData.set(
            "category_id",
            normalizedData.category_id as unknown as string,
        );
        formData.set("skin_ids", normalizedData.skin_ids as unknown as
string);
        formData.set(
            "body_part_ids",
            normalizedData.body_part_ids as unknown as string,
        );
        formData.set("pack_ids", normalizedData.pack_ids as unknown as
string);

        //console.log(formData);

        // Requête HTTP vers l'API
        const process = dataToUpdate
            ? await new ProductApiService().update(formData)
            : await new ProductApiService().insert(formData);

        // Si la requête HTTP à réussie
        /* est ce que le code de status est dans cette liste = different
de -1 (-1= absent de la liste)
        useNavigate = on redirige vers une autre page sans click */
        if ([200, 201].indexOf(process.status) !== -1) {
            navigate("/admin/products");
            // Si la requête HTTP échoue
        } else if ([400].indexOf(process.status) !== -1) {
            // Afficher le message
            setMessage(process.message as unknown as string);
        }
    };

    return (
        <section className={styles.formWrapper}>
            /* Afficher le message via une condition ternaire*/
            {message ? <p role="alert">{message}</p> : null}

            /* onSubmit: evenement javascript
handleSubmit: react hook form
handleSubmit: fonction que l'on crée*/
            <form
                className={styles.orientalForm}
                encType="multipart/form-data"
                onSubmit={handleSubmit(submitForm)}
            >

```

```

<div>
  <label htmlFor={nameId}>
    <input
      type="text"
      id={nameId}
      placeholder="Nom"
      {...register("name", {
        required: "Le nom est obligatoire",
        minLength: {
          value: 2,
          message: "Un nom doit comporter au
minimum 2 caractères",
        },
        maxLength: {
          value: 50,
          message: "Un nom doit comporté au
maximum 50 caracteres",
        },
      })}
    />
  </label>
  { /* AFFICHER LES MESSAGES D'ERREURS Utiliser le nom de
champs définit dans register */ }
  <small role="alert">
    { " " }
    { errors.name?.message ?? serverErrors?.name }
  </small>
</div>

<div>
  <label htmlFor={imageId}>
    <input
      type="file"
      id={imageId}
      placeholder="Image"
      {...register(
        "image",
        dataToUpdate
          ? {}
          : {
            required: "Le nom de l'image
est obligatoire",
            minLength: {
              value: 2,
              message:
                "Une image doit
comporter au minimum 2 caractères",
            },
            maxLength: {
              value: 50,
              message:
                "Un image peut
comporter au maximum 50 caracteres",
            },
          }
      )}
    </input>
  </label>
  { /* AFFICHER LES MESSAGES D'ERREURS Utiliser le nom de
champs définit dans register */ }
  <small role="alert">
    { " " }
    { errors.image?.message ?? serverErrors?.image }
  </small>
</div>

```



```

    },
    })
  />
</label>
<small role="alert">
  {" "}
  {errors.image?.message ?? serverErrors?.image}
</small>
</div>

<div>
  <label htmlFor={priceId}>
    {
      <input
        type="text"
        id={priceId}
        placeholder="Prix"
        {...register("price", {
          required: "Le prix est obligatoire",
          min: {
            value: 1,
            message: "Un prix doit comporter
au minimum 1 chiffre",
          },
          max: {
            value: 99.99,
            message:
              "Un prix peut comporter au
maximum 4 caracteres dont 2 decimales",
          },
        })}
      />
    }
  </label>
  <small role="alert">
    {" "}
    {errors.price?.message ?? serverErrors?.price}
  </small>
</div>

<div>
  <label htmlFor={descriptionId}>
    <textarea
      id={descriptionId}
      placeholder="Décrivez les rituels et les
vertus de ce soin..."
      rows={5}
      {...register("description", {
        required: "La description est
obligatoire",
        minLength: {
          value: 20,
          message: "Une description doit
comporter au minimum 20 ",

```

```

        },
        }}}
    </textarea>
</label>
<small role="alert">
    {" "}
    {errors.description?.message ??
serverErrors?.description}
</small>
</div>

    { /* CRUD ==> CREATE
- si le formulaire contient un champ de fichier : ajouter l'attribut
enctype="multipart/form-data"
- pour les champs en relation :
FK : créer, soit une liste déroulante <select>, soit des boutons radio
> sélection d'un seul choix
table de jointure : cases à cocher
> sélection de plusieurs choix
*/ }

    <div className={styles.formGroup}>
        <label htmlFor={categoryId}>Catégorie</label>
        <select
            className={styles.title}
            id={categoryId}
            {...register("category_id", {
                required: "Veuillez selectionner une
category",
            })}
            >
            <option value="">-- Choisir une catégorie --
</option>

            {categories.map((item) => {
                return (
                    <option key={item.id} value={item.id}>
                        {" "}
                        {item.name}{" "}
                    </option>
                );
            })}
        </select>
        <small role="alert">
            {" "}
            {errors.category_id?.message ??
serverErrors?.category_id}
        </small>
    </div>

    <div className={styles.formGroup}>
        <p className={styles.groupTitle}>Type de peau</p>
        <div className={styles.checkboxGrid}>
            {skins.map((item) => (
                <label key={item.id} className=

```

```

{styles.checkboxItem}>
    <input
      type="checkbox"
      value={item.id}
      id={`skin-${item.id}`}
      {...register("skin_ids", {
        required:
          "Veuillez selectionner un ou
plusieurs type de peau",
      })}
    />
    <small role="alert">
      {" "}
      {errors.skin_ids?.message ??
serverErrors?.skin_ids}
    </small>
    <span>{item.type}</span>
  </label>
  )})
</div>
</div>

<div className={styles.formGroup}>
  <p className={styles.groupTitle}>Zone du corps</p>
  <div className={styles.checkboxGrid}>
    {bodyparts.map((item) => (
      <label key={item.id} className=
{styles.checkboxItem}>
        <input
          type="checkbox"
          value={item.id}
          id={`bodyparts-${item.id}`}
          {...register("body_part_ids", {
            required:
              "Veuillez selectionner une ou
plusieurs zone du corps",
          })}
        />
        <small role="alert">
          {" "}
          {errors.body_part_ids?.message ??
serverErrors?.body_part_ids}
        </small>
        <span>{item.name}</span>
      </label>
    ))}
  </div>
</div>

<div className={styles.formGroup}>
  <p className={styles.groupTitle}>Packs</p>
  <div className={styles.checkboxGrid}>
    {packs.map((item) => (
      <label key={item.id} className=

```

```

{styles.checkboxItem}>
    <input
      type="checkbox"
      value={item.id}
      id={`packs-${item.id}`}
      {...register("pack_ids", {
        required: "Veuillez selectionner
un ou plusieurs pack",
      })}
    />
    <small role="alert">
      {" "}
      {errors.pack_ids?.message ??
serverErrors?.pack_ids}
    </small>
    <span>{item.name}</span>
  </label>
  )})
</div>
</div>

<div>
  <input type="hidden" id={idId} {...register("id")} />
  <button type="submit">
    {dataToUpdate ? "Modifier produit " : "Cr  er
Produit"}
  </button>
</div>
</form>
</section>
);
};

export default AdminForm;

```

FICHER : code/src/components/admin_home.tsx

```

import { use } from "react";
import { Link, NavLink } from "react-router";
import styles from
"../../assets/css/admin_css/admin_product_index.module.css";
import ProductApiService from "../../src/services/product_api_service";

const AdminProductHome = () => {
  const results = use(new ProductApiService().selectAll()).data;
  // console.log(results);

  return (
    <div className={styles.crudContainer}>

```

```

        <header className={styles.crudHeader}>
            <div>
                <h1>Gestion des Produits</h1>
                <p>Consultez et gérez votre catalogue de soins
orientaux.</p>
            </div>

            <NavLink to="/admin/products/add" className=
{styles.addButton}>
                + Ajouter un produit
            </NavLink>
        </header>

        {results?.map((item) => {
            return (
                <div key={item.id}>
                    <div>
                        <p>{item.id} </p>
                        <p>{item.name} </p>
                    </div>

                    <div className={styles.actions}>
                        <Link to=
{`/admin/products/add/${item.id}`}>Modifier</Link>
                        <Link to=
{`/admin/products/add/${item.id}`}>Supprimer</Link>
                    </div>
                </div>
            );
        })}

        /* <section className={styles.tableWrapper}>
            <table className={styles.crudTable}>
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Nom</th>
                        <th>Prix</th>
                        <th>Stock</th>
                        <th>Actions</th>
                    </tr>
                </thead>

                <tbody>
                    {products.map((p) => (
                        <tr key={p.id}>
                            <td>#{p.id}</td>
                            <td className={styles.productName}>
{p.name}</td>

                            <td>{p.price} €</td>
                            <td>
                                <span className={p.stock < 15 ?
styles.lowStock : ""}>
                                    {p.stock}

```

```

                </span>
            </td>
        </td>
        <div className={styles.actions}>
            <button className=
{styles.editBtn}>Modifier</button>
            <button className=
{styles.deleteBtn}>Supprimer</button>
        </div>
    </td>
</tr>
    </tbody>
</table>
</section> */}
</div>
);
};

export default AdminProductHome;

```

FICHER : code/src/components/favicon.tsx

FICHER : code/src/components/footer.tsx

```

import { FaFacebookSquare } from "react-icons/fa";
import { NavLink } from "react-router";
import styles from "../../assets/css/footer.module.css";
import Logo from "../logo";

const Footer = () => {
    return (
        <footer className={styles.footer}>
            <div className={styles.logWrapper}>
                <Logo />
            </div>
            <div className={styles.nav}>
                <ul className={styles.menu}>
                    <li>
                        <NavLink to={"/"}> Accueil </NavLink>
                    </li>
                    <li>
                        <NavLink to={"/produits"}> Produits </NavLink>
                    </li>
                </ul>
            </div>
        </footer>
    );
};

```

```

        </li>
        <li>
            <NavLink to={"/register"}> Inscription </NavLink>
        </li>
        <li>
            <NavLink to={"/login"}> Connexion </NavLink>
        </li>
        <li>
            <NavLink to={"/contact"}> Contact </NavLink>
        </li>
        <li>
            <NavLink to={"/a_propos"}> A propos </NavLink>
        </li>
        <li>
            <NavLink to={"/mentions_legales"}> Mentions
légales </NavLink>
        </li>
    </ul>
</div>
<div className={styles.icone}>
    <FaFacebookSquare />
</div>
</footer>
);
};

export default Footer;

```

FICHER : code/src/components/form.tsx

```

"use client";
import type { FormProps } from "../../models/props/form_props";
import styles from "../../assets/css/form.module.css";

const PublicForm = ({ title, buttonText, type }: FormProps) => {

    return (
        <section className={styles.formWrapper}>
            <h2>{title}</h2>
            <form className={styles.orientalForm} onSubmit={(e) =>
e.preventDefault()}>

                </* Rendu conditionnel des champs */>
                {type === "contact" && (
                    <>
                        <input type="email" placeholder="Email" required />
                        <input type="text" placeholder="Sujet" required />
                        <textarea placeholder="Message"></textarea>
                    </>
                )}

            </form>
        </section>
    );
}

```

```

    })

    {type === "login" && (
      <>
        <input type="email" placeholder="Email" required />
        <input type="password" placeholder="Mot de passe" required />
      </>
    )}

    {type === "register" && (
      <>
        <input type="email" placeholder="Email" required />
        <input type="password" placeholder="Mot de passe" required />
        <input type="password" placeholder="Confirmer mot de passe"
required />
      </>
    )}

    <button type="submit">{buttonText}</button>
  </form>

  <div className={styles.formFooter}>
    {type === "login" && <p>Mot de passe oublié ?</p>}
    {type === "register" && <p>En vous inscrivant, vous rejoignez
notre rituel de fidélité.</p>}
  </div>

</section>
);
}

export default PublicForm;

```

FICHER : code/src/components/header.tsx

```

import styles from "../../assets/css/header.module.css";
import IconNavbar from "../icon_navbar";
import Logo from "../logo";
import NavBar from "../navbar";

const Header = () => {
  return (
    <header className={styles.header}>
      <NavBar />
      <Logo />
      <IconNavbar />
    </header>
  );
};

```



```
export default Header;
```

FICHER : code/src/components/home/featuredProducts.tsx

```
import { use } from "react";
import styles from "../../assets/css/featured_products.module.css";
import ProductApiService from "../../services/product_api_service";
import ProductCard from "../products/product_card";

const FeaturedProducts = () => {
  const results = use(new ProductApiService().selectAll()).data;
  const title = "Sélectionnés pour vous";
  const subtitle = "Nos Incontournables";

  return (
    <section className={styles.featuredProducts}>
      <div className={styles.sectionHeader}>
        <span>{subtitle}</span>
        <h2>{title}</h2>
      </div>

      <div className={styles.productGrid}>
        {results?.map((item) => (
          <ProductCard key={item.id} data={item} />
        ))}
      </div>
    </section>
  );
};

export default FeaturedProducts;
```

FICHER : code/src/components/home/features.tsx

```
import styles from "../../assets/css/homepage.module.css";

const Features = () => {
  return (
    <section className={styles.featuresSection}>
      <div className={styles.featureCard}>
        <span className={styles.icon}>☐</span>
        <h3>100% Naturel</h3>
        <p>Des ingrédients sourcés avec respect, sans additifs
chimiques.</p>
      </div>
    </section>
  );
};
```

```

        <div className={styles.featureCard}>
          <span className={styles.icon}>❖</span>
          <h3>Savoir-faire</h3>
          <p>
            Des recettes traditionnelles transmises de génération
en génération.
          </p>
        </div>

        <div className={styles.featureCard}>
          <span className={styles.icon}>❖</span>
          <h3>Éclat Durable</h3>
          <p>Une efficacité prouvée par les rituels de beauté
orientaux.</p>
        </div>
      </section>
    );
  };

  export default Features;

```

FICHER : code/src/components/home/hero.tsx

```

import { Link } from "react-router";
import styles from "../../assets/css/homepage.module.css";

const Hero = () => {
  return (
    <section className={styles.hero}>
      <div className={styles.heroContent}>
        <h2>L'Essence du Rituel Ancestral</h2>
        <p>
          Découvrez des soins purs et naturels pour sublimer
votre peau au
          quotidien.
        </p>
        <Link to="/produits" className={styles.ctaButton}>
          Explorer la collection
        </Link>
      </div>
    </section>
  );
};

export default Hero;

```

FICHER : code/src/components/home/product_skeleton.tsx

```
import styles from "../../../assets/css/product_skeleton.module.css";

const ProductSkeleton = () => {
  return (
    <div className={styles.skeletonCard}>
      <div className={styles.skeletonImage}></div>
      <div className={styles.skeletonInfo}>
        <div className={styles.skeletonTitle}></div>
        <div className={styles.skeletonPrice}></div>
        <div className={styles.skeletonButton}></div>
      </div>
    </div>
  );
};

export const FeaturedSkeleton = () => {
  return (
    <section style={{ padding: "40px 20px" }}>
      </* On imite la grille de produits (par exemple 4 cartes) */>
      <div
        style={{
          display: "grid",
          gridTemplateColumns: "repeat(auto-fill, minmax(250px, 1fr))",
          gap: "20px",
          maxWidth: "1200px",
          margin: "0 auto",
        }}
        >
        {[1, 2, 3, 4].map((i) => (
          <ProductSkeleton key={i} />
        ))}
      </div>
    </section>
  );
};
```

FICHER : code/src/components/home/shopPreview.tsx

```
import { Link } from "react-router";
import styles from "../../../assets/css/shop_preview.module.css";

const ShopPreview = () => {
  return (
    <section className={styles.shopPreview}>
      <div className={styles.previewText}>
```

```
        <h2>Le Hammam à la Maison</h2>
        <p>
            Savon noir, huile d'argan, gant kessa... Retrouvez
tous vos
            indispensables.
        </p>
        <Link to="/produits" className={styles.secondaryLink}>
            Voir tous les produits →
        </Link>
    </div>
</section>
);
};

export default ShopPreview;
```

FICHER : code/src/components/icon_navbar.tsx

```
import { FaUser } from "react-icons/fa";
import { FaBasketShopping, FaMagnifyingGlass } from "react-icons/fa6";
import styles from "../assets/css/icon_navbar.module.css";

const IconNavbar = () => {
    return (
        <div className={styles.icon_navbar}>
            <FaMagnifyingGlass />
            <FaUser />
            <FaBasketShopping />
        </div>
    );
};

export default IconNavbar;
```

FICHER : code/src/components/lien_d_evitement.tsx

FICHER : code/src/components/logo.tsx

```
import styles from "../assets/css/logo.module.css";
```

```

const Logo = () => {
  return (
    <div className={styles.logo}>
      <h1>Secret de Hammam</h1>
      
    </div>
  );
};

export default Logo;

```

FICHER : code/src/components/navbar.tsx

```

"use client";

import { useState } from "react";
import { NavLink } from "react-router";
import styles from "../../assets/css/navbar.module.css";

const NavBar = () => {
  // const navVisible: boolean = false;
  const [navVisible, setNavVisible] = useState<boolean>(false);

  const click = () => setNavVisible(!navVisible);

  const closeMenu = () => setNavVisible(false);

  // console.log(navVisible)

  return (
    <div className={styles.navContainer}>
      <button type="button" onClick={click}>

        <div className={` ${styles.line} ${styles.line1}
          ${navVisible ? styles.one : ""}`}></div>

        <div className={` ${styles.line} ${styles.line2}
          ${navVisible ? styles.two : ""}`}></div>

        <div className={` ${styles.line} ${styles.line3}
          ${navVisible ? styles.three : ""}`}></div>

      </button>
    </div>
  );
};

```

```

        <nav className={` ${styles.navbar} ${navVisible ? "" :
styles["nav-visible"]} `}>

            <ul className={styles.menu}>

                <li> <NavLink to={"/"} onClick={closeMenu}> Accueil
</NavLink> </li>
                <li><NavLink to={"/produits"} onClick={closeMenu}>
Produits </NavLink></li>
                <li> <NavLink to={"/register"} onClick={closeMenu}>
Inscription </NavLink> </li>
                <li> <NavLink to={"/login"} onClick={closeMenu}>
Connexion </NavLink> </li>
                <li> <NavLink to={"/contact"} onClick={closeMenu}>
Contact </NavLink> </li>
                <li> <NavLink to={"/a_propos"} onClick={closeMenu}> A
propos </NavLink> </li>
                <li> <NavLink to={"/mentions_legales"} onClick=
{closeMenu}> Mentions légales </NavLink> </li>
                <li><NavLink to="/admin" onClick={closeMenu}
className={styles.adminLink}>Admin</NavLink></li>

            </ul>

        </nav>

    </div>

);

};

export default NavBar;

```

FICHER : code/src/components/products/product_card.tsx

```

import { Link } from "react-router";
import styles from "../../assets/css/product_card.module.css";
import type { ProductsListItemProps } from
"../../models/props/products_list_item_props";

const ProductCard = ({ data }: ProductsListItemProps) => {
    const formattedPrice = data.price ? Number(data.price).toFixed(2) :
"0.00";
    return (
        <div className={styles.productCard}>
            <div className={styles.imageContainer}>
                <img src={`/img/products/${data.image}`} alt={data.name}

```

```

    />
    </div>
    <div className={styles.productInfo}>
      <h4>{data.name}</h4>
      <p>{formattedPrice}€</p>
      <Link to={` /produits/${data.id}`} className=
{styles.productLink}>
        Découvrir
      </Link>
    </div>
  </div>
);
};

export default ProductCard;

```

FICHER : code/src/components/products/products_contents.tsx

```

import { Link } from "react-router";
import styles from "../../assets/css/products.module.css";
import type { ProductsContentsProps } from
"../../models/props/products_contents_props";

// Récupération de la props data envoyé par le parent (page) à l'enfant
(composant)
const ProductsContents = ({ data }: ProductsContentsProps) => {
  return (
    <section className={styles.productsSection}>
      <div className={styles.productDetailContainer}>
        <article className={styles.productCard}>
          <div className={styles.imageWrapper}>
            <img src={` /img/products/${data.image}`} alt=
{data.name} />
          </div>

          <div className={styles.productInfo}>
            <h3>{data.name}</h3>
            <p className={styles.price}>{data.price} €</p>
            <p className={styles.descriptionText}>
{data.description}</p>
            <button type="button" className=
{styles.viewButton}>
              Ajouter au panier
            </button>
            <Link to="/produits">
              <span>Retour aux produits</span>
            </Link>
          </div>
        </article>
      </div>
    </section>
  );
};

```

```
        <div className={styles.Info}></div>
      </article>
    </div>
  </section>
);
};

export default ProductsContents;
```

FICHER : code/src/components/products/products_list.tsx

```
import { use } from "react";
import type { Product } from "../../models/product";
import styles from "../../assets/css/products.module.css";
import type { ApiResponse } from "../../models/api_response";
import ProductApiService from "../../services/product_api_service";
import ProductsListItem from "../../products_list_item";

const ProductsList = () => {
  // uniquement des composants
  // SEO
  // use : permet de récupérer les données d'une promesse dans un
  // composant server de react

  const results = use<ApiResponse<Product[]>>(
    new ProductApiService().selectAll(),
  );

  console.log(results);
  return (
    <section className={styles.productsSection}>
      <h2 className={styles.mainTitle}> Nos Soins Orientaux </h2>

      <div className={styles.productsGrid}>
        {results.data?.map((item) => (
          <ProductsListItem key={item.id} data={item} />
        ))}
      </div>
    </section>
  );
};

export default ProductsList;
```

FICHER : code/src/components/products/products_list_item.tsx


```

import { Link } from "react-router";
import styles from "../../assets/css/products.module.css";
import type { ProductsListItemProps } from
"../../models/props/products_list_item_props";

const ProductsListItem = ({ data }: ProductsListItemProps) => {
  return (
    <article className={styles.productCard}>
      <div className={styles.imageWrapper}>
        {/* / cible directement le dossier public */}
        <img src={`/img/products/${data.image}`} alt={data.name}
/>
      </div>

      <div className={styles.productInfo}>
        <h3>{data.name}</h3>

        <p className={styles.price}>{data.price}</p>

        <Link to={`/produits/${data.id}`}>
          <button type="button" className={styles.viewButton}>
            Découvrir
          </button>
        </Link>
      </div>
    </article>
  );
};

export default ProductsListItem;

```

FICHER : code/src/components/seo.tsx

```

import type { SeoProps } from "../../models/props/seo_props";

// récupéré les props dans les parametres () de la fonction du composant
const Seo = ({ title, description, url }: SeoProps) => {
  return (
    <>
      {/* A peu pres 50 caratcere au maximum */}
      <title>{`Sercret de Hammam - ${title}`}</title>

      {/* A peu pres 150 caratcere au maximum */}
      <meta name="description" content={`Secret de Hammam -
${description}`} />

      {/* Open Graph Your content's social media identity*/}
      <meta property="og:title" content={`Sercret de Hammam -

```

```

    ${title}` } />
        <meta property="og:type" content="website" />
        <meta property="og:url" content=
    {`https://secretsdehammam.com/${url}` } />
        { /* Image format 1200 x 630 pixels (taille optimale pour
Facebook) */ }
        <meta property="og:image"
content="https://yoursite.com/image.jpg" />
        <meta
            property="og:description"
            content={`Secret de Hammam - ${description}`}
        />

        { /* Twitter */ }

        <meta name="twitter:card" content="summary" />
        <meta name="twitter:site" content="@yourusername" />
        <meta name="twitter:title" content={`Sercet de Hammam -
    ${title}`} />
        <meta
            name="twitter:description"
            content={`Secret de Hammam - ${description}`}
        />
        { /* Utilisez des images carrées */ }
        <meta name="twitter:image"
content="https://yoursite.com/image.jpg" />
        </>
    );
};

export default Seo;

```

FICHER : code/src/components/sidebar.tsx

```

"use client";
import { NavLink } from "react-router";
import styles from "/app/src/assets/css/admin_css/sidebar.module.css";

const Sidebar = () => {
    return (
        <aside className={styles.sidebar}>
            <div className={styles.logoSection}>
                <h2>Secret de Hammam</h2>
                <span className={styles.badge}>Admin Panel</span>
            </div>

            <nav className={styles.navLinks}>
                <NavLink
                    to="/admin"

```

```

        end
        className=(({ isActive }) => isActive ? styles.active :
styles.link}
      >
        Tableau de bord
      </NavLink>

      <NavLink
        to="/admin/products"
        className=(({ isActive }) => isActive ? styles.active :
styles.link}
      >
        Gestion Produits
      </NavLink>

      <NavLink
        to="/admin/orders"
        className=(({ isActive }) => isActive ? styles.active :
styles.link}
      >
        Commandes
      </NavLink>
    </nav>

    <div className={styles.sidebarFooter}>
      <NavLink to="/" className={styles.exitLink}>
        Quitter l'admin
      </NavLink>
    </div>
  </aside>
);
};

export default Sidebar;

```

FICHER : code/src/framework/entry.browser.tsx

```

import {
  createFromReadableStream,
  createTemporaryReferenceSet,
  encodeReply,
  setServerCallback,
} from "@vitejs/plugin-rsc/browser";
import { StrictMode, startTransition } from "react";
import { hydrateRoot } from "react-dom/client";
import {
  unstable_createCallServer as createCallServer,
  unstable_getRSCStream as getRSCStream,
  unstable_RSCHydratedRouter as RSCHydratedRouter,
  type unstable_RSCPayload as RSCServerPayload,

```

```

} from "react-router/dom";

// Create and set the callServer function to support post-hydration server
actions.
setServerCallback(
  createCallServer({
    createFromReadableStream,
    createTemporaryReferenceSet,
    encodeReply,
  }),
);

// Get and decode the initial server payload.
createFromReadableStream<RSCServerPayload>(getRSCStream()).then((payload)
=> {
  startTransition(async () => {
    const formState =
      payload.type === "render" ? await payload.formState :
undefined;

    hydrateRoot(
      document,
      <StrictMode>
        <RSCHydratedRouter
          createFromReadableStream={createFromReadableStream}
          payload={payload}
        />
      </StrictMode>,
      {
        formState,
      },
    );
  });
});

```

FICHER : code/src/framework/entry.rsc.tsx

```

import {
  createTemporaryReferenceSet,
  decodeAction,
  decodeFormState,
  decodeReply,
  loadServerAction,
  renderToReadableStream,
} from "@vitejs/plugin-rsc/rsc";
import { unstable_matchRSCServerRequest as matchRSCServerRequest } from
"react-router";
import RouterService from "../services/router_service";

```

```
function fetchServer(request: Request) {
  return matchRSCServerRequest({
    // Provide the React Server touchpoints.
    createTemporaryReferenceSet,
    decodeAction,
    decodeFormState,
    decodeReply,
    loadServerAction,
    // The incoming request.
    request,
    // The app routes.
    routes: new RouterService().getRouter(),
    // Encode the match with the React Server implementation.
    generateResponse(match) {
      return new Response(renderToReadableStream(match.payload), {
        status: match.statusCode,
        headers: match.headers,
      });
    },
  });
}

export default async function handler(request: Request) {
  // Import the generateHTML function from the client environment
  const ssr = await import.meta.viteRsc.loadModule<
    typeof import("./entry.ssr")
  >("ssr", "index");

  return ssr.generateHTML(request, fetchServer);
}
```

FICHER : code/src/framework/entry.ssr.tsx

```
import { createFromReadableStream } from "@vitejs/plugin-rsc/ssr";
import { renderToReadableStream as renderHTMLToReadableStream } from
"react-dom/server.edge";
import {
  unstable_RSCStaticRouter as RSCStaticRouter,
  unstable_routeRSCServerRequest as routeRSCServerRequest,
} from "react-router";

export async function generateHTML(
  request: Request,
  fetchServer: (request: Request) => Promise<Response>,
): Promise<Response> {
  return await routeRSCServerRequest({
    // The incoming request.
    request,
    // How to call the React Server.
  });
}
```

```

    fetchServer,
    // Provide the React Server touchpoints.
    createFromReadableStream,
    // Render the router to HTML.
    async renderHTML(getPayload) {
      const payload = getPayload();

      const bootstrapScriptContent =
        await
import.meta.viteRsc.loadBootstrapScriptContent("index");

      return await renderHTMLToReadableStream(
        <RSCStaticRouter getPayload={getPayload} />,
        {
          bootstrapScriptContent,
          formState: payload.formState,
        },
      );
    },
  });
}

```

FICHER : code/src/framework/error-boundary.tsx

```

'use client'

import React from 'react'

// Minimal ErrorBoundary example to handle errors globally on browser
export function GlobalErrorBoundary(props: { children?: React.ReactNode })
{
  return (
    <ErrorBoundary errorComponent={DefaultGlobalErrorPage}>
      {props.children}
    </ErrorBoundary>
  )
}

//
https://github.com/vercel/next.js/blob/33f8428f7066bf8b2ec61f025427ceb2a54c4bdf/packages/next/src/client/components/error-boundary.tsx
// https://react.dev/reference/react/Component#catching-rendering-errors-with-an-error-boundary
class ErrorBoundary extends React.Component<{
  children?: React.ReactNode
  errorComponent: React.FC<{
    error: Error
    reset: () => void
  }>
}>

```

```

}> {
  state: { error?: Error } = {}

  static getDerivedStateFromError(error: Error) {
    return { error }
  }

  reset = () => {
    this.setState({ error: null })
  }

  render() {
    const error = this.state.error
    if (error) {
      return <this.props.errorComponent error={error} reset={this.reset}
    />
    }
    return this.props.children
  }
}

//
https://github.com/vercel/next.js/blob/677c9b372faef680d17e9ba224743f44e1107661/packages/next/src/build/webpack/loaders/next-app-loader.ts#L73
//
https://github.com/vercel/next.js/blob/677c9b372faef680d17e9ba224743f44e1107661/packages/next/src/client/components/error-boundary.tsx#L145
function DefaultGlobalErrorPage(props: { error: Error; reset: () => void
}) {
  return (
    <html>
      <head>
        <title>Unexpected Error</title>
      </head>
      <body
        style={{
          height: '100vh',
          display: 'flex',
          flexDirection: 'column',
          placeContent: 'center',
          placeItems: 'center',
          fontSize: '16px',
          fontWeight: 400,
          lineHeight: '24px',
        }}
      >
        <p>Caught an unexpected error</p>
        <pre>
          Error:{' '}
          {import.meta.env.DEV && 'message' in props.error
            ? props.error.message
            : '(Unknown)'}
        </pre>
        <button

```

```

        onClick={() => {
          React.startTransition(() => {
            props.reset()
          })
        }}
      >
        Reset
      </button>
    </body>
  </html>
)
}

```

FICHER : code/src/framework/request.tsx

```

// Framework conventions (arbitrary choices for this demo):
// - Use `_.rsc` URL suffix to differentiate RSC requests from SSR
  requests
// - Use `x-rsc-action` header to pass server action ID
const URL_POSTFIX = `_.rsc`
const HEADER_ACTION_ID = `x-rsc-action`

// Parsed request information used to route between RSC/SSR rendering and
  action handling.
// Created by parseRenderRequest() from incoming HTTP requests.
type RenderRequest = {
  isRsc: boolean // true if request should return RSC payload (via `_.rsc`
    suffix)
  isAction: boolean // true if this is a server action call (POST request)
  actionId?: string // server action ID from x-rsc-action header
  request: Request // normalized Request with `_.rsc` suffix removed from
    URL
  url: URL // normalized URL with `_.rsc` suffix removed
}

export function createRscRenderRequest(
  urlString: string,
  action?: { id: string; body: BodyInit },
): Request {
  const url = new URL(urlString)
  url.pathname += URL_POSTFIX
  const headers = new Headers()
  if (action) {
    headers.set(HEADER_ACTION_ID, action.id)
  }
  return new Request(url.toString(), {
    method: action ? 'POST' : 'GET',
    headers,
    body: action?.body,
  })
}

```



```

    })
  }

export function parseRenderRequest(request: Request): RenderRequest {
  const url = new URL(request.url)
  const isAction = request.method === 'POST'
  if (url.pathname.endsWith(URL_POSTFIX)) {
    url.pathname = url.pathname.slice(0, -URL_POSTFIX.length)
    const actionId = request.headers.get(HEADER_ACTION_ID) || undefined
    if (request.method === 'POST' && !actionId) {
      throw new Error('Missing action id header for RSC action request')
    }
    return {
      isRsc: true,
      isAction,
      actionId,
      request: new Request(url, request),
      url,
    }
  } else {
    return {
      isRsc: false,
      isAction,
      request,
      url,
    }
  }
}

```

FICHER : code/src/layouts/admin_layout.tsx

```

import "../assets/css/reset.css";
import "../assets/css/base.css";
import styles from "/app/src/assets/css/admin_layout.module.css";
import { Outlet } from "react-router";
import Sidebar from "../components/sidebar";

const AdminLayout = () => {
  return (
    <>
      { /* <Lien d'évitement /> */ }
      <div className={styles.adminLayout}>
        <Sidebar />
        <main className={styles.mainContent}>
          <Outlet />
        </main>
      </div>
    </>
  )
}

```

```
    );  
};  
  
export default AdminLayout;
```

FICHER : code/src/layouts/public_layout.tsx

```
import "../assets/css/reset.css";  
import "../assets/css/base.css";  
import { Outlet } from "react-router";  
import Footer from "../components/footer";  
import Header from "../components/header";  
import styles from "../assets/css/public_layout.module.css";  
  
const PublicLayout = () => {  
  return (  
    <div className={styles.appContainer}>  
      <Header />  
  
      <main className={styles.mainContent}>  
  
        { /* <Lien d'évitement /> */ }  
  
        <Outlet />  
  
      </main>  
  
      <Footer />  
  
    </div>  
  );  
};  
  
export default PublicLayout;
```

FICHER : code/src/layouts/root_layout.tsx

```
import "../assets/css/reset.css";  
import "../assets/css/base.css";  
  
import { Outlet } from "react-router";  
  
const RootLayout = () => {  
  return (  
    <html lang="fr">
```

```

        <head>
            <meta charset="UTF-8" />
            { /*<link rel="icon" type="image/svg+xml" href="/vite.svg"
/>*/}
            <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
        </head>
        <body>
            { /* Zone rempli par un autre contenu */}
            <Outlet />
        </body>
    </html>
    );
};

export default RootLayout;

```

FICHER : code/src/models/api_response.ts

```

// T represente un type générique : type variable
// on utilise le T chaque fois que l'on doit typer quelque chose mais
qu'on ne peut pas définir le type dans le sens ou cela peut être un
string, un number, un array, un objet.....

type ApiResponse<T> = {
    status: number;
    message: string;
    // ? : optionnel
    data?: T;
};

export type { ApiResponse };

```

FICHER : code/src/models/params/admin_products_params.ts

```

type AdminProductsParams = {
    // reprendre les variables d'URL créées dans le routeur
    params: {
        id: number
    }
}

export type { AdminProductsParams };

```

FICHER : code/src/models/params/produits_details_params.ts

```
type ProduitsDetailsParams = {  
  // reprendre les variables d'URL créés dans le routeur  
  params: {  
    id: number  
  }  
}  
  
export type { ProduitsDetailsParams };
```

FICHER : code/src/models/props/admin_products_add_props.ts

```
import type { ZodError } from "zod";  
import type { Body_part } from "../../../models/body_part";  
import type { Category } from "../../../models/category";  
import type { Pack } from "../../../models/pack";  
import type { Product } from "../../../models/product";  
import type { Skin } from "../../../models/skin";  
  
type AdminProductsAddProps = {  
  // Reprendre strictement le nom des props définis sur le composant  
  categories: Category[];  
  skins: Skin[];  
  bodyparts: Body_part[];  
  packs: Pack[];  
  validator: (data: Partial<Product>) => Promise<Partial<Product> |  
ZodError>;  
  dataToUpdate: Product | undefined;  
};  
  
export type { AdminProductsAddProps };
```

FICHER : code/src/models/props/featured_products_props.ts

```
import type { Product } from "../../../models/product";  
  
type FeaturedProductsProps = {  
  title: string;  
  subtitle: string;  
  products: Product[];  
};
```

```
export type { FeaturedProductsProps };
```

FICHIER : code/src/models/props/form_props.ts

```
type FormProps = {  
  // Reprendre strictement le nom des props définis sur le composant  
  title: string;  
  buttonText: string;  
  type: "contact" | "login" | "register" | "product";  
};  
  
export type { FormProps };
```

FICHIER : code/src/models/props/products_contents_props.ts

```
import type { Product } from "../../models/product";  
  
type ProductsContentsProps = {  
  // Reprendre strictement le nom des props définis sur le composant  
  data: Product  
};  
  
export type { ProductsContentsProps };
```

FICHIER : code/src/models/props/products_list_item_props.ts

```
import type { Product } from "../../models/product";  
  
type ProductsListItemProps = {  
  // Reprendre strictement le nom des props définis sur le composant  
  data: Product  
};  
  
export type { ProductsListItemProps };
```

FICHIER : code/src/models/props/seo_props.ts

```
type SeoProps = {
  title: string;
  description: string;
  url: string;
};

export type { SeoProps };
```

FICHER : code/src/pages/admin/index.tsx

```
import styles from "/app/src/assets/css/admin_layout.module.css";

const AdminHome = () => {
  return (
    <div className={styles.container}>
      <header className={styles.header}>
        <h1>Tableau de bord</h1>
        <p>Bienvenue dans la gestion de votre boutique de soins.
</p>
      </header>

      <div className={styles.statsGrid}>
        <div className={styles.card}>
          <h3>Produits actifs</h3>
          <p className={styles.number}>24</p>
        </div>
        <div className={styles.card}>
          <h3>Commandes en cours</h3>
          <p className={styles.number}>8</p>
        </div>
        <div className={styles.card}>
          <h3>Chiffre d'affaires</h3>
          <p className={styles.number}>1 240 €</p>
        </div>
      </div>

      <div className={styles.recentActivity}>
        <h2>Derniers produits ajoutés</h2>
        <ul>
          <li>Savon noir à l'Eucalyptus - <span className=
{styles.date}>Il y a 2h</span></li>
          <li>Huile d'Argan Bio - <span className=
{styles.date}>Hier</span></li>
        </ul>
      </div>
    </div>
  );
};
```

```
export default AdminHome;
```

FICHER : code/src/pages/admin/login.tsx

```
const Login = () => {
  return (
    <h2>admin login</h2>
  );
};

export default Login;
```

FICHER : code/src/pages/admin/product/add.tsx

```
import { NavLink } from "react-router";
import styles from "../../../assets/css/admin_css/add.module.css";
import React, { use } from "react";
import CategoryApiService from "../../../services/category_api_service";
import type { Category } from "../../../models/category";
import AdminForm from "../../../components/admin_formulaire";
import type { Body_part } from "../../../models/body_part";
import type { Skin } from "../../../models/skin";
import type { Pack } from "../../../models/pack";
import SkinApiService from "../../../services/skin_api_service";
import BodyPartApiService from "../../../services/body_part_api_service";
import PackApiService from "../../../services/pack_api_service";
import AdminFormulaireValidator from
"../../../validators/admin/admin_formulaire_validator";
import type { AdminProductsParams } from
"../../../models/params/admin_products_params";
import type { Product } from "../../../models/product";
import ProductApiService from "../../../services/product_api_service";

const AddProduct = ({ params }: AdminProductsParams): React.JSX.Element =>
{
  // console.log(params);

  // Récupérer la variable d'URL
  const { id } = params;
  // console.log(id);

  // Récupérer les données à mettre à jour
  let dataToUpdate: Product | undefined;
  // Si un identifiant est présent dans l'URL
  if (id) {
    // La methode Then equivaut a Await lorsque les conditions d'await ne
```

```

sont pas rempli( etre dans une fonction et asynchrone)
// new ProductApiService().selectOne(id).then(item => {
//   dataToUpdate = item.data as Product;
//   console.log(dataToUpdate);
// });
dataToUpdate = use(new ProductApiService().selectOne(id)).data as
Product;

}

// console.log(dataToUpdate);

// recuperer les produits
const categories = use(new CategoryApiService().selectAll()).data as
Category[];
const skins = use(new SkinApiService().selectAll()).data as Skin[];
const bodyparts = use(new BodyPartApiService().selectAll()).data as
Body_part[];
const packs = use(new PackApiService().selectAll()).data as Pack[];

return (
  <div className={styles.formContainer}>
    <header className={styles.header}>
      <div>
        <h1>Administration</h1>
        <p>Gestion du catalogue Secret de Hammam</p>
      </div>
      <NavLink to="/admin/products" className={styles.backBtn}>
        Annuler
      </NavLink>
    </header>

    <AdminForm categories={categories} skins={skins} bodyparts=
{bodyparts} packs={packs}
      validator={new AdminFormulaireValidator().validate} dataToUpdate=
{dataToUpdate} />

  </div>
);
};

export default AddProduct;

```

FICHER : code/src/pages/admin/product/delete.tsx

```
"use client";
```



```
import type React from "react";
import type { AdminProductsParams } from
"../../models/params/admin_products_params";
import { useEffect } from "react";
import ProductApiService from "../../services/product_api_service";
import { useNavigate } from "react-router";

const AdminProductDelete = ({ params }: AdminProductsParams):
React.JSX.Element => {

    // Récupérer la variable d'URL
    const { id } = params;

    // useNavigate permet de créer une redirection
    const navigate = useNavigate();

    // Executer la suppression à l'affichage de la page

    // La methode Then equivaut a Await lorsque les conditions d'await ne
sont pas rempli( etre dans une fonction et asynchrone)
    useEffect(() => {
        new ProductApiService().delete({ id: id }).then(() => {
            navigate("/admin/products");
            return;
        });
    }, [id, navigate]);

    return (

        <>
            <title> Delete</title>
        </>

    );
};

export default AdminProductDelete;
```

FICHER : code/src/pages/admin/product/index.tsx

```
import { Link, NavLink } from "react-router";
import styles from
"../../assets/css/admin_css/admin_product_index.module.css";
import { use } from "react";
import ProductApiService from "../../services/product_api_service";
```

```

const AdminProductHome = () => {
  const results = use(new ProductApiService().selectAll()).data;

  return (
    <div className={styles.crudContainer}>
      <header className={styles.crudHeader}>
        <div>
          <h1>Gestion des Produits</h1>
          <p>Consultez et gérez votre catalogue de soins
orientaux.</p>
        </div>

        <NavLink to="/admin/products/add" className=
{styles.addButton}>
          + Ajouter un produit
        </NavLink>
      </header>

      {/* {results?.map((item) => {
        return (
          <div key={item.id}>
            <div>
              <p>{item.id} </p>
              <p>{item.name} </p>
            </div>

            <div className={styles.actions}>
              <Link to=
{'/admin/products/add/${item.id}`}>Modifier</Link>
              <Link to=
{'/admin/products/add/${item.id}`}>Supprimer</Link>
            </div>

          </div>

        )
      })
    } */}

      <section className={styles.tableWrapper}>
        <table className={styles.crudTable}>
          <thead>
            <tr>
              <th>ID</th>
              <th>Nom</th>
              <th>Prix</th>
              <th>Actions</th>
            </tr>
          </thead>

          <tbody>
            {results.map((item) => {
              return (
                <tr key={item.id}>

```

```

        <td>#{item.id}</td>
        <td className={styles.productName}>
{item.name}</td>
        <td>{item.price} €</td>
        <td>
            <div className={styles.actions}>
                <Link to=
{`/admin/products/add/${item.id}`} className=
{styles.modif}>Modifier</Link>
                <Link to=
{`/admin/products/delete/${item.id}`} className=
{styles.suppr}>Supprimer</Link>
            </div>
        </td>
    </tr>
  )
  )}
</tbody>
</table>
</section>
</div>
);
};

export default AdminProductHome;

```

FICHER : code/src/pages/public/a_propos.tsx

```

import { Link } from "react-router";
import styles from "../../assets/css/products.module.css";

const APropos = () => {
  // uniquement des composants
  // SEO

  return (
    <section className={styles.productsSection}>
      <h2 className={styles.mainTitle}>Notre Histoire</h2>

      <div className={styles.productDetailContainer}>
        /* On réutilise la carte pour encadrer le texte
proprement */
        <article className={styles.productCard} style={{ padding:
"30px" }}>
          <div className={styles.descriptionText}>
            <h3>L'Essence de Secret de hammam</h3>
            <p>
              Né d'une passion pour les rituels ancestraux,
            {" "}

```

```

    <strong>Secret de hammam</strong> est une
invitation au voyage et
    à la sérénité. Nous avons puisé au cœur des
traditions marocaines
    pour vous offrir une sélection de soins
authentiques.
    </p>

    <h3>L'Héritage du Maroc</h3>
    <p>
        Derrière chaque produit se cache un savoir-
faire séculaire. De
        l'extraction de l'huile d'Argan dans les
coopératives du sud, à la
        récolte de la Figue de Barbarie, nous
sélectionnons nos
        ingrédients pour leur pureté absolue.
    </p>

    <ul>
        <li>
            <strong>Authenticité :</strong> Produits
sourcés en direct.
        </li>
        <li>
            <strong>Pureté :</strong> Sans additifs
superflus.
        </li>
        <li>
            <strong>Bien-être :</strong> L'expérience
du hammam à domicile.
        </li>
    </ul>

    <Link to="/produits" className={styles.backLink}>
        ← Découvrir la collection
    </Link>
    </div>
  </article>
</div>
</section>
);
};

export default APropos;
```

FICHER : code/src/pages/public/contact.tsx

```
import PublicForm from "../../components/form";
import Seo from "../../components/seo";

const Contact = () => {
  return (
    <>
      <Seo
        title="Contact"
        description="Nous contacter"
        url="/contact"
      />

      <PublicForm
        title="Envoyez-nous un message"
        buttonText="Envoyer"
        type="contact"
      />
    </>
  );
};

export default Contact;
```

FICHER : code/src/pages/public/homepage.tsx

```
import { Suspense } from "react";
import FeaturedProducts from "../../components/home/featuredProducts";
import Features from "../../components/home/features";
import Hero from "../../components/home/hero";
import { FeaturedSkeleton } from "../../components/home/product_skeleton";
import ShopPreview from "../../components/home/shopPreview";
import Seo from "../../components/seo";

const HomePage = () => {
  // uniquement des composants
  // SEO

  return (
    <main>
      <Seo title="Accueil" description="Accueil - Desk" url="/" />
      <Hero />
      <Features />
      <Suspense fallback={<FeaturedSkeleton />}>
        <FeaturedProducts />
      </Suspense>
      <ShopPreview />
    </main>
  );
};
```

```
    );  
  };  
  
  export default HomePage;
```

FICHER : code/src/pages/public/login.tsx

```
import PublicForm from "../../components/form";  
import Seo from "../../components/seo";  
  
const Login = () => {  
  return (  
    <>  
      <Seo  
        title="Connexion"  
        description="Connectez-vous à votre espace client"  
        url="/login"  
      />  
  
      <PublicForm  
        title="Mon Compte"  
        buttonText="Se connecter"  
        type="login"  
      />  
    </>  
  );  
};  
  
export default Login;
```

FICHER : code/src/pages/public/mentions_legales.tsx

```
import { Link } from "react-router";  
import styles from "../../assets/css/products.module.css";  
  
const MentionsLegales = () => {  
  // uniquement des composants  
  // SEO  
  
  return (  
    <section className={styles.productsSection}>  
      <h2 className={styles.mainTitle}>Informations Légales</h2>  
  
      <div className={styles.productDetailContainer}>  
        /* L'utilisation de productCard permet de garder le cadre
```

```
blanc et l'ombre portée */}
    <article className={styles.productCard} style={{ padding:
"30px" }}>
        <div className={styles.descriptionText}>
            <h2>Mentions Légales</h2>
            <hr
                style={{
                    border: "0",
                    borderTop: "1px solid #f3efe4",
                    margin: "20px 0",
                }}
            />

            <h3>1. Édition du Site</h3>
            <p>
                Le site <strong>Secret de hammam</strong> est
                édité par [Votre
                Nom/Entreprise], [Forme juridique] au capital
                de [Montant]€, dont
                le siège social est situé au [Adresse
                Complète]. Immatriculée au
                RCS sous le numéro SIRET [14 chiffres].
            </p>

            <h3>2. Hébergement</h3>
            <p>
                Ce site est hébergé par la société [Nom de
                l'hébergeur], située au
                [Adresse de l'hébergeur].
            </p>

            <h3>3. Protection des données (RGPD)</h3>
            <p>
                Les données collectées via le site (email,
                type de peau) sont
                exclusivement destinées à la gestion de vos
                commandes et à la
                personnalisation de votre expérience client.
                Vous disposez d'un
                droit d'accès et de suppression de vos données
                personnelles.
            </p>

            <br />

            <h2>Conditions Générales de Vente</h2>
            <hr
                style={{
                    border: "0",
                    borderTop: "1px solid #f3efe4",
                    margin: "20px 0",
                }}
            />
```

```

        <h3>Livraison</h3>
        <p>
            Nous expédions nos soins sous [48h] ouvrés.
            Les frais de port sont
            calculés lors de la validation du panier et
            sont offerts dès [50€]
            d'achat.
        </p>

        <h3>Retours et Hygiène</h3>
        <p>
            Conformément à la loi, vous disposez de 14
            jours pour retourner
            vos produits.
            <strong> Important :</strong> Pour des raisons
            d'hygiène, seuls
            les produits
            <strong> non ouverts et scellés</strong>
            seront acceptés en
            retour.
        </p>

        <Link to="/produits" className={styles.backLink}>
            ← Retour à la boutique
        </Link>
    </div>
</article>
</div>
</section>
);
};

export default MentionsLegales;

```

FICHER : code/src/pages/public/pack.tsx

FICHER : code/src/pages/public/produits.tsx

```

import ProductsList from "../../components/products/products_list";
import Seo from "../../components/seo";

const Produits = () => {

    return (

```



```

        <>
        <Seo
            title="Produits"
            description="Liste produits proposés"
            url="/produits"
        />

        <ProductsList/>

    </>
);
};

export default Produits;

```

FICHER : code/src/pages/public/produits_details.tsx

```

import { use } from "react";
import type { ProduitsDetailsParams } from
"../../models/params/produits_details_params";
import ProductApiService from "../../services/product_api_service";
import Seo from "../../components/seo";
import ProductsContents from
"../../components/products/products_contents";
import type { Product } from "../../models/product";

// param permet de recuperer une variable d'URL
const ProduitsDetails = ({ params }: ProduitsDetailsParams) => {
    // récupérer l'identifiant dans les paramètres
    // déconstruction d'un objet: permet de créer des variables pour chaque
propriété d'un objet
    const { id } = params

    // Récupération des données
    const result = use(new ProductApiService().selectOne(id));

    console.log(result);

    return (
        <>
        <Seo
            // ? indique une data optionnelle. Si la data existe il
faut typer avec "as", si elle n'existe pas aucune erreur ne s'affichera.
            title={result.data?.name as string}
            description={result.data?.description as string}
            url={` /produits/${id}`}
        />
    )

```

```

        <ProductsContents data={result.data as Product} />

    </>
  );
};

export default ProduitsDetails;

```

FICHER : code/src/pages/public/register.tsx

```

import PublicForm from "../../components/form";
import Seo from "../../components/seo";

const Register = () => {
  return (
    <>
      <Seo
        title="Inscription"
        description="Créez votre compte pour profiter de nos
rituels de soin"
        url="/register"
      />

      <PublicForm
        title="Devenir Membre"
        buttonText="Créer mon compte"
        type="register"
      />
    </>
  );
};

export default Register;

```

FICHER : code/src/services/body_part_api_service.ts

```

import type { Body_part } from "../../models/body_part";
import type { ApiResponse } from "../../models/api_response";

class BodyPartApiService {
  // Préfixe de l'API
  private préfix = "/api/body_part";

  // selection de tous les enregistrement
  // On recoit une promesse (car mode async) qui est typer: on recoit

```

```
un type ApiResponse qui est lui même un Array( un type dans un type)
  public selectAll = async () :Promise<ApiResponse<Body_part[]>> => {

    // configurer la requête HTTP
    // import.meta.env permet d'importer une variable d'environnement
    dans vite/react. Il n'y à pas d'espace.
    const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}`);

    // exécuter la requête HTTP
    const response = await fetch(request);

    // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
    // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
    // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
    const results = await response.json();

    // retourner le resultat
    return results;

  };

  // Selection d'un enregistrement
  public selectOne = async (id: number)
:Promise<ApiResponse<Body_part>> => {

    // configurer la requête HTTP
    // import.meta.env permet d'importer une variable d'environnement
    dans vite/react. Il n'y à pas d'espace.
    const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}/${id}`);

    // exécuter la requête HTTP
    const response = await fetch(request);

    // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
    // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
    // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
    const results = await response.json();

    // retourner le resultat
    return results;

  };
}

export default BodyPartApiService;
```

FICHER : code/src/services/category_api_service.ts

```
import type { Category } from "../../models/category";
import type { ApiResponse } from "../../models/api_response";

class CategoryApiService {
  // Préfixe de l'API
  private préfix = "/api/category";

  // selection de tous les enregistrement
  // On nreçoit une promesse (car mode async) qui est typer: on recoit
  un type ApiResponse qui est lui même un Array( un type dans un type)
  public selectAll = async () :Promise<ApiResponse<Category[]>> => {

    // configurer la requête HTTP
    // import.meta.env permet d'importer une variable d'environnement
    dans vite/react. Il n'y à pas d'espace.
    const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}`);

    // exécuter la requête HTTP
    const response = await fetch(request);

    // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
    // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
    // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
    const results = await response.json();

    // retourner le resultat
    return results;
  };

  // Selection d'un enregistrement
  public selectOne = async (id: number) :Promise<ApiResponse<Category>>
=> {

    // configurer la requête HTTP
    // import.meta.env permet d'importer une variable d'environnement
    dans vite/react. Il n'y à pas d'espace.
    const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}/${id}`);

    // exécuter la requête HTTP
    const response = await fetch(request);

    // convertir la reponse en JSON en utilisant fonction json: ()
```

```

obligatoire
    // sérialiser: convertir des données complexes (onjet, array) en
    chaine de caractère
    // désérialiser: convertir une chaine de caractère en données
    complexes (objet, array...)
    const results = await response.json();

    // retourner le resultat
    return results;

};
}

export default CategoryApiService;

```

FICHER : code/src/services/pack_api_service.ts

```

import type { Pack } from "../../models/pack";
import type { ApiResponse } from "../models/api_response";

class PackApiService {
    // Préfixe de l'API
    private préfix = "/api/pack";

    // selection de tous les enregistrement
    // On nrecoit une promesse (car mode async) qui est typer: on recoit
    un type ApiResponse qui est lui même un Array( un type dans un type)
    public selectAll = async () :Promise<ApiResponse<Pack[]>> => {

        // configurer la requête HTTP
        // import.meta.env permet d'importer une variable d'environnement
        dans vite/react. Il n'y à pas d'espace.
        const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}`);

        // exécuter la requête HTTP
        const response = await fetch(request);

        // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
        // sérialiser: convertir des données complexes (onjet, array) en
        chaine de caractère
        // désérialiser: convertir une chaine de caractère en données
        complexes (objet, array...)
        const results = await response.json();

        // retourner le resultat
        return results;

    };
}

```

```

    // Selection d'un enregistrement
    public selectOne = async (id: number) :Promise<ApiResponse<Pack>> =>
    {

        // configurer la requête HTTP
        // import.meta.env permet d'importer une variable d'environnement
        dans vite/react. Il n'y à pas d'espace.
        const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}/${id}`);

        // exécuter la requête HTTP
        const response = await fetch(request);

        // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
        // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
        // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
        const results = await response.json();

        // retourner le resultat
        return results;

    };
}

export default PackApiService;

```

FICHER : code/src/services/product_api_service.ts

```

import type { Product } from "../../models/product";
import type { ApiResponse } from "../../models/api_response";

class ProductApiService {
    // Préfixe de l'API
    private préfix = "/api/product";

    // selection de tous les enregistrement
    // On nreçoit une promesse (car mode async) qui est typer: on recoit
un type ApiResponse qui est lui même un Array( un type dans un type)
    public selectAll = async () :Promise<ApiResponse<Product[]>> => {

        // configurer la requête HTTP
        // import.meta.env permet d'importer une variable d'environnement
dans vite/react. Il n'y à pas d'espace.
        const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}`);

```

```

        // exécuter la requête HTTP
        const response = await fetch(request);

        // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
        // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
        // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
        const results = await response.json();

        // retourner le resultat
        return results;

    };

    // Selection d'un enregistrement
    public selectOne = async (id: number) :Promise<ApiResponse<Product>>
=> {

        // configurer la requête HTTP
        // import.meta.env permet d'importer une variable d'environnement
dans vite/react. Il n'y à pas d'espace.
        const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}/${id}`);

        // exécuter la requête HTTP
        const response = await fetch(request);

        // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
        // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
        // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
        const results = await response.json();

        // retourner le resultat
        return results;

    };

    // Insertion d'un enregistrement
    /* Si le formulaire contient un champ de fichier: utiliser FormData en
parametre
    Si le formulaire ne contient pas de champ de gichier: utyliser le
stype */
    public insert = async (data: FormData)
:Promise<ApiResponse<Product>> => {

        // configurer la requête HTTP
        // import.meta.env permet d'importer une variable d'environnement
dans vite/react. Il n'y à pas d'espace.
        const request = new Request(

```

```

        `${import.meta.env.VITE_API_URL}${this.préfix}`,
        {
            method: "post",
            /* Si le formulaire contient un champs de fichier
               la propriété body renvoi un objet formData
            */
            body: data

            /*
               Si le formulaire ne contient pas de champs de
fichier
               la propriété body renvoi du JSON :
JSON.stringify(...)
               ajouter l'en-tête Content-Type:
applicatio,/json

            body: JSON.stringify(data)
            headers:{
                "Content-Type": "application/json",
            },
            */
        }
    );

    // exécuter la requête HTTP
    const response = await fetch(request);

    // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
    // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
    // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
    const results = await response.json();

    // retourner le resultat
    return results;
};

// Mise à jour d'un enregistrement
/* Si le formulaire contient un champ de fichier: utiliser FormData en
parametre
   Si le formulaire ne contient pas de champ de gichier: utiliser le
stype */
    public update = async (data: FormData) :Promise<ApiResponse<Product>>
=> {

        // configurer la requête HTTP
        // import.meta.env permet d'importer une variable d'environnement
dans vite/react. Il n'y à pas d'espace.
        const request = new Request(
            `${import.meta.env.VITE_API_URL}${this.préfix}`,

```



```

        {
            method: "put",
            /* Si le formulaire contient un champs de fichier
               la propriété body renvoi un objet formData
            */
            body: data

            /*
               Si le formulaire ne contient pas de champs de
fichier
               la propriété body renvoi du JSON :
JSON.stringify(...)
               ajouter l'en-tête Content-Type:
applicatio,/json

            body: JSON.stringify(data)
            headers:{
                "Content-Type": "application/json",
            },
            */
        }
    );

    // exécuter la requête HTTP
    const response = await fetch(request);

    // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
    // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
    // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
    const results = await response.json();

    // retourner le resultat
    return results;

};

    public delete = async (data: Partial<Product>)
:Promise<ApiResponse<Product>> => {

        // configurer la requête HTTP
        // import.meta.env permet d'importer une variable d'environnement
dans vite/react. Il n'y à pas d'espace.
        const request = new Request(
            `${import.meta.env.VITE_API_URL}${this.préfix}`,
            {
                method: "delete",
                /* Si le formulaire contient un champs de fichier
                   la propriété body renvoi un objet formData

                body: data */
            }
        );
    }
}

```

```

        /*
        Si le formulaire ne contient pas de champs de
fichier
        la propriété body renvoie du JSON :
JSON.stringify(...)
ajouter l'en-tête Content-Type:
applicatio,/json */

        headers:{
            "Content-Type": "application/json",
            /* Serialiser = stringify ==> Transformer une
donnée complexe (array, objet) en chaîne de caractères
            Désérialiser = parse ==> Transformer une
chaîne de caractère en donnée complexe (array, objet) */
        },
        body: JSON.stringify(data),

    }

    );

    // exécuter la requête HTTP
    const response = await fetch(request);

    // convertir la réponse en JSON en utilisant fonction json: ()
obligatoire
    // sérialiser: convertir des données complexes (objet, array) en
chaîne de caractère
    // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
    const results = await response.json();

    // retourner le résultat
    return results;

};
}

export default ProductApiService;

```

FICHER : code/src/services/router_service.ts

```

import type { unstable_RSCRouteConfig as RSCRouteConfig } from "react-
router";

class RouterService {
    public getRouter = () => {
        return [
            {
                // Identifiant unique de la route
            }
        ];
    };
}

```

```

    id: "root",
    // Préfixe des routes
    path: "/",
    // Importation de la mise en page
    lazy: () => import("../layouts/root_layout"),
    children: [
      {
        id: "public",
        path: "",
        lazy: () => import("../layouts/public_layout"),
        children: [
          {
            id: "home",
            index: true,
            path: "",
            lazy: () =>
import("../pages/public/homepage"),
          },
          {
            id: "produits",
            path: "produits",
            lazy: () =>
import("../pages/public/produits"),
          },
          {
            id: "produits_details",
            // path représente la route
            // variable d'URL est préfixée d'in :
            path: "produits/:id",
            lazy: () =>
import("../pages/public/produits_details"),
          },
          {
            id: "register",
            path: "register",
            lazy: () =>
import("../pages/public/register"),
          },
          {
            id: "login",
            path: "login",
            lazy: () =>
import("../pages/public/login"),
          },
          {
            id: "a_propos",
            path: "a_propos",
            lazy: () =>
import("../pages/public/a_propos"),
          },
          {
            id: "contact",
            path: "contact",
            lazy: () =>

```

```

import("../pages/public/contact"),
      },
      {
        id: "mentions_legales",
        path: "mentions_legales",
        lazy: () =>
import("../pages/public/mentions_legales"),
      },
    ],
  },
  {
    id: "admin",
    path: "/admin",
    lazy: () => import("../layouts/admin_layout"),
    children: [
      {
        id: "admin-home",
        path: "",
        lazy: () => import("../pages/admin/index"),
      },
      {
        id: "admin-product",
        path: "products",
        lazy: () =>
import("../pages/admin/product/index"),
      },
      {
        id: "add-product",
        // path représente la route
        // variable d'URL est préfixée d'un :
        // variable d'URL optionnelle est suffixée
d'un ?
        path: "products/add/:id?",
        lazy: () =>
import("../pages/admin/product/add"),
      },
      {
        id: "admin-product-delete",
        path: "products/delete/:id",
        lazy: () =>
import("../pages/admin/product/delete"),
      },
    ],
  },
  /*
  {
    id: "about",
    path: "about",
    lazy: () => import("../about/route"),
  },*/
],
},
] satisfies RSCRouteConfig;
};

```

```
}  
  
export default RouterService;
```

FICHER : code/src/services/skin_api_service.ts

```
import type { Skin } from "../../models/skin";  
import type { ApiResponse } from "../../models/api_response";  
  
class SkinApiService {  
  // Préfixe de l'API  
  private préfix = "/api/skin";  
  
  // selection de tous les enregistrement  
  // On nreçoit une promesse (car mode async) qui est typer: on recoit  
  un type ApiResponse qui est lui même un Array( un type dans un type)  
  public selectAll = async () :Promise<ApiResponse<Skin[]>> => {  
  
    // configurer la requête HTTP  
    // import.meta.env permet d'importer une variable d'environnement  
    dans vite/react. Il n'y à pas d'espace.  
    const request = new  
Request(`${import.meta.env.VITE_API_URL}${this.préfix}`);  
  
    // exécuter la requête HTTP  
    const response = await fetch(request);  
  
    // convertir la reponse en JSON en utilisant fonction json: ()  
obligatoire  
    // sérialiser: convertir des données complexes (onjet, array) en  
chaîne de caractère  
    // désérialiser: convertir une chaîne de caractère en données  
complexes (objet, array...)  
    const results = await response.json();  
  
    // retourner le resultat  
    return results;  
  
  };  
  
  // Selection d'un enregistrement  
  public selectOne = async (id: number) :Promise<ApiResponse<Skin>> =>  
{  
  
    // configurer la requête HTTP  
    // import.meta.env permet d'importer une variable d'environnement  
    dans vite/react. Il n'y à pas d'espace.  
    const request = new  
Request(`${import.meta.env.VITE_API_URL}${this.préfix}/${id}`);
```

```

        // exécuter la requête HTTP
        const response = await fetch(request);

        // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
        // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
        // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
        const results = await response.json();

        // retourner le resultat
        return results;

    };
}

export default SkinApiService;

```

FICHER : code/src/validators/admin/admin_formulaire_validator.ts

```

import { z, type ZodError } from "zod";
import type { Product } from "../../models/product";

class AdminFormulaireValidator {

    // validation des données du formulaire
    public validate = async (data: Partial<Product>,
    ): Promise<Partial<Product> | ZodError> => {
        // La méthode doit etre executée coté serveur
        "use server";

        // Contraintes de validation
        // Reprendre strictement les propriétés du type à valider
        const constraints = z.object({
            id: z.union([
                z.string().nullable(),
                // coerce : permet de transtyper les donnés (string
devient number)
                z.coerce
                    .number()
                    .positive(),
            ]),

            name: z.string("Le nom est obligatoire").min(2, "Un nom doit
comporter au minimum 2 caractères").max(50, "Un nom doit comporté au
maximum 50 caracteres"),

            image: z.union([

```

```

        z.string().nullable(),
        z.file("L'image est obligatoire"),
    ]),

    price: z.coerce.number("Le prix est obligatoire").min(1, "Un
prix doit comporter au minimum 1 chiffre").max(99.99, "Un prix peut
comporter au maximum 4 caracteres dont 2 decimales"),

    description: z.string("La description est
obligatoire").min(20, "Une description doit comporter au minimum 20 "),

    category_id: z.coerce.number("Veuillez selectionner une
category").positive(),

    skin_ids: z.string("Veuillez selectionner un ou plusieurs type
de peau").min(1, "Selectionnez au moins un type de peau"),

    body_part_ids: z.string("Veuillez selectionner une ou
plusieurs zone du corps").min(1, "Selectionnez au moins une zone du
corps"),

    pack_ids: z.string("Veuillez selectionner une ou plusieurs
pack").min(1, "Selectionnez au moins un pack"),
    });

    // Validation de la saisie du formulaire

    const validation = await constraints.safeParseAsync(data);

    // Si la validation échoue
    if (!validation.success) {
        return validation.error;
    }
    // Si la validation réussie
    return validation.data as Partial<Product>;
};
}

export default AdminFormulaireValidator;

```

FICHER : code/tsconfig.json

```

{
  "compilerOptions": {
    "erasableSyntaxOnly": true,
    "allowImportingTsExtensions": true,
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "skipLibCheck": true,

```

```

    "verbatimModuleSyntax": true,
    "noEmit": true,
    "moduleResolution": "Bundler",
    "module": "ESNext",
    "target": "ESNext",
    "lib": ["ESNext", "DOM", "DOM.Iterable"],
    "types": ["vite/client", "@vitejs/plugin-rsc/types"],
    "jsx": "react-jsx"
  }
}

```

FICHER : code/vite.config.ts

```

import react from "@vitejs/plugin-react";
import rsc from "@vitejs/plugin-rsc";
import { defineConfig } from "vite";

export default defineConfig(() => {
  return {
    server: {
      port: 5173,
      host: true,
    },
    plugins: [
      rsc({
        // `entries` option is only a shorthand for specifying
        each `rollupOptions.input` below
        // > entries: { rsc, ssr, client },
        //
        // by default, the plugin setup request handler based on
        `default export` of `rsc` environment `rollupOptions.input.index`.
        // This can be disabled when setting up own server handler
        e.g. `@cloudflare/vite-plugin`.
        // > serverHandler: false
      }),
      // use any of react plugins https://github.com/vitejs/vite-
      plugin-react
      // to enable client component HMR
      react(),
      // use https://github.com/antfu-collective/vite-plugin-inspect
      // to understand internal transforms required for RSC.
      // import("vite-plugin-inspect").then(m => m.default()),
    ],
    // specify entry point for each environment.
    // (currently the plugin assumes `rollupOptions.input.index` for
    some features.)
  };
});

```



```

    environments: {
      // `rsc` environment loads modules with `react-server`
condition.
      // this environment is responsible for:
      // - RSC stream serialization (React VDOM -> RSC stream)
      // - server functions handling
      rsc: {
        build: {
          rollupOptions: {
            input: {
              index: "./src/framework/entry.rsc.tsx",
            },
          },
        },
      },

      // `ssr` environment loads modules without `react-server`
condition.
      // this environment is responsible for:
      // - RSC stream deserialization (RSC stream -> React VDOM)
      // - traditional SSR (React VDOM -> HTML string/stream)
      ssr: {
        build: {
          rollupOptions: {
            input: {
              index: "./src/framework/entry.ssr.tsx",
            },
          },
        },
      },

      // client environment is used for hydration and client-side
rendering
      // this environment is responsible for:
      // - RSC stream deserialization (RSC stream -> React VDOM)
      // - traditional CSR (React VDOM -> Browser DOM tree
mount/hydration)
      // - refetch and re-render RSC
      // - calling server functions
      client: {
        build: {
          rollupOptions: {
            input: {
              index: "./src/framework/entry.browser.tsx",
            },
          },
        },
      },
    },
  };
});

```

FICHER : mon_projet_certification.md

```
# ANALYSE DU PROJET DWWM
```

```
---
```

```
### FICHER : .devcontainer/app/devcontainer.json
```

```
```json
```

```
{
 "name": "Secret-de-Hammam - app",
 "service": "app",
 "dockerComposeFile": "../..../docker-compose.dev.yaml",
 "workspaceFolder": "/app",
 "customizations": {
 "vscode": {
 "extensions": [
 "christian-kohler.npm-intellisense",
 "christian-kohler.path-intellisense",
 "biomejs.biome",
 "dsznajder.es7-react-js-snippets",
 "VASubasRaj.flashpost",
 "ritwickdey.LiveServer"
],
 "settings": {
 "[typescript]": {
 "editor.defaultFormatter": "biomejs.biome",
 "editor.formatOnSave": true,
 "editor.formatOnType": true
 },
 "[typescriptreact]": {
 "editor.defaultFormatter": "biomejs.biome",
 "editor.formatOnSave": true,
 "editor.formatOnType": true
 },
 "[javascript]": {
 "editor.defaultFormatter": "biomejs.biome",
 "editor.formatOnSave": true,
 "editor.formatOnType": true
 },
 "[javascriptreact]": {
 "editor.defaultFormatter": "biomejs.biome",
 "editor.formatOnSave": true,
 "editor.formatOnType": true
 },
 "[css]": {
 "editor.defaultFormatter": "biomejs.biome",
 "editor.formatOnSave": true,
 "editor.formatOnType": true
 },
 "editor.codeActionsOnSave": {
 "source.fixAll": "always"
 }
 }
 }
 }
}
```

```

 }
 }
}

```

FICHER : .devcontainer/mongodb/devcontainer.json

```

// Reprendre les infos du fichier yaml
{
 "name": "Secret-de-Hammam - MongoDB",
 "service": "mongodb", // reprendre strictement le nom du fichier
 "dockerComposeFile": "../../docker-compose.dev.yaml", // reprendre
strictement le nom du fichier yaml
 "workspaceFolder": "/app" // reprendre strictement le nom de
working_dir
}

```

FICHER : .devcontainer/mysql/devcontainer.json

```

{
 "name": "Secret-de-Hammam - MySQL",
 "service": "mysql",
 "dockerComposeFile": "../../docker-compose.dev.yaml",
 "workspaceFolder": "/app",
 "customizations": {
 "vscode": {
 "extensions": [],
 "settings": {}
 }
 }
}

```

FICHER : README.md

```

Secret-de-Hammam

```

FICHER : bundle-code.js

```

const fs = require('fs');
const path = require('path');

// 1. Configuration des fichiers/dossiers à ignorer
const IGNORE_LIST = [
 'node_modules', '.git', 'dist', 'build', '.next',
 'package-lock.json', 'yarn.lock', '.gitignore', '.env', 'mysql-data',
 'venv', '.vscode'
];

// 2. Extensions de fichiers que nous voulons inclure
const ALLOWED_EXTENSIONS = ['.ts', '.tsx', '.js', '.jsx', '.css', '.json',
 '.html', '.md'];

const outputFile = 'mon_projet_certification.md';

/**
 * Fonction principale qui parcourt les dossiers récursivement
 */
function generateMarkdown(dir, fileList = []) {
 const files = fs.readdirSync(dir);

 files.forEach(file => {
 const filePath = path.join(dir, file);
 const stat = fs.statSync(filePath);

 // Si c'est un dossier et qu'il n'est pas dans la liste noire, on
 descend dedans
 if (stat.isDirectory()) {
 if (!IGNORE_LIST.includes(file)) {
 generateMarkdown(filePath, fileList);
 }
 } else {
 // Si c'est un fichier, on vérifie son extension
 const ext = path.extname(file);
 if (ALLOWED_EXTENSIONS.includes(ext) &&
!IGNORE_LIST.includes(file)) {
 const content = fs.readFileSync(filePath, 'utf8');
 const relativePath = path.relative(process.cwd(),
filePath);

 // On formate le fichier pour le Markdown
 const markdownBlock = `\n---\n### FICHER :
${relativePath}\n\n\`\`\`${ext.replace('.', '')}\n${content}\n\`\`\`\n`;
 fs.appendFileSync(outputFile, markdownBlock);
 }
 }
 });
}

// Nettoyage du fichier de sortie s'il existe déjà
if (fs.existsSync(outputFile)) fs.unlinkSync(outputFile);

```

```
fs.writeFileSync(outputFile, "# ANALYSE DU PROJET DWWM\n");
console.log("Extraction en cours...");
generateMarkdown('.');
console.log(`Terminé ! Votre fichier est prêt : ${outputFile}`);
```

---

FICHER : code/README.md

## # Vite + RSC

This example shows how to set up a React application with [Server Component](<https://react.dev/reference/rsc/server-components>) features on Vite using [`@vitejs/plugin-rsc`](<https://github.com/vitejs/vite-plugin-react/tree/main/packages/plugin-rsc>).

[![Open in StackBlitz]([https://developer.stackblitz.com/img/open\\_in\\_stackblitz.svg](https://developer.stackblitz.com/img/open_in_stackblitz.svg))](<https://stackblitz.com/github/vitejs/vite-plugin-react/tree/main/packages/plugin-rsc/examples/starter>)

```
```sh
```

```
# run dev server
npm run dev
```

```
# build for production and preview
npm run build
npm run preview
```

API usage

See [@vitejs/plugin-rsc](#) for the documentation.

- `vite.config.ts`
 - `@vitejs/plugin-rsc/plugin`
- `./src/framework/entry.rsc.tsx`
 - `@vitejs/plugin-rsc/rsc`
 - `import.meta.viteRsc.loadModule`
- `./src/framework/entry.ssr.tsx`
 - `@vitejs/plugin-rsc/ssr`
 - `import.meta.viteRsc.loadBootstrapScriptContent`
 - `rsc-html-stream/server`
- `./src/framework/entry.browser.tsx`
 - `@vitejs/plugin-rsc/browser`
 - `rsc-html-stream/client`

Notes

- `./src/framework/entry.{browser,rsc,ssr}.tsx` (with inline comments) provides an overview of how low level RSC (React flight) API can be used to build RSC framework.
- You can use `vite-plugin-inspect` to understand how `"use client"` and `"use server"` directives are transformed internally.

Deployment

See [vite-plugin-rsc-deploy-example](#)

```

---
### FICHER : code/Test.ts

```ts
//REPOSITORY

// sélectionner un enregistrement
// data représente une partie des propriétés du type
public
selectOne = async (data: Partial<Role>): Promise<Role | unknown> => {
 // connexion au serveur MySQL
 const connection = await new MySQLService().connect(); // requete SQL
 // where category.id=....
 // SELECT level.* FROM arabizy_dev.level;
 // variable de requete : précédée d'un :, suivi du nom de la variable
 // requetes préparées :sécurité;(utilisation des varibales de
 requetes)la requete est exécutée si elle ne représente pas de risque de
 sécurité
 const sql = SELECT;
 $;
 this.table;
 .*
 FROM $
 process.env.MYSQL_DATABASE;
 .$
 this.table;
 where;
 $;
 this.table;
 .id= :id
 // try / catch: récupérer les résultats de la requete ou un erreur
 try {
 // exécution de la requete
 // si la requete possède des variables, utiliser le paramètre de
 la méthode
 const [query] = await connection.execute(sql, data);
 // récupérer le premier indice d'un array
 const result = (query as Role[]).shift();
 // retourner les résultats
 return result;
 } catch (error) {
 return error;
 }
}

```

```

 }
};

//controllers;

public
selectOne = async (req: Request, res: Response) => {
 // récupérer la variable de route
 // req.params :rrécupère les variables de route
 console.log(req.params);
 // récupération des résultats de la requete
 const results = await new RoleRepository().selectOne(req.params);
 // si la requete renvoie une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === "production" ? "Error" :
results.message,
 });
 return;
 }
 // envoyer une réponse
 // res.send("coucou");
 // renvoyer une réponse avec un code de statut HTTP et au format JSON
 res.status(200).json({
 status: 200,
 message: "ok level",
 data: results,
 });
};

//router;
// // variable de route:précédée par un ;; suivie du nom de la variable
// appel 1
this.router.get("/:id", new RoleController().selectOne);

```

---

FICHER : code/\_flashpost/flashpost-collection-body\_part.json

```

{"app": "Flashpost", "id": "f4c9fbc8-0fec-4e25-bab3-b2f42db98c17", "name": "body_part", "version": "1.0", "type": "collections", "createdAt": "05-Dec-2025 14:16:23", "exportedDate": "05-Dec-2025 14:16:48", "collections": [{"id": "f4c9fbc8-0fec-4e25-bab3-b2f42db98c17", "parent": "0", "text": "body_part", "createdAt": "05-Dec-2025 14:16:23", "draggable": true, "data": {"treeNodeType": "collection", "variableId": ""}, "relativeIndex": 0},

```

```

{"id": "874c7867-14c1-4060-b6d1-073ec44b5235", "parent": "f4c9fbc8-0fec-4e25-bab3-b2f42db98c17", "text": "127.0.0.1:3000/api/body_part/3", "createdTime": "05-Dec-2025 15:13:11", "draggable": false, "data": {"id": "f8e1841f-5a2c-4cea-8008-0519516254fb", "method": "get", "name": "127.0.0.1:3000/api/body_part/3", "url": "127.0.0.1:3000/api/body_part/3", "createdTime": "05-Dec-2025 15:13:11", "$loki": 14, "treeNodeType": "request"}}, {"id": "4ffb410a-3504-4131-a613-c442e14174de", "parent": "f4c9fbc8-0fec-4e25-bab3-b2f42db98c17", "text": "127.0.0.1:3000/api/body_part", "createdTime": "05-Dec-2025 15:12:35", "draggable": false, "data": {"id": "af61a4c0-f02e-4d24-9528-f839684ac4b0", "method": "get", "name": "127.0.0.1:3000/api/body_part", "url": "127.0.0.1:3000/api/body_part", "createdTime": "05-Dec-2025 15:12:35", "$loki": 13, "treeNodeType": "request"}}], "requests": [{"id": "874c7867-14c1-4060-b6d1-073ec44b5235", "url": "127.0.0.1:3000/api/body_part/3", "name": "127.0.0.1:3000/api/body_part/3", "createdTime": "05-Dec-2025 15:13:11", "method": "get", "params": [{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth": {"authType": "noauth", "userName": "", "password": "", "addTo": "queryParams", "showPwd": false, "tokenPrefix": "", "aws": {"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "none", "formdata": [{"isChecked": false, "key": "", "value": ""}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"}, {"id": "4ffb410a-3504-4131-a613-c442e14174de", "url": "127.0.0.1:3000/api/body_part", "name": "127.0.0.1:3000/api/body_part", "createdTime": "05-Dec-2025 15:12:35", "method": "get", "params": [{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth": {"authType": "noauth", "userName": "", "password": "", "addTo": "queryParams", "showPwd": false, "tokenPrefix": "", "aws": {"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "none", "formdata": [{"isChecked": false, "key": "", "value": ""}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"}]}

```



FICHER : code/\_flashpost/flashpost-collection-category.json

```
{
 "app": "Flashpost",
 "id": "b6bd3bce-70c8-43a3-8cd7-11c8ecd2e917",
 "name": "category",
 "version": "1.0",
 "type": "collections",
 "createdAt": "05-Dec-2025 13:38:19",
 "exportedDate": "05-Dec-2025 13:39:54",
 "collections": [
 {
 "id": "b6bd3bce-70c8-43a3-8cd7-11c8ecd2e917",
 "parent": "0",
 "text": "category",
 "createdAt": "05-Dec-2025 13:38:19",
 "draggable": true,
 "data": {
 "treeNodeType": "collection",
 "variableId": "",
 "relativeIndex": 0
 },
 "id": "c86f2434-3063-4230-b343-acf21492a1af",
 "parent": "b6bd3bce-70c8-43a3-8cd7-11c8ecd2e917",
 "text": "127.0.0.1:3000/api/category/1",
 "createdAt": "05-Dec-2025 14:38:57",
 "draggable": false,
 "data": {
 "id": "177799a8-d92f-4480-87a5-37b24b6afd6d",
 "method": "get",
 "name": "127.0.0.1:3000/api/category/1",
 "url": "127.0.0.1:3000/api/category/1",
 "createdAt": "05-Dec-2025 14:38:57",
 "$loki": 7,
 "treeNodeType": "request"
 },
 "id": "dd92977e-8538-45c9-8889-b7366d8898c2",
 "parent": "b6bd3bce-70c8-43a3-8cd7-11c8ecd2e917",
 "text": "127.0.0.1:3000/api/category",
 "createdAt": "05-Dec-2025 14:37:38",
 "draggable": false,
 "data": {
 "id": "2715445d-c94c-4135-aa26-8c43797b6fbe",
 "method": "get",
 "name": "127.0.0.1:3000/api/category",
 "url": "127.0.0.1:3000/api/category/1",
 "createdAt": "05-Dec-2025 14:37:38",
 "$loki": 6,
 "treeNodeType": "request"
 }
 },
 {
 "id": "c86f2434-3063-4230-b343-acf21492a1af",
 "url": "127.0.0.1:3000/api/category/1",
 "name": "127.0.0.1:3000/api/category/1",
 "createdAt": "05-Dec-2025 14:38:57",
 "method": "get",
 "params": {
 "isChecked": false,
 "key": "",
 "value": ""
 },
 "pathParams": [],
 "auth": {
 "authType": "noauth",
 "userName": "",
 "password": "",
 "addTo": "queryParams",
 "showPwd": false,
 "tokenPrefix": "",
 "aws": {
 "service": "",
 "region": "",
 "accessKey": "",
 "secretAccessKey": "",
 "sessionToken": ""
 },
 "headers": [
 {
 "key": "Accept",
 "value": "*/*",
 "isChecked": true
 },
 {
 "key": "User-Agent",
 "value": "Flashpost",
 "isChecked": true
 },
 {
 "key": "",
 "value": "",
 "isChecked": false
 }
],
 "body": {
 "bodyType": "none",
 "formdata": {
 "isChecked": false,
 "key": "",
 "value": ""
 },
 "urlencoded": {
 "isChecked": false,
 "key": "",
 "value": ""
 },
 "raw": {
 "data": "",
 "lang": "json",
 "binary": {
 "fileName": "",
 "data": {}
 },
 "contentTypeOption": "manual",
 "graphql": {
 "query": "",
 "variables": ""
 },
 "tests": [
 {
 "parameter": "",
 "action": "",
 "expectedValue": ""
 },
 {
 "parameter": "",
 "key": "",
 "variableName": ""
 }
],
 "notes": "",
 "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"
 },
 "id": "dd92977e-8538-45c9-8889-b7366d8898c2",
 "url": "127.0.0.1:3000/api/category/1",
 "name": "127.0.0.1:3000/api/category",
 "createdAt": "05-Dec-2025 14:37:38",
 "method": "get",
 "params": {
 "isChecked": false,
 "key": "",
 "value": ""
 },
 "pathParams": [],
 "auth": {
 "authType": "noauth",
 "userName": "",
 "password": "",
 "addTo": "queryParams",
 "showPwd": false,
 "tokenPrefix": "",
 "aws": {
 "service": "",
 "region": "",
 "accessKey": "",
 "secretAccessKey": "",
 "sessionToken": ""
 },
 "headers": [
 {
 "key": "Accept",
 "value": "*/*",
 "isChecked": true
 },
 {
 "key": "User-Agent",
 "value": "Flashpost",
 "isChecked": true
 },
 {
 "key": "",
 "value": "",
 "isChecked": false
 }
],
 "body": {
 "bodyType": "none",
 "formdata": {
 "isChecked": false,
 "key": "",
 "value": ""
 },
 "urlencoded": {
 "isChecked": false,
 "key": "",
 "value": ""
 },
 "raw": {
 "data": "",
 "lang": "json",
 "binary": {
 "fileName": "",
 "data": {}
 },
 "contentTypeOption": "manual",
 "graphql": {
 "query": "",
 "variables": ""
 },
 "tests": [
 {
 "parameter": "",
 "action": "",
 "expectedValue": ""
 },
 {
 "parameter": "",
 "key": "",
 "variableName": ""
 }
],
 "notes": "",
 "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"
 }
 }
 }
 }
 }
 }
]
}
```

```

owPwd": false, "tokenPrefix": "", "aws":
{"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "none", "formdata": [{"isChecked": false, "key": "", "value": ""}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"]}]

```

FICHER : code/\_flashpost/flashpost-collection-homepage.json

```

{"app": "Flashpost", "id": "05ca44cd-054a-44b6-a922-9b7e8830b5a3", "name": "homepage", "version": "1.0", "type": "collections", "createdAt": "05-Dec-2025 12:51:22", "exportedDate": "05-Dec-2025 12:52:48", "collections": [{"id": "05ca44cd-054a-44b6-a922-9b7e8830b5a3", "parent": "0", "text": "homepage", "createdAt": "05-Dec-2025 12:51:22", "draggable": true, "data": {"treeNodeType": "collection", "variableId": ""}, "relativeIndex": 0}, {"id": "bb2a5775-780f-4ea9-946c-33a60527eef3", "parent": "05ca44cd-054a-44b6-a922-9b7e8830b5a3", "text": "127.0.0.1:3000/api", "createdAt": "05-Dec-2025 13:50:00", "draggable": false, "data": {"id": "5b7dd5ed-c995-447f-bf5c-cec9018e9fee", "method": "get", "name": "127.0.0.1:3000/api", "url": "127.0.0.1:3000/api", "createdAt": "05-Dec-2025 13:50:00", "$loki": 3, "treeNodeType": "request"}}], "requests": [{"id": "bb2a5775-780f-4ea9-946c-33a60527eef3", "url": "127.0.0.1:3000/api", "name": "127.0.0.1:3000/api", "createdAt": "05-Dec-2025 13:50:00", "method": "get", "params": [{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth": {"authType": "noauth", "userName": "", "password": "", "addTo": "queryParams", "showPwd": false, "tokenPrefix": "", "aws": {"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "none", "formdata": [{"isChecked": false, "key": "", "value": ""}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"}]}

```

FICHER : code/\_flashpost/flashpost-collection-pack.json

```
{
 "app": "Flashpost",
 "id": "91a7ecfc-2514-4439-a207-63a88d331b5e",
 "name": "pack",
 "version": "1.0",
 "type": "collections",
 "createdTime": "05-Dec-2025 14:09:39",
 "exportedDate": "05-Dec-2025 14:10:41",
 "collections": [
 {
 "id": "91a7ecfc-2514-4439-a207-63a88d331b5e",
 "parent": "0",
 "text": "pack",
 "createdTime": "05-Dec-2025 14:09:39",
 "droppable": true,
 "data": {
 "treeNodeType": "collection",
 "variableId": "",
 "relativeIndex": 0
 },
 "id": "6b422a11-ffde-42cf-a832-016146ea36dd",
 "parent": "91a7ecfc-2514-4439-a207-63a88d331b5e",
 "text": "127.0.0.1:3000/api/pack/2",
 "createdTime": "05-Dec-2025 15:10:04",
 "droppable": false,
 "data": {
 "id": "d8413749-9b22-44b6-8ef9-da49d25ebac5",
 "method": "get",
 "name": "127.0.0.1:3000/api/pack/2",
 "url": "127.0.0.1:3000/api/pack/2",
 "createdTime": "05-Dec-2025 15:10:04",
 "$loki": 12,
 "treeNodeType": "request"
 }
 },
 {
 "id": "ddd3583a-8387-4871-ba69-f7e551ec5c92",
 "parent": "91a7ecfc-2514-4439-a207-63a88d331b5e",
 "text": "127.0.0.1:3000/api/pack",
 "createdTime": "05-Dec-2025 15:09:10",
 "droppable": false,
 "data": {
 "id": "7f1e9c16-60c5-4d92-82d1-52076346fcc2",
 "method": "get",
 "name": "127.0.0.1:3000/api/pack",
 "url": "127.0.0.1:3000/api/pack/2",
 "createdTime": "05-Dec-2025 15:09:10",
 "$loki": 10,
 "treeNodeType": "request"
 }
 }
],
 "requests": [
 {
 "id": "6b422a11-ffde-42cf-a832-016146ea36dd",
 "url": "127.0.0.1:3000/api/pack/2",
 "name": "127.0.0.1:3000/api/pack/2",
 "createdTime": "05-Dec-2025 15:10:04",
 "method": "get",
 "params": [
 {
 "isChecked": false,
 "key": "",
 "value": ""
 }
],
 "pathParams": [],
 "auth": {
 "authType": "noauth",
 "userName": "",
 "password": "",
 "addTo": "queryparams",
 "showPwd": false,
 "tokenPrefix": "",
 "aws": {
 "service": "",
 "region": "",
 "accessKey": "",
 "secretAccessKey": "",
 "sessionToken": ""
 }
 },
 "headers": [
 {
 "key": "Accept",
 "value": "*/*",
 "isChecked": true
 },
 {
 "key": "User-Agent",
 "value": "Flashpost",
 "isChecked": true
 },
 {
 "key": "",
 "value": "",
 "isChecked": false
 }
],
 "body": {
 "bodyType": "none",
 "formdata": [
 {
 "isChecked": false,
 "key": "",
 "value": ""
 }
],
 "urlencoded": [
 {
 "isChecked": false,
 "key": "",
 "value": ""
 }
],
 "raw": {
 "data": "",
 "lang": "json",
 "binary": {
 "fileName": "",
 "data": {}
 },
 "contentTypeOption": "manual",
 "graphql": {
 "query": "",
 "variables": ""
 },
 "tests": [
 {
 "parameter": "",
 "action": "",
 "expectedValue": ""
 }
],
 "setvar": [
 {
 "parameter": "",
 "key": "",
 "variableName": ""
 }
],
 "notes": "",
 "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"
 },
 {
 "id": "ddd3583a-8387-4871-ba69-f7e551ec5c92",
 "url": "127.0.0.1:3000/api/pack/2",
 "name": "127.0.0.1:3000/api/pack",
 "createdTime": "05-Dec-2025 15:09:10",
 "method": "get",
 "params": [
 {
 "isChecked": false,
 "key": "",
 "value": ""
 }
],
 "pathParams": [],
 "auth": {
 "authType": "noauth",
 "userName": "",
 "password": "",
 "addTo": "queryparams",
 "showPwd": false,
 "tokenPrefix": "",
 "aws": {
 "service": "",
 "region": "",
 "accessKey": "",
 "secretAccessKey": "",
 "sessionToken": ""
 }
 },
 "headers": [
 {
 "key": "Accept",
 "value": "*/*",
 "isChecked": true
 },
 {
 "key": "User-Agent",
 "value": "Flashpost",
 "isChecked": true
 },
 {
 "key": "",
 "value": "",
 "isChecked": false
 }
],
 "body": {

```

```
{
 "bodyType": "none",
 "formdata": [
 {
 "isChecked": false,
 "key": "",
 "value": ""
 },
 {
 "isChecked": false,
 "key": "",
 "value": ""
 }
],
 "urlencoded": [
 {
 "isChecked": false,
 "key": "",
 "value": ""
 }
],
 "raw": {
 "data": "",
 "lang": "json",
 "binary": {
 "fileName": "",
 "data": {}
 },
 "contentTypeOption": "manual",
 "graphql": {
 "query": "",
 "variables": ""
 },
 "tests": [
 {
 "parameter": "",
 "action": "",
 "expectedValue": ""
 }
],
 "setvar": [
 {
 "parameter": "",
 "key": "",
 "variableName": ""
 }
],
 "notes": "",
 "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"
]
}
```

FICHER : code/\_flashpost/flashpost-collection-product.json

```
{
 "app": "Flashpost",
 "id": "40583afe-8a76-4b94-9d8f-51ed03cac317",
 "name": "product",
 "version": "1.0",
 "type": "collections",
 "createdAt": "05-Dec-2025 14:19:35",
 "exportedDate": "18-Dec-2025 21:25:09",
 "collections": [
 {
 "id": "40583afe-8a76-4b94-9d8f-51ed03cac317",
 "parent": "0",
 "text": "product",
 "createdTime": "05-Dec-2025 14:19:35",
 "draggable": true,
 "data": {
 "treeNodeType": "collection",
 "variableId": "",
 "relativeIndex": 0
 }
 },
 {
 "id": "2f95e3ef-8be8-459f-9adb-0990aa71a20a",
 "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
 "text": "127.0.0.1:3000/api/product",
 "createdTime": "18-Dec-2025 10:06:39",
 "draggable": false,
 "data": {
 "id": "db85cbbd-ff4f-4497-92a1-df1f7ad6e7f6",
 "method": "put",
 "name": "127.0.0.1:3000/api/product",
 "url": "127.0.0.1:3000/api/product",
 "createdTime": "18-Dec-2025 10:06:39",
 "$loki": 23,
 "treeNodeType": "request"
 }
 },
 {
 "id": "b7031931-d679-4564-8a5a-cdb95b0c7a8d",
 "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
 "text": "127.0.0.1:3000/api/product",
 "createdTime": "18-Dec-2025 14:19:36",
 "draggable": false,
 "data": {
 "id": "8ddcffad-3c8f-4435-ba2a-72aab86b7019",
 "method": "delete",
 "name": "127.0.0.1:3000/api/product",
 "url": "127.0.0.1:3000/api/product",
 "createdTime": "18-Dec-2025 14:19:36",
 "$loki": 25,
 "treeNodeType": "request"
 }
 },
 {
 "id": "2c6b49dc-0897-42c0-8a60-c6fcd27e1e11",
 "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
 "text": "127.0.0.1:3000/api/product",
 "createdTime": "17-Dec-2025 09:19:47",
 "draggable": false,
 "data": {
 "id": "2335fdca-2643-4f01-ab74-42a6c954be5c",
 "method": "post",
 "name": "127.0.0.1:3000/api/product",
 "url": "127.0.0.1:3000/api/product",
 "createdTime": "17-Dec-2025 09:19:47",
 "$loki": 18,
 "treeNodeType": "request"
 }
 },
 {
 "id": "a177c61d-ee44-4994-948f-fc99f49323d6",
 "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
 "text": "127.0.0.1:3000/api/product/7",
 "createdTime": "05-Dec-2025 15:19:21",
 "draggable": false,
 "data": {
 "id": "47a0b5d5-5c07-4571-b5ce-d267a5e1b34c",
 "method": "get",
 "name": "127.0.0.1:3000/api/product/7",
 "url": "127.0.0.1:3000/api/product/7",
 "createdTime": "05-Dec-2025 15:19:21",
 "$loki": 16,
 "treeNodeType": "request"
 }
 },
 {
 "id": "c4ca3ced-fec0-4814-96f5-4bd0c0b34bed",
 "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
 "text": "127.0.0.1:3000/api/product",
 "createdTime": "05-Dec-2025 15:19:05",
 "draggable": false,
 "data": {
 "id": "7778d6e4-3918-4f2e-9436-1740adb140e6",
 "method": "get",
 "name": "127.0.0.1:3000/api/product",
 "url": "127.0.0.1:3000/api/product",
 "createdTime": "05-Dec-2025 15:19:05",
 "$loki": 15,
 "treeNodeType": "request"
 }
 }
],
 "requests": [
 {
 "id": "40583afe-8a76-4b94-9d8f-51ed03cac317",
 "parent": "0",
 "text": "product",
 "createdTime": "05-Dec-2025 14:19:35",
 "draggable": true,
 "data": {
 "treeNodeType": "collection",
 "variableId": "",
 "relativeIndex": 0
 }
 },
 {
 "id": "2f95e3ef-8be8-459f-9adb-0990aa71a20a",
 "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
 "text": "127.0.0.1:3000/api/product",
 "createdTime": "18-Dec-2025 10:06:39",
 "draggable": false,
 "data": {
 "id": "db85cbbd-ff4f-4497-92a1-df1f7ad6e7f6",
 "method": "put",
 "name": "127.0.0.1:3000/api/product",
 "url": "127.0.0.1:3000/api/product",
 "createdTime": "18-Dec-2025 10:06:39",
 "$loki": 23,
 "treeNodeType": "request"
 }
 },
 {
 "id": "b7031931-d679-4564-8a5a-cdb95b0c7a8d",
 "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
 "text": "127.0.0.1:3000/api/product",
 "createdTime": "18-Dec-2025 14:19:36",
 "draggable": false,
 "data": {
 "id": "8ddcffad-3c8f-4435-ba2a-72aab86b7019",
 "method": "delete",
 "name": "127.0.0.1:3000/api/product",
 "url": "127.0.0.1:3000/api/product",
 "createdTime": "18-Dec-2025 14:19:36",
 "$loki": 25,
 "treeNodeType": "request"
 }
 },
 {
 "id": "2c6b49dc-0897-42c0-8a60-c6fcd27e1e11",
 "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
 "text": "127.0.0.1:3000/api/product",
 "createdTime": "17-Dec-2025 09:19:47",
 "draggable": false,
 "data": {
 "id": "2335fdca-2643-4f01-ab74-42a6c954be5c",
 "method": "post",
 "name": "127.0.0.1:3000/api/product",
 "url": "127.0.0.1:3000/api/product",
 "createdTime": "17-Dec-2025 09:19:47",
 "$loki": 18,
 "treeNodeType": "request"
 }
 },
 {
 "id": "a177c61d-ee44-4994-948f-fc99f49323d6",
 "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
 "text": "127.0.0.1:3000/api/product/7",
 "createdTime": "05-Dec-2025 15:19:21",
 "draggable": false,
 "data": {
 "id": "47a0b5d5-5c07-4571-b5ce-d267a5e1b34c",
 "method": "get",
 "name": "127.0.0.1:3000/api/product/7",
 "url": "127.0.0.1:3000/api/product/7",
 "createdTime": "05-Dec-2025 15:19:21",
 "$loki": 16,
 "treeNodeType": "request"
 }
 },
 {
 "id": "c4ca3ced-fec0-4814-96f5-4bd0c0b34bed",
 "parent": "40583afe-8a76-4b94-9d8f-51ed03cac317",
 "text": "127.0.0.1:3000/api/product",
 "createdTime": "05-Dec-2025 15:19:05",
 "draggable": false,
 "data": {
 "id": "7778d6e4-3918-4f2e-9436-1740adb140e6",
 "method": "get",
 "name": "127.0.0.1:3000/api/product",
 "url": "127.0.0.1:3000/api/product",
 "createdTime": "05-Dec-2025 15:19:05",
 "$loki": 15,
 "treeNodeType": "request"
 }
 }
]
}
```

```
[{"id": "2f95e3ef-8be8-459f-9adb-0990aa71a20a", "url": "127.0.0.1:3000/api/product", "name": "127.0.0.1:3000/api/product", "createdTime": "18-Dec-2025 10:06:39", "method": "put", "params": [{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth": {"authType": "noauth", "userName": "", "password": "", "addTo": "queryParams", "showPwd": false, "tokenPrefix": "", "aws": {"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "Content-Type", "value": "multipart/form-data", "isChecked": true, "isFixed": false}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "formdata", "formdata": [{"isChecked": true, "key": "id", "value": "48"}, {"isChecked": true, "key": "name", "value": "update product", "type": "Text"}, {"isChecked": true, "key": "image", "value": "update img", "type": "Text"}, {"isChecked": true, "key": "description", "value": "update desc", "type": "Text"}, {"isChecked": true, "key": "price", "value": "12", "type": "Text"}, {"isChecked": true, "key": "category_id", "value": "2", "type": "Text"}, {"isChecked": true, "key": "skin_ids", "value": "3,4", "type": "Text"}, {"isChecked": true, "key": "pack_ids", "value": "3,4", "type": "Text"}, {"isChecked": true, "key": "body_part_ids", "value": "3,4", "type": "Text"}, {"isChecked": false, "key": "", "value": "", "type": "Text"}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"}, {"id": "b7031931-d679-4564-8a5a-cdb95b0c7a8d", "url": "127.0.0.1:3000/api/product", "name": "127.0.0.1:3000/api/product", "createdTime": "18-Dec-2025 14:19:36", "method": "delete", "params": [{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth": {"authType": "noauth", "userName": "", "password": "", "addTo": "queryParams", "showPwd": false, "tokenPrefix": "", "aws": {"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "Content-Type", "value": "application/json", "isChecked": true, "isFixed": false}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "raw", "formdata": [{"isChecked": false, "key": "", "value": ""}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "{\r\n \id\r\n : 48\r\n}", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"}, {"id": "2c6b49dc-0897-42c0-8a60-c6fcd27e1e11", "url": "127.0.0.1:3000/api/product", "name": "127.0.0.1:3000/ap
```



```

i/product", "createdTime": "17-Dec-2025 09:19:47", "method": "post", "params":
[{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth":
{"authType": "noauth", "userName": "", "password": "", "addTo": "queryparams", "sh
owPwd": false, "tokenPrefix": "", "aws":
{"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToke
n": ""}}, "headers": [{"isChecked": true, "key": "Content-
Type", "value": "multipart/form-data", "isFixed": false},
{"key": "", "value": "", "isChecked": false}], "body":
{"bodyType": "formdata", "formdata":
[{"isChecked": true, "key": "name", "value": "nouveau product 3"},
{"isChecked": true, "key": "image", "value": "image .jpg", "type": "Text"},
{"isChecked": true, "key": "description", "value": "desc nouveau product
", "type": "Text"},
{"isChecked": true, "key": "price", "value": "10", "type": "Text"},
{"isChecked": true, "key": "category_id", "value": "1", "type": "Text"},
{"isChecked": true, "key": "skin_ids", "value": "1,2", "type": "Text"},
{"isChecked": true, "key": "body_part_ids", "value": "1,2", "type": "Text"},
{"isChecked": true, "key": "pack_ids", "value": "1,2", "type": "Text"},
{"isChecked": false, "key": "", "value": "", "type": "Text"}], "urlencoded":
[{"isChecked": false, "key": "", "value": ""}], "raw":
{"data": "", "lang": "json"}, "binary": {"fileName": "", "data":
{}}, "contentTypeOption": "manual", "graphql":
{"query": "", "variables": ""}}, "tests":
[{"parameter": "", "action": "", "expectedValue": ""}], "setvar":
[{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironm
entId": "5cfb74f5-0183-401a-a720-9780a2b57fd4", {"id": "a177c61d-ee44-4994-
948f-
fc99f49323d6", "url": "127.0.0.1:3000/api/product/7", "name": "127.0.0.1:3000/
api/product/7", "createdTime": "05-Dec-2025
15:19:21", "method": "get", "params":
[{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth":
{"authType": "noauth", "userName": "", "password": "", "addTo": "queryparams", "sh
owPwd": false, "tokenPrefix": "", "aws":
{"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToke
n": ""}}, "headers": [{"key": "Accept", "value": "*//*", "isChecked": true},
{"key": "User-Agent", "value": "Flashpost", "isChecked": true},
{"key": "", "value": "", "isChecked": false}], "body":
{"bodyType": "none", "formdata":
[{"isChecked": false, "key": "", "value": ""}], "urlencoded":
[{"isChecked": false, "key": "", "value": ""}], "raw":
{"data": "", "lang": "json"}, "binary": {"fileName": "", "data":
{}}, "contentTypeOption": "manual", "graphql":
{"query": "", "variables": ""}}, "tests":
[{"parameter": "", "action": "", "expectedValue": ""}], "setvar":
[{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironm
entId": "5cfb74f5-0183-401a-a720-9780a2b57fd4", {"id": "c4ca3ced-fec0-4814-
96f5-
4bd0c0b34bed", "url": "127.0.0.1:3000/api/product", "name": "127.0.0.1:3000/ap
i/product", "createdTime": "05-Dec-2025 15:19:05", "method": "get", "params":
[{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth":
{"authType": "noauth", "userName": "", "password": "", "addTo": "queryparams", "sh
owPwd": false, "tokenPrefix": "", "aws":
{"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToke
n": ""}}, "headers": [{"key": "Accept", "value": "*//*", "isChecked": true},

```

```
{
 "key": "User-Agent", "value": "Flashpost", "isChecked": true,
 "key": "", "value": "", "isChecked": false,
 "body": {
 "bodyType": "none", "formdata": {
 "isChecked": false, "key": "", "value": ""
 }, "urlencoded": {
 "isChecked": false, "key": "", "value": ""
 }, "raw": {
 "data": "", "lang": "json", "binary": {
 "fileName": "", "data": {}
 }, "contentTypeOption": "manual", "graphql": {
 "query": "", "variables": ""
 }, "tests": {
 "parameter": "", "action": "", "expectedValue": ""
 }, "setvar": {
 "parameter": "", "key": "", "variableName": ""
 }, "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"
 }
 }
}
```

FICHER : code/\_flashpost/flashpost-collection-role.json

```
{
 "app": "Flashpost", "id": "56fc6175-1f58-456f-b478-f4068571010b", "name": "role", "version": "1.0", "type": "collections", "createdAt": "05-Dec-2025 12:54:38", "exportedDate": "05-Dec-2025 12:57:50", "collections": [
 {
 "id": "56fc6175-1f58-456f-b478-f4068571010b", "parent": "0", "text": "role", "createdAt": "05-Dec-2025 12:54:38", "draggable": true, "data": {
 "treeNodeType": "collection", "variableId": "", "relativeIndex": 0,
 "id": "50293731-cdbc-4eef-a4a7-23406b494626", "parent": "56fc6175-1f58-456f-b478-f4068571010b", "text": "127.0.0.1:3000/api/role/1", "createdAt": "05-Dec-2025 13:55:26", "draggable": false, "data": {
 "id": "57399b58-95ca-482e-a09c-272a3fa41fe7", "method": "get", "name": "127.0.0.1:3000/api/role/1", "url": "127.0.0.1:3000/api/role/1", "createdAt": "05-Dec-2025 13:55:26", "$loki": 5, "treeNodeType": "request"
 }
 }, {
 "id": "8c4982af-b0e4-4784-ae83-676d3182d87c", "parent": "56fc6175-1f58-456f-b478-f4068571010b", "text": "127.0.0.1:3000/api/role", "createdAt": "05-Dec-2025 13:54:16", "draggable": false, "data": {
 "id": "b8f097a1-15ba-4935-9ddc-864c49927d4a", "method": "get", "name": "127.0.0.1:3000/api/role", "url": "127.0.0.1:3000/api/role", "createdAt": "05-Dec-2025 13:54:16", "$loki": 4, "treeNodeType": "request"
 }
 }
], "requests": [
 {
 "id": "50293731-cdbc-4eef-a4a7-23406b494626", "url": "127.0.0.1:3000/api/role/1", "name": "127.0.0.1:3000/api/role/1", "createdAt": "05-Dec-2025 13:55:26", "method": "get", "params": {
 "isChecked": false, "key": "", "value": ""
 }, "pathParams": [], "auth": {
 "authType": "noauth", "userName": "", "password": "", "addTo": "queryParams", "showPwd": false, "tokenPrefix": "", "aws": {
 "service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""
 }
 }, "headers": [
 {
 "key": "Accept", "value": "*/*", "isChecked": true
 },
 {
 "key": "User-Agent", "value": "Flashpost", "isChecked": true
 },
 {
 "key": "", "value": "", "isChecked": false
 }
], "body": {
 "bodyType": "none", "formdata": {
 "isChecked": false, "key": "", "value": ""
 }, "urlencoded": {
 "isChecked": false, "key": "", "value": ""
 }, "raw": {
 "data": "", "lang": "json", "binary": {
 "fileName": "", "data": {}
 }, "contentTypeOption": "manual", "graphql": {
 "query": "", "variables": ""
 }, "tests": {

```

```
[{"parameter":"","action":"","expectedValue":""},"setvar":
[{"parameter":"","key":"","variableName":""},"notes":"","selectedEnvironm
entId":"5cfb74f5-0183-401a-a720-9780a2b57fd4"}, {"id":"8c4982af-b0e4-4784-
ae83-
676d3182d87c","url":"127.0.0.1:3000/api/role","name":"127.0.0.1:3000/api/r
ole","createdTime":"05-Dec-2025 13:54:16","method":"get","params":
[{"isChecked":false,"key":"","value":""},"pathParams":[],"auth":
{"authType":"noauth","userName":"","password":"","addTo":"queryparams","sh
owPwd":false,"tokenPrefix":"","aws":
{"service":"","region":"","accessKey":"","secretAccessKey":"","sessionToke
n":""}}, {"headers":[{"key":"Accept","value":"*/*","isChecked":true},
{"key":"User-Agent","value":"Flashpost","isChecked":true},
{"key":"","value":"","isChecked":false}],"body":
{"bodyType":"none","formdata":
[{"isChecked":false,"key":"","value":""},"urlencoded":
[{"isChecked":false,"key":"","value":""},"raw":
{"data":"","lang":"json"},"binary":{"fileName":"","data":
{},"contentTypeOption":"manual"},"graphql":
{"query":"","variables":""},"tests":
[{"parameter":"","action":"","expectedValue":""},"setvar":
[{"parameter":"","key":"","variableName":""},"notes":"","selectedEnvironm
entId":"5cfb74f5-0183-401a-a720-9780a2b57fd4"}]]}
```

FICHER : code/\_flashpost/flashpost-collection-skin.json

```
{"app":"Flashpost","id":"ca3fda4a-c6da-49c4-8d25-
9f082e9b41b7","name":"skin","version":"1.0","type":"collections","createdT
ime":"05-Dec-2025 14:07:23","exportedDate":"05-Dec-2025
14:08:03","collections":[{"id":"ca3fda4a-c6da-49c4-8d25-
9f082e9b41b7","parent":"0","text":"skin","createdTime":"05-Dec-2025
14:07:23","draggable":true,"data":
{"treeNodeType":"collection","variableId":"","relativeIndex":0},
{"id":"bd4747dd-aed2-4b63-ae15-2c94229cda94","parent":"ca3fda4a-c6da-49c4-
8d25-9f082e9b41b7","text":"127.0.0.1:3000/api/skin/3","createdTime":"05-
Dec-2025 15:06:53","draggable":false,"data":{"id":"40b38ee2-51cd-4f67-
9272-
46ce7ee8dbc2","method":"get","name":"127.0.0.1:3000/api/skin/3","url":"127
.0.0.1:3000/api/skin/3","createdTime":"05-Dec-2025
15:06:53","$loki":9,"treeNodeType":"request"}}, {"id":"ba924c8c-9d2c-4e83-
b07a-94e20736bbcf","parent":"ca3fda4a-c6da-49c4-8d25-
9f082e9b41b7","text":"127.0.0.1:3000/api/skin","createdTime":"05-Dec-2025
15:06:14","draggable":false,"data":{"id":"d642455b-7ee4-4a5a-9950-
a4b32f56758f","method":"get","name":"127.0.0.1:3000/api/skin","url":"127.0
.0.1:3000/api/skin","createdTime":"05-Dec-2025
15:06:14","$loki":8,"treeNodeType":"request"}]]},"requests":
[{"id":"bd4747dd-aed2-4b63-ae15-
2c94229cda94","url":"127.0.0.1:3000/api/skin/3","name":"127.0.0.1:3000/api
/skin/3","createdTime":"05-Dec-2025 15:06:53","method":"get","params":
[{"isChecked":false,"key":"","value":""},"pathParams":[],"auth":
{"authType":"noauth","userName":"","password":"","addTo":"queryparams","sh
```



```

owPwd": false, "tokenPrefix": "", "aws":
{"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "none", "formdata": [{"isChecked": false, "key": "", "value": ""}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"}, {"id": "ba924c8c-9d2c-4e83-b07a-94e20736bbcf", "url": "127.0.0.1:3000/api/skin", "name": "127.0.0.1:3000/api/skin", "createdTime": "05-Dec-2025 15:06:14", "method": "get", "params": [{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth": {"authType": "noauth", "userName": "", "password": "", "addTo": "queryparams", "showPwd": false, "tokenPrefix": "", "aws": {"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToken": ""}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true}, {"key": "User-Agent", "value": "Flashpost", "isChecked": true}, {"key": "", "value": "", "isChecked": false}], "body": {"bodyType": "none", "formdata": [{"isChecked": false, "key": "", "value": ""}], "urlencoded": [{"isChecked": false, "key": "", "value": ""}], "raw": {"data": "", "lang": "json"}, "binary": {"fileName": "", "data": {}}, "contentTypeOption": "manual", "graphql": {"query": "", "variables": ""}, "tests": [{"parameter": "", "action": "", "expectedValue": ""}], "setvar": [{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironmentId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"}]}

```

FICHER : code/\_flashpost/flashpost-collection-user.json

```

{"app": "Flashpost", "id": "28f0a610-03ad-46b9-9ed5-a916d659558d", "name": "user", "version": "1.0", "type": "collections", "createdTime": "15-Dec-2025 11:43:21", "exportedDate": "15-Dec-2025 11:43:43", "collections": [{"id": "28f0a610-03ad-46b9-9ed5-a916d659558d", "parent": "0", "text": "user", "createdTime": "15-Dec-2025 11:43:21", "draggable": true, "data": {"treeNodeType": "collection", "variableId": "", "relativeIndex": 0}, {"id": "e40e4cee-d331-4c7e-a890-10d1545385ee", "parent": "28f0a610-03ad-46b9-9ed5-a916d659558d", "text": "127.0.0.1:3000/api/user", "createdTime": "15-Dec-2025 12:42:14", "draggable": false, "data": {"id": "6e284180-86ed-4742-9085-e40fb31ec1a0", "method": "get", "name": "127.0.0.1:3000/api/user", "url": "127.0.0.1:3000/api/user", "createdTime": "15-Dec-2025 12:42:14", "$loki": 17, "treeNodeType": "request"}}, {"requests": [{"id": "e40e4cee-d331-4c7e-a890-

```

```

10d1545385ee", "url": "127.0.0.1:3000/api/user", "name": "127.0.0.1:3000/api/u
ser", "createdTime": "15-Dec-2025 12:42:14", "method": "get", "params":
[{"isChecked": false, "key": "", "value": ""}], "pathParams": [], "auth":
{"authType": "noauth", "userName": "", "password": "", "addTo": "queryParams", "sh
owPwd": false, "tokenPrefix": "", "aws":
{"service": "", "region": "", "accessKey": "", "secretAccessKey": "", "sessionToke
n": ""}}, "headers": [{"key": "Accept", "value": "*/*", "isChecked": true},
{"key": "User-Agent", "value": "Flashpost", "isChecked": true},
{"key": "", "value": "", "isChecked": false}], "body":
{"bodyType": "none", "formdata":
[{"isChecked": false, "key": "", "value": ""}], "urlencoded":
[{"isChecked": false, "key": "", "value": ""}], "raw":
{"data": "", "lang": "json"}, "binary": {"fileName": "", "data":
{}}, "contentTypeOption": "manual", "graphql":
{"query": "", "variables": ""}}, "tests":
[{"parameter": "", "action": "", "expectedValue": ""}], "setvar":
[{"parameter": "", "key": "", "variableName": ""}], "notes": "", "selectedEnvironm
entId": "5cfb74f5-0183-401a-a720-9780a2b57fd4"}]]

```

---

FICHER : code/models/body\_part.ts

```

import type { Product } from "../product";

// Reprendre strictement les noms des colonnes de la table SQL
type Body_part = {
 id: number;
 name: string;

 // liste concaténée des identifiants des produits
 body_part_ids: string;
 body_parts: Body_part[];

 product_ids: string;
 products: Product[];
};

export type { Body_part };

```

---

FICHER : code/models/category.ts

```

// Reprendre strictement les noms des colonnes de la table SQL
type Category = {
 id: number;
 name: string;
};

```

```
export type { Category };
```

---

FICHER : code/models/pack.ts

```
import type { Product } from "../product";

// Reprendre strictement les noms des colonnes de la table SQL
type Pack = {
 id: number;
 name: string;
 price: number;

 // liste concaténée des identifiants des produits
 product_ids: string;
 products: Product[];
};

export type { Pack };
```

---

FICHER : code/models/product.ts

```
import type { Body_part } from "../body_part";
import type { Category } from "../category";
import type { Pack } from "../pack";
import type { Skin } from "../skin";

// Reprendre strictement les noms des colonnes de la table SQL
type Product = {
 id: number;
 name: string;
 image: string;
 description: string;
 price: number;

 category: Category;
 category_id: number;

 skin_ids: string | string[];
 skins: Skin[];

 body_part_ids: string | string[];
 body_parts: Body_part[];

 pack_ids: string | string[];
 packs: Pack[];
```

```
};

export type { Product };
```

---

FICHER : code/models/role.ts

```
// Reprendre strictement les noms des colonnes de la table SQL
type Role = {
 id: number;
 name: string;
};

export type { Role };
```

---

FICHER : code/models/skin.ts

```
import type { Product } from "../product";

// Reprendre strictement les noms des colonnes de la table SQL
type Skin = {
 id: number;
 type: string;

 // liste concaténée des identifiants des produits
 skin_ids: string ;
 skins: Skin[];

 product_ids: string;
 products: Product[];
};

export type { Skin };
```

---

FICHER : code/models/user.ts

```
// Reprendre strictement les noms des colonnes de la table SQL
type User = {
 id: number;
 email: string;
 password: string;
};
```

```
export type { User };
```

---

FICHER : code/package.json

```
{
 "name": "@vitejs/plugin-rsc-examples-starter",
 "version": "0.0.0",
 "private": true,
 "license": "MIT",
 "type": "module",
 "scripts": {
 "dev": "vite",
 "build": "vite build",
 "preview": "vite preview",
 "server": "npx tsx watch --env-file .env.development server/index.ts"
 },
 "dependencies": {
 "@types/cors": "^2.8.19",
 "@types/express": "^5.0.6",
 "@types/multer": "^2.0.0",
 "cors": "^2.8.5",
 "express": "^5.2.1",
 "file-type": "^21.3.0",
 "multer": "^2.0.2",
 "mysql2": "^3.15.3",
 "react": "^19.2.0",
 "react-dom": "^19.2.0",
 "react-hook-form": "^7.70.0",
 "react-icons": "^5.5.0",
 "react-router": "^7.10.1",
 "tsx": "^4.21.0",
 "zod": "^4.3.5"
 },
 "devDependencies": {
 "@biomejs/biome": "2.3.8",
 "@types/react": "^19.2.7",
 "@types/react-dom": "^19.2.3",
 "@vitejs/plugin-react": "latest",
 "@vitejs/plugin-rsc": "latest",
 "rsc-html-stream": "^0.0.7",
 "vite": "^7.2.4"
 }
}
```

---

FICHER : code/server/controller/body\_part\_controller.ts

```

import type { Request, Response } from "express";
import Body_partRepository from "../repository/body_part_repository";

class Body_PartController {
 // Méthode relié à la route en GET située dans le routeur
 public index = async (_req: Request, res: Response) => {
 // Récupération des relultats de la rête
 const results = await new Body_partRepository().selectAll();

 // Si la requête renvoie une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === `production` ? `Error` :
results.message,
 });
 return; // Permet de stopper l'execution de la fonction
 }

 // renvoyer une reponse avec un code de status HTTP et au format
json
 res.status(200).json({
 status: 200,
 message: "OK",
 data: results,
 });
 };

 public selectOne = async (req: Request, res: Response) => {
 // récupérer la variable de route
 // req.params : récupère les variables de route
 console.log(req.params);
 // récupération des resultats de la requete
 const results = await new
Body_partRepository().selectOne(req.params);
 // Si la raquete renvoi une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === "production" ? "Error" :
results.message,
 });
 return;
 }
 // envoyer une réponse
 // res.send("coucou");
 // renvoyer une réponse avec un code de statut HTTP et au format
JSON
 res.status(200).json({
 status: 200,

```

```

 message: "ok body_part",
 data: results,
 });
};
}

export default Body_PartController;

```

---

FICHER : code/server/controller/category\_controller.ts

```

import type { Request, Response } from "express";
import CategoryRepository from "../repository/category_repository";

class CategoryController {
 // Méthode relié à la route en GET située dans le routeur
 public index = async (_req: Request, res: Response) => {
 // Récupération des relultats de la rêquete
 const results = await new CategoryRepository().selectAll();

 // Si la requête renvoie une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === `production` ? `Error` :
results.message,
 });
 return; // Permet de stopper l'execution de la fonction
 }

 // renvoyer une reponse avec un code de status HTTP et au format
json
 res.status(200).json({
 status: 200,
 message: "OK",
 data: results,
 });
 };

 public selectOne = async (req: Request, res: Response) => {
 // récupérer la variable de route
 // req.params : récupère les variables de route
 console.log(req.params);
 // récupération des resultats de la requete
 const results = await new
CategoryRepository().selectOne(req.params);
 // Si la raquete renvoi une erreur
 if (results instanceof Error) {
 res.status(400).json({

```

```

 status: 400,
 message:
 process.env.NODE_ENV === "production" ? "Error" :
results.message,
 });
 return;
}
// envoyer une réponse
// res.send("coucou");
// renvoyer une réponse avec un code de statut HTTP et au format
JSON
res.status(200).json({
 status: 200,
 message: "ok category",
 data: results,
});
};
}

export default CategoryController;

```

---

FICHER : code/server/controller/homepage\_controller.ts

```

import type { Request, Response } from "express";

class HomepageController {
 // Méthode relié à la route en GET située dans le routeur
 public index = (_req: Request, res: Response) => {
 // renvoyer une reponse avec un code de status HTTP et au format
 json
 res.status(200).json({
 status: 200,
 message: "OK",
 });
 };
}

export default HomepageController;

```

---

FICHER : code/server/controller/pack\_controller.ts

```

import type { Request, Response } from "express";
import PackRepository from "../repository/pack_repository";

class PackController {

```



```

// Méthode relié à la route en GET située dans le routeur
public index = async (_req: Request, res: Response) => {
 // Récupération des relultats de la rêquete
 const results = await new PackRepository().selectAll();

 // Si la requête renvoie une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === `production` ? `Error` :
results.message,
 });
 return; // Permet de stopper l'execution de la fonction
 }

 // renvoyer une reponse avec un code de status HTTP et au format
json
 res.status(200).json({
 status: 200,
 message: "OK",
 data: results,
 });
};

public selectOne = async (req: Request, res: Response) => {
 // récupérer la variable de route
 // req.params : récupère les variables de route
 console.log(req.params);
 // récupération des resultats de la requete
 const results = await new PackRepository().selectOne(req.params);
 // Si la raquete renvoi une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === "production" ? "Error" :
results.message,
 });
 return;
 }
 // envoyer une réponse
 // res.send("coucou");
 // renvoyer une réponse avec un code de statut HTTP et au format
JSON
 res.status(200).json({
 status: 200,
 message: "ok pack",
 data: results,
 });
};
}

```

```
export default PackController;
```

FICHER : code/server/controller/product\_controller.ts

```
import type { Request, Response } from "express";
import ProductRepository from "../repository/product_repository";
import FileService from "../service/file_service";
import type { Product } from "../../models/product";

class ProductController {
 // Méthode relié à la route en GET située dans le routeur
 public index = async (_req: Request, res: Response) => {
 // Récupération des relultats de la requête
 const results = await new ProductRepository().selectAll();

 // Si la requête renvoie une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === `production` ? `Error` :
results.message,
 });
 return; // Permet de stopper l'execution de la fonction
 }

 // renvoyer une reponse avec un code de status HTTP et au format
 json
 res.status(200).json({
 status: 200,
 message: "OK",
 data: results,
 });
 };

 public selectOne = async (req: Request, res: Response) => {
 // récupérer la variable de route
 // req.params : récupère les variables de route
 console.log(req.params);
 // récupération des resultats de la requete
 const results = await new
ProductRepository().selectOne(req.params);
 // Si la raquete renvoi une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === "production" ? "Error" :
results.message,
```

```

 });
 return;
 }
 // envoyer une réponse
 // res.send("coucou");
 // renvoyer une réponse avec un code de statut HTTP et au format
JSON
 res.status(200).json({
 status: 200,
 message: "ok product",
 data: results,
 });
};

public insert = async (req: Request, res: Response) => {
 // req.files: permet de récupérer les fichiers transférés
 // console.log(req.files);
 // const file = req.files[0];
 const file = (req.files as Express.Multer.File[]).shift() as
Express.Multer.File;

 // Instancier le service de fichiers
 const fileService = new FileService();

 // Renommer le fichier transféré et on recupere le nom complet
avec extension
 const fullname = await fileService.rename(file);

 // req.body : récupérer les données contenues dans la requête HTTP
 // console.log(req.body);

 // insertion d'un enregistrement
 const results = await new ProductRepository().insert({...req.body,
image: fullname,});

 // Si la raquete renvoi une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === "production" ? "Error" :
results.message,
 });
 return;
 }
 // envoyer une réponse
 // res.send("coucou");
 // renvoyer une réponse avec un code de statut HTTP et au format
JSON
 res.status(201).json({
 status: 201,
 message: "created product",
 data: results,

```

```
});
};

public update = async (req: Request, res: Response) => {
 // req.body : récupérer les données contenues dans la requête HTTP
 // console.log(req.body);
 const file = (req.files as Express.Multer.File[]).shift() as
Express.Multer.File;

 // Instancier le service de fichiers
 const fileService = new FileService();

 let fullname: string;

 if (file) {
 // Renommer le fichier transféré et on recupere le nom complet
avec extension
 fullname = await fileService.rename(file);

 } else {
 fullname = (await new ProductRepository().selectOne(req.body) as
Product).image;
 }

 console.log(fullname);

 // modification d'un enregistrement
 const results = await new ProductRepository().update({...req.body,
image: fullname,});
 // console.log(file);

 // Si la raquete renvoi une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === "production" ? "Error" :
results.message,
 });
 return;
 }
 // envoyer une réponse
 // res.send("coucou");
 // renvoyer une réponse avec un code de statut HTTP et au format
JSON
 res.status(200).json({
 status: 200,
 message: "update product",
 data: results,
 });
 }
};
```

```

};

public delete = async (req: Request, res: Response) => {
 // req.body : récupérer les données contenues dans la requête HTTP
 console.log(req.body);

 // insertion d'un enregistrement
 const results = await new ProductRepository().delete(req.body);

 // Si la raquete renvoi une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === "production" ? "Error" :
results.message,
 });
 return;
 }
 // envoyer une réponse
 // res.send("coucou");
 // renvoyer une réponse avec un code de statut HTTP et au format
JSON
 res.status(201).json({
 status: 201,
 message: "delete product",
 data: results,
 });
};
}

export default ProductController;

```

---

FICHER : code/server/controller/role\_controller.ts

```

import type { Request, Response } from "express";
import RoleRepository from "../repository/role_repository";

class RoleController {
 // Méthode relié à la route en GET située dans le routeur
 public index = async (_req: Request, res: Response) => {
 // Récupération des relultats de la rêquete
 const results = await new RoleRepository().selectAll();

 // Si la requête renvoie une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:

```

```

 process.env.NODE_ENV === `production` ? `Error` :
results.message,
 });
 return; // Permet de stopper l'execution de la fonction
}

// renvoyer une reponse avec un code de status HTTP et au format
json
res.status(200).json({
 status: 200,
 message: "OK",
 data: results,
});
});

public selectOne = async (req: Request, res: Response) => {
 // récupérer la variable de route
 // req.params : récupère les variables de route
 console.log(req.params);
 // récupération des resultats de la requete
 const results = await new RoleRepository().selectOne(req.params);
 // Si la raquete renvoi une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === "production" ? "Error" :
results.message,
 });
 return;
 }
 // envoyer une réponse
 // res.send("coucou");
 // renvoyer une réponse avec un code de statut HTTP et au format
JSON
res.status(200).json({
 status: 200,
 message: "ok role",
 data: results,
});
});
}

export default RoleController;

```

---

FICHER : code/server/controller/skin\_controller.ts

```

import type { Request, Response } from "express";
import SkinRepository from "../repository/skin_repository";

```

```

class SkinController {
 // Méthode relié à la route en GET située dans le routeur
 public index = async (_req: Request, res: Response) => {
 // Récupération des relultats de la requête
 const results = await new SkinRepository().selectAll();

 // Si la requête renvoie une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === `production` ? `Error` :
results.message,
 });
 return; // Permet de stopper l'execution de la fonction
 }

 // renvoyer une reponse avec un code de status HTTP et au format
json
 res.status(200).json({
 status: 200,
 message: "OK skin",
 data: results,
 });
 };

 public selectOne = async (req: Request, res: Response) => {
 // récupérer la variable de route
 // req.params : récupère les variables de route
 console.log(req.params);
 // récupération des resultats de la requete
 const results = await new SkinRepository().selectOne(req.params);
 // Si la raquete renvoi une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === "production" ? "Error" :
results.message,
 });
 return;
 }
 // envoyer une réponse
 // res.send("coucou");
 // renvoyer une réponse avec un code de statut HTTP et au format
JSON
 res.status(200).json({
 status: 200,
 message: "ok skin",
 data: results,
 });
 };
}

```

```
export default SkinController;
```

FICHER : code/server/controller/user\_controller.ts

```
import type { Request, Response } from "express";
import UserRepository from "../repository/user_repository";

class UserController {
 // Méthode relié à la route en GET située dans le routeur
 public index = async (_req: Request, res: Response) => {
 // Récupération des résultats de la requête
 const results = await new UserRepository().selectAll();

 // Si la requête renvoie une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === `production` ? `Error` :
results.message,
 });
 return; // Permet de stopper l'exécution de la fonction
 }

 // renvoyer une réponse avec un code de status HTTP et au format
 json
 res.status(200).json({
 status: 200,
 message: "OK",
 data: results,
 });
 };

 public selectOne = async (req: Request, res: Response) => {
 // récupérer la variable de route
 // req.params : récupère les variables de route
 console.log(req.params);
 // récupération des résultats de la requête
 const results = await new UserRepository().selectOne(req.params);
 // Si la requête renvoie une erreur
 if (results instanceof Error) {
 res.status(400).json({
 status: 400,
 message:
 process.env.NODE_ENV === "production" ? "Error" :
results.message,
 });
 return;
 }
 };
}
```



```
 }
 // envoyer une réponse
 // res.send("coucou");
 // renvoyer une réponse avec un code de statut HTTP et au format
JSON
 res.status(200).json({
 status: 200,
 message: "ok user",
 data: results,
 });
};
}

export default UserController;
```

---

FICHER : code/server/core/server.ts

```
import express from "express";
import cors from "cors";
import Body_partRouter from "../router/body_part_router";
import CategoryRouter from "../router/category_router";
import HomepageRouter from "../router/homepage_router";
import PackRouter from "../router/pack_router";
import ProductRouter from "../router/product_router";
import RoleRouter from "../router/role_router";
import SkinRouter from "../router/skin_router";
import UserRouter from "../router/user_router";

class Server {
 // Propriétés
 private app = express();
 private router = express.Router();

 // constructeur
 constructor() {
 // middleware JSON permet de récupérer la propriété body en JSON
 // de la requête HTTP
 this.app.use(express.json());

 // intégrer le middleare CORS qui permet d'autoriser l'accès aux
 // ressources à des origines différents (protocoles, port, sous-domaine)
 this.app.use(cors({
 origin: process.env.ORIGINS?.split(",")
 }));

 // relier le router à l'application
 this.app.use(this.router);

 // appel des routeurs
```

```
 this.routersList();
 }

 // Méthodes
 // listes des routeurs
 private routersList = () => {
 // créer un préfixe à toutes les routes incluses dans un routeur
 this.router.use("/api", new HomepageRouter().getRoutes());
 this.router.use("/api/role", new RoleRouter().getRoutes());
 this.router.use("/api/category", new
CategoryRouter().getRoutes());
 this.router.use("/api/skin", new SkinRouter().getRoutes());
 this.router.use("/api/pack", new PackRouter().getRoutes());
 this.router.use("/api/body_part", new
Body_partRouter().getRoutes());
 this.router.use("/api/product", new ProductRouter().getRoutes());
 this.router.use("/api/user", new UserRouter().getRoutes());
 };

 // créer méthode pour démarrer le serveur
 public startServer = () => {
 return this.app;
 };
}

export default Server;
```

---

FICHER : code/server/index.ts

```
import Server from "../core/server";

// Créer le serveur (instanciation)
const server = new Server().startServer();

// Démarrer le serveur
// process.env.VAR : acceder aux variables d'environnement
server.listen(process.env.PORT);
```

---

FICHER : code/server/repository/body\_part\_repository.ts

```
import type { Body_part } from "../../models/body_part";
import type { Product } from "../../models/product";
import MySQLService from "../service/mysql_service";
import ProductRepository from "../product_repository";
```

```

class Body_partRepository {
 // Nom de la table SQL
 private table = "body_part";

 // Selectionner tous les enregistrements
 public selectAll = async (): Promise<Body_part[] | unknown> => {
 // connexion au server SQL
 const connection = await new MySQLService().connect();

 // requête SQL : SELECT role.* FROM secretsDeHamam_dev.body_part;
 /* const sql = `SELECT ${this.table}.*
 FROM ${process.env.MYSQL_DATABASE}.${this.table};`
 */

 // requete SQL
 /*
 SELECT
 body_part.*,
 GROUP_CONCAT(product_id) AS product_ids
 FROM
 secretsDeHamam_dev.body_part
 JOIN
 secretsDeHamam_dev.product_body_part
 ON
 product_body_part.body_part_id = body_part.id
 JOIN
 secretsDeHamam_dev.product
 ON
 product.id = product_body_part.product_id
 GROUP BY
 body_part.id `
 */

 const sql = `
 SELECT ${this.table}.*,
 GROUP_CONCAT(product_id) AS product_ids
 FROM ${process.env.MYSQL_DATABASE}.${this.table}
 JOIN ${process.env.MYSQL_DATABASE}.product_body_part
 ON product_body_part.body_part_id = ${this.table}.id
 JOIN ${process.env.MYSQL_DATABASE}.product
 ON product.id = product_body_part.product_id
 GROUP BY ${this.table}.id ;
 `;

 // Try / Catch : récupérer les résultats de la requête ou un erreur
 try {
 // Execution de la requete
 const [query] = await connection.execute(sql);

 for (let i = 0; i < (query as Body_part[]).length; i++) {
 // Récupérer le resultat
 const result = (query as Body_part[])[i] as Body_part;
 }
 }
 }
}

```

```

 // tables de jointure
 result.products = (await new
ProductRepository().selectInList(
 result.product_ids,
)) as Product[];
 }

 // Retourner reultats
 return query;
} catch (error) {
 return error;
}
};

// Sélectionner un enregistrement
// data représente une partie des propriété du type
public selectOne = async (
 data: Partial<Body_part>,
): Promise<Body_part | unknown> => {
 // connexion au server MySQL
 const connection = await new MySQLService().connect();
 // requete SQL
 // where category.id=....
 // SELECT level.* FROM secretsDeHamam_dev;
 // variable de requete : précédée d'un :, suivi du nom de la
variable
 // requetes préparées :sécurité;(utilisation des varibales de
requetes)la requete est exécutée si elle ne représente pas de risque de
sécurité
 // const sql = `
 // SELECT ${this.table}.*
 // FROM ${process.env.MYSQL_DATABASE}.${this.table}
 // WHERE ${this.table}.id = :id;
 // `;

 const sql = `
 SELECT ${this.table}.*,
GROUP_CONCAT(product_id) AS product_ids
FROM ${process.env.MYSQL_DATABASE}.${this.table}
JOIN ${process.env.MYSQL_DATABASE}.product_body_part
ON product_body_part.body_part_id = ${this.table}.id
JOIN ${process.env.MYSQL_DATABASE}.product
ON product.id = product_body_part.product_id
WHERE ${this.table}.id = :id
GROUP BY ${this.table}.id ;
 `;

 // try / catch pour récupérer les resultats de la requete ou une
erreur
 try {
 // execution de la requete
 // si la requete possede des variables, utiliser le parametre
de la méthode
 const [query] = await connection.execute(sql, data);

```

```

 // récupérer le premier indice d'un array
 const result = (query as Body_part[]).shift() as Body_part;
 result.products = (await new ProductRepository().selectInList(
 result.product_ids,
)) as Product[];
 // retourner les résultats
 return result;
 } catch (error) {
 return error;
 }
};

public selectInList = async (list: string): Promise<Body_part[] |
unknown> => {
 // connexion au server SQL
 const connection = await new MySQLService().connect();

 // requête SQL : SELECT role.* FROM secretsDeHamman_dev.product;
 const sql = `SELECT ${this.table}.*
 FROM ${process.env.MYSQL_DATABASE}.${this.table}
 WHERE ${this.table}.id IN (${list})
 `;

 // Try / Catch : récupérer les résultats de la requête ou un erreur
 try {
 // Execution de la requete
 const [query] = await connection.execute(sql);

 // Retourner reultats
 return query;
 } catch (error) {
 return error;
 }
};
}

export default Body_partRepository;

```

---

FICHER : code/server/repository/category\_repository.ts

```

import type { Category } from "../../models/category";
import MySQLService from "../service/mysql_service";

class CategoryRepository {
 // Nom de la table SQL
 private table = "category";

 // Selectionner tous les enregistrements

```

```

public selectAll = async (): Promise<Category[] | unknown> => {
 // connexion au server SQL
 const connection = await new MySQLService().connect();

 // requête SQL : SELECT role.* FROM secretsDeHamam_dev.catégorie;
 const sql = `SELECT ${this.table}.*
 FROM ${process.env.MYSQL_DATABASE}.${this.table}`;

 // Try / Catch : récupérer les résultats de la requête ou un erreur
 try {
 // Execution de la requete
 const [query] = await connection.execute(sql);

 // Retourner reultats
 return query;
 } catch (error) {
 return error;
 }
};

// Sélectionner un enregistrement
// data représente une partie des propriété du type
public selectOne = async (
 data: Partial<Category>,
): Promise<Category | unknown> => {
 // connexion au server MySQL
 const connection = await new MySQLService().connect();
 // requete SQL
 // where category.id=....
 // SELECT level.* FROM secretsDeHamam_dev;
 // variable de requete : précédée d'un :, suivi du nom de la
variable
 // requetes préparées :sécurité;(utilisation des varibales de
requetes)la requete est exécutée si elle ne représente pas de risque de
sécurité
 const sql = `
SELECT ${this.table}.*
FROM ${process.env.MYSQL_DATABASE}.${this.table}
WHERE ${this.table}.id = :id;
`;
 // try / catch pour récupérer les resultats de la requete ou une
erreur
 try {
 // execution de la requete
 // si la requete possede des variables, utiliser le parametre
de la méthode
 const [query] = await connection.execute(sql, data);
 // récupérer le premier indice d'un array
 const result = (query as Category[]).shift();
 // retourner les résultats
 return result;
 } catch (error) {
 return error;
 }
}

```

```

 };
}

export default CategoryRepository;

```

FICHER : code/server/repository/pack\_repository.ts

```

import type { Pack } from "../../models/pack";
import type { Product } from "../../models/product";
import MySQLService from "../service/mysql_service";
import ProductRepository from "../product_repository";

class PackRepository {
 // Nom de la table SQL
 private table = "pack";

 // Selectionner tous les enregistrements
 public selectAll = async (): Promise<Pack[] | unknown> => {
 // connexion au server SQL
 const connection = await new MySQLService().connect();

 /* requête SQL : SELECT role.* FROM secretsDeHamam_dev.pack;
 const sql = `SELECT ${this.table}.*
 FROM ${process.env.MYSQL_DATABASE}.${this.table};`;
 */

 // requete SQL
 /*
 SELECT
 pack.*,
 GROUP_CONCAT(product_id) AS product_ids
 FROM
 secretsDeHamam_dev.pack
 JOIN
 secretsDeHamam_dev.product_pack
 ON
 product_pack.pack_id = pack.id
 JOIN
 secretsDeHamam_dev.product
 ON
 product.id = product_pack.product_id
 GROUP BY
 pack.id `
 */

 const sql = `
 SELECT ${this.table}.*,
 GROUP_CONCAT(product_id) AS product_ids
 FROM ${process.env.MYSQL_DATABASE}.${this.table}

```

```

JOIN ${process.env.MYSQL_DATABASE}.product_pack
ON product_pack.pack_id = ${this.table}.id
JOIN ${process.env.MYSQL_DATABASE}.product
ON product.id = product_pack.product_id
GROUP BY ${this.table}.id ;
 `;

// Try / Catch : récupérer les résultats de la requête ou un erreur
try {
 // Execution de la requete
 const [query] = await connection.execute(sql);

 for (let i = 0; i < (query as Pack[]).length; i++) {
 // Récupérer le resultat
 const result = (query as Pack[])[i] as Pack;

 // tables de jointure
 result.products = (await new
ProductRepository().selectInList(
 result.product_ids,
)) as Product[];
 }

 // Retourner reultats
 return query;
} catch (error) {
 return error;
}
};

// Sélectionner un enregistrement
// data représente une partie des propriété du type
public selectOne = async (data: Partial<Pack>): Promise<Pack |
unknown> => {
 // connexion au server MySQL
 const connection = await new MySQLService().connect();
 // requete SQL
 // where category.id=....
 // SELECT level.* FROM secretsDeHamam_dev;
 // variable de requete : précédée d'un :, suivi du nom de la
variable
 // requetes préparées :sécurité;(utilisation des varibales de
requetes)la requete est exécutée si elle ne représente pas de risque de
sécurité
 // const sql = `
 // SELECT ${this.table}.*
 // FROM ${process.env.MYSQL_DATABASE}.${this.table}
 // WHERE ${this.table}.id = :id;
 // `;

 const sql = `
 SELECT ${this.table}.*,
 GROUP_CONCAT(product_id) AS product_ids

```



```

FROM ${process.env.MYSQL_DATABASE}.${this.table}
JOIN ${process.env.MYSQL_DATABASE}.product_pack
ON product_pack.pack_id = ${this.table}.id
JOIN ${process.env.MYSQL_DATABASE}.product
ON product.id = product_pack.product_id
WHERE ${this.table}.id = :id
GROUP BY ${this.table}.id ;
`;

// try / catch pour récupérer les resultats de la requete ou une
erreur
try {
 // execution de la requete
 // si la requete possede des variables, utiliser le parametre
de la méthode
 const [query] = await connection.execute(sql, data);
 // récupérer le premier indice d'un array
 const result = (query as Pack[]).shift() as Pack;
 result.products = (await new ProductRepository().selectInList(
 result.product_ids,
)) as Product[];
 // retourner les résultats
 return result;
} catch (error) {
 return error;
}

};

public selectInList = async (list: string): Promise<Pack[] | unknown>
=> {
 // connexion au server SQL
 const connection = await new MySQLService().connect();

 // requête SQL : SELECT role.* FROM secretsDeHamam_dev.product;
 const sql = `SELECT ${this.table}.*
 FROM ${process.env.MYSQL_DATABASE}.${this.table}
 WHERE ${this.table}.id IN (${list})
 `;

 // Try / Catch : récupérer les résultats de la requête ou un erreur
 try {
 // Execution de la requete
 const [query] = await connection.execute(sql);

 // Retourner reultats
 return query;
 } catch (error) {
 return error;
 }
};

```

```
}

export default PackRepository;
```

---

FICHER : code/server/repository/product\_repository.ts

```
import type { QueryResult } from "mysql2";
import type { Category } from "../../models/category";
import type { Product } from "../../models/product";
import MySQLService from "../service/mysql_service";
import CategoryRepository from "../category_repository";
import SkinRepository from "../skin_repository";
import type { Skin } from "../../models/skin";
import PackRepository from "../pack_repository";
import Body_partRepository from "../body_part_repository";
import type { Body_part } from "../../models/body_part";
import type { Pack } from "../../models/pack";

class ProductRepository {
 // Nom de la table SQL
 private table = "product";

 // Selectionner tous les enregistrements
 public selectAll = async (): Promise<Product[] | unknown> => {
 // connexion au server SQL
 const connection = await new MySQLService().connect();

 // requête SQL : SELECT role.* FROM secretsDeHamman_dev.product;
 const sql = `SELECT ${this.table}.*,
 GROUP_CONCAT(DISTINCT skin.id) AS skin_ids,
 GROUP_CONCAT(DISTINCT pack.id) AS pack_ids,
 GROUP_CONCAT(DISTINCT body_part.id) AS body_part_ids
 FROM ${process.env.MYSQL_DATABASE}.${this.table}

 JOIN ${process.env.MYSQL_DATABASE}.product_skin
 ON product_skin.product_id = product.id
 JOIN ${process.env.MYSQL_DATABASE}.skin
 ON product_skin.skin_id = skin.id

 JOIN ${process.env.MYSQL_DATABASE}.product_pack
 ON product_pack.product_id = product.id
 JOIN ${process.env.MYSQL_DATABASE}.pack
 ON product_pack.pack_id = pack.id

 JOIN ${process.env.MYSQL_DATABASE}.product_body_part
 ON product_body_part.product_id = product.id
 JOIN ${process.env.MYSQL_DATABASE}.body_part
 ON product_body_part.body_part_id = body_part.id`
 }
}
```

```

 GROUP BY ${this.table}.id;
 `;

 // Try / Catch : récupérer les résultats de la requête ou un erreur
 try {
 // Execution de la requete
 const [query] = await connection.execute(sql);

 // Boucler sur les resultats pour recuperer les objets en
 relation (composition en Programmation Orientée Objet)
 for (let i = 0; i < (query as Product[]).length; i++) {
 // Récupérer le resultat
 const result = (query as Product[])[i] as Product;

 // Cles etrangère
 result.category = (await new
CategoryRepository().selectOne({
 id: result.category_id,
 })) as Category;

 // tables de jointure
 result.skins = (await new SkinRepository().selectInList(
 result.skin_ids as string
)) as Skin[];

 }

 // Retourner reultats
 return query;
 } catch (error) {
 return error;
 }
};

// Sélectionner un enregistrement
// data représente une partie des propriété du type
public selectOne = async (
 data: Partial<Product>,
): Promise<Product | unknown> => {
 // connexion au server MySQL
 const connection = await new MySQLService().connect();
 // requete SQL
 // where category.id=....
 // SELECT level.* FROM secretsDeHamam_dev;
 // variable de requete : précédée d'un :, suivi du nom de la
variable
 // requetes préparées :sécurité;(utilisation des varibales de
requetes)la requete est exécutée si elle ne représente pas de risque de
sécurité
 // DISTINCT: Evite les doublons
 const sql = `SELECT ${this.table}.*,
 GROUP_CONCAT(DISTINCT skin.id) AS skin_ids,
 GROUP_CONCAT(DISTINCT pack.id) AS pack_ids,

```

```

GROUP_CONCAT(DISTINCT body_part.id) AS body_part_ids
FROM ${process.env.MYSQL_DATABASE}.${this.table}

JOIN ${process.env.MYSQL_DATABASE}.product_skin
ON product_skin.product_id = product.id
LEFT JOIN ${process.env.MYSQL_DATABASE}.skin
ON product_skin.skin_id = skin.id

JOIN ${process.env.MYSQL_DATABASE}.product_pack
ON product_pack.product_id = product.id
LEFT JOIN ${process.env.MYSQL_DATABASE}.pack
ON product_pack.pack_id = pack.id

JOIN ${process.env.MYSQL_DATABASE}.product_body_part
ON product_body_part.product_id = product.id
JOIN ${process.env.MYSQL_DATABASE}.body_part
ON product_body_part.body_part_id = body_part.id

WHERE ${this.table}.id = :id
GROUP BY ${this.table}.id;
`;

// try / catch pour récupérer les résultats de la requête ou une
erreur
try {
 // execution de la requête
 // si la requête possède des variables, utiliser le paramètre
de la méthode
 const [query] = await connection.execute(sql, data);
 // récupérer le premier indice d'un array
 const result = (query as Product[]).shift() as Product;

 // Cles étrangère
 result.category = (await new CategoryRepository().selectOne({
 id: result.category_id,
 })) as Category;

 // tables de jointure
 result.skins = (await new SkinRepository().selectInList(
 result.skin_ids as string
)) as Skin[];

 result.packs = (await new PackRepository().selectInList(
 result.pack_ids as string
)) as Pack[];

 result.body_parts = (await new
Body_partRepository().selectInList(
 result.body_part_ids as string
)) as Body_part[];

 // retourner les résultats
 return result;

```

```

 } catch (error) {
 return error;
 }
 };

 // Selectionner plusieurs éléments dans une liste
 public selectInList = async (list: string): Promise<Product[] |
unknown> => {
 // connexion au server SQL
 const connection = await new MySQLService().connect();

 // requête SQL : SELECT role.* FROM secretsDeHamam_dev.product;
 const sql = `SELECT ${this.table}.*
 FROM ${process.env.MYSQL_DATABASE}.${this.table}
 WHERE ${this.table}.id IN (${list})
 `;

 // Try / Catch : récupérer les résultats de la requête ou un erreur
 try {
 // Execution de la requete
 const [query] = await connection.execute(sql);

 // Boucler sur les resultats pour recuperer les objets en
 relation (composition en Programmation Orientée Objet)
 for (let i = 0; i < (query as Product[]).length; i++) {
 // Récupérer le resultat
 const result = (query as Product[])[i] as Product;

 // Cles étrangère
 result.category = (await new
CategoryRepository().selectOne({
 id: result.category_id,
 })) as Category;
 }

 // Retourner reultats
 return query;
 } catch (error) {
 return error;
 }
 };

 // Insérer un enregistrement
 public insert = async (
 data: Partial<Product>,
): Promise<QueryResult | unknown> => {
 // connexion au server SQL
 const connection = await new MySQLService().connect();

 // requête SQL : SELECT role.* FROM secretsDeHamam_dev.product;
 let sql = `
 INSERT INTO ${process.env.MYSQL_DATABASE}.${this.table}
 VALUE (

```

```

 NULL,
 :name,
 :image,
 :description,
 :price,
 :category_id
);
`;

// Try / Catch : récupérer les résultats de la requête ou un erreur
try {
 // Demarrer une transaction
 connection.beginTransaction();

 // Execution de la première requete
 await connection.execute(sql, data);

 // Seconde Requete
 sql = `SET @id = LAST_INSERT_ID();`;
 await connection.execute(sql, data);

 // Troisième requete
 /*
 1,2,3
 >>>
 INSERT INTO secretsDeHamman_dev.product_pack
 VALUES
 (1, @id),
 (2, @id),
 (3, @id)
 ;

 split: extraire les données d'une chaine de caractères en
array
 1,2,3 >>>> [1,2,3]
 map
 [1,2,3] >> [(1, @id), (2, @id), (3, @id)]
 join
 [(1, @id), (2, @id), (3, @id)] >> (1, @id), (2, @id), (3,
@id)

 */

 let joinIds = (data.skin_ids as string)
 ?.split(',')
 .map((value) => `(@id, ${value})`)
 .join();

 sql = `
 INSERT INTO
 ${process.env.MYSQL_DATABASE}.product_skin
 VALUES
 ${joinIds}
 ;
 `;

```

```

 await connection.execute(sql, data);

 joinIds = (data.pack_ids as string)
 ?.split(',')
 .map((value) => `(@id, ${value})`)
 .join();

 sql = `
 INSERT INTO
 ${process.env.MYSQL_DATABASE}.product_pack
 VALUES
 ${joinIds}
 `;

 await connection.execute(sql, data);

 joinIds = (data.body_part_ids as string)
 ?.split(',')
 .map((value) => `(@id, ${value})`)
 .join();

 sql = `
 INSERT INTO
 ${process.env.MYSQL_DATABASE}.product_body_part
 VALUES
 ${joinIds}
 `;

 // query doit etre placer sur la dernière requete
 const [query] = await connection.execute(sql);

 // Valider transaction SQL
 connection.commit();

 // Retourner reultats
 return query;
 } catch (error) {
 // Annuler une transaction
 connection.rollback();

 return error;
 }
};

public update = async (
 data: Partial<Product>,
): Promise<QueryResult | unknown> => {
 // connexion au server SQL
 const connection = await new MySQLService().connect();

 // requête SQL : SELECT role.* FROM secretsDeHamam_dev.product;

```

```
let sql = `
 Update
 ${process.env.MYSQL_DATABASE}.${this.table}
 set
 ${this.table}.name = :name,
 ${this.table}.image = :image,
 ${this.table}.description = :description,
 ${this.table}.price = :price,
 ${this.table}.category_id = :category_id
 where
 ${this.table}.id = :id;
`;

// Try / Catch : récupérer les résultats de la requête ou un erreur
try {
 // Demarrer une transaction
 connection.beginTransaction();

 // Execution de la première requete
 await connection.execute(sql, data);

 sql = `
 DELETE FROM
 ${process.env.MYSQL_DATABASE}.product_skin
 WHERE
 product_skin.product_id = :id;
 `;
 await connection.execute(sql, data);

 sql = `
 DELETE FROM
 ${process.env.MYSQL_DATABASE}.product_pack
 WHERE
 product_pack.product_id = :id;
 `;
 await connection.execute(sql, data);

 sql = `
 DELETE FROM
 ${process.env.MYSQL_DATABASE}.product_body_part
 WHERE
 product_body_part.product_id = :id;
 `;
 await connection.execute(sql, data);

 let joinIds = (data.skin_ids as string)
 ?.split(',')
 .map((value) => `(:id, ${value})`)
 .join();

 sql = `
 INSERT INTO
 ${process.env.MYSQL_DATABASE}.product_skin
 VALUES
```



```

 ${joinIds}
 `;

 await connection.execute(sql, data);

 joinIds = (data.pack_ids as string)
 ?.split(',')
 .map((value) => `(:id, ${value})`)
 .join();

 sql = `
 INSERT INTO
 ${process.env.MYSQL_DATABASE}.product_pack
 VALUES
 ${joinIds}
 `;

 await connection.execute(sql, data);

 joinIds = (data.body_part_ids as string)
 ?.split(',')
 .map((value) => `(:id, ${value})`)
 .join();

 sql = `
 INSERT INTO
 ${process.env.MYSQL_DATABASE}.product_body_part
 VALUES
 ${joinIds}
 `;

 // // Seconde Requete
 // sql = `
 // DELETE FROM
 // ${process.env.MYSQL_DATABASE}.${this.table}
 // WHERE
 // ${this.table}.id = :id;
 // `;

 // // Seconde Requete
 // sql = `SET @id = LAST_INSERT_ID();`;
 // await connection.execute(sql);

 // Troisième requete
 /*
 1,2,3
 >>>
 INSERT INTO secretsDeHammam_dev.product_pack
 VALUES
 (1, @id),
 (2, @id),

```

```

 (3, @id)
 ;

 split: extraire les données d'une chaîne de caractères en
array
 1,2,3 >>>> [1,2,3]
 map
 [1,2,3] >> [(1, @id), (2, @id), (3, @id)]
 join
 [(1, @id), (2, @id), (3, @id)] >> (1, @id), (2, @id), (3,
@id)

 */

 // const joinIds = data.skin_ids
 // ?.split(',')
 // .map((value) => `(@id, ${value})`)
 // .join();

 // sql = `
 // INSERT INTO
 // ${process.env.MYSQL_DATABASE}.product_skin
 // VALUES
 // ${joinIds}
 // ;
 // `;

 // query doit être placé sur la dernière requête
 const [query] = await connection.execute(sql, data);

 // Valider transaction SQL
 connection.commit();

 // Retourner résultats
 return query;
 } catch (error) {
 // Annuler une transaction
 connection.rollback();

 return error;
 }
};

public delete = async (
 data: Partial<Product>,
): Promise<QueryResult | unknown> => {
 // connexion au serveur SQL
 const connection = await new MySQLService().connect();

 // requête SQL : SELECT role.* FROM secretsDeHamam_dev.product;
 let sql = `
 DELETE FROM
 ${process.env.MYSQL_DATABASE}.product_skin
 WHERE
 product_skin.product_id = :id;
 `;

```

```
`;

// Try / Catch : récupérer les résultats de la requête ou un erreur
try {
 // Demarrer une transaction
 connection.beginTransaction();

 // Execution de la première requete
 await connection.execute(sql, data);

 sql = `
 DELETE FROM
 ${process.env.MYSQL_DATABASE}.product_pack
 WHERE
 product_pack.product_id = :id;
 `;
 await connection.execute(sql, data);

 sql = `
 DELETE FROM
 ${process.env.MYSQL_DATABASE}.product_body_part
 WHERE
 product_body_part.product_id = :id;
 `;
 await connection.execute(sql, data);

 // // Seconde Requete
 sql = `
 DELETE FROM
 ${process.env.MYSQL_DATABASE}.${this.table}
 WHERE
 ${this.table}.id = :id;
 `;

 // query doit etre placer sur la dernière requete
 const [query] = await connection.execute(sql, data);

 // Valider transaction SQL
 connection.commit();

 // Retourner reultats
 return query;
} catch (error) {
 // Annuler une transaction
 connection.rollback();

 return error;
}
};
}

export default ProductRepository;
```

FICHER : code/server/repository/role\_repository.ts

```
import type { Role } from "../../models/role";
import MySQLService from "../service/mysql_service";

class RoleRepository {
 // Nom de la table SQL
 private table = "role";

 // Selectionner tous les enregistrements
 public selectAll = async (): Promise<Role[] | unknown> => {
 // connexion au server SQL
 const connection = await new MySQLService().connect();

 // requête SQL : SELECT role.* FROM secretsDeHamam_dev.role;
 const sql = `SELECT ${this.table}.*
 FROM ${process.env.MYSQL_DATABASE}.${this.table};`;

 // Try / Catch : récupérer les résultats de la requête ou un erreur
 try {
 // Execution de la requete
 const [query] = await connection.execute(sql);

 // Retourner reultats
 return query;
 } catch (error) {
 return error;
 }
 };

 // Sélectionner un enregistrement
 // data représente une partie des propriété du type
 public selectOne = async (data: Partial<Role>): Promise<Role |
unknown> => {
 // connexion au server MySQL
 const connection = await new MySQLService().connect();
 // requete SQL
 // where category.id=....
 // SELECT level.* FROM secretsDeHamam_dev;
 // variable de requete : précédée d'un :, suivi du nom de la
variable
 // requetes préparées :sécurité;(utilisation des varibales de
requetes)la requete est exécutée si elle ne représente pas de risque de
sécurité
 const sql = `
 SELECT ${this.table}.*
 FROM ${process.env.MYSQL_DATABASE}.${this.table}
 WHERE ${this.table}.id = :id;
 `;
 // try / catch pour récupérer les resultats de la requete ou une

```

```

 erreur
 try {
 // execution de la requete
 // si la requete possede des variables, utiliser le parametre
de la méthode
 const [query] = await connection.execute(sql, data);
 // récupérer le premier indice d'un array
 const result = (query as Role[]).shift();
 // retourner les résultats
 return result;
 } catch (error) {
 return error;
 }
};
}

export default RoleRepository;

```

---

FICHER : code/server/repository/skin\_repository.ts

```

import type { Product } from "../../models/product";
import type { Skin } from "../../models/skin";
import MySQLService from "../service/mysql_service";
import ProductRepository from "../product_repository";

class SkinRepository {
 // Nom de la table SQL
 private table = "skin";

 // Selectionner tous les enregistrements
 public selectAll = async (): Promise<Skin[] | unknown> => {
 // connexion au server SQL
 const connection = await new MySQLService().connect();

 /* requête SQL : SELECT role.* FROM secretsDeHamam_dev.skin;
const sql = `SELECT ${this.table}.*
 FROM ${process.env.MYSQL_DATABASE}.${this.table}`;
 */

 // requete SQL
 /*
 SELECT
 skin.*,
 GROUP_CONCAT(product_id) AS product_ids
 FROM
 secretsDeHamam_dev.skin
 JOIN
 secretsDeHamam_dev.product_skin
 ON

```

```

 product_skin.skin_id = skin.id
 JOIN
 secretsDeHamam_dev.product
 ON
 product.id = product_skin.product_id
 GROUP BY
 skin.id `
 */

 const sql = `
 SELECT ${this.table}.*,
 GROUP_CONCAT(product_id) AS product_ids
 FROM ${process.env.MYSQL_DATABASE}.${this.table}
 JOIN ${process.env.MYSQL_DATABASE}.product_skin
 ON product_skin.skin_id = ${this.table}.id
 JOIN ${process.env.MYSQL_DATABASE}.product
 ON product.id = product_skin.product_id
 GROUP BY ${this.table}.id ;
 `;

 // Try / Catch : récupérer les résultats de la requête ou un erreur
 try {
 // Execution de la requete
 const [query] = await connection.execute(sql);

 for (let i = 0; i < (query as Skin[]).length; i++) {
 // Récuperer le resultat
 const result = (query as Skin[])[i] as Skin;

 // tables de jointure
 result.products = (await new
ProductRepository().selectInList(
 result.product_ids,
)) as Product[];
 }

 // Retourner reultats
 return query;
 } catch (error) {
 return error;
 }
};

// Sélectionner un enregistrement
// data représente une partie des propriété du type
public selectOne = async (data: Partial<Skin>): Promise<Skin |
unknown> => {
 // connexion au server MySQL
 const connection = await new MySQLService().connect();
 // requete SQL
 // where category.id=....
 // SELECT level.* FROM secretsDeHamam_dev;
 // variable de requete : précédée d'un :, suivi du nom de la

```

```

variable
 // requetes préparées :sécurité;(utilisation des varibales de
 requetes)la requete est exécutée si elle ne représente pas de risque de
 sécurité
 // const sql = `
 // SELECT ${this.table}.*
 // FROM ${process.env.MYSQL_DATABASE}.${this.table}
 // WHERE ${this.table}.id = :id;
 // `;

 const sql = `
 SELECT ${this.table}.*,
 GROUP_CONCAT(product_id) AS product_ids
 FROM ${process.env.MYSQL_DATABASE}.${this.table}
 JOIN ${process.env.MYSQL_DATABASE}.product_skin
 ON product_skin.skin_id = ${this.table}.id
 JOIN ${process.env.MYSQL_DATABASE}.product
 ON product.id = product_skin.product_id
 WHERE ${this.table}.id = :id
 GROUP BY ${this.table}.id ;
 `;

 // try / catch pour récupérer les resultats de la requete ou une
 erreur
 try {
 // execution de la requete
 // si la requete possede des variables, utiliser le parametre
 de la méthode
 const [query] = await connection.execute(sql, data);
 // récupérer le premier indice d'un array
 const result = (query as Skin[]).shift() as Skin;
 result.products = (await new ProductRepository().selectInList(
 result.product_ids,
)) as Product[];
 // retourner les résultats
 return result;
 } catch (error) {
 return error;
 }
};

public selectInList = async (list: string): Promise<Skin[] | unknown>
=> {
 // connexion au server SQL
 const connection = await new MySQLService().connect();

 // requête SQL : SELECT role.* FROM secretsDeHamman_dev.product;
 const sql = `SELECT ${this.table}.*
 FROM ${process.env.MYSQL_DATABASE}.${this.table}
 WHERE ${this.table}.id IN (${list})
 `;

```

```

 // Try / Catch : récupérer les résultats de la requête ou un erreur
 try {
 // Execution de la requete
 const [query] = await connection.execute(sql);

 // Retourner reultats
 return query;
 } catch (error) {
 return error;
 }
 };
}

export default SkinRepository;

```

---

FICHER : code/server/repository/user\_repository.ts

```

import type { User } from "../../models/user";
import MySQLService from "../service/mysql_service";

class UserRepository {
 // Nom de la table SQL
 private table = "user";

 // Selectionner tous les enregistrements
 public selectAll = async (): Promise<User[] | unknown> => {
 // connexion au server SQL
 const connection = await new MySQLService().connect();

 // requête SQL : SELECT role.* FROM secretsDeHamam_dev.pack;
 const sql = `SELECT ${this.table}.*
 FROM ${process.env.MYSQL_DATABASE}.${this.table};`;

 // Try / Catch : récupérer les résultats de la requête ou un erreur
 try {
 // Execution de la requete
 const [query] = await connection.execute(sql);

 // Retourner reultats
 return query;
 } catch (error) {
 return error;
 }
 };

 // Sélectionner un enregistrement
 // data représente une partie des propriété du type
 public selectOne = async (data: Partial<User>): Promise<User |
unknown> => {

```



```

 // connexion au server MySQL
 const connection = await new MySQLService().connect();
 // requete SQL
 // where category.id=....
 // SELECT level.* FROM secretsDeHamam_dev;
 // variable de requete : précédée d'un :, suivi du nom de la
variable
 // requetes préparées :sécurité;(utilisation des varibales de
requetes)la requete est exécutée si elle ne représente pas de risque de
sécurité
 const sql = `
 SELECT ${this.table}.*
 FROM ${process.env.MYSQL_DATABASE}.${this.table}
 WHERE ${this.table}.id = :id;
 `;
 // try / catch pour récupérer les resultats de la requete ou une
erreur
 try {
 // execution de la requete
 // si la requete possede des variables, utiliser le parametre
de la méthode
 const [query] = await connection.execute(sql, data);
 // récupérer le premier indice d'un array
 const result = (query as User[]).shift();
 // retourner les résultats
 return result;
 } catch (error) {
 return error;
 }
};
}

export default UserRepository;

```

---

FICHER : code/server/router/body\_part\_router.ts

```

import express from "express";
import Body_PartController from "../controller/body_part_controller";

class Body_partRouter {
 // router express
 private router = express.Router();

 // liste des routes
 public getRoutes = () => {
 // Créer une route en GET simple
 this.router.get("/", new Body_PartController().index);

 // variable de route: précédé par ;; suivi du nom de la variable

```

```
 // appel 1
 this.router.get("/:id", new Body_PartController().selectOne);

 // retourner le routeur
 return this.router;
 };
}

export default Body_partRouter;
```

---

FICHER : code/server/router/category\_router.ts

```
import express from "express";
import CategoryController from "../controller/category_controller";

class CategoryRouter {
 // router express
 private router = express.Router();

 // liste des routes
 public getRoutes = () => {
 // Créer une route en GET simple
 this.router.get("/", new CategoryController().index);

 // variable de route: précédé par ;; suivi du nom de la variable
 // appel 1
 this.router.get("/:id", new CategoryController().selectOne);

 // retourner le routeur
 return this.router;
 };
}

export default CategoryRouter;
```

---

FICHER : code/server/router/homepage\_router.ts

```
import express from "express";
import HomepageController from "../controller/homepage_controller";

class HomepageRouter {
 // router express
 private router = express.Router();

 // liste des routes
```

```
 public getRoutes = () => {
 // Créer une route en GET simple
 this.router.get("/", new HomepageController().index);

 // retourner le routeur
 return this.router;
 };
}

export default HomepageRouter;
```

---

FICHER : code/server/router/pack\_router.ts

```
import express from "express";
import PackController from "../controller/pack_controller";

class PackRouter {
 // router express
 private router = express.Router();

 // liste des routes
 public getRoutes = () => {
 // Créer une route en GET simple
 this.router.get("/", new PackController().index);

 // variable de route: précédé par ;; suivi du nom de la variable
 // appel 1
 this.router.get("/:id", new PackController().selectOne);

 // retourner le routeur
 return this.router;
 };
}

export default PackRouter;
```

---

FICHER : code/server/router/product\_router.ts

```
import express from "express";
import multer from "multer";
import ProductController from "../controller/product_controller";

class ProductRouter {
 // router express
 private router = express.Router();
```

```
// multer permet de gérer le transfert de fichier
// private multer = multer({ dest: "public" });
// Si fichier ranger dans sous dossier, modifier le chemin
private multer = multer({ dest:
`${process.env.PUBLIC_DIR}/img/products` });

// liste des routes
public getRoutes = () => {
 // Créer une route en GET simple
 this.router.get("/", new ProductController().index);

 // variable de route: précédé par ;; suivi du nom de la variable
 // appel 1
 this.router.get("/:id", new ProductController().selectOne);

 // middleware multer
 // Creation: inserer un enregistrement => Post
 this.router.post("/", this.multer.any(), new
ProductController().insert);
 // Modification: mettre à jour un enregistrement => Put
 this.router.put("/", this.multer.any(), new
ProductController().update);
 // Supprimer => Delete
 this.router.delete("/", new ProductController().delete);

 // retourner le routeur
 return this.router;
};
}

export default ProductRouter;
```

---

FICHER : code/server/router/role\_router.ts

```
import express from "express";
import RoleController from "../controller/role_controller";

class RoleRouter {
 // router express
 private router = express.Router();

 // liste des routes
 public getRoutes = () => {
 // Créer une route en GET simple
 this.router.get("/", new RoleController().index);

 // variable de route: précédé par ;; suivi du nom de la variable
 // appel 1
 }
}
```

```
 this.router.get("/:id", new RoleController().selectOne);

 // retourner le routeur
 return this.router;
 };
}

export default RoleRouter;
```

---

FICHER : code/server/router/skin\_router.ts

```
import express from "express";
import SkinController from "../controller/skin_controller";

class SkinRouter {
 // router express
 private router = express.Router();

 // liste des routes
 public getRoutes = () => {
 // Créer une route en GET simple
 this.router.get("/", new SkinController().index);

 // variable de route: précédé par ;; suivi du nom de la variable
 // appel 1
 this.router.get("/:id", new SkinController().selectOne);

 // retourner le routeur
 return this.router;
 };
}

export default SkinRouter;
```

---

FICHER : code/server/router/user\_router.ts

```
import express from "express";
import UserController from "../controller/user_controller";

class UserRouter {
 // router express
 private router = express.Router();

 // liste des routes
 public getRoutes = () => {
```

```

 // Créer une route en GET simple
 this.router.get("/", new UserController().index);

 // variable de route: précédé par ;; suivi du nom de la variable
 // appel 1
 this.router.get("/:id", new UserController().selectOne);

 // retourner le routeur
 return this.router;
 };
}

export default UserRouter;

```

---

FICHER : code/server/service/file\_service.ts

```

// fs = file system
import { fileTypeFromFile } from 'file-type';
import fs from 'node:fs/promises';

//
// [
// {
// fieldname: 'image',
// originalname: 'Argile_et_Rhassoul.png',
// encoding: '7bit',
// mimetype: 'image/png',
// destination: 'public/img/products',
// filename: 'bde194790217794769729886886c077b',
// path: 'public/img/products/bde194790217794769729886886c077b',
// size: 3468635
// }
//]

class FileService {

 // fonction qui renomme le fichier transferé en ajoutant l'extension
 // Cette fonction doit retourner le nom complet du fichier
 // C'est a nous de le faire pour des raisons de sécurité
 public rename = async (file: Express.Multer.File): Promise<string> =>
 {
 // Ajouter l'extension au fichier
 const fullname = `${file.filename}.${(await
fileTypeFromFile(file.path)).ext}`;
 console.log(fullname);
 // Renommer le fichier avec son nom complet
 await fs.rename(file.path, `${file.destination}/${fullname}`);
 // Retourner le nom complet
 return fullname;
 }
}

```

```
 }

 }

 export default FileService;
```

---

FICHER : code/server/service/mysql\_service.ts

```
import mysql, { type PoolConnection } from "mysql2/promise";

class MySQLService {
 // propriété statique : accessible sans objet
 private static instance: PoolConnection;

 // Créer fonction de connexion au server MySQL
 public connect = async () => {
 // si aucune connexion existe
 if (!MySQLService.instance) {
 MySQLService.instance = await mysql
 .createPool({
 host: process.env.MYSQL_HOST,
 user: process.env.MYSQL_USER,
 password: process.env.MYSQL_PASSWORD,
 database: process.env.MYSQL_DATABASE,
 namedPlaceholders: true,
 })
 .getConnection();
 }

 // Retourner la connexion
 return MySQLService.instance;
 };
}

export default MySQLService;
```

---

FICHER : code/src/assets/css/admin\_css/add.module.css

```
/* Conteneur principal pour centrer le formulaire */
.formContainer {
 max-width: 850px;
 margin: 0 auto;
 animation: fadeIn 0.5s ease-in-out;
}

/* En-tête du formulaire */
```

```
.header {
 display: flex;
 justify-content: space-between;
 align-items: center;
 margin-bottom: 2.5rem;
}

.header h1 {
 font-family: 'Playfair Display', serif;
 color: #4a3f35;
 margin: 0;
}

.header p {
 color: #8c725b;
 margin: 5px 0 0 0;
 font-style: italic;
}

/* Bouton Retour / Annuler */
.backBtn {
 text-decoration: none;
 color: #8c725b;
 border: 1px solid #eaddca;
 padding: 10px 20px;
 border-radius: 8px;
 font-size: 0.9rem;
 transition: all 0.3s ease;
 background: white;
}

.backBtn:hover {
 background: #fdf2f2;
 color: #d9534f;
 border-color: #fca5a5;
}

/* Le bloc formulaire */
.productForm {
 background: white;
 padding: 2.5rem;
 border-radius: 15px;
 box-shadow: 0 10px 30px rgba(0, 0, 0, 0.05);
 border: 1px solid #f3efe4;
}

/* Groupes de champs (Label + Input) */
.formGroup {
 display: flex;
 flex-direction: column;
 margin-bottom: 1.5rem;
}

.formGroup label {
```



```
 font-weight: 600;
 color: #645945;
 margin-bottom: 0.6rem;
 font-size: 0.95rem;
}

/* Style des Inputs, Select et Textarea */
.formGroup input,
.formGroup select,
.formGroup textarea {
 padding: 12px 15px;
 border: 1px solid #eaddca;
 border-radius: 8px;
 font-size: 1rem;
 color: #4a3f35;
 background-color: #fff;
 transition: border-color 0.3s ease, box-shadow 0.3s ease;
}

.formGroup input:focus,
.formGroup select:focus,
.formGroup textarea:focus {
 outline: none;
 border-color: #c5a059;
 box-shadow: 0 0 0 3px rgba(197, 160, 89, 0.1);
}

/* Mise en page sur deux colonnes pour Prix et Stock */
.formRow {
 display: grid;
 grid-template-columns: 1fr 1fr;
 gap: 20px;
}

/* Bouton de validation final */
.submitBtn {
 width: 100%;
 padding: 15px;
 background-color: #c5a059;
 color: white;
 border: none;
 border-radius: 8px;
 font-size: 1.1rem;
 font-weight: bold;
 cursor: pointer;
 margin-top: 1rem;
 transition: background 0.3s ease, transform 0.2s active;
}

.submitBtn:hover {
 background-color: #a68648;
}

.submitBtn:active {
```

```
 transform: scale(0.98);
 }

 /* Petit effet d'entrée */
 @keyframes fadeIn {
 from { opacity: 0; transform: translateY(10px); }
 to { opacity: 1; transform: translateY(0); }
 }

 /* Responsive pour les petits écrans */
 @media (max-width: 600px) {
 .formRow {
 grid-template-columns: 1fr;
 }
 }
}
```

---

FICHER : code/src/assets/css/admin\_css/admin\_product\_index.module.css

```
.crudContainer {
 padding: 20px;
 max-width: 1200px;
 margin: 0 auto;
}

.crudHeader {
 display: flex;
 justify-content: space-between;
 align-items: flex-end;
 margin-bottom: 30px;
 gap: 20px;
}

.addButton {
 background-color: #645945;
 color: white;
 padding: 12px 24px;
 border-radius: 8px;
 text-decoration: none;
 font-weight: bold;
 transition: all 0.3s;
 white-space: nowrap; /*Empêche le texte du bouton de revenir à la
ligne*/
 display: inline-block;
}

.addButton:hover {
 background-color: #c5a059;
 transform: translateY(-2px);
}
```

```
.tableWrapper {
 background: white;
 padding: 20px;
 border-radius: 12px;
 box-shadow: 0 4px 20px rgba(0,0,0,0.05);
 overflow-x: auto;
}

.crudTable {
 width: 100%;
 border-collapse: collapse;
}

.crudTable th {
 text-align: left;
 padding: 15px;
 border-bottom: 2px solid #f3efe4;
 color: #8c725b;
}

.crudTable td {
 padding: 15px;
 border-bottom: 1px solid #f3efe4;
}

.productName {
 font-weight: 600;
 color: #8c725b;
}

.actions {
 display: flex;
 gap: 10px;
}

.modif {
 color: #8c725b;
}

.suppl {
 color: red;
}

.editBtn, .deleteBtn {
 padding: 6px 12px;
 border: none;
 border-radius: 4px;
 cursor: pointer;
 font-size: 0.85rem;
}

.editBtn {
```

```
 background-color: #eaddca;
 color: #645945;
}

.deleteBtn {
 background-color: #fdf2f2;
 color: #d9534f;
}

.deleteBtn:hover {
 background-color: #d9534f;
 color: white;
}
```

---

FICHER : code/src/assets/css/admin\_css/sidebar.module.css

```
*{
 box-sizing: border-box;
}

.sidebar {
 width: 260px;
 height: 100vh;
 background-color: #645945;
 color: #f3efe4;
 text-decoration: none;
 font-family: 'Playfair Display', serif;
 font-size: 1.1rem;
 display: flex;
 flex-direction: column;
 position: fixed;
 left: 0;
 top: 0;
 padding: 2rem 1rem;
}

.logoSection {
 margin-bottom: 3rem;
 text-align: center;
}

.logoSection h2 {
 color: #c5a059;
 font-family: 'Playfair Display', serif;
 margin: 0;
}

.badge {
 font-size: 0.7rem;
 text-transform: uppercase;
 letter-spacing: 2px;
}
```

```
 color: #8c725b;
}

.navLinks {
 display: flex;
 flex-direction: column;
 gap: 0.8rem;
}

.link, .active {
 display: flex;
 align-items: center;
 padding: 12px 15px;
 text-decoration: none;
 border-radius: 8px;
 transition: 0.3s;
 color: #a89f91;
}

.link:hover {
 background: rgba(197, 160, 89, 0.1);
 color: #c5a059;
}

.active {
 background: #c5a059;
 color: white;
}

.icon {
 margin-right: 12px;
 font-size: 1.2rem;
}

.sidebarFooter {
 margin-top: auto;
 padding-top: 1rem;
 border-top: 1px solid #333;
}

.exitLink {
 background-color: #c5a059;
 color: white;
 display: flex; /* Aligne le texte comme les autres liens */
 align-items: center;
 padding: 12px 15px; /* Même espacement interne que .link */
 text-decoration: none;
 font-family: 'Playfair Display', serif;
 font-size: 1.1rem;
 border-radius: 8px;
 transition: 0.3s;
}

.exitLink:hover {
```

```
background-color: #d9534f; /* Devient rouge plein au survol */
color: white;
} /* Texte blanc au survol */
```

---

FICHER : code/src/assets/css/admin\_layout.module.css

```
/* Container principal */
.container {
 padding: 2rem;
 color: #4a3f35;
 background-color: #fcfaf7;
 min-height: 100%;
}

.adminLayout {
 display: flex;
 width: 100%;
 min-height: 100vh;
}

.mainContent {
 /* Épouse la largeur restante de l'écran */
 flex: 1;

 /* TRÈS IMPORTANT : Pousse le contenu pour qu'il commence
 après la sidebar qui fait 260px */
 margin-left: 260px;

 background-color: #fcfaf7; /* Fond clair pour le confort visuel */
 padding: 40px;
 box-sizing: border-box;
 min-height: 100vh;
}

/* En-tête de la page */
.header {
 margin-bottom: 3rem;
 border-bottom: 2px solid #eaddca;
 padding-bottom: 1rem;
}

.header h1 {
 font-family: 'Playfair Display', serif; /* Si tu l'as importée, sinon
serif */
 font-size: 2.2rem;
 color: #645945;
 margin: 0;
}

.header p {
 font-style: italic;
```

```
 color: #8c725b;
}

/* Grille de statistiques (les cartes) */
.statsGrid {
 display: grid;
 grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
 gap: 1.5rem;
 margin-bottom: 3rem;
}

.card {
 background: white;
 padding: 1.5rem;
 border-radius: 15px;
 box-shadow: 0 4px 15px rgba(0, 0, 0, 0.05);
 border: 1px solid #f3efe4;
 transition: transform 0.2s ease;
}

.card:hover {
 transform: translateY(-5px);
 border-color: #c5a059;
}

.card h3 {
 font-size: 1rem;
 text-transform: uppercase;
 letter-spacing: 1px;
 color: #8c725b;
 margin-bottom: 1rem;
}

.number {
 font-size: 2.5rem;
 font-weight: bold;
 color: #645945;
 margin: 0;
}

.warning {
 color: #b33a3a; /* Pour les alertes de stock */
}

/* Zone d'activité récente */
.recentActivity {
 background: white;
 padding: 2rem;
 border-radius: 15px;
 box-shadow: 0 4px 15px rgba(0, 0, 0, 0.05);
}

.recentActivity h2 {
 font-size: 1.4rem;
}
```

```
 color: #645945;
 margin-bottom: 1.5rem;
}

.recentActivity ul {
 list-style: none;
 padding: 0;
}

.recentActivity li {
 padding: 1rem 0;
 border-bottom: 1px solid #f3efe4;
 display: flex;
 justify-content: space-between;
 align-items: center;
}

.recentActivity li:last-child {
 border-bottom: none;
}

.date {
 font-size: 0.85rem;
 color: #a89f91;
 background: #f3efe4;
 padding: 4px 10px;
 border-radius: 20px;
}

/* Responsive */
@media (max-width: 768px) {
 .container {
 padding: 1rem;
 }

 .header h1 {
 font-size: 1.8rem;
 }
}
```

---

FICHER : code/src/assets/css/base.css

```
/* Polices */

/* Variables CSS */
:root {
 --background-color: #f3efe4;
 --padding-left: 2rem;
 --padding-right: 2rem;
 --margin: 10px 0 10px 0;
```



```
--color: #7e5835;
--font-weight: bold;
--font-style: italic;
--line-height: 32px;
}

/* Balises HTML */
* {
 box-sizing: border-box;
}

img {
 width: 100%;
}

body {
 background-color: var(--background-color);
 color: var(--color);
}
```

---

FICHER : code/src/assets/css/featured\_products.module.css

```
.featuredProducts {
 max-width: 1200px;
 margin: 100px auto;
 padding: 0 20px;
}

.sectionHeader {
 text-align: center;
 margin-bottom: 60px;
}

.sectionHeader span {
 color: #c5a059;
 text-transform: uppercase;
 letter-spacing: 3px;
 font-size: 0.85rem;
 font-weight: 700;
 display: block;
 margin-bottom: 10px;
}

.sectionHeader h2 {
 font-family: "Playfair Display", serif;
 color: #645945;
 font-size: 2.8rem;
 margin: 0;
}
```

```
.productGrid {
 display: grid;
 /* Création automatique de colonnes : minimum 280px par carte */
 grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
 gap: 50px 30px; /* 50px d'espace vertical, 30px horizontal */
}

/* Version Tablette / Mobile */
@media (max-width: 768px) {
 .sectionHeader h2 {
 font-size: 2rem;
 }
 .featuredProducts {
 margin: 60px auto;
 }
}
```

---

FICHER : code/src/assets/css/features.module.css

```
.featuresSection {
 display: grid;
 grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
 gap: 30px;
 max-width: 1200px;
 margin: -50px auto 80px;
 position: relative;
 z-index: 10;
 padding: 0 20px;
}

.featureCard {
 background: white;
 padding: 40px;
 border-radius: 20px;
 text-align: center;
 box-shadow: 0 20px 40px rgba(100, 89, 69, 0.1);
 transition:
 transform 0.4s ease,
 box-shadow 0.4s ease;
}

.featureCard:hover {
 transform: translateY(-8px);
 box-shadow: 0 30px 60px rgba(100, 89, 69, 0.15);
}

.icon {
 font-size: 2.5rem;
```

```
 display: block;
 margin-bottom: 15px;
}

.featureCard h3 {
 font-family: "Playfair Display", serif;
 color: #645945;
 margin-bottom: 10px;
}

.featureCard p {
 font-weight: 300;
 color: #8a7d6a;
 font-size: 0.95rem;
 margin: 0;
}

@media (max-width: 768px) {
 .featuresSection {
 margin: -30px auto 40px;
 grid-template-columns: 1fr;
 }
}
```

---

FICHER : code/src/assets/css/footer.module.css

```
.footer {
 background-color: #645945;
 color: #f3efe4;
 padding: 40px 20px;
 display: flex;
 flex-direction: column;
 align-items: center;
 gap: 30px;
 margin-top: 50px; /* Espace avec le contenu de la page */
}

/* Logo */
.logo {
 width: 100px;
 transition: transform 0.3s ease;
}

.logo img {
 width: 100%;
 height: auto;
 filter: brightness(0) invert(1); /* Rend le logo blanc pour ressortir
sur le marron */
}
```

```
.logo:hover {
 transform: scale(1.05);
}

/* Navigation */
.nav {
 width: 100%;
}

.menu {
 list-style: none;
 padding: 0;
 margin: 0;
 display: flex;
 flex-wrap: wrap; /* Pour mobile : les liens passent à la ligne */
 justify-content: center;
 gap: 20px;
}

.menu li a {
 color: #f3efe4;
 text-decoration: none;
 font-size: 0.9rem;
 text-transform: uppercase;
 letter-spacing: 1px;
 transition: color 0.3s ease;
}

.menu li a:hover {
 color: #c5a059;
}

/* Icône Sociale */
.icone {
 font-size: 1.8rem;
 color: #f3efe4;
 cursor: pointer;
 transition: all 0.3s ease;
}

.icone:hover {
 color: #c5a059;
 transform: translateY(-3px);
}

/* Responsive Ordinateur */
@media (min-width: 768px) {
 .footer {
 flex-direction: row;
 justify-content: space-between;
 padding: 40px 60px;
 }
}
```

```
.menu {
 gap: 30px;
}

.nav {
 width: auto;
}
}
```

---

FICHER : code/src/assets/css/form.module.css

```
.formWrapper {
 max-width: 500px;
 margin: 60px auto;
 padding: 40px;
 background: #ffffff;
 border-radius: 20px;
 /* Ombre douce pour un effet aérien */
 box-shadow: 0 15px 35px rgba(140, 114, 91, 0.1);
 border: 1px solid rgba(197, 160, 89, 0.1);
 position: relative;
 overflow: hidden;
}

/* Petit motif floral ou organique en fond (optionnel) */
.formWrapper::after {
 content: "";
 position: absolute;
 top: -50px;
 right: -50px;
 width: 150px;
 height: 150px;
 background: rgba(197, 160, 89, 0.05);
 border-radius: 50%;
}

.formWrapper h2 {
 font-family: 'Playfair Display', serif;
 color: #5d6d5e; /* Vert sauge profond */
 text-align: center;
 font-size: 2rem;
 margin-bottom: 30px;
 font-weight: 600;
}

.orientalForm {
 display: flex;
 flex-direction: column;
 gap: 20px;
}
```

```
/* Style des labels s'ils sont ajoutés, ou placeholder */
.orientalForm input,
.orientalForm textarea {
 font-family: 'Poppins', sans-serif;
 width: 100%;
 padding: 14px;
 border: 1px solid #e0e0e0;
 border-radius: 10px;
 background-color: #faf9f7; /* Couleur sable très clair */
 font-size: 0.95rem;
 transition: all 0.4s ease;
 color: #4a4a4a;
}

.orientalForm input:focus,
.orientalForm textarea:focus {
 border-color: #c5a059; /* Or brossé */
 background-color: #ffffff;
 box-shadow: 0 4px 12px rgba(197, 160, 89, 0.1);
 outline: none;
}

/* Style spécifique pour le bouton "Rituel de Beauté" */
.orientalForm button {
 font-family: 'Poppins', sans-serif;
 background: linear-gradient(135deg, #8c725b 0%, #a68b74 100%);
 color: white;
 padding: 16px;
 border: none;
 border-radius: 50px; /* Bouton arrondi pour la douceur */
 font-size: 1rem;
 font-weight: 500;
 letter-spacing: 1.5px;
 text-transform: uppercase;
 cursor: pointer;
 transition: transform 0.2s, box-shadow 0.2s;
 margin-top: 10px;
}

.orientalForm button:hover {
 transform: translateY(-2px);
 box-shadow: 0 7px 20px rgba(140, 114, 91, 0.3);
}

/* Texte d'aide ou liens sous le formulaire */
.formFooter {
 text-align: center;
 margin-top: 20px;
 font-family: 'Poppins', sans-serif;
 font-size: 0.85rem;
 color: #8c725b;
}
```

```
/*TEMPORAIRE*/

.groupTitle {
 font-family: 'Playfair Display', serif;
 font-size: 0.85rem;
 color: #8c725b;
 font-weight: 600;
 text-transform: uppercase;
 letter-spacing: 1px;
 margin-bottom: 10px;
}

/* Style spécifique pour le SELECT dans le formGroup */
.formGroup select {
 width: 100%;
 border: none;
 background: transparent;
 font-family: 'Poppins', sans-serif;
 font-size: 1rem;
 color: #4a4a4a;
 cursor: pointer;
 outline: none;
 padding: 5px 0;
}

/* Conteneur pour aligner les checkbox proprement */
.checkboxGrid {
 display: grid;
 grid-template-columns: repeat(auto-fill, minmax(140px, 1fr));
 gap: 10px;
 padding-top: 5px;
}

/* Style de chaque ligne de checkbox */
.checkboxItem {
 display: flex;
 align-items: center;
 gap: 10px;
 cursor: pointer;
 font-size: 0.9rem;
 color: #4a4a4a;
 transition: color 0.2s;
}

.checkboxItem:hover {
 color: #c5a059;
}

.checkboxItem input[type="checkbox"] {
 width: auto; /* Empêche la checkbox de prendre 100% de largeur */
 accent-color: #c5a059; /* Colore la case en doré */
}
```

```
 cursor: pointer;
}
```

---

FICHER : code/src/assets/css/header.module.css

```
.header {
 display: flex;
 justify-content: space-between;
 align-items: center;
 padding: 10px 40px;
 background-color: #ffffff;
 box-shadow: 0 2px 10px rgba(0, 0, 0, 0.05);
 position: sticky;
 top: 0;
 z-index: 1000;
}

h1 {
 display: none;
}

/* RESPONSIVE ECRAN ORDINATEUR
@media only screen and (min-width: 769px) {
 .header {
 }
} */
```

---

FICHER : code/src/assets/css/hero.module.css

```
.hero {
 height: 80vh;
 display: flex;
 align-items: center;
 justify-content: center;
 background:
 linear-gradient(rgba(0, 0, 0, 0.4), rgba(0, 0, 0, 0.4)),
 url("/img/products/Hammam.png") center / cover no-repeat;
 color: #645945;
 text-align: center;
 border-radius: 0 0 40px 40px;
}

.heroContent {
 color: white;
}
```



```
 max-width: 800px;
 padding: 20px;
 position: relative;
 z-index: 5;
 }

.heroContent h2 {
 font-family: "Playfair Display", serif;
 font-size: 4rem;
 font-weight: 700;
 margin-bottom: 25px;
 letter-spacing: 2px;
 line-height: 1.1;
 text-shadow: 0 2px 15px rgba(0, 0, 0, 0.2);
}

.heroContent p {
 font-size: 1.2rem;
 font-weight: 300;
 line-height: 1.8;
 margin-bottom: 45px;
 max-width: 650px;
 margin: 0 auto 45px;
 color: rgba(255, 255, 255, 0.9);
}

.ctaButton {
 background-color: #c5a059;
 color: white;
 padding: 15px 40px;
 border-radius: 50px;
 text-decoration: none;
 font-weight: bold;
 display: inline-block;
 transition: 0.3s ease;
}

.ctaButton:hover {
 background-color: white;
 color: #c5a059;
}

@media (max-width: 768px) {
 .hero {
 height: 70vh;
 border-radius: 0 0 30px 30px;
 }
 .heroContent h2 {
 font-size: 2.2rem;
 }
}
```

FICHER : code/src/assets/css/homepage.module.css

```
.homeWrapper {
 background-color: #fffdf9;
}

.hero {
 height: 80vh;
 display: flex;
 align-items: center;
 justify-content: center;
 background:
 linear-gradient(rgba(0, 0, 0, 0.4), rgba(0, 0, 0, 0.4)),
 url("/img/products/Hammam.png") center / cover no-repeat;
 color: #645945;
 text-align: center;
 border-radius: 0 0 40px 40px;
 margin-bottom: 0;
}

.heroContent {
 color: white;
 max-width: 800px;
 padding: 20px;
 position: relative;
 z-index: 5;
}

.heroContent h2 {
 font-family: "Playfair Display", serif;
 font-size: 4rem;
 font-weight: 700;
 margin-bottom: 25px;
 letter-spacing: 2px;
 line-height: 1.1;
 text-align: center;
 text-shadow: 0 2px 15px rgba(0, 0, 0, 0.2);
}

.heroContent p {
 font-size: 1.2rem;
 font-weight: 300;
 line-height: 1.8;
 margin-bottom: 45px;
 max-width: 650px;
 margin-left: auto;
 margin-right: auto;
 color: rgba(255, 255, 255, 0.9);
}

.ctaButton {
```

```
 background-color: #c5a059;
 color: white;
 padding: 15px 40px;
 border-radius: 50px;
 text-decoration: none;
 font-weight: bold;
 display: inline-block;
 transition: 0.3s ease;
}

.ctaButton:hover {
 background-color: white;
}

.featuresSection {
 display: grid;
 grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
 gap: 30px;
 max-width: 1200px;
 margin: -50px auto 80px;
 position: relative;
 z-index: 10;
 padding: 0 20px;
}

.featureCard {
 background: white;
 padding: 40px;
 border-radius: 20px;
 text-align: center;
 box-shadow: 0 20px 40px rgba(100, 89, 69, 0.1);
 transition:
 transform 0.4s ease,
 box-shadow 0.4s ease;
 position: relative;
 z-index: 10;
}

.featureCard p {
 font-weight: 300;
 color: #8a7d6a;
 font-size: 0.95rem;
}

.featureCard:hover {
 transform: translateY(-8px);
 box-shadow: 0 30px 60px rgba(100, 89, 69, 0.15);
}

.icon {
 font-size: 2.5rem;
 display: block;
 margin-bottom: 15px;
}
```

```
.featureCard h3 {
 font-family: "Playfair Display", serif;
 color: #645945;
 margin-bottom: 10px;
}

.shopPreview {
 padding: 100px 20px;
 text-align: center;
 background-color: #f3efe4;
}

.secondaryLink {
 color: #645945;
 text-decoration: none;
 font-weight: bold;
 border-bottom: 2px solid #c5a059;
 padding-bottom: 5px;
}

.featuredProducts {
 max-width: 1200px;
 margin: 100px auto;
 padding: 0 20px;
}

.sectionHeader {
 text-align: center;
 margin-bottom: 60px;
}

.sectionHeader span {
 color: #c5a059;
 text-transform: uppercase;
 letter-spacing: 3px;
 font-size: 0.9rem;
 font-weight: bold;
}

.sectionHeader h2 {
 font-family: "Playfair Display", serif;
 color: #645945;
 font-size: 2.5rem;
 margin-top: 10px;
}

.productGrid {
 display: grid;
 grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
 gap: 40px;
}

.productCard {
```

```
 text-align: center;
 transition: 0.3s;
}

.imageContainer {
 background-color: #f9f6f0;
 border-radius: 15px;
 padding: 40px;
 margin-bottom: 20px;
 position: relative;
 overflow: hidden;
}

.imageContainer img {
 width: 100%;
 height: 300px;
 object-fit: contain; /* Garde les proportions du flacon/produit */
 transition: transform 0.5s ease;
}

.productCard:hover img {
 transform: scale(1.05);
}

.badge {
 position: absolute;
 top: 15px;
 right: 15px;
 background-color: #645945;
 color: white;
 padding: 5px 12px;
 font-size: 0.7rem;
 border-radius: 20px;
 text-transform: uppercase;
}

.productCard h4 {
 font-family: "Playfair Display", serif;
 font-size: 1.3rem;
 color: #333;
 margin-bottom: 8px;
}

.productCard p {
 color: #8a7d6a;
 font-weight: 300;
 margin-bottom: 20px;
}

.productLink {
 text-decoration: none;
 color: #c5a059;
 font-weight: bold;
 border-bottom: 1px solid transparent;
```

```
 transition: 0.3s;
 }

.productLink:hover {
 border-bottom: 1px solid #c5a059;
}

@media (max-width: 768px) {
 .hero {
 height: 70vh;
 border-radius: 0 0 30px 30px;
 }

 .heroContent h2 {
 font-size: 2.2rem;
 padding: 0 10px;
 }

 .heroContent p {
 font-size: 1rem;
 margin-bottom: 30px;
 padding: 0 15px;
 }

 .featuresSection {
 margin: -30px auto 40px;
 grid-template-columns: 1fr;
 gap: 20px;
 }

 .featureCard {
 padding: 30px 20px;
 }
}
```

---

FICHER : code/src/assets/css/icon\_navbar.module.css

```
.icon_navbar {
 display: flex;
 gap: 20px;
 align-items: center;
 color: #645945;
}

.icon_navbar svg {
 font-size: 1.3rem;
 cursor: pointer;
 transition: color 0.3s;
}
```

```
.icon_navbar svg:hover {
 color: #c5a059;
}
```

---

FICHER : code/src/assets/css/logo.module.css

```
.logo {
 display: flex;
 justify-content: center;
 align-items: center;
 width: auto;
 height: 50px;
 transition: transform 0.3s ease;
}

.logo img {
 width: auto;
 height: 100%;
 display: block;
 object-fit: contain;
}

.logo h1 {
 display: none;
}
```

---

FICHER : code/src/assets/css/navbar.module.css

```
/* Polices */

/* Variables CSS */

/* Balises HTML */
/* MOBILE FIRST */

* {
 box-sizing: border-box;
}

.navContainer {
 display: flex;
 align-items: center;
}

.navContainer button {
```

```
 display: flex;
 flex-direction: column;
 gap: 0.35rem;
 background: none;
 border: none;
 cursor: pointer;
 padding: 10px;
 z-index: 1100;
 }

 .line {
 width: 1.8rem;
 height: 2px;
 background-color: #645945;
 transition:
 transform 0.3s ease,
 opacity 0.3s ease;
 }

 .one {
 transform-origin: left top;
 transform: rotate(45deg) translate(0px, -3px);
 }
 .two {
 opacity: 0;
 }
 .three {
 transform-origin: left bottom;
 transform: rotate(-45deg) translate(0px, 3px);
 }

 .navbar {
 position: fixed;
 top: 125px;
 left: 0;
 width: 250px;
 height: calc(100vh-125px);
 background-color: #645945;
 transition: transform 0.4s ease;
 z-index: 1050;
 box-shadow: 5px 0 15px rgba(0,0,0,0.1);
 }

 .nav-visible {
 transform: translateX(-100%);
 }

 .menu li a {
 display: block;
 padding: 15px 30px;
 color: #f3efe4;
 text-decoration: none;
 font-family: 'Playfair Display', serif;
 font-size: 1.1rem;
```



```
 transition: all 0.3s;
}

.menu li a:hover {
 background-color: rgba(197, 160, 89, 0.2);
 color: #c5a059;
 padding-left: 40px;
}

.adminLink {
 margin-top: 20px;
 border-top: 1px solid rgba(243, 239, 228, 0.2);
 font-weight: bold;
 color: #c5a059 !important;
}
```

---

FICHER : code/src/assets/css/product\_card.module.css

```
.productCard {
 display: flex;
 flex-direction: column;
 text-align: center;
 transition: transform 0.3s ease;
}

.imageContainer {
 background-color: #fcfaf7; /* Fond crème très doux */
 border-radius: 20px;
 margin-bottom: 20px;
 position: relative;
 overflow: hidden;
 aspect-ratio: 1 / 1; /* Force un carré parfait pour l'unité visuelle */
 /*
 display: flex;
 align-items: center;
 justify-content: center;
 */
}

.imageContainer img {
 width: 100%;
 height: 100%;
 object-fit: cover;
 transition: transform 0.6s cubic-bezier(0.33, 1, 0.68, 1);
}

/* Effet au survol de la carte */
.productCard:hover .imageContainer img {
 transform: scale(1.1);
}
```

```
.productInfo h4 {
 font-family: "Playfair Display", serif;
 font-size: 1.25rem;
 color: #2d2d2d;
 margin: 0 0 8px 0;
}

.productInfo p {
 color: #8a7d6a;
 font-weight: 300;
 font-size: 0.95rem;
 margin-bottom: 15px;
}

.productLink {
 text-decoration: none;
 color: #c5a059;
 font-weight: 700;
 font-size: 0.9rem;
 text-transform: uppercase;
 letter-spacing: 1px;
 display: inline-block;
 transition: color 0.3s ease;
}

.productLink:hover {
 color: #645945;
}
```

---

FICHER : code/src/assets/css/product\_skeleton.module.css

```
/* src/components/products/product_skeleton.module.css */
.skeletonCard {
 background: #fdfcf9;
 border-radius: 8px;
 overflow: hidden;
 border: 1px solid #eee;
}

.skeletonImage {
 width: 100%;
 height: 200px;
 background: #e0e0e0;
}

.skeletonInfo {
 padding: 15px;
 display: flex;
 flex-direction: column;
```

```
 gap: 10px;
 }

 .skeletonTitle,
 .skeletonPrice,
 .skeletonButton {
 background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0
75%);
 background-size: 200% 100%;
 animation: shimmer 1.5s infinite;
 border-radius: 4px;
 }

 .skeletonTitle {
 height: 20px;
 width: 80%;
 }
 .skeletonPrice {
 height: 15px;
 width: 40%;
 }
 .skeletonButton {
 height: 35px;
 width: 100%;
 margin-top: 10px;
 }

 @keyframes shimmer {
 0% {
 background-position: 200% 0;
 }
 100% {
 background-position: -200% 0;
 }
 }
}
```

---

FICHER : code/src/assets/css/products.module.css

```
.productsSection {
 padding: 60px 20px;
 max-width: 1200px;
 margin: 0 auto;
}

.mainTitle {
 text-align: center;
 font-family: "Playfair Display", serif;
 color: #645945;
 font-size: 2.5rem;
}
```

```
 margin-bottom: 40px;
 position: relative;
}

/* Petit trait décoratif sous le titre */
.mainTitle::after {
 content: "";
 display: block;
 width: 60px;
 height: 3px;
 background: #c5a059;
 margin: 10px auto;
}

/* Grille Responsive */
.productsGrid {
 display: grid;
 grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
 gap: 20px;
}

/* Carte Produit */
.productCard {
 background: white;
 border-radius: 15px;
 overflow: hidden;
 box-shadow: 0 4px 15px rgba(0, 0, 0, 0.05);
 transition:
 transform 0.3s ease,
 box-shadow 0.3s ease;
 display: flex;
 flex-direction: column;
}

.productCard:hover {
 transform: translateY(-10px);
 box-shadow: 0 10px 25px rgba(100, 89, 69, 0.1);
}

/* Image */
.imageWrapper {
 height: auto;
 width: 100%;
 background-color: #f3efe4;
}

.imageWrapper img {
 width: 100%;
 height: 250px;
 object-fit: cover;
 transition: transform 0.5s ease;
}

.productCard:hover .imageWrapper img {
```

```
 transform: scale(1.1);
 }

 /* Infos */
 .productInfo {
 padding: 15px 20px;
 text-align: left;
 align-items: stretch;
 display: flex;
 flex-direction: column;
 gap: 8px;
 }

 .productInfo h3 {
 font-family: "Playfair Display", serif;
 color: #4a3f35;
 font-size: 1.1rem;
 margin: 0;
 }

 .productInfo a {
 text-decoration: none;
 }

 .price {
 color: #c5a059;
 font-weight: bold;
 font-size: 1rem;
 }

 /* Bouton */
 .viewButton {
 margin-top: auto;
 background-color: #645945;
 color: white;
 border: none;
 padding: 8px;
 border-radius: 5px;
 cursor: pointer;
 transition: background 0.3s;
 font-family: "Playfair Display", serif;
 font-size: 0.9rem;
 }

 .viewButton:hover {
 background-color: #c5a059;
 }

 /* centre la fiche */
 .productDetailContainer {
 max-width: 600px;
 margin: 0 auto;
 }
}
```

```
.productDetailContainer .productCard {
 flex-direction: row-reverse;
}

.descriptionText {
 color: #4a3f35;
 font-size: 1rem;
 line-height: 1.7;
 padding: 15px 0;
 text-align: left;
}

.backLink {
 display: inline-block;
 margin-top: 20px;
 color: #645945;
 text-decoration: none;
 font-size: 0.9rem;
 font-family: "Playfair Display", serif;
 transition: color 0.3s;
}

.backLink:hover {
 color: #c5a059;
}

.descriptionText h2 {
 font-family: "Playfair Display", serif;
 color: #645945;
 font-size: 1.8rem;
 margin-top: 30px;
 border-bottom: 1px solid #f3efe4;
 padding-bottom: 10px;
}

.descriptionText h3 {
 font-family: "Playfair Display", serif;
 color: #c5a059;
 font-size: 1.3rem;
 margin-top: 25px;
 margin-bottom: 10px;
}

.descriptionText p {
 margin-bottom: 15px;
}

.descriptionText ul {
 list-style: none;
 padding-left: 10px;
 margin-bottom: 20px;
}

.descriptionText li {
```

```
 position: relative;
 padding-left: 25px;
 margin-bottom: 10px;
 }

.descriptionText li::before {
 content: "◆";
 position: absolute;
 left: 0;
 color: #c5a059;
}

.descriptionText strong {
 color: #645945;
 font-weight: 600;
}

span {
 gap: 15px;
}
```

---

FICHER : code/src/assets/css/public\_layout.module.css

```
.appContainer {
 display: flex;
 flex-direction: column;
 /* Force le site à faire au moins la hauteur de l'écran */
 min-height: 100vh;
}

.mainContent {
 /* Dit au contenu de prendre tout l'espace vide, poussant le footer en
bas */
 flex: 1;
}
```

---

FICHER : code/src/assets/css/reset.css

```
/* http://meyerweb.com/eric/tools/css/reset/
v2.0 | 20110126
License: none (public domain)
*/

html,
body,
div,
```

span,  
applet,  
object,  
iframe,  
h1,  
h2,  
h3,  
h4,  
h5,  
h6,  
p,  
blockquote,  
pre,  
a,  
abbr,  
acronym,  
address,  
big,  
cite,  
code,  
del,  
dfn,  
em,  
img,  
ins,  
kbd,  
q,  
s,  
samp,  
small,  
strike,  
strong,  
sub,  
sup,  
tt,  
var,  
b,  
u,  
i,  
center,  
dl,  
dt,  
dd,  
ol,  
ul,  
li,  
fieldset,  
form,  
label,  
legend,  
table,  
caption,  
tbody,  
tfoot,



```
thead,
tr,
th,
td,
article,
aside,
canvas,
details,
embed,
figure,
figcaption,
footer,
header,
hgroup,
menu,
nav,
output,
ruby,
section,
summary,
time,
mark,
audio,
video {
 margin: 0;
 padding: 0;
 border: 0;
 font: inherit;
 font-size: 100%;
 vertical-align: baseline;
}
/* HTML5 display-role reset for older browsers */
article,
aside,
details,
figcaption,
figure,
footer,
header,
hgroup,
menu,
nav,
section {
 display: block;
}
body {
 line-height: 1;
}
ol,
ul {
 list-style: none;
}
blockquote,
q {
```

```
 quotes: none;
 }
 blockquote:before,
 blockquote:after,
 q:before,
 q:after {
 content: none;
 }
 table {
 border-collapse: collapse;
 border-spacing: 0;
 }
```

---

FICHER : code/src/assets/css/shop\_preview.module.css

```
.shopPreview {
 padding: 100px 20px;
 text-align: center;
 background-color: #f3efe4;
}

.shopPreview h2 {
 font-family: "Playfair Display", serif;
 color: #645945;
 margin-bottom: 15px;
}

.shopPreview p {
 color: #8a7d6a;
 margin-bottom: 30px;
}

.secondaryLink {
 color: #645945;
 text-decoration: none;
 font-weight: bold;
 border-bottom: 2px solid #c5a059;
 padding-bottom: 5px;
 transition: color 0.3s;
}

.secondaryLink:hover {
 color: #c5a059;
}
```

---

FICHER : code/src/components/admin\_formulaire.tsx

```
"use client";

import { useEffect, useId, useState } from "react";
import { useForm } from "react-hook-form";
import { useNavigate } from "react-router";
import type { ZodIssue } from "zod/v3";
import type { Product } from "../../models/product";
import styles from "../../assets/css/form.module.css";
import type { AdminProductsAddProps } from
"../../models/props/admin_products_add_props";
import ProductApiService from "../../services/product_api_service";

const AdminForm = ({
 categories,
 skins,
 bodyparts,
 packs,
 validator,
 dataToUpdate,
}: AdminProductsAddProps) => {
 const idId = useId();
 const nameId = useId();
 const imageId = useId();
 const priceId = useId();
 const descriptionId = useId();
 const categoryId = useId();

 // console.log(dataToUpdate);

 // useNaviagate hook: permet de créer une redirection
 const navigate = useNavigate();

 // Stocker les messages d'erreur de validation côté serveur
 const [serverErrors, setserverErrors] = useState<Partial<Product>>>();

 // Message lié à la soumission du formulaire en cas d'echec
 const [message, setMessage] = useState<string>("");

 const {
 register,
 handleSubmit,
 reset,
 formState: { errors },
 } = useForm<Partial<Product>>>();

 // Pré-remplir le formulaire avant l'affichage du composant
 // Use effect ne s'execute que dans un composantclient: "use client";

 useEffect(() => {
 // Si des données sont à mettre à jour (update)
 if (dataToUpdate) {
 // Normaliser les données de saisies: se baser sur les données
```

```

testees dans flashpost pour que les donnés soient identiques.
 // Modifier le models si besoin
 const normalizedData = {
 ...dataToUpdate,
 skin_ids: (dataToUpdate.skin_ids as string).split(","),
 pack_ids: (dataToUpdate.pack_ids as string).split(","),
 body_part_ids: (dataToUpdate.body_part_ids as
string).split(","),
 };
 reset(normalizedData);
}
}, [dataToUpdate, reset]);

const submitForm = async (data: Partial<Product>) => {
 // Normaliser les donnés de saisies: se baser sur les donnés
testees dans flashpost pour que les donnés soient identiques.
 const normalizedData = {
 ...data,
 skin_ids: (data.skin_ids as unknown as string[]).join(),
 pack_ids: (data.pack_ids as unknown as string[]).join(),
 body_part_ids: (data.body_part_ids as unknown as
string[]).join(),
 image: (data.image as string)[0],
 };

 // validation de la saisie avec le validateur côté serveur
 const validation = await validator(normalizedData);

 // Si la validation échoue
 if (validation instanceof Error) {
 // Stocker les messages d'erreurs dans un objet
 let errors = {};

 // récupérer les messages d'erreur
 (JSON.parse(validation.message) as ZodIssue[]).map((item) => {
 errors = { ...errors, [item.path.shift() as string]:
item.message };
 return errors;
 });

 // Définir l'état
 setserverErrors(errors);

 return;
 }

 // Si la validation réussie
 // // console.log(normalizedData);
 // Si le formulaire contient un champs de fichier: envoyer vers
l'API un objet de type formData

 const formData = new FormData();
 // Reprendre strictement le nom des champs de formulaire tester
sur flashpost

```

```

 formData.set("id", normalizedData.id as unknown as string);
 formData.set("name", normalizedData.name as unknown as string);
 formData.set("image", normalizedData.image as unknown as string);
 formData.set(
 "description",
 normalizedData.description as unknown as string,
);
 formData.set("price", normalizedData.price as unknown as string);
 formData.set(
 "category_id",
 normalizedData.category_id as unknown as string,
);
 formData.set("skin_ids", normalizedData.skin_ids as unknown as
string);
 formData.set(
 "body_part_ids",
 normalizedData.body_part_ids as unknown as string,
);
 formData.set("pack_ids", normalizedData.pack_ids as unknown as
string);

 //console.log(formData);

 // Requête HTTP vers l'API
 const process = dataToUpdate
 ? await new ProductApiService().update(formData)
 : await new ProductApiService().insert(formData);

 // Si la requête HTTP à réussie
 /* est ce que le code de status est dans cette liste = different
de -1 (-1= absent de la liste)
 useNavigate = on redirige vers une autre page sans click */
 if ([200, 201].indexOf(process.status) !== -1) {
 navigate("/admin/products");
 // Si la requête HTTP échoue
 } else if ([400].indexOf(process.status) !== -1) {
 // Afficher le message
 setMessage(process.message as unknown as string);
 }
 };

 return (
 <section className={styles.formWrapper}>
 /* Afficher le message via une condition ternaire*/
 {message ? <p role="alert">{message}</p> : null}

 /* onSubmit: evenement javascript
handleSubmit: react hook form
handleSubmit: fonction que l'on crée*/
 <form
 className={styles.orientalForm}
 encType="multipart/form-data"
 onSubmit={handleSubmit(submitForm)}
 >

```

```

<div>
 <label htmlFor={nameId}>
 <input
 type="text"
 id={nameId}
 placeholder="Nom"
 {...register("name", {
 required: "Le nom est obligatoire",
 minLength: {
 value: 2,
 message: "Un nom doit comporter au
minimum 2 caractères",
 },
 maxLength: {
 value: 50,
 message: "Un nom doit comporté au
maximum 50 caracteres",
 },
 })}
 />
 </label>
 { /* AFFICHER LES MESSAGES D'ERREURS Utiliser le nom de
champs définit dans register */ }
 <small role="alert">
 { " " }
 { errors.name?.message ?? serverErrors?.name }
 </small>
</div>

<div>
 <label htmlFor={imageId}>
 <input
 type="file"
 id={imageId}
 placeholder="Image"
 {...register(
 "image",
 dataToUpdate
 ? {}
 : {
 required: "Le nom de l'image
est obligatoire",
 minLength: {
 value: 2,
 message:
 "Une image doit
comporter au minimum 2 caractères",
 },
 maxLength: {
 value: 50,
 message:
 "Un image peut
comporter au maximum 50 caracteres",
 },
 }
)}
 </input>
 </label>

```

```

 },
 })
 />
</label>
<small role="alert">
 {" "}
 {errors.image?.message ?? serverErrors?.image}
</small>
</div>

<div>
 <label htmlFor={priceId}>
 {
 <input
 type="text"
 id={priceId}
 placeholder="Prix"
 {...register("price", {
 required: "Le prix est obligatoire",
 min: {
 value: 1,
 message: "Un prix doit comporter
au minimum 1 chiffre",
 },
 max: {
 value: 99.99,
 message:
 "Un prix peut comporter au
maximum 4 caracteres dont 2 decimales",
 },
 })}
 />
 }
 </label>
 <small role="alert">
 {" "}
 {errors.price?.message ?? serverErrors?.price}
 </small>
</div>

<div>
 <label htmlFor={descriptionId}>
 <textarea
 id={descriptionId}
 placeholder="Décrivez les rituels et les
vertus de ce soin..."
 rows={5}
 {...register("description", {
 required: "La description est
obligatoire",
 minLength: {
 value: 20,
 message: "Une description doit
comporter au minimum 20 ",

```

```

 },
 }}}
 </textarea>
</label>
<small role="alert">
 {" "}
 {errors.description?.message ??
serverErrors?.description}
</small>
</div>

 { /* CRUD ==> CREATE
- si le formulaire contient un champ de fichier : ajouter l'attribut
enctype="multipart/form-data"
- pour les champs en relation :
FK : créer, soit une liste déroulante <select>, soit des boutons radio
> sélection d'un seul choix
table de jointure : cases à cocher
> sélection de plusieurs choix
*/ }

 <div className={styles.formGroup}>
 <label htmlFor={categoryId}>Catégorie</label>
 <select
 className={styles.title}
 id={categoryId}
 {...register("category_id", {
 required: "Veuillez selectionner une
category",
 })}
 >
 <option value="">-- Choisir une catégorie --
</option>

 {categories.map((item) => {
 return (
 <option key={item.id} value={item.id}>
 {" "}
 {item.name}{" "}
 </option>
);
 })}
 </select>
 <small role="alert">
 {" "}
 {errors.category_id?.message ??
serverErrors?.category_id}
 </small>
 </div>

 <div className={styles.formGroup}>
 <p className={styles.groupTitle}>Type de peau</p>
 <div className={styles.checkboxGrid}>
 {skins.map((item) => (
 <label key={item.id} className=

```



```

{styles.checkboxItem}>
 <input
 type="checkbox"
 value={item.id}
 id={`skin-${item.id}`}
 {...register("skin_ids", {
 required:
 "Veuillez selectionner un ou
plusieurs type de peau",
 })}
 />
 <small role="alert">
 {" "}
 {errors.skin_ids?.message ??
serverErrors?.skin_ids}
 </small>
 {item.type}
 </label>
)})
</div>
</div>

<div className={styles.formGroup}>
 <p className={styles.groupTitle}>Zone du corps</p>
 <div className={styles.checkboxGrid}>
 {bodyparts.map((item) => (
 <label key={item.id} className=
{styles.checkboxItem}>
 <input
 type="checkbox"
 value={item.id}
 id={`bodyparts-${item.id}`}
 {...register("body_part_ids", {
 required:
 "Veuillez selectionner une ou
plusieurs zone du corps",
 })}
 />
 <small role="alert">
 {" "}
 {errors.body_part_ids?.message ??
serverErrors?.body_part_ids}
 </small>
 {item.name}
 </label>
))}
 </div>
</div>

<div className={styles.formGroup}>
 <p className={styles.groupTitle}>Packs</p>
 <div className={styles.checkboxGrid}>
 {packs.map((item) => (
 <label key={item.id} className=

```

```

{styles.checkboxItem}>
 <input
 type="checkbox"
 value={item.id}
 id={`packs-${item.id}`}
 {...register("pack_ids", {
 required: "Veuillez selectionner
un ou plusieurs pack",
 })}
 />
 <small role="alert">
 {" "}
 {errors.pack_ids?.message ??
serverErrors?.pack_ids}
 </small>
 {item.name}
 </label>
)})
</div>
</div>

<div>
 <input type="hidden" id={idId} {...register("id")} />
 <button type="submit">
 {dataToUpdate ? "Modifier produit " : "Cr  er
Produit"}
 </button>
</div>
</form>
</section>
);
};

export default AdminForm;

```

---

FICHER : code/src/components/admin\_home.tsx

```

import { use } from "react";
import { Link, NavLink } from "react-router";
import styles from
"../../assets/css/admin_css/admin_product_index.module.css";
import ProductApiService from "../../src/services/product_api_service";

const AdminProductHome = () => {
 const results = use(new ProductApiService().selectAll()).data;
 // console.log(results);

 return (
 <div className={styles.crudContainer}>

```

```

 <header className={styles.crudHeader}>
 <div>
 <h1>Gestion des Produits</h1>
 <p>Consultez et gérez votre catalogue de soins
orientaux.</p>
 </div>

 <NavLink to="/admin/products/add" className=
{styles.addButton}>
 + Ajouter un produit
 </NavLink>
 </header>

 {results?.map((item) => {
 return (
 <div key={item.id}>
 <div>
 <p>{item.id} </p>
 <p>{item.name} </p>
 </div>

 <div className={styles.actions}>
 <Link to=
{`/admin/products/add/${item.id}`}>Modifier</Link>
 <Link to=
{`/admin/products/add/${item.id}`}>Supprimer</Link>
 </div>
 </div>
);
 })}

 /* <section className={styles.tableWrapper}>
 <table className={styles.crudTable}>
 <thead>
 <tr>
 <th>ID</th>
 <th>Nom</th>
 <th>Prix</th>
 <th>Stock</th>
 <th>Actions</th>
 </tr>
 </thead>

 <tbody>
 {products.map((p) => (
 <tr key={p.id}>
 <td>#{p.id}</td>
 <td className={styles.productName}>
{p.name}</td>

 <td>{p.price} €</td>
 <td>
 <span className={p.stock < 15 ?
styles.lowStock : ""}>
 {p.stock}

```

```


 </td>
 </td>
 <div className={styles.actions}>
 <button className=
{styles.editBtn}>Modifier</button>
 <button className=
{styles.deleteBtn}>Supprimer</button>
 </div>
 </td>
</tr>
 </tbody>
</table>
</section> */}
</div>
);
};

export default AdminProductHome;

```

---

FICHER : code/src/components/favicon.tsx

---

FICHER : code/src/components/footer.tsx

```

import { FaFacebookSquare } from "react-icons/fa";
import { NavLink } from "react-router";
import styles from "../assets/css/footer.module.css";
import Logo from "../logo";

const Footer = () => {
 return (
 <footer className={styles.footer}>
 <div className={styles.logWrapper}>
 <Logo />
 </div>
 <div className={styles.nav}>
 <ul className={styles.menu}>

 <NavLink to={"/"}> Accueil </NavLink>

 <NavLink to={"/produits"}> Produits </NavLink>

 </div>
 </footer>
);
};

```

```


 <NavLink to={"/register"}> Inscription </NavLink>

 <NavLink to={"/login"}> Connexion </NavLink>

 <NavLink to={"/contact"}> Contact </NavLink>

 <NavLink to={"/a_propos"}> A propos </NavLink>

 <NavLink to={"/mentions_legales"}> Mentions
légales </NavLink>

</div>
<div className={styles.icone}>
 <FaFacebookSquare />
</div>
</footer>
);
};

export default Footer;

```

---

FICHER : code/src/components/form.tsx

```

"use client";
import type { FormProps } from "../../models/props/form_props";
import styles from "../../assets/css/form.module.css";

const PublicForm = ({ title, buttonText, type }: FormProps) => {

 return (
 <section className={styles.formWrapper}>
 <h2>{title}</h2>
 <form className={styles.orientalForm} onSubmit={(e) =>
e.preventDefault()}>

 </* Rendu conditionnel des champs */>
 {type === "contact" && (
 <>
 <input type="email" placeholder="Email" required />
 <input type="text" placeholder="Sujet" required />
 <textarea placeholder="Message"></textarea>
 </>
)}

 </form>
 </section>
);
}

```

```

 })

 {type === "login" && (
 <>
 <input type="email" placeholder="Email" required />
 <input type="password" placeholder="Mot de passe" required />
 </>
)}

 {type === "register" && (
 <>
 <input type="email" placeholder="Email" required />
 <input type="password" placeholder="Mot de passe" required />
 <input type="password" placeholder="Confirmer mot de passe"
required />
 </>
)}

 <button type="submit">{buttonText}</button>
 </form>

 <div className={styles.formFooter}>
 {type === "login" && <p>Mot de passe oublié ?</p>}
 {type === "register" && <p>En vous inscrivant, vous rejoignez
notre rituel de fidélité.</p>}
 </div>

</section>
);
}

export default PublicForm;

```

---

FICHER : code/src/components/header.tsx

```

import styles from "../../assets/css/header.module.css";
import IconNavbar from "../icon_navbar";
import Logo from "../logo";
import NavBar from "../navbar";

const Header = () => {
 return (
 <header className={styles.header}>
 <NavBar />
 <Logo />
 <IconNavbar />
 </header>
);
};

```

```
export default Header;
```

FICHER : code/src/components/home/featuredProducts.tsx

```
import { use } from "react";
import styles from "../../assets/css/featured_products.module.css";
import ProductApiService from "../../services/product_api_service";
import ProductCard from "../products/product_card";

const FeaturedProducts = () => {
 const results = use(new ProductApiService().selectAll()).data;
 const title = "Sélectionnés pour vous";
 const subtitle = "Nos Incontournables";

 return (
 <section className={styles.featuredProducts}>
 <div className={styles.sectionHeader}>
 {subtitle}
 <h2>{title}</h2>
 </div>

 <div className={styles.productGrid}>
 {results?.map((item) => (
 <ProductCard key={item.id} data={item} />
))}
 </div>
 </section>
);
};

export default FeaturedProducts;
```

FICHER : code/src/components/home/features.tsx

```
import styles from "../../assets/css/homepage.module.css";

const Features = () => {
 return (
 <section className={styles.featuresSection}>
 <div className={styles.featureCard}>
 ❏
 <h3>100% Naturel</h3>
 <p>Des ingrédients sourcés avec respect, sans additifs
chimiques.</p>
 </div>
 </section>
);
};
```

```
 <div className={styles.featureCard}>
 ❏
 <h3>Savoir-faire</h3>
 <p>
 Des recettes traditionnelles transmises de génération
en génération.
 </p>
 </div>

 <div className={styles.featureCard}>
 ❏
 <h3>Éclat Durable</h3>
 <p>Une efficacité prouvée par les rituels de beauté
orientaux.</p>
 </div>
 </section>
);
 };

 export default Features;
```

---

FICHER : code/src/components/home/hero.tsx

```
import { Link } from "react-router";
import styles from "../../assets/css/homepage.module.css";

const Hero = () => {
 return (
 <section className={styles.hero}>
 <div className={styles.heroContent}>
 <h2>L'Essence du Rituel Ancestral</h2>
 <p>
 Découvrez des soins purs et naturels pour sublimer
votre peau au
 quotidien.
 </p>
 <Link to="/produits" className={styles.ctaButton}>
 Explorer la collection
 </Link>
 </div>
 </section>
);
};

export default Hero;
```



FICHER : code/src/components/home/product\_skeleton.tsx

```
import styles from "../../../assets/css/product_skeleton.module.css";

const ProductSkeleton = () => {
 return (
 <div className={styles.skeletonCard}>
 <div className={styles.skeletonImage}></div>
 <div className={styles.skeletonInfo}>
 <div className={styles.skeletonTitle}></div>
 <div className={styles.skeletonPrice}></div>
 <div className={styles.skeletonButton}></div>
 </div>
 </div>
);
};

export const FeaturedSkeleton = () => {
 return (
 <section style={{ padding: "40px 20px" }}>
 </* On imite la grille de produits (par exemple 4 cartes) */>
 <div
 style={{
 display: "grid",
 gridTemplateColumns: "repeat(auto-fill, minmax(250px, 1fr))",
 gap: "20px",
 maxWidth: "1200px",
 margin: "0 auto",
 }}
 >
 {[1, 2, 3, 4].map((i) => (
 <ProductSkeleton key={i} />
))}
 </div>
 </section>
);
};
```

FICHER : code/src/components/home/shopPreview.tsx

```
import { Link } from "react-router";
import styles from "../../../assets/css/shop_preview.module.css";

const ShopPreview = () => {
 return (
 <section className={styles.shopPreview}>
 <div className={styles.previewText}>
```

```
 <h2>Le Hammam à la Maison</h2>
 <p>
 Savon noir, huile d'argan, gant kessa... Retrouvez
tous vos
 indispensables.
 </p>
 <Link to="/produits" className={styles.secondaryLink}>
 Voir tous les produits →
 </Link>
 </div>
</section>
);
};

export default ShopPreview;
```

---

FICHER : code/src/components/icon\_navbar.tsx

```
import { FaUser } from "react-icons/fa";
import { FaBasketShopping, FaMagnifyingGlass } from "react-icons/fa6";
import styles from "../assets/css/icon_navbar.module.css";

const IconNavbar = () => {
 return (
 <div className={styles.icon_navbar}>
 <FaMagnifyingGlass />
 <FaUser />
 <FaBasketShopping />
 </div>
);
};

export default IconNavbar;
```

---

FICHER : code/src/components/lien\_d\_evitement.tsx

---

FICHER : code/src/components/logo.tsx

```
import styles from "../assets/css/logo.module.css";
```

```

const Logo = () => {
 return (
 <div className={styles.logo}>
 <h1>Secret de Hammam</h1>

 </div>
);
};

export default Logo;

```

---

FICHER : code/src/components/navbar.tsx

```

"use client";

import { useState } from "react";
import { NavLink } from "react-router";
import styles from "../../assets/css/navbar.module.css";

const NavBar = () => {
 // const navVisible: boolean = false;
 const [navVisible, setNavVisible] = useState<boolean>(false);

 const click = () => setNavVisible(!navVisible);

 const closeMenu = () => setNavVisible(false);

 // console.log(navVisible)

 return (
 <div className={styles.navContainer}>
 <button type="button" onClick={click}>
 <div className={` ${styles.line} ${styles.line1}
 ${navVisible ? styles.one : ""}`}></div>
 <div className={` ${styles.line} ${styles.line2}
 ${navVisible ? styles.two : ""}`}></div>
 <div className={` ${styles.line} ${styles.line3}
 ${navVisible ? styles.three : ""}`}></div>
 </button>
 </div>
);
};

```

```

 <nav className={` ${styles.navbar} ${navVisible ? "" :
styles["nav-visible"]} `}>

 <ul className={styles.menu}>

 <NavLink to={"/"} onClick={closeMenu}> Accueil
</NavLink>
 <NavLink to={"/produits"} onClick={closeMenu}>
Produits </NavLink>
 <NavLink to={"/register"} onClick={closeMenu}>
Inscription </NavLink>
 <NavLink to={"/login"} onClick={closeMenu}>
Connexion </NavLink>
 <NavLink to={"/contact"} onClick={closeMenu}>
Contact </NavLink>
 <NavLink to={"/a_propos"} onClick={closeMenu}> A
propos </NavLink>
 <NavLink to={"/mentions_legales"} onClick=
{closeMenu}> Mentions légales </NavLink>
 <NavLink to="/admin" onClick={closeMenu}
className={styles.adminLink}>Admin</NavLink>

 </nav>

 </div>

);

};

export default NavBar;

```

---

FICHER : code/src/components/products/product\_card.tsx

```

import { Link } from "react-router";
import styles from "../../assets/css/product_card.module.css";
import type { ProductsListItemProps } from
"../../models/props/products_list_item_props";

const ProductCard = ({ data }: ProductsListItemProps) => {
 const formattedPrice = data.price ? Number(data.price).toFixed(2) :
"0.00";
 return (
 <div className={styles.productCard}>
 <div className={styles.imageContainer}>
 <img src={` /img/products/${data.image}`} alt={data.name}

```

```

 />
 </div>
 <div className={styles.productInfo}>
 <h4>{data.name}</h4>
 <p>{formattedPrice}€</p>
 <Link to={` /produits/${data.id}`} className=
{styles.productLink}>
 Découvrir
 </Link>
 </div>
 </div>
);
};

export default ProductCard;

```

---

FICHER : code/src/components/products/products\_contents.tsx

```

import { Link } from "react-router";
import styles from "../../assets/css/products.module.css";
import type { ProductsContentsProps } from
"../../models/props/products_contents_props";

// Récupération de la props data envoyé par le parent (page) à l'enfant
(composant)
const ProductsContents = ({ data }: ProductsContentsProps) => {
 return (
 <section className={styles.productsSection}>
 <div className={styles.productDetailContainer}>
 <article className={styles.productCard}>
 <div className={styles.imageWrapper}>
 <img src={` /img/products/${data.image}`} alt=
{data.name} />
 </div>

 <div className={styles.productInfo}>
 <h3>{data.name}</h3>
 <p className={styles.price}>{data.price} €</p>
 <p className={styles.descriptionText}>
{data.description}</p>
 <button type="button" className=
{styles.viewButton}>
 Ajouter au panier
 </button>
 <Link to="/produits">
 Retour aux produits
 </Link>
 </div>
 </article>
 </div>
 </section>
);
};

```

```
 <div className={styles.Info}></div>
 </article>
 </div>
 </section>
);
};

export default ProductsContents;
```

---

FICHER : code/src/components/products/products\_list.tsx

```
import { use } from "react";
import type { Product } from "../../models/product";
import styles from "../../assets/css/products.module.css";
import type { ApiResponse } from "../../models/api_response";
import ProductApiService from "../../services/product_api_service";
import ProductsListItem from "../../products_list_item";

const ProductsList = () => {
 // uniquement des composants
 // SEO
 // use : permet de récupérer les données d'une promesse dans un
 // composant server de react

 const results = use<ApiResponse<Product[]>>(
 new ProductApiService().selectAll(),
);

 console.log(results);
 return (
 <section className={styles.productsSection}>
 <h2 className={styles.mainTitle}> Nos Soins Orientaux </h2>

 <div className={styles.productsGrid}>
 {results.data?.map((item) => (
 <ProductsListItem key={item.id} data={item} />
))}
 </div>
 </section>
);
};

export default ProductsList;
```

---

FICHER : code/src/components/products/products\_list\_item.tsx

```

import { Link } from "react-router";
import styles from "../../assets/css/products.module.css";
import type { ProductsListItemProps } from
"../../models/props/products_list_item_props";

const ProductsListItem = ({ data }: ProductsListItemProps) => {
 return (
 <article className={styles.productCard}>
 <div className={styles.imageWrapper}>
 { /* / cible directement le dossier public */ }
 <img src={`/img/products/${data.image}`} alt={data.name}
 />
 </div>

 <div className={styles.productInfo}>
 <h3>{data.name}</h3>

 <p className={styles.price}>{data.price}</p>

 <Link to={`/produits/${data.id}`}>
 <button type="button" className={styles.viewButton}>
 Découvrir
 </button>
 </Link>
 </div>
 </article>
);
};

export default ProductsListItem;

```

---

FICHER : code/src/components/seo.tsx

```

import type { SeoProps } from "../../models/props/seo_props";

// récupéré les props dans les parametres () de la fonction du composant
const Seo = ({ title, description, url }: SeoProps) => {
 return (
 <>
 { /* A peu pres 50 caratcere au maximum */ }
 <title>{`Sercret de Hammam - ${title}`}</title>

 { /* A peu pres 150 caratcere au maximum */ }
 <meta name="description" content={`Sercret de Hammam -
 ${description}`} />

 { /* Open Graph Your content's social media identity */ }
 <meta property="og:title" content={`Sercret de Hammam -

```

```

 ${title}` } />
 <meta property="og:type" content="website" />
 <meta property="og:url" content=
 {`https://secretsdehammam.com/${url}` } />
 { /* Image format 1200 x 630 pixels (taille optimale pour
Facebook) */ }
 <meta property="og:image"
content="https://yoursite.com/image.jpg" />
 <meta
 property="og:description"
 content={`Secret de Hammam - ${description}`}
 />

 { /* Twitter */ }

 <meta name="twitter:card" content="summary" />
 <meta name="twitter:site" content="@yourusername" />
 <meta name="twitter:title" content={`Serecret de Hammam -
 ${title}` } />
 <meta
 name="twitter:description"
 content={`Secret de Hammam - ${description}`}
 />
 { /* Utilisez des images carrées */ }
 <meta name="twitter:image"
content="https://yoursite.com/image.jpg" />
 </>
);
};

export default Seo;

```

---

FICHER : code/src/components/sidebar.tsx

```

"use client";
import { NavLink } from "react-router";
import styles from "/app/src/assets/css/admin_css/sidebar.module.css";

const Sidebar = () => {
 return (
 <aside className={styles.sidebar}>
 <div className={styles.logoSection}>
 <h2>Secret de Hammam</h2>
 Admin Panel
 </div>

 <nav className={styles.navLinks}>
 <NavLink
 to="/admin"

```



```

 end
 className=(({ isActive }) => isActive ? styles.active :
styles.link}
 >
 Tableau de bord
 </NavLink>

 <NavLink
 to="/admin/products"
 className=(({ isActive }) => isActive ? styles.active :
styles.link}
 >
 Gestion Produits
 </NavLink>

 <NavLink
 to="/admin/orders"
 className=(({ isActive }) => isActive ? styles.active :
styles.link}
 >
 Commandes
 </NavLink>
 </nav>

 <div className={styles.sidebarFooter}>
 <NavLink to="/" className={styles.exitLink}>
 Quitter l'admin
 </NavLink>
 </div>
 </aside>
);
};

export default Sidebar;

```

---

FICHER : code/src/framework/entry.browser.tsx

```

import {
 createFromReadableStream,
 createTemporaryReferenceSet,
 encodeReply,
 setServerCallback,
} from "@vitejs/plugin-rsc/browser";
import { StrictMode, startTransition } from "react";
import { hydrateRoot } from "react-dom/client";
import {
 unstable_createCallServer as createCallServer,
 unstable_getRSCStream as getRSCStream,
 unstable_RSCHydratedRouter as RSCHydratedRouter,
 type unstable_RSCPayload as RSCServerPayload,

```

```

} from "react-router/dom";

// Create and set the callServer function to support post-hydration server
actions.
setServerCallback(
 createCallServer({
 createFromReadableStream,
 createTemporaryReferenceSet,
 encodeReply,
 }),
);

// Get and decode the initial server payload.
createFromReadableStream<RSCServerPayload>(getRSCStream()).then((payload)
=> {
 startTransition(async () => {
 const formState =
 payload.type === "render" ? await payload.formState :
undefined;

 hydrateRoot(
 document,
 <StrictMode>
 <RSCHydratedRouter
 createFromReadableStream={createFromReadableStream}
 payload={payload}
 />
 </StrictMode>,
 {
 formState,
 },
);
 });
});

```

---

FICHER : code/src/framework/entry.rsc.tsx

```

import {
 createTemporaryReferenceSet,
 decodeAction,
 decodeFormState,
 decodeReply,
 loadServerAction,
 renderToReadableStream,
} from "@vitejs/plugin-rsc/rsc";
import { unstable_matchRSCServerRequest as matchRSCServerRequest } from
"react-router";
import RouterService from "../services/router_service";

```

```
function fetchServer(request: Request) {
 return matchRSCServerRequest({
 // Provide the React Server touchpoints.
 createTemporaryReferenceSet,
 decodeAction,
 decodeFormState,
 decodeReply,
 loadServerAction,
 // The incoming request.
 request,
 // The app routes.
 routes: new RouterService().getRouter(),
 // Encode the match with the React Server implementation.
 generateResponse(match) {
 return new Response(renderToReadableStream(match.payload), {
 status: match.statusCode,
 headers: match.headers,
 });
 },
 });
}

export default async function handler(request: Request) {
 // Import the generateHTML function from the client environment
 const ssr = await import.meta.viteRsc.loadModule<
 typeof import("./entry.ssr")
 >("ssr", "index");

 return ssr.generateHTML(request, fetchServer);
}
```

---

FICHER : code/src/framework/entry.ssr.tsx

```
import { createFromReadableStream } from "@vitejs/plugin-rsc/ssr";
import { renderToReadableStream as renderHTMLToReadableStream } from
"react-dom/server.edge";
import {
 unstable_RSCStaticRouter as RSCStaticRouter,
 unstable_routeRSCServerRequest as routeRSCServerRequest,
} from "react-router";

export async function generateHTML(
 request: Request,
 fetchServer: (request: Request) => Promise<Response>,
): Promise<Response> {
 return await routeRSCServerRequest({
 // The incoming request.
 request,
 // How to call the React Server.
 });
}
```

```

 fetchServer,
 // Provide the React Server touchpoints.
 createFromReadableStream,
 // Render the router to HTML.
 async renderHTML(getPayload) {
 const payload = getPayload();

 const bootstrapScriptContent =
 await
import.meta.viteRsc.loadBootstrapScriptContent("index");

 return await renderHTMLToReadableStream(
 <RSCStaticRouter getPayload={getPayload} />,
 {
 bootstrapScriptContent,
 formState: payload.formState,
 },
);
 },
 });
}

```

---

FICHER : code/src/framework/error-boundary.tsx

```

'use client'

import React from 'react'

// Minimal ErrorBoundary example to handle errors globally on browser
export function GlobalErrorBoundary(props: { children?: React.ReactNode })
{
 return (
 <ErrorBoundary errorComponent={DefaultGlobalErrorPage}>
 {props.children}
 </ErrorBoundary>
)
}

//
https://github.com/vercel/next.js/blob/33f8428f7066bf8b2ec61f025427ceb2a54c4bdf/packages/next/src/client/components/error-boundary.tsx
// https://react.dev/reference/react/Component#catching-rendering-errors-with-an-error-boundary
class ErrorBoundary extends React.Component<{
 children?: React.ReactNode
 errorComponent: React.FC<{
 error: Error
 reset: () => void
 }>
}>

```

```

}> {
 state: { error?: Error } = {}

 static getDerivedStateFromError(error: Error) {
 return { error }
 }

 reset = () => {
 this.setState({ error: null })
 }

 render() {
 const error = this.state.error
 if (error) {
 return <this.props.errorComponent error={error} reset={this.reset}
 />
 }
 return this.props.children
 }
}

//
https://github.com/vercel/next.js/blob/677c9b372faef680d17e9ba224743f44e1107661/packages/next/src/build/webpack/loaders/next-app-loader.ts#L73
//
https://github.com/vercel/next.js/blob/677c9b372faef680d17e9ba224743f44e1107661/packages/next/src/client/components/error-boundary.tsx#L145
function DefaultGlobalErrorPage(props: { error: Error; reset: () => void
}) {
 return (
 <html>
 <head>
 <title>Unexpected Error</title>
 </head>
 <body
 style={{
 height: '100vh',
 display: 'flex',
 flexDirection: 'column',
 placeContent: 'center',
 placeItems: 'center',
 fontSize: '16px',
 fontWeight: 400,
 lineHeight: '24px',
 }}
 >
 <p>Caught an unexpected error</p>
 <pre>
 Error: {' '}
 {import.meta.env.DEV && 'message' in props.error
 ? props.error.message
 : '(Unknown)'}
 </pre>
 <button

```

```

 onClick={() => {
 React.startTransition(() => {
 props.reset()
 })
 }}
 >
 Reset
 </button>
 </body>
 </html>
)
}

```

---

FICHER : code/src/framework/request.tsx

```

// Framework conventions (arbitrary choices for this demo):
// - Use `_.rsc` URL suffix to differentiate RSC requests from SSR
 requests
// - Use `x-rsc-action` header to pass server action ID
const URL_POSTFIX = '_.rsc'
const HEADER_ACTION_ID = 'x-rsc-action'

// Parsed request information used to route between RSC/SSR rendering and
 action handling.
// Created by parseRenderRequest() from incoming HTTP requests.
type RenderRequest = {
 isRsc: boolean // true if request should return RSC payload (via _.rsc
 suffix)
 isAction: boolean // true if this is a server action call (POST request)
 actionId?: string // server action ID from x-rsc-action header
 request: Request // normalized Request with _.rsc suffix removed from
 URL
 url: URL // normalized URL with _.rsc suffix removed
}

export function createRscRenderRequest(
 urlString: string,
 action?: { id: string; body: BodyInit },
): Request {
 const url = new URL(urlString)
 url.pathname += URL_POSTFIX
 const headers = new Headers()
 if (action) {
 headers.set(HEADER_ACTION_ID, action.id)
 }
 return new Request(url.toString(), {
 method: action ? 'POST' : 'GET',
 headers,
 body: action?.body,
 })
}

```

```

 })
 }

export function parseRenderRequest(request: Request): RenderRequest {
 const url = new URL(request.url)
 const isAction = request.method === 'POST'
 if (url.pathname.endsWith(URL_POSTFIX)) {
 url.pathname = url.pathname.slice(0, -URL_POSTFIX.length)
 const actionId = request.headers.get(HEADER_ACTION_ID) || undefined
 if (request.method === 'POST' && !actionId) {
 throw new Error('Missing action id header for RSC action request')
 }
 return {
 isRsc: true,
 isAction,
 actionId,
 request: new Request(url, request),
 url,
 }
 } else {
 return {
 isRsc: false,
 isAction,
 request,
 url,
 }
 }
}

```

---

FICHER : code/src/layouts/admin\_layout.tsx

```

import "../assets/css/reset.css";
import "../assets/css/base.css";
import styles from "/app/src/assets/css/admin_layout.module.css";
import { Outlet } from "react-router";
import Sidebar from "../components/sidebar";

const AdminLayout = () => {
 return (
 <>
 { /* <Lien d'évitement /> */ }
 <div className={styles.adminLayout}>
 <Sidebar />
 <main className={styles.mainContent}>
 <Outlet />
 </main>
 </div>
 </>
)
}

```

```
);
};

export default AdminLayout;
```

---

FICHER : code/src/layouts/public\_layout.tsx

```
import "../assets/css/reset.css";
import "../assets/css/base.css";
import { Outlet } from "react-router";
import Footer from "../components/footer";
import Header from "../components/header";
import styles from "../assets/css/public_layout.module.css";

const PublicLayout = () => {
 return (
 <div className={styles.appContainer}>
 <Header />

 <main className={styles.mainContent}>

 { /* <Lien d'évitement /> */ }

 <Outlet />

 </main>

 <Footer />

 </div>
);
};

export default PublicLayout;
```

---

FICHER : code/src/layouts/root\_layout.tsx

```
import "../assets/css/reset.css";
import "../assets/css/base.css";

import { Outlet } from "react-router";

const RootLayout = () => {
 return (
 <html lang="fr">
```



```

 <head>
 <meta charSet="UTF-8" />
 { /*<link rel="icon" type="image/svg+xml" href="/vite.svg"
/>*/}
 <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
 </head>
 <body>
 { /* Zone rempli par un autre contenu */}
 <Outlet />
 </body>
 </html>
);
};

export default RootLayout;

```

---

FICHER : code/src/models/api\_response.ts

```

// T represente un type générique : type variable
// on utilise le T chaque fois que l'on doit typer quelque chose mais
qu'on ne peut pas définir le type dans le sens ou cela peut être un
string, un number, un array, un objet.....

type ApiResponse<T> = {
 status: number;
 message: string;
 // ? : optionnel
 data?: T;
};

export type { ApiResponse };

```

---

FICHER : code/src/models/params/admin\_products\_params.ts

```

type AdminProductsParams = {
 // reprendre les variables d'URL créées dans le routeur
 params: {
 id: number
 }
}

export type { AdminProductsParams };

```

FICHER : code/src/models/params/produits\_details\_params.ts

```
type ProduitsDetailsParams = {
 // reprendre les variables d'URL créés dans le routeur
 params: {
 id: number
 }
}

export type { ProduitsDetailsParams };
```

FICHER : code/src/models/props/admin\_products\_add\_props.ts

```
import type { ZodError } from "zod";
import type { Body_part } from "../../../models/body_part";
import type { Category } from "../../../models/category";
import type { Pack } from "../../../models/pack";
import type { Product } from "../../../models/product";
import type { Skin } from "../../../models/skin";

type AdminProductsAddProps = {
 // Reprendre strictement le nom des props définis sur le composant
 categories: Category[];
 skins: Skin[];
 bodyparts: Body_part[];
 packs: Pack[];
 validator: (data: Partial<Product>) => Promise<Partial<Product> |
ZodError>;
 dataToUpdate: Product | undefined;
};

export type { AdminProductsAddProps };
```

FICHER : code/src/models/props/featured\_products\_props.ts

```
import type { Product } from "../../../models/product";

type FeaturedProductsProps = {
 title: string;
 subtitle: string;
 products: Product[];
};
```

```
export type { FeaturedProductsProps };
```

---

FICHIER : code/src/models/props/form\_props.ts

```
type FormProps = {
 // Reprendre strictement le nom des props définis sur le composant
 title: string;
 buttonText: string;
 type: "contact" | "login" | "register" | "product";
};

export type { FormProps };
```

---

FICHIER : code/src/models/props/products\_contents\_props.ts

```
import type { Product } from "../../models/product";

type ProductsContentsProps = {
 // Reprendre strictement le nom des props définis sur le composant
 data: Product
};

export type { ProductsContentsProps };
```

---

FICHIER : code/src/models/props/products\_list\_item\_props.ts

```
import type { Product } from "../../models/product";

type ProductsListItemProps = {
 // Reprendre strictement le nom des props définis sur le composant
 data: Product
};

export type { ProductsListItemProps };
```

---

FICHIER : code/src/models/props/seo\_props.ts

```
type SeoProps = {
 title: string;
 description: string;
 url: string;
};

export type { SeoProps };
```

---

FICHER : code/src/pages/admin/index.tsx

```
import styles from "/app/src/assets/css/admin_layout.module.css";

const AdminHome = () => {
 return (
 <div className={styles.container}>
 <header className={styles.header}>
 <h1>Tableau de bord</h1>
 <p>Bienvenue dans la gestion de votre boutique de soins.
</p>
 </header>

 <div className={styles.statsGrid}>
 <div className={styles.card}>
 <h3>Produits actifs</h3>
 <p className={styles.number}>24</p>
 </div>
 <div className={styles.card}>
 <h3>Commandes en cours</h3>
 <p className={styles.number}>8</p>
 </div>
 <div className={styles.card}>
 <h3>Chiffre d'affaires</h3>
 <p className={styles.number}>1 240 €</p>
 </div>
 </div>

 <div className={styles.recentActivity}>
 <h2>Derniers produits ajoutés</h2>

 Savon noir à l'Eucalyptus - <span className={
styles.date}>Il y a 2h
 Huile d'Argan Bio - <span className={
styles.date}>Hier

 </div>
 </div>
);
};
```

```
export default AdminHome;
```

---

FICHER : code/src/pages/admin/login.tsx

```
const Login = () => {
 return (
 <h2>admin login</h2>
);
};

export default Login;
```

---

FICHER : code/src/pages/admin/product/add.tsx

```
import { NavLink } from "react-router";
import styles from "../../../assets/css/admin_css/add.module.css";
import React, { use } from "react";
import CategoryApiService from "../../../services/category_api_service";
import type { Category } from "../../../models/category";
import AdminForm from "../../../components/admin_formulaire";
import type { Body_part } from "../../../models/body_part";
import type { Skin } from "../../../models/skin";
import type { Pack } from "../../../models/pack";
import SkinApiService from "../../../services/skin_api_service";
import BodyPartApiService from "../../../services/body_part_api_service";
import PackApiService from "../../../services/pack_api_service";
import AdminFormulaireValidator from
 "../../../validators/admin/admin_formulaire_validator";
import type { AdminProductsParams } from
 "../../../models/params/admin_products_params";
import type { Product } from "../../../models/product";
import ProductApiService from "../../../services/product_api_service";

const AddProduct = ({ params }: AdminProductsParams): React.JSX.Element =>
{
 // console.log(params);

 // Récupérer la variable d'URL
 const { id } = params;
 // console.log(id);

 // Récupérer les données à mettre à jour
 let dataToUpdate: Product | undefined;
 // Si un identifiant est présent dans l'URL
 if (id) {
 // La methode Then equivaut a Await lorsque les conditions d'await ne
```

```

sont pas rempli(etre dans une fonction et asynchrone)
// new ProductApiService().selectOne(id).then(item => {
// dataToUpdate = item.data as Product;
// console.log(dataToUpdate);
// });
dataToUpdate = use(new ProductApiService().selectOne(id)).data as
Product;

}

// console.log(dataToUpdate);

// recuperer les produits
const categories = use(new CategoryApiService().selectAll()).data as
Category[];
const skins = use(new SkinApiService().selectAll()).data as Skin[];
const bodyparts = use(new BodyPartApiService().selectAll()).data as
Body_part[];
const packs = use(new PackApiService().selectAll()).data as Pack[];

return (
 <div className={styles.formContainer}>
 <header className={styles.header}>
 <div>
 <h1>Administration</h1>
 <p>Gestion du catalogue Secret de Hammam</p>
 </div>
 <NavLink to="/admin/products" className={styles.backBtn}>
 Annuler
 </NavLink>
 </header>

 <AdminForm categories={categories} skins={skins} bodyparts=
{bodyparts} packs={packs}
 validator={new AdminFormulaireValidator().validate} dataToUpdate=
{dataToUpdate} />

 </div>
);
};

export default AddProduct;

```

---

FICHER : code/src/pages/admin/product/delete.tsx

```
"use client";
```

```
import type React from "react";
import type { AdminProductsParams } from
"../../models/params/admin_products_params";
import { useEffect } from "react";
import ProductApiService from "../../services/product_api_service";
import { useNavigate } from "react-router";

const AdminProductDelete = ({ params }: AdminProductsParams):
React.JSX.Element => {

 // Récupérer la variable d'URL
 const { id } = params;

 // useNavigate permet de créer une redirection
 const navigate = useNavigate();

 // Executer la suppression à l'affichage de la page

 // La methode Then equivaut a Await lorsque les conditions d'await ne
sont pas rempli(etre dans une fonction et asynchrone)
 useEffect(() => {
 new ProductApiService().delete({ id: id }).then(() => {
 navigate("/admin/products");
 return;
 });
 }, [id, navigate]);

 return (

 <>
 <title> Delete</title>
 </>

);
};

export default AdminProductDelete;
```

---

FICHER : code/src/pages/admin/product/index.tsx

```
import { Link, NavLink } from "react-router";
import styles from
"../../assets/css/admin_css/admin_product_index.module.css";
import { use } from "react";
import ProductApiService from "../../services/product_api_service";
```

```

const AdminProductHome = () => {
 const results = use(new ProductApiService().selectAll()).data;

 return (
 <div className={styles.crudContainer}>
 <header className={styles.crudHeader}>
 <div>
 <h1>Gestion des Produits</h1>
 <p>Consultez et gérez votre catalogue de soins
orientaux.</p>
 </div>

 <NavLink to="/admin/products/add" className=
{styles.addButton}>
 + Ajouter un produit
 </NavLink>
 </header>

 {/* {results?.map((item) => {
 return (
 <div key={item.id}>
 <div>
 <p>{item.id} </p>
 <p>{item.name} </p>
 </div>

 <div className={styles.actions}>
 <Link to=
{`/admin/products/add/${item.id}`}>Modifier</Link>
 <Link to=
{`/admin/products/add/${item.id}`}>Supprimer</Link>
 </div>

 </div>

)
 })
 } */}

 <section className={styles.tableWrapper}>
 <table className={styles.crudTable}>
 <thead>
 <tr>
 <th>ID</th>
 <th>Nom</th>
 <th>Prix</th>
 <th>Actions</th>
 </tr>
 </thead>

 <tbody>
 {results.map((item) => {
 return (
 <tr key={item.id}>

```



```

 <td>#{item.id}</td>
 <td className={styles.productName}>
{item.name}</td>
 <td>{item.price} €</td>
 <td>
 <div className={styles.actions}>
 <Link to=
{`/admin/products/add/${item.id}`} className=
{styles.modif}>Modifier</Link>
 <Link to=
{`/admin/products/delete/${item.id}`} className=
{styles.suppr}>Supprimer</Link>
 </div>
 </td>
 </tr>
)
)}
</tbody>
</table>
</section>
</div>
);
};

export default AdminProductHome;

```

---

FICHER : code/src/pages/public/a\_propos.tsx

```

import { Link } from "react-router";
import styles from "../../assets/css/products.module.css";

const APropos = () => {
 // uniquement des composants
 // SEO

 return (
 <section className={styles.productsSection}>
 <h2 className={styles.mainTitle}>Notre Histoire</h2>

 <div className={styles.productDetailContainer}>
 /* On réutilise la carte pour encadrer le texte
proprement */
 <article className={styles.productCard} style={{ padding:
"30px" }}>
 <div className={styles.descriptionText}>
 <h3>L'Essence de Secret de hammam</h3>
 <p>
 Né d'une passion pour les rituels ancestraux,
 {" "}

```

```

 Secret de hammam est une
invitation au voyage et
 à la sérénité. Nous avons puisé au cœur des
traditions marocaines
 pour vous offrir une sélection de soins
authentiques.
 </p>

 <h3>L'Héritage du Maroc</h3>
 <p>
 Derrière chaque produit se cache un savoir-
faire séculaire. De
 l'extraction de l'huile d'Argan dans les
coopératives du sud, à la
 récolte de la Figue de Barbarie, nous
sélectionnons nos
 ingrédients pour leur pureté absolue.
 </p>

 Authenticité : Produits
sourcés en direct.

 Pureté : Sans additifs
superflus.

 Bien-être : L'expérience
du hammam à domicile.

 <Link to="/produits" className={styles.backLink}>
 ← Découvrir la collection
 </Link>
 </div>
 </article>
</div>
</section>
);
};

export default APropos;

```

---

FICHER : code/src/pages/public/contact.tsx

```
import PublicForm from "../../components/form";
import Seo from "../../components/seo";

const Contact = () => {
 return (
 <>
 <Seo
 title="Contact"
 description="Nous contacter"
 url="/contact"
 />

 <PublicForm
 title="Envoyez-nous un message"
 buttonText="Envoyer"
 type="contact"
 />
 </>
);
};

export default Contact;
```

---

FICHER : code/src/pages/public/homepage.tsx

```
import { Suspense } from "react";
import FeaturedProducts from "../../components/home/featuredProducts";
import Features from "../../components/home/features";
import Hero from "../../components/home/hero";
import { FeaturedSkeleton } from "../../components/home/product_skeleton";
import ShopPreview from "../../components/home/shopPreview";
import Seo from "../../components/seo";

const HomePage = () => {
 // uniquement des composants
 // SEO

 return (
 <main>
 <Seo title="Accueil" description="Accueil - Desk" url="/" />
 <Hero />
 <Features />
 <Suspense fallback={<FeaturedSkeleton />}>
 <FeaturedProducts />
 </Suspense>
 <ShopPreview />
 </main>
);
};
```

```
);
 };

 export default HomePage;
```

---

FICHER : code/src/pages/public/login.tsx

```
import PublicForm from "../../components/form";
import Seo from "../../components/seo";

const Login = () => {
 return (
 <>
 <Seo
 title="Connexion"
 description="Connectez-vous à votre espace client"
 url="/login"
 />

 <PublicForm
 title="Mon Compte"
 buttonText="Se connecter"
 type="login"
 />
 </>
);
};

export default Login;
```

---

FICHER : code/src/pages/public/mentions\_legales.tsx

```
import { Link } from "react-router";
import styles from "../../assets/css/products.module.css";

const MentionsLegales = () => {
 // uniquement des composants
 // SEO

 return (
 <section className={styles.productsSection}>
 <h2 className={styles.mainTitle}>Informations Légales</h2>

 <div className={styles.productDetailContainer}>
 /* L'utilisation de productCard permet de garder le cadre
```

```
blanc et l'ombre portée */}
 <article className={styles.productCard} style={{ padding:
"30px" }}>
 <div className={styles.descriptionText}>
 <h2>Mentions Légales</h2>
 <hr
 style={{
 border: "0",
 borderTop: "1px solid #f3efe4",
 margin: "20px 0",
 }}
 />

 <h3>1. Édition du Site</h3>
 <p>
 Le site Secret de hammam est
 édité par [Votre
 Nom/Entreprise], [Forme juridique] au capital
 de [Montant]€, dont
 le siège social est situé au [Adresse
 Complète]. Immatriculée au
 RCS sous le numéro SIRET [14 chiffres].
 </p>

 <h3>2. Hébergement</h3>
 <p>
 Ce site est hébergé par la société [Nom de
 l'hébergeur], située au
 [Adresse de l'hébergeur].
 </p>

 <h3>3. Protection des données (RGPD)</h3>
 <p>
 Les données collectées via le site (email,
 type de peau) sont
 exclusivement destinées à la gestion de vos
 commandes et à la
 personnalisation de votre expérience client.
 Vous disposez d'un
 droit d'accès et de suppression de vos données
 personnelles.
 </p>

 <h2>Conditions Générales de Vente</h2>
 <hr
 style={{
 border: "0",
 borderTop: "1px solid #f3efe4",
 margin: "20px 0",
 }}
 />
```

```

 <h3>Livraison</h3>
 <p>
 Nous expédions nos soins sous [48h] ouvrés.
 Les frais de port sont
 calculés lors de la validation du panier et
 sont offerts dès [50€]
 d'achat.
 </p>

 <h3>Retours et Hygiène</h3>
 <p>
 Conformément à la loi, vous disposez de 14
 jours pour retourner
 vos produits.
 Important : Pour des raisons
 d'hygiène, seuls
 les produits
 non ouverts et scellés
 seront acceptés en
 retour.
 </p>

 <Link to="/produits" className={styles.backLink}>
 ← Retour à la boutique
 </Link>
 </div>
</article>
</div>
</section>
);
};

export default MentionsLegales;

```

---

FICHER : code/src/pages/public/pack.tsx

---

FICHER : code/src/pages/public/produits.tsx

```

import ProductsList from "../../components/products/products_list";
import Seo from "../../components/seo";

const Produits = () => {

 return (

```

```

 <>
 <Seo
 title="Produits"
 description="Liste produits proposés"
 url="/produits"
 />

 <ProductsList/>

 </>
);
};

export default Produits;

```

---

FICHER : code/src/pages/public/produits\_details.tsx

```

import { use } from "react";
import type { ProduitsDetailsParams } from
"../../models/params/produits_details_params";
import ProductApiService from "../../services/product_api_service";
import Seo from "../../components/seo";
import ProductsContents from
"../../components/products/products_contents";
import type { Product } from "../../models/product";

// param permet de recuperer une variable d'URL
const ProduitsDetails = ({ params }: ProduitsDetailsParams) => {
 // récupérer l'identifiant dans les paramètres
 // déconstruction d'un objet: permet de créer des variables pour chaque
propriété d'un objet
 const { id } = params

 // Récupération des données
 const result = use(new ProductApiService().selectOne(id));

 console.log(result);

 return (
 <>
 <Seo
 // ? indique une data optionnelle. Si la data existe il
faut typer avec "as", si elle n'existe pas aucune erreur ne s'affichera.
 title={result.data?.name as string}
 description={result.data?.description as string}
 url={` /produits/${id}`}
 />
)

```

```
 <ProductsContents data={result.data as Product} />

 </>
);
};

export default ProduitsDetails;
```

---

FICHER : code/src/pages/public/register.tsx

```
import PublicForm from "../../components/form";
import Seo from "../../components/seo";

const Register = () => {
 return (
 <>
 <Seo
 title="Inscription"
 description="Créez votre compte pour profiter de nos
rituels de soin"
 url="/register"
 />

 <PublicForm
 title="Devenir Membre"
 buttonText="Créer mon compte"
 type="register"
 />
 </>
);
};

export default Register;
```

---

FICHER : code/src/services/body\_part\_api\_service.ts

```
import type { Body_part } from "../../models/body_part";
import type { ApiResponse } from "../../models/api_response";

class BodyPartApiService {
 // Préfixe de l'API
 private préfix = "/api/body_part";

 // selection de tous les enregistrement
 // On nrecoit une promesse (car mode async) qui est typer: on recoit
```



```
un type ApiResponse qui est lui même un Array(un type dans un type)
 public selectAll = async () :Promise<ApiResponse<Body_part[]>> => {

 // configurer la requête HTTP
 // import.meta.env permet d'importer une variable d'environnement
 dans vite/react. Il n'y à pas d'espace.
 const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}`);

 // exécuter la requête HTTP
 const response = await fetch(request);

 // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
 // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
 // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
 const results = await response.json();

 // retourner le resultat
 return results;

 };

 // Selection d'un enregistrement
 public selectOne = async (id: number)
:Promise<ApiResponse<Body_part>> => {

 // configurer la requête HTTP
 // import.meta.env permet d'importer une variable d'environnement
 dans vite/react. Il n'y à pas d'espace.
 const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}/${id}`);

 // exécuter la requête HTTP
 const response = await fetch(request);

 // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
 // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
 // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
 const results = await response.json();

 // retourner le resultat
 return results;

 };
}

export default BodyPartApiService;
```

FICHER : code/src/services/category\_api\_service.ts

```
import type { Category } from "../../models/category";
import type { ApiResponse } from "../../models/api_response";

class CategoryApiService {
 // Préfixe de l'API
 private préfix = "/api/category";

 // selection de tous les enregistrement
 // On nreçoit une promesse (car mode async) qui est typer: on recoit
 un type ApiResponse qui est lui même un Array(un type dans un type)
 public selectAll = async () :Promise<ApiResponse<Category[]>> => {

 // configurer la requête HTTP
 // import.meta.env permet d'importer une variable d'environnement
 dans vite/react. Il n'y à pas d'espace.
 const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}`);

 // exécuter la requête HTTP
 const response = await fetch(request);

 // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
 // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
 // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
 const results = await response.json();

 // retourner le resultat
 return results;
 };

 // Selection d'un enregistrement
 public selectOne = async (id: number) :Promise<ApiResponse<Category>>
=> {

 // configurer la requête HTTP
 // import.meta.env permet d'importer une variable d'environnement
 dans vite/react. Il n'y à pas d'espace.
 const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}/${id}`);

 // exécuter la requête HTTP
 const response = await fetch(request);

 // convertir la reponse en JSON en utilisant fonction json: ()
```

```

obligatoire
 // sérialiser: convertir des données complexes (onjet, array) en
 chaine de caractère
 // désérialiser: convertir une chaine de caractère en données
 complexes (objet, array...)
 const results = await response.json();

 // retourner le resultat
 return results;

};
}

export default CategoryApiService;

```

---

FICHER : code/src/services/pack\_api\_service.ts

```

import type { Pack } from "../../models/pack";
import type { ApiResponse } from "../../models/api_response";

class PackApiService {
 // Préfixe de l'API
 private préfix = "/api/pack";

 // selection de tous les enregistrement
 // On nreçoit une promesse (car mode async) qui est typer: on recoit
 un type ApiResponse qui est lui même un Array(un type dans un type)
 public selectAll = async () :Promise<ApiResponse<Pack[]>> => {

 // configurer la requête HTTP
 // import.meta.env permet d'importer une variable d'environnement
 dans vite/react. Il n'y à pas d'espace.
 const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}`);

 // exécuter la requête HTTP
 const response = await fetch(request);

 // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
 // sérialiser: convertir des données complexes (onjet, array) en
 chaine de caractère
 // désérialiser: convertir une chaine de caractère en données
 complexes (objet, array...)
 const results = await response.json();

 // retourner le resultat
 return results;

 };
}

```

```

 // Selection d'un enregistrement
 public selectOne = async (id: number) :Promise<ApiResponse<Pack>> =>
 {

 // configurer la requête HTTP
 // import.meta.env permet d'importer une variable d'environnement
 dans vite/react. Il n'y à pas d'espace.
 const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}/${id}`);

 // exécuter la requête HTTP
 const response = await fetch(request);

 // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
 // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
 // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
 const results = await response.json();

 // retourner le resultat
 return results;

 };
}

export default PackApiService;

```

---

FICHER : code/src/services/product\_api\_service.ts

```

import type { Product } from "../../models/product";
import type { ApiResponse } from "../../models/api_response";

class ProductApiService {
 // Préfixe de l'API
 private préfix = "/api/product";

 // selection de tous les enregistrement
 // On nreçoit une promesse (car mode async) qui est typer: on recoit
un type ApiResponse qui est lui même un Array(un type dans un type)
 public selectAll = async () :Promise<ApiResponse<Product[]>> => {

 // configurer la requête HTTP
 // import.meta.env permet d'importer une variable d'environnement
dans vite/react. Il n'y à pas d'espace.
 const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}`);

```

```

 // exécuter la requête HTTP
 const response = await fetch(request);

 // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
 // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
 // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
 const results = await response.json();

 // retourner le resultat
 return results;

 };

 // Selection d'un enregistrement
 public selectOne = async (id: number) :Promise<ApiResponse<Product>>
=> {

 // configurer la requête HTTP
 // import.meta.env permet d'importer une variable d'environnement
dans vite/react. Il n'y à pas d'espace.
 const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}/${id}`);

 // exécuter la requête HTTP
 const response = await fetch(request);

 // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
 // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
 // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
 const results = await response.json();

 // retourner le resultat
 return results;

 };

 // Insertion d'un enregistrement
 /* Si le formulaire contient un champ de fichier: utiliser FormData en
parametre
 Si le formulaire ne contient pas de champ de gichier: utyliser le
stype */
 public insert = async (data: FormData)
:Promise<ApiResponse<Product>> => {

 // configurer la requête HTTP
 // import.meta.env permet d'importer une variable d'environnement
dans vite/react. Il n'y à pas d'espace.
 const request = new Request(

```

```

 `${import.meta.env.VITE_API_URL}${this.préfix}`,
 {
 method: "post",
 /* Si le formulaire contient un champs de fichier
 la propriété body renvoi un objet formData
 */
 body: data

 /*
 Si le formulaire ne contient pas de champs de
fichier
 la propriété body renvoir du JSON :
JSON.stringify(...)
 ajouter l'en-tête Content-Type:
applicatio,/json

 body: JSON.stringify(data)
 headers:{
 "Content-Type": "application/json",
 },
 */
 }
);

 // exécuter la requête HTTP
 const response = await fetch(request);

 // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
 // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
 // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
 const results = await response.json();

 // retourner le resultat
 return results;
};

// Mise à jour d'un enregistrement
/* Si le formulaire contient un champ de fichier: utiliser FormData en
parametre
 Si le formulaire ne contient pas de champ de gichier: utiliser le
stype */
 public update = async (data: FormData) :Promise<ApiResponse<Product>>
=> {

 // configurer la requête HTTP
 // import.meta.env permet d'importer une variable d'environnement
dans vite/react. Il n'y à pas d'espace.
 const request = new Request(
 `${import.meta.env.VITE_API_URL}${this.préfix}`,

```

```

 {
 method: "put",
 /* Si le formulaire contient un champs de fichier
 la propriété body renvoi un objet formData
 */
 body: data

 /*
 Si le formulaire ne contient pas de champs de
fichier
 la propriété body renvoi du JSON :
JSON.stringify(...)
 ajouter l'en-tête Content-Type:
applicatio,/json

 body: JSON.stringify(data)
 headers:{
 "Content-Type": "application/json",
 },
 */
 }
);

 // exécuter la requête HTTP
 const response = await fetch(request);

 // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
 // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
 // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
 const results = await response.json();

 // retourner le resultat
 return results;

};

 public delete = async (data: Partial<Product>)
:Promise<ApiResponse<Product>> => {

 // configurer la requête HTTP
 // import.meta.env permet d'importer une variable d'environnement
dans vite/react. Il n'y à pas d'espace.
 const request = new Request(
 `${import.meta.env.VITE_API_URL}${this.préfix}`,
 {
 method: "delete",
 /* Si le formulaire contient un champs de fichier
 la propriété body renvoi un objet formData

 body: data */
 }
);
 }
}

```

```

 /*
 Si le formulaire ne contient pas de champs de
fichier
 la propriété body renvoie du JSON :
JSON.stringify(...)
ajouter l'en-tête Content-Type:
applicatio,/json */

 headers:{
 "Content-Type": "application/json",
 /* Serialiser = stringify ==> Transformer une
donnée complexe (array, objet) en chaîne de caractères
Deserialiser = parse ==> Transformer une
chaîne de caractère en donnée complexe (array, objet) */
 },
 body: JSON.stringify(data),

 }

);

 // exécuter la requête HTTP
 const response = await fetch(request);

 // convertir la réponse en JSON en utilisant fonction json: ()
obligatoire
 // sérialiser: convertir des données complexes (objet, array) en
chaîne de caractère
 // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
 const results = await response.json();

 // retourner le résultat
 return results;

};
}

export default ProductApiService;

```

---

FICHER : code/src/services/router\_service.ts

```

import type { unstable_RSCRouteConfig as RSCRouteConfig } from "react-
router";

class RouterService {
 public getRouter = () => {
 return [
 {
 // Identifiant unique de la route
 }
]
 }
}

```



```
 id: "root",
 // Préfixe des routes
 path: "/",
 // Importation de la mise en page
 lazy: () => import("../layouts/root_layout"),
 children: [
 {
 id: "public",
 path: "",
 lazy: () => import("../layouts/public_layout"),
 children: [
 {
 id: "home",
 index: true,
 path: "",
 lazy: () =>
import("../pages/public/homepage"),
 },
 {
 id: "produits",
 path: "produits",
 lazy: () =>
import("../pages/public/produits"),
 },
 {
 id: "produits_details",
 // path représente la route
 // variable d'URL est préfixée d'in :
 path: "produits/:id",
 lazy: () =>
import("../pages/public/produits_details"),
 },
 {
 id: "register",
 path: "register",
 lazy: () =>
import("../pages/public/register"),
 },
 {
 id: "login",
 path: "login",
 lazy: () =>
import("../pages/public/login"),
 },
 {
 id: "a_propos",
 path: "a_propos",
 lazy: () =>
import("../pages/public/a_propos"),
 },
 {
 id: "contact",
 path: "contact",
 lazy: () =>
```

```

import("../pages/public/contact"),
 },
 {
 id: "mentions_legales",
 path: "mentions_legales",
 lazy: () =>
import("../pages/public/mentions_legales"),
 },
],
 },
 {
 id: "admin",
 path: "/admin",
 lazy: () => import("../layouts/admin_layout"),
 children: [
 {
 id: "admin-home",
 path: "",
 lazy: () => import("../pages/admin/index"),
 },
 {
 id: "admin-product",
 path: "products",
 lazy: () =>
import("../pages/admin/product/index"),
 },
 {
 id: "add-product",
 // path représente la route
 // variable d'URL est préfixée d'un :
 // variable d'URL optionnelle est suffixée
d'un ?
 path: "products/add/:id?",
 lazy: () =>
import("../pages/admin/product/add"),
 },
 {
 id: "admin-product-delete",
 path: "products/delete/:id",
 lazy: () =>
import("../pages/admin/product/delete"),
 },
],
 },
 /*
 {
 id: "about",
 path: "about",
 lazy: () => import("../about/route"),
 },*/
],
},
] satisfies RSCRouteConfig;
};

```

```
}

export default RouterService;
```

---

FICHER : code/src/services/skin\_api\_service.ts

```
import type { Skin } from "../../models/skin";
import type { ApiResponse } from "../models/api_response";

class SkinApiService {
 // Préfixe de l'API
 private préfix = "/api/skin";

 // selection de tous les enregistrement
 // On nreçoit une promesse (car mode async) qui est typer: on recoit
 un type ApiResponse qui est lui même un Array(un type dans un type)
 public selectAll = async () :Promise<ApiResponse<Skin[]>> => {

 // configurer la requête HTTP
 // import.meta.env permet d'importer une variable d'environnement
 dans vite/react. Il n'y à pas d'espace.
 const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}`);

 // exécuter la requête HTTP
 const response = await fetch(request);

 // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
 // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
 // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
 const results = await response.json();

 // retourner le resultat
 return results;

 };

 // Selection d'un enregistrement
 public selectOne = async (id: number) :Promise<ApiResponse<Skin>> =>
{

 // configurer la requête HTTP
 // import.meta.env permet d'importer une variable d'environnement
 dans vite/react. Il n'y à pas d'espace.
 const request = new
Request(`${import.meta.env.VITE_API_URL}${this.préfix}/${id}`);
```

```

 // exécuter la requête HTTP
 const response = await fetch(request);

 // convertir la reponse en JSON en utilisant fonction json: ()
obligatoire
 // sérialiser: convertir des données complexes (onjet, array) en
chaîne de caractère
 // désérialiser: convertir une chaîne de caractère en données
complexes (objet, array...)
 const results = await response.json();

 // retourner le resultat
 return results;

 };
}

export default SkinApiService;

```

---

FICHER : code/src/validators/admin/admin\_formulaire\_validator.ts

```

import { z, type ZodError } from "zod";
import type { Product } from "../../models/product";

class AdminFormulaireValidator {

 // validation des données du formulaire
 public validate = async (data: Partial<Product>,
): Promise<Partial<Product> | ZodError> => {
 // La méthode doit être exécutée côté serveur
 "use server";

 // Contraintes de validation
 // Reprendre strictement les propriétés du type à valider
 const constraints = z.object({
 id: z.union([
 z.string().nullable(),
 // coerce : permet de transtyper les donnés (string
devient number)
 z.coerce
 .number()
 .positive(),
]),

 name: z.string("Le nom est obligatoire").min(2, "Un nom doit
comporter au minimum 2 caractères").max(50, "Un nom doit comporté au
maximum 50 caracteres"),

 image: z.union([

```

```

 z.string().nullable(),
 z.file("L'image est obligatoire"),
]),

 price: z.coerce.number("Le prix est obligatoire").min(1, "Un
prix doit comporter au minimum 1 chiffre").max(99.99, "Un prix peut
comporter au maximum 4 caracteres dont 2 decimales"),

 description: z.string("La description est
obligatoire").min(20, "Une description doit comporter au minimum 20 "),

 category_id: z.coerce.number("Veuillez selectionner une
category").positive(),

 skin_ids: z.string("Veuillez selectionner un ou plusieurs type
de peau").min(1, "Selectionnez au moins un type de peau"),

 body_part_ids: z.string("Veuillez selectionner une ou
plusieurs zone du corps").min(1, "Selectionnez au moins une zone du
corps"),

 pack_ids: z.string("Veuillez selectionner une ou plusieurs
pack").min(1, "Selectionnez au moins un pack"),
 });

 // Validation de la saisie du formulaire

 const validation = await constraints.safeParseAsync(data);

 // Si la validation échoue
 if (!validation.success) {
 return validation.error;
 }
 // Si la validation réussie
 return validation.data as Partial<Product>;
};
}

export default AdminFormulaireValidator;

```

---

FICHER : code/tsconfig.json

```

{
 "compilerOptions": {
 "erasableSyntaxOnly": true,
 "allowImportingTsExtensions": true,
 "strict": true,
 "noUnusedLocals": true,
 "noUnusedParameters": true,
 "skipLibCheck": true,

```

```
 "verbatimModuleSyntax": true,
 "noEmit": true,
 "moduleResolution": "Bundler",
 "module": "ESNext",
 "target": "ESNext",
 "lib": ["ESNext", "DOM", "DOM.Iterable"],
 "types": ["vite/client", "@vitejs/plugin-rsc/types"],
 "jsx": "react-jsx"
 }
}
```

---

FICHER : code/vite.config.ts

```
import react from "@vitejs/plugin-react";
import rsc from "@vitejs/plugin-rsc";
import { defineConfig } from "vite";

export default defineConfig(() => {
 return {
 server: {
 port: 5173,
 host: true,
 },
 plugins: [
 rsc({
 // `entries` option is only a shorthand for specifying
 each `rollupOptions.input` below
 // > entries: { rsc, ssr, client },
 //
 // by default, the plugin setup request handler based on
 `default export` of `rsc` environment `rollupOptions.input.index`.
 // This can be disabled when setting up own server handler
 e.g. `@cloudflare/vite-plugin`.
 // > serverHandler: false
 }),

 // use any of react plugins https://github.com/vitejs/vite-
 plugin-react
 // to enable client component HMR
 react(),

 // use https://github.com/antfu-collective/vite-plugin-inspect
 // to understand internal transforms required for RSC.
 // import("vite-plugin-inspect").then(m => m.default()),
],

 // specify entry point for each environment.
 // (currently the plugin assumes `rollupOptions.input.index` for
 some features.)
 }
})
```

```

 environments: {
 // `rsc` environment loads modules with `react-server`
condition.
 // this environment is responsible for:
 // - RSC stream serialization (React VDOM -> RSC stream)
 // - server functions handling
 rsc: {
 build: {
 rollupOptions: {
 input: {
 index: "./src/framework/entry.rsc.tsx",
 },
 },
 },
 },

 // `ssr` environment loads modules without `react-server`
condition.
 // this environment is responsible for:
 // - RSC stream deserialization (RSC stream -> React VDOM)
 // - traditional SSR (React VDOM -> HTML string/stream)
 ssr: {
 build: {
 rollupOptions: {
 input: {
 index: "./src/framework/entry.ssr.tsx",
 },
 },
 },
 },

 // client environment is used for hydration and client-side
rendering
 // this environment is responsible for:
 // - RSC stream deserialization (RSC stream -> React VDOM)
 // - traditional CSR (React VDOM -> Browser DOM tree
mount/hydration)
 // - refetch and re-render RSC
 // - calling server functions
 client: {
 build: {
 rollupOptions: {
 input: {
 index: "./src/framework/entry.browser.tsx",
 },
 },
 },
 },
 },
 };
});

```

